



MODUL PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

[SVPL214610]

Oleh:

**Dr. Eng. Ir. Ganjar Alfian, S.T., M. Eng.,
dan Tim Mahasiswa**

NIP. 111198701202201101

TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2025

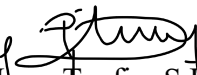
HALAMAN PENGESAHAN

MODUL PRAKTIKUM

Nama mata kuliah : Praktikum Penambangan Data
Kode Mata Kuliah : SVPL214610
Nama Penulis : Dr.Eng. Ir. Ganjar Alfian, S.T., M.Eng.
NIP : 111198701202201101
Departemen / Prodi : Teknik Elektro dan Informatika/ Teknologi Rekayasa Perangkat Lunak

Yogyakarta, 09 – 12 – 2025

Menyetujui,
Ketua Program Studi


Dr. Umar Taufiq, S.Kom., M.Cs.
NIP. 111198212201610101

Penulis


Dr.Eng. Ir. Ganjar Alfian, S.T., M.Eng.
NIP. 111198701202201101

Mengetahui,
Ketua Departemen Teknik Elektro dan Informatika,


Ir. Nur Rohman Rosyid, S.T., M.T., D.Eng., IPM.
NIP. 111197510201206101

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga Modul Praktikum Penambangan Data ini dapat diselesaikan dengan baik. Penyusunan modul ini bertujuan untuk memberikan panduan pembelajaran yang sistematis bagi mahasiswa dalam memahami konsep, teknik, dan implementasi penambangan data, khususnya pada Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi Universitas Gadjah Mada.

Modul ini diharapkan dapat menjadi referensi pembelajaran yang efektif dalam kegiatan praktikum serta membantu mahasiswa meningkatkan kemampuan analisis data, pengolahan data, dan penerapan algoritma penambangan data menggunakan berbagai pendekatan dan perangkat lunak pendukung. Selain itu, modul ini diharapkan juga dapat mendorong mahasiswa untuk lebih aktif mengeksplorasi penelitian dan inovasi di bidang data mining.

Ucapan terima kasih kami sampaikan kepada semua pihak yang telah berkontribusi dalam penyusunan modul ini, termasuk dosen, asisten laboratorium, dan rekan-rekan yang telah memberikan masukan serta dukungan selama proses penyusunan berlangsung.

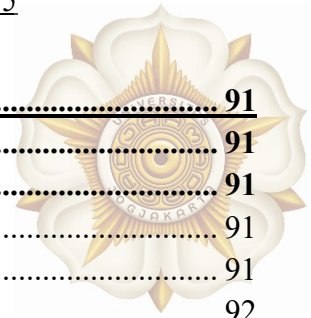
Kami menyadari bahwa modul ini masih jauh dari sempurna. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari para pembaca demi penyempurnaan modul ini pada edisi berikutnya.

Yogyakarta, 09-12-2025

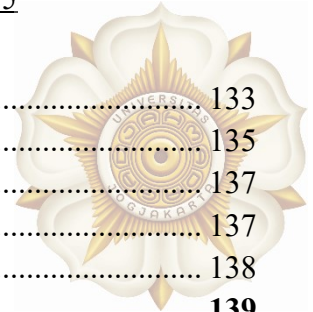
Penyusun

DAFTAR ISI

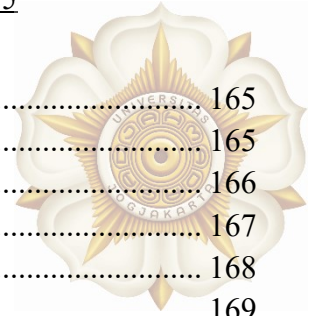
HALAMAN PENGESAHAN.....	2
KATA PENGANTAR.....	3
DAFTAR ISI.....	4
DAFTAR GAMBAR.....	9
DAFTAR TABEL	17
PERTEMUAN I NUMPY DAN PANDAS	18
1.1. TUJUAN PEMBELAJARAN	18
1.2. DASAR TEORI.....	18
A. NUMPY	18
B. PANDAS	18
1.3. ALAT DAN BAHAN.....	19
A. PERANGKAT KERAS	19
B. PERANGKAT LUNAK	19
1.4. LANGKAH PERCOBAAN.....	19
A. NUMPY.....	19
B. DATAFRAME PANDAS	32
1.5. TUGAS & ANALISIS.....	55
1.6. REFERENSI.....	55
PERTEMUAN II EXPLORATORY DATA ANALYSIS DAN DATA PREPROCESSING	56
2.1. TUJUAN PEMBELAJARAN	56
2.2. DASAR TEORI.....	56
A. ARTIFICIAL INTELLIGENCE	56
B. MACHINE LEARNING	56
C. EXPLORATORY DATA ANALYSIS	57
D. DATA PREPROCESSING	57
2.3. ALAT DAN BAHAN.....	58
A. PERANGKAT KERAS	58
B. PERANGKAT LUNAK	58
2.4. LANGKAH PERCOBAAN.....	58
A. EXPLORATORY DATA ANALYSIS (EDA).....	58
B. DATA PREPROCESSING	76
2.5. TUGAS & ANALISIS.....	89
A. EXPLORATORY DATA ANALYSIS	89
B. DATA PREPROCESSING	90
2.6. REFERENSI.....	90



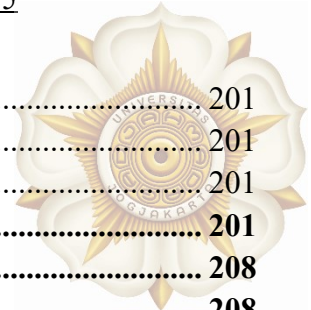
PERTEMUAN III CLASSIFICATION.....	91
3.1. TUJUAN PEMBELAJARAN	91
3.2. DASAR TEORI.....	91
A. CLASSIFICATION	91
B. ALGORITMA CLASSIFICATION.....	91
C. METRIK EVALUASI CLASSIFICATION	92
3.3. ALAT DAN BAHAN.....	93
A. PERANGKAT KERAS	93
B. PERANGKAT LUNAK	93
3.4. LANGKAH PERCOBAAN.....	94
3.5. TUGAS & ANALISIS.....	109
3.6. REFERENSI.....	109
PERTEMUAN IV REGRESSION	110
4.1. TUJUAN PEMBELAJARAN	110
4.2. DASAR TEORI.....	110
A. REGRESI.....	110
B. ALGORITMA REGRESI	110
C. METRIK EVALUASI REGRESI	112
4.3. ALAT DAN BAHAN.....	113
A. PERANGKAT KERAS	113
B. PERANGKAT LUNAK	113
4.4. LANGKAH PERCOBAAN.....	113
4.5. TUGAS & ANALISIS.....	126
4.6. REFERENSI.....	126
PERTEMUAN V FORECASTING.....	128
5.1 TUJUAN PEMBELAJARAN	128
5.2 DASAR TEORI	128
A. SUPERVISED LEARNING	128
B. FORECASTING	128
C. RMSE	129
D. PEARSON CORRELATION COEFFICIENT	129
5.3 ALAT DAN BAHAN.....	129
A. PERANGKAT KERAS	129
B. PERANGKAT LUNAK	129
5.4 LANGKAH PERCOBAAN.....	129
1. PERSIAPAN IDE	130
2. IMPORT LIBRARY	130
3. IMPORT DATASET	131
4. MEMBENTUK DATE TIME SERIES.....	131
5. MENAMBAHKAN FITUR STATISTIK	132



6. PEMROSESAN DATA	133
7. PELATIHAN MODEL MACHINE LEARNING	135
8. MELAKUKAN PREDIKSI	137
9. VISUALISASI HASIL	137
10. EVALUASI MODEL	138
5.5 TUGAS DAN ANALISIS.....	139
5.6 REFERENSI.....	139
<u>PERTEMUAN VI CLUSTERING.....</u>	<u>141</u>
6.1 TUJUAN PEMBELAJARAN	141
6.2 DASAR TEORI	141
A. UNSUPERVISED LEARNING	141
B. CLUSTERING	141
C. K-MEANS	142
D. K-MEDOIDS.....	142
E. DBSCAN	143
F. ASSOCIATION RULE	143
G. INERTIA	143
H. SILHOUTTE SCROE.....	143
6.3 ALAT DAN BAHAN.....	144
A. PERANGKAT KERAS	144
B. PERANGKAT LUNAK	144
6.4 LANGKAH PERCOBAAN.....	144
1. PERSIAPKAN IDE	144
2. INSTALASI DAN IMPOR LIBRARY	144
3. READ DAN LOAD DATASET INFORMATION.....	145
4. DATA CLEANING.....	146
5. STANDARISASI DATA	147
6. MENENTUKAN JUMLAH CLUSTER (ELBOW METHOD & SILHOUTTE SCORE).....	148
7. MENERAPKAN ALGORITMA K-MEANS.....	153
8. MENERAPKAN ALGORITMA K-MEDOIDS	156
9. ANALISIS DAN INTERPRETASI CLUSTER	158
6.5 TUGAS DAN ANALISIS.....	159
6.6 REFERENSI.....	160
<u>PERTEMUAN VII IMBALANCED DATASET</u>	<u>162</u>
7.1 TUJUAN PEMBELAJARAN	162
7.2 DASAR TEORI	162
7.3 ALAT DAN BAHAN.....	165
A. PERANGKAT KERAS	165
B. PERANGKAT LUNAK	165
7.4 LANGKAH PERCOBAAN.....	165



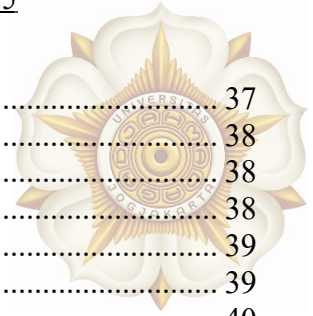
1. PERSIAPKAN IDE	165
2. INSTALASI DAN IMPOR LIBRARY	165
3. IMPORT DAN LOAD DATASET INFORMATION	166
4. EXPLORATORY DATA ANALYSIS.....	167
5. DATA CLEANING.....	168
6. RANDOM UNDER SAMPLING	169
7. RANDOM OVER SAMPLING	172
8. SMOTE.....	175
9. ADASYN.....	178
7.5 TUGAS DAN ANALISIS.....	180
7.6 REFERENSI.....	180
PERTEMUAN VIII NEURAL NETWORK.....	182
8.1 TUJUAN PEMBELAJARAN	182
8.2 DASAR TEORI	182
A. NEURAL NETWORK.....	182
B. MULTILAYER PERCEPTRON	182
C. BACKPROPAGATION LEARNING.....	183
D. ACTIVATION FUNCTION.....	183
E. KLASIFIKASI	183
F. REGRESI.....	184
8.3 ALAT DAN BAHAN.....	184
A. PERANGKAT KERAS	184
B. PERANGKAT LUNAK	184
8.4 LANGKAH PERCOBAAN.....	184
1. PERSIAPKAN IDE	184
2. INSTALASI DAN IMPOR LIBRARY	184
3. IMPORT DAN LOAD DATASET INFORMATION	185
4. EXPLORATORY DATA ANALYSIS.....	185
5. DATA CLEANING.....	185
6. MLP UNTUK KLASIFIKASI.....	185
7. MLP UNTUK REGRESI	191
8.5 TUGAS DAN ANALISIS.....	194
8.6 REFERENSI.....	194
PERTEMUAN IX CONVOLUTIONAL NEURAL NETWORK.....	196
9.1. TUJUAN PEMBELAJARAN	196
9.2. DASAR TEORI	196
A. REPRESENTASI CITRA DAN KETERBATASAN MLP	196
B. ARSITEKTUR CONVOLUTIONAL NEURAL NETWORK (CNN).....	199
C. DROP OUT	200
9.3. ALAT DAN BAHAN.....	200



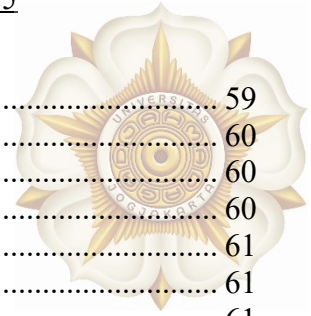
A. PERANGKAT KERAS	201
B. PERANGKAT LUNAK	201
C. PUSTAKA PYTHON.....	201
9.4. LANGKAH PERCOBAAN.....	201
9.5. TUGAS & ANALISIS.....	208
9.6. REFERENSI.....	208
<u>PERTEMUAN X MODEL DEPLOYMENT (FLASK)</u>	<u>210</u>
10.1. TUJUAN PEMBELAJARAN	210
10.2. DASAR TEORI.....	210
A. TAHAPAN PENGEMBANGAN MACHINE LEARNING	210
B. MODEL DEPLOYMENT	211
C. FLASK FRAMEWORK.....	212
D. MODEL SERIALIZATION.....	213
10.3. ALAT DAN BAHAN.....	213
A. PERANGKAT KERAS	213
B. PERANGKAT LUNAK	213
C. PUSTAKA PYTHON.....	213
10.4. LANGKAH PERCOBAAN.....	213
A. TUGAS & ANALISIS.....	222
B. REFERENSI	223
<u>PERTEMUAN XI MODEL DEPLOYMENT (STREAMLIT).....</u>	<u>224</u>
11.1. TUJUAN PEMBELAJARAN	224
C. DASAR TEORI	224
A. MENGENAL STREAMLIT.....	224
B. PERBEDAAN FLASK DAN STREAMLIT.....	224
C. KOMPONEN UTAMA STREAMLIT	226
11.2. ALAT DAN BAHAN.....	228
A. PERANGKAT KERAS	228
B. PERANGKAT LUNAK	228
C. PUSTAKA PYTHON.....	228
11.3. LANGKAH PERCOBAAN.....	228
11.4. TUGAS & ANALISIS.....	238
11.5. REFERENSI.....	238

DAFTAR GAMBAR

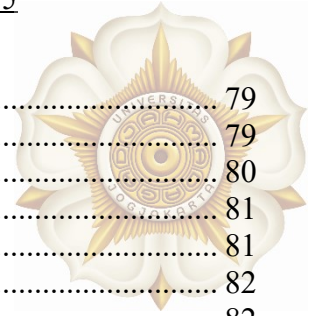
Gambar 1.4.1 Instalasi NumPy	19
Gambar 1.4.2 Impor pustaka NumPy	20
Gambar 1.4.3 Membuat array 1 dimensi	20
Gambar 1.4.4 Melihat bentuk array	20
Gambar 1.4.5 Membuat array 2 dimensi dan mengecek bentuk array	21
Gambar 1.4.6 Membuat array 3 dimensi	22
Gambar 1.4.7 Array String.....	22
Gambar 1.4.8 Indexing array 1 dimensi.....	23
Gambar 1.4.9 Indexing array 2 dimensi.....	23
Gambar 1.4.10 Indexing array 3 dimensi.....	23
Gambar 1.4.11 Contoh array 3 dimensi	24
Gambar 1.4.12 Implementasi Array.....	24
Gambar 1.4.13 Target slicing.....	25
Gambar 1.4.14 Contoh penerapan Slicing	25
Gambar 1.4.15 Konversi tipe data di NumPy	26
Gambar 1.4.16 Contoh Reshape ke dimensi lebih tinggi.....	27
Gambar 1.4.17 Reshape ke dimensi 1	28
Gambar 1.4.18 Len vs Shape	28
Gambar 1.4.19 Iterasi langsung	28
Gambar 1.4.20 Iterasi dengan indeks.....	29
Gambar 1.4.21 Concatenate 2 array.....	29
Gambar 1.4.22 Menambahkan Array di sumbu 1	29
Gambar 1.4.23 Menambahkan Array di sumbu 0	30
Gambar 1.4.24 Array Split.....	30
Gambar 1.4.25 Mencari elemen dalam array	30
Gambar 1.4.26 Sorting NumPy.....	31
Gambar 1.4.27 Numpy Random	31
Gambar 1.4.28 Penjumlahan Array	31
Gambar 1.4.29 Summation	32
Gambar 1.4.30 Product of elements NumPy	32
Gambar 1.4.31 Instalasi pandas	32
Gambar 1.4.32 Import Library pandas.....	32
Gambar 1.4.33 Membuat Dataframe	33
Gambar 1.4.34 Membuat Dataframe	33
Gambar 1.4.35 Menambahkan Kolom.....	34
Gambar 1.4.36 Menambahkan Kolom.....	34
Gambar 1.4.37 Menambahkan Kolom.....	34
Gambar 1.4.38 Menambahkan Multi Kolom.....	35
Gambar 1.4.39 Menambahkan Multi Kolom.....	35
Gambar 1.4.40 Menghapus Kolom.....	36
Gambar 1.4.41 Menghapus Kolom dengan Fungsi pop	36
Gambar 1.4.42 Menghapus Kolom dengan Perintah del	37



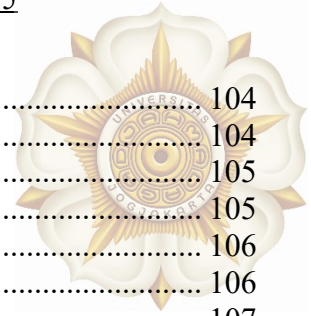
Gambar 1.4.43 Menghapus Multi Kolom	37
Gambar 1.4.44 Membuat Dataframe	38
Gambar 1.4.45 Sorting Data	38
Gambar 1.4.46 Sorting Data	38
Gambar 1.4.47 Sorting Data Berdasarkan Kolom Tertentu.....	39
Gambar 1.4.48 Sorting Data Berdasarkan Kolom Tertentu.....	39
Gambar 1.4.49 Filtering Data dengan Kondisi	40
Gambar 1.4.50 Filtering Data dengan Kondisi	40
Gambar 1.4.51 Filtering Data dengan Fungsi isin()	41
Gambar 1.4.52 Filtering Data dengan Query.....	41
Gambar 1.4.53 Filtering Data dengan Query.....	41
Gambar 1.4.54 Filtering Data dengan fungsi str.contains().....	41
Gambar 1.4.55 Membuat Dataframe	42
Gambar 1.4.56 Menggabungkan Dataframe	42
Gambar 1.4.57 Menampilkan Data Menurut Pekerjaan	42
Gambar 1.4.58 Menampilkan Satu Kategori Tertentu.....	43
Gambar 1.4.59 Aggregation.....	43
Gambar 1.4.60 Aggregation.....	44
Gambar 1.4.61 Aggregation.....	44
Gambar 1.4.62 Merging Dataframe	45
Gambar 1.4.63 Merging Dataframe	45
Gambar 1.4.64 Merging Dataframe	46
Gambar 1.4.65 Joining Dataframe	46
Gambar 1.4.66 Joining Dataframe	47
Gambar 1.4.67 Concatenate	47
Gambar 1.4.68 Concatenate	48
Gambar 1.4.69 Concatenate	48
Gambar 1.4.70 Reset Index.....	49
Gambar 1.4.71 Reset Index.....	49
Gambar 1.4.72 Set Index	50
Gambar 1.4.73 Set Index	50
Gambar 1.4.74 Set Index	51
Gambar 1.4.75 Membuat Dataframe	51
Gambar 1.4.76 Pivot Table	52
Gambar 1.4.77 Pivot Table	52
Gambar 1.4.78 Membuat Dataframe	52
Gambar 1.4.79 Melt Table	53
Gambar 1.4.80 Lambda Function	53
Gambar 1.4.81 Lambda Function	53
Gambar 1.4.82 Lambda Function	54
Gambar 1.4.83 Lambda Function	54
Gambar 1.4.84 Menghubungkan Google Colaboratory dengan Google Drive	54
Gambar 1.4.85 Membaca File dari Google Drive.....	55
Gambar 2.4.1 Import Library.....	58
Gambar 2.4.2 Lima Baris Pertama Dataset.....	59
Gambar 2.4.3 Lima Baris Terakhir Dataset	59



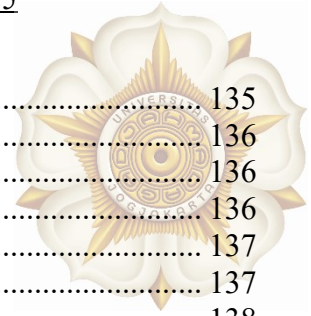
Gambar 2.4.4 Pemisahan Dataset Bank Churner	59
Gambar 2.4.5 Pembuatan Line Chart dengan Data Dummy	60
Gambar 2.4.6 Line Chart dari Data Dummy	60
Gambar 2.4.7 Load Dataset IoT	60
Gambar 2.4.8 Sampel Data IoT	61
Gambar 2.4.9 Pembuatan Line Chart dengan Data IoT	61
Gambar 2.4.10 Line Chart dengan Data IoT	61
Gambar 2.4.11 Pembuatan Pie Chart dengan Data Dummy	62
Gambar 2.4.12 Pie Chart dengan Data Dummy	62
Gambar 2.4.13 Pembuatan Pie Chart dengan Data Bank Churner	63
Gambar 2.4.14 Pie Chart Marital Status	63
Gambar 2.4.15 Pembuatan Bar Plot dengan Data Dummy	64
Gambar 2.4.16 Bar Plot dengan Data Dummy	64
Gambar 2.4.17 Pembuaran Bar Plot Marital Status	65
Gambar 2.4.18 Bar Plot Marital Status	65
Gambar 2.4.19 Pembuatan Stacked Bar Plot Marital Status	66
Gambar 2.4.20 Stacked Bar Plot Marital Status	66
Gambar 2.4.21 Pembuatan Histogram dengan Data Dummy	67
Gambar 2.4.22 Histogram dengan Data Dummy	67
Gambar 2.4.23 Pembuatan Histogram Customer Age	67
Gambar 2.4.24 Histogram Customer Age	68
Gambar 2.4.25 Pembuatan Scatter Plot dengan plt.scatter	68
Gambar 2.4.26 Scatter Plot Customer Age vs Months on Book	69
Gambar 2.4.27 Pembuatan Scatter Plot dengan fungsi plot.scatter	69
Gambar 2.4.28 Scatter Plot Customer Age vs Months on Book	70
Gambar 2.4.29 Pembuatan Box Plot dengan plt.boxplot	70
Gambar 2.4.30 Box Plot Customer Age	70
Gambar 2.4.31 Pembuatan Box Plot dengan fungsi plot.box	71
Gambar 2.4.32 Box Plot Customer Age	71
Gambar 2.4.33 Pembuatan Violin Plot	71
Gambar 2.4.34 Violin Plot Customer Age	72
Gambar 2.4.35 Pembuatan Sub Plot	72
Gambar 2.4.36 Sub Plot	73
Gambar 2.4.37 Pembuatan Annotation	73
Gambar 2.4.38 Annotation pada Histogram	74
Gambar 2.4.39 Pengaturan Axis	74
Gambar 2.4.40 Line Chart	75
Gambar 2.4.41 Pembuatan Legend	75
Gambar 2.4.42 Histogram dengan Legend	76
Gambar 2.4.43 Import Library	76
Gambar 2.4.44 Lima Baris Pertama Dataset	77
Gambar 2.4.45 Lima Baris Terakhir Dataset	77
Gambar 2.4.46 Melihat Bentuk Dataset dengan df.shape	77
Gambar 2.4.47 Membuat Pie Chart	77
Gambar 2.4.48 Pie Chart Distribusi Data	78
Gambar 2.4.49 Membuat Histogram	78



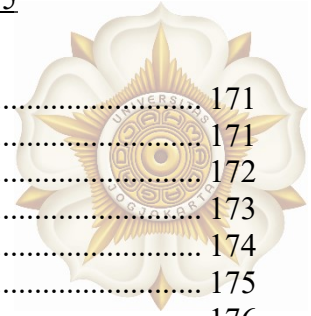
Gambar 2.4.50 Histogram Kolom Numerik	79
Gambar 2.4.51 Korelasi Antar Variabel	79
Gambar 2.4.52 Deskripsi Statistik	80
Gambar 2.4.53 Cek Missing Value.....	81
Gambar 2.4.54 Cek Data Duplikat.....	81
Gambar 2.4.55 Imputasi Missing Value	82
Gambar 2.4.56 Missing Value	82
Gambar 2.4.57 Memeriksa Atribut Kategorikal	83
Gambar 2.4.58 Atribut Kategorikal	83
Gambar 2.4.59 One Hot Encoding.....	83
Gambar 2.4.60 Hasil One Hot Encoding	84
Gambar 2.4.61 One Hot Encoding dengan OneHotEncoder	84
Gambar 2.4.62 Hasil One Hot Encoding	84
Gambar 2.4.63 Label Encoding	85
Gambar 2.4.64 Hasil Label Encoding.....	85
Gambar 2.4.65 Label Encoding untuk y	86
Gambar 2.4.66 Hasil Label Encoding.....	86
Gambar 2.4.67 Cek Korelasi Atribut	87
Gambar 2.4.68 Korelasi Antar Variabel	87
Gambar 2.4.69 Pembagian Dataset.....	87
Gambar 2.4.70 Sampel Data Training	88
Gambar 2.4.71 Scaling dengan StandardScaler.....	88
Gambar 2.4.72 Sampel Data Training Setelah Scaling.....	89
Gambar 2.4.73 Scaling dengan MinMaxScaler	89
Gambar 2.4.74 Sampel Data Training Setelah Scaling.....	89
Gambar 3.4.1 Import Pustaka	94
Gambar 3.4.2 Lima Baris Pertama Dataset.....	94
Gambar 3.4.3 Lima Baris Terakhir Dataset	95
Gambar 3.4.4 Hasil Deskripsi Statistik Dataset.....	95
Gambar 3.4.5 Presentase Target Kelas	95
Gambar 3.4.6 Kode untuk Menampilkan Histogram.....	96
Gambar 3.4.7 Histogram Seluruh Variabel Numerik	96
Gambar 3.4.8 Pengecekan Missing Value	97
Gambar 3.4.9 Pengecekan Atribut Kategorikal	97
Gambar 3.4.10 Kode untuk Melakukan Data Preprocessing.....	98
Gambar 3.4.11 Variabel X Setelah Dikonversi Menjadi Numerik.....	99
Gambar 3.4.12 Variabel y Setelah Proses Label Encoding	100
Gambar 3.4.13 Hasil Transformasi pada Data Training	100
Gambar 3.4.14 Hasil Transformasi pada Data Testing.....	100
Gambar 3.4.15 Kode untuk Logistic Regression	101
Gambar 3.4.16 Output Performa Model	101
Gambar 3.4.17 Confusion Matrix	102
Gambar 3.4.18 Output F1 Score	102
Gambar 3.4.19 Pengecekan Nilai Koefisien dari Model Logistic Rgression	102
Gambar 3.4.20 Pengecekan Nilai Konstanta pada Model Logistic Regression	103
Gambar 3.4.21 Kode untuk Melatih Model KNN	103



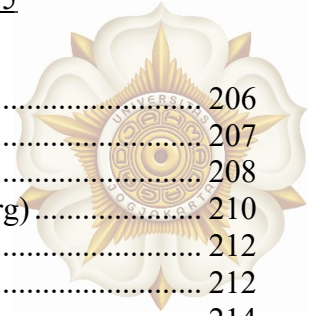
Gambar 3.4.22 Output Performa Model	104
Gambar 3.4.23 Confusion Matrix	104
Gambar 3.4.24 Kode untuk Melatih Model Decision Tree	105
Gambar 3.4.25 Output Performa Model	105
Gambar 3.4.26 Confusion Matrix	106
Gambar 3.4.27 Kode untuk Melatih Model Random Forest	106
Gambar 3.4.28 Output Performa Model	107
Gambar 3.4.29 Confusion Matrix	107
Gambar 3.4.30 Kode untuk Melatih Model Ada Boost	108
Gambar 3.4.31 Output Performa Model	108
Gambar 3.4.32 Confusion Matrix	109
Gambar 4.4.1 Import Library	114
Gambar 4.4.2 Lima Baris Pertama Dataset	114
Gambar 4.4.3 Lima Baris Terakhir Dataset	114
Gambar 4.4.4 Hasil Deskripsi Statistik Dataset	115
Gambar 4.4.5 Kode untuk Menampilkan Histogram	115
Gambar 4.4.6 Histogram Seluruh Variabel Numerik	115
Gambar 4.4.7 Korelasi Atribut Numerik	116
Gambar 4.4.8 Pengecekan Missing Value	116
Gambar 4.4.9 Pengecekan Atribut Kategorikal	117
Gambar 4.4.10 Kode untuk Membuat Model Linear Regression	117
Gambar 4.4.11 Hasil Prediksi Model Linear Regression	119
Gambar 4.4.12 Kode Evaluasi Model Menggunakan RMSE dan R^2	119
Gambar 4.4.13 Hasil Evaluasi Menggunakan Metrik RMSE dan R^2	120
Gambar 4.4.14 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya	120
Gambar 4.4.15 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya	121
Gambar 4.4.16 Kode untuk Membuat Model Decision Tree Regression	121
Gambar 4.4.17 Hasil Prediksi Model Decision Tree Regression	122
Gambar 4.4.18 Kode Evaluasi Model Menggunakan RMSE dan R^2	123
Gambar 4.4.19 Hasil Evaluasi Model	123
Gambar 4.4.20 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya	123
Gambar 4.4.21 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya	123
Gambar 4.4.22 Kode untuk Membuat Model Random Forest Regression	124
Gambar 4.4.23 Hasil Prediksi Model Random Forest Regression	125
Gambar 4.4.24 Kode Evaluasi Model Menggunakan RMSE dan R^2	125
Gambar 4.4.25 Hasil Evaluasi Model	125
Gambar 4.4.26 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya	126
Gambar 4.4.27 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya	126
Gambar 5.4.1 Impor Pustaka	130
Gambar 5.4.2 Pembacaan Dataset	131
Gambar 5.4.3 Konversi tanggal ke format datetime	131
Gambar 5.4.4 Fungsi sliding window	132
Gambar 5.4.5 Fungsi statistik	133
Gambar 5.4.6 Preprocessing Dataset	134
Gambar 5.4.7 Preprocessing Dataset	134
Gambar 5.4.8 Preprocessing Dataset	135



Gambar 5.4.9 Visualisasi Dataset	135
Gambar 5.4.10 Fungsi Multilayer Perceptron.....	136
Gambar 5.4.11 Fungsi KNN	136
Gambar 5.4.12 Fungsi Decision Tree	136
Gambar 5.4.13 Fungsi Random Forest	137
Gambar 5.4.14 Lakukan prediksi.....	137
Gambar 5.4.15 Visualisasi hasil prediksi.....	138
Gambar 5.4.16 Model Evaluation	139
Gambar 6.4.1 Impor Pustaka	145
Gambar 6.4.2 Membaca dataset.....	145
Gambar 6.4.3 Melihat struktur Dataset.....	146
Gambar 6.4.4 Melihat korelasi antar kolom	146
Gambar 6.4.5 Menghapus kolom CUST_ID	146
Gambar 6.4.6 Menghitung jumlah kolom null.....	147
Gambar 6.4.7 Imputasi Data	147
Gambar 6.4.8 Cek hasil Imputasi Data	147
Gambar 6.4.9 Data Scaling	148
Gambar 6.4.10 Elbow Method.....	148
Gambar 6.4.11 Visualisasi Elbow Method	149
Gambar 6.4.12 Menentukan titik optimal menggunakan kneed	149
Gambar 6.4.13 Visualisasi Knee Point	150
Gambar 6.4.14 Mencari nilai k elbow method	150
Gambar 6.4.15 Visualisasi elbow method	151
Gambar 6.4.16 Visualisasi Knee Point	151
Gambar 6.4.17 Menentukan silhouette score.....	152
Gambar 6.4.18 Visualisasi Silhouette score.....	152
Gambar 6.4.19 Kmedoid.....	153
Gambar 6.4.20 Visualisasi Kmedoid	153
Gambar 6.4.21 Training K-Mean sesuai kluster optimal.....	154
Gambar 6.4.22 Cek Centroids.....	154
Gambar 6.4.23 Visualisasi K Means Clustering.....	155
Gambar 6.4.24 Visualisasi klustering	155
Gambar 6.4.25 Visualisasi Scatter	156
Gambar 6.4.26 Evaluasi jumlah cluster menggunakan inertia	156
Gambar 6.4.27 Pelatihan model K-Medoids.....	157
Gambar 6.4.28 Visualisasi K-Medoids clustering	157
Gambar 6.4.29 Visualisasi Kmedoid Scatter	158
Gambar 6.4.30 Interpretasi Cluster.....	159
Gambar 6.4.31 Analisis Cluster	159
Gambar 7.4.1 Impor Pustaka	166
Gambar 7.4.2 Pembacaan Dataset	166
Gambar 7.4.3 Mengecek Null Value	167
Gambar 7.4.4 Distribusi data	168
Gambar 7.4.5 Data Cleaning.....	168
Gambar 7.4.6 Mengecek jumlah data null.....	169
Gambar 7.4.7 Random Sampling.....	170



Gambar 7.4.8 Melihat data.....	171
Gambar 7.4.9 Perbandingan sebelum random sampling dan sesudah	171
Gambar 7.4.10 Visualisasi Scatter	172
Gambar 7.4.11 Random over sampling	173
Gambar 7.4.12 Sebelum vs Sesudah over sampling	174
Gambar 7.4.13 Visualisasi scatter.....	175
Gambar 7.4.14 SMOTE	176
Gambar 7.4.15 Sebelum vs Sesudah SMOTE	177
Gambar 7.4.16 Visualisasi Scatter	177
Gambar 7.4.17 ADASYN	178
Gambar 7.4.18 Sebelum dan Sesudah Adasyn	179
Gambar 7.4.19 Visualisasi scatter.....	180
Gambar 8.4.1 Impor Pustaka	185
Gambar 8.4.2 Membaca dataset.....	186
Gambar 8.4.3 Menganalisis dataset	186
Gambar 8.4.4 Mengecek proporsi data	187
Gambar 8.4.5 Mengecek nilai null.....	187
Gambar 8.4.6 Mengidentifikasi atribut kategorikal dan numerik.....	187
Gambar 8.4.7 Pembagian data training dan testing	188
Gambar 8.4.8 Koefisien model	188
Gambar 8.4.9 Visualisasi Data.....	189
Gambar 8.4.10 Visualisasi Data.....	189
Gambar 8.4.11 Visualisasi Confusion Matrix.....	190
Gambar 8.4.12 Melatih model MLP	190
Gambar 8.4.13 Melihat hasil prediksi.....	191
Gambar 8.4.14 Membaca dataset.....	191
Gambar 8.4.15 Mengecek missing value.....	192
Gambar 8.4.16 Histogram fitur numerik.....	192
Gambar 8.4.17 Mengecek Performa	192
Gambar 8.4.18 Mengecek performa	193
Gambar 8.4.19 Evaluasi model	193
Gambar 8.4.20 Visualisasi hasil prediksi.....	194
Gambar 9.2.1 Penglihatan Manusia vs Komputer	196
Gambar 9.2.2 Hidden Layer MLP	197
Gambar 9.2.3 Pergeseran Citra pada MLP	198
Gambar 9.2.4 Ilustrasi Alur Tradisional vs Deep Learning.....	198
Gambar 9.2.5 Ilustrasi Alur Tradisional vs Deep Learning.....	198
Gambar 9.2.6 Alur CNN secara keseluruhan.....	200
Gambar 9.2.7 Dropout	200
Gambar 9.4.1 Unduh Dataset Dog vs Cat.....	202
Gambar 9.4.2 Import Pustaka	202
Gambar 9.4.3 Data Preparation dan Splitting	203
Gambar 9.4.4 Pengujian pembacaan data	204
Gambar 9.4.5 Pendefinisian IMG_SHAPE dan kelas	205
Gambar 9.4.6 Model CNN sederhana.....	205
Gambar 9.4.7 Ringkasan model CNN sederhana	206



Gambar 9.4.8 Latih model CNN sederhana.....	206
Gambar 9.4.9 Kode plot evaluasi.....	207
Gambar 9.4.10 Plot evaluasi	208
Gambar 10.2.1 Machine Learning Lifecycle (Sumber: www.geeksforgeeks.org).....	210
Gambar 10.2.2 Skema alur data client-server	212
Gambar 10.2.3 Logo Flask.....	212
Gambar 10.4.1 Folder yang perlu disalin.....	214
Gambar 10.4.2 Preprocessing Dataset	215
Gambar 10.4.3 Decision Tree Model.....	216
Gambar 10.4.4 Menyimpan model hasil training	216
Gambar 10.4.5 Pembuatan Model DT Liver	217
Gambar 10.4.6 Menjalankan script python.....	217
Gambar 10.4.7 File yang perlu disalin.....	218
Gambar 10.4.8 Pustaka Deployment Flask.....	218
Gambar 10.4.9 Minimal Flask Application	219
Gambar 10.4.10 Endpoint Prediksi (tabular_api.py)	221
Gambar 10.4.11 Struktur Proyek Flask.....	221
Gambar 10.4.12 Menjalankan Flask	221
Gambar 10.4.13 Endpoint Tabular.....	222
Gambar 10.4.14 Endpoint Result Tabular	222
Gambar C.1 Contoh penggunaan dekorator st.cache_resource	227
Gambar 11.3.1 Struktur Proyek Praktikum Streamlit.....	229
Gambar 11.3.2 Aplikasi Streamlit berhasil dijalankan	230
Gambar 11.3.3 Menambahkan Konfigurasi Web, Title, Caption	230
Gambar 11.3.4 Halaman yang tertampil di Streamlit	230
Gambar 11.3.5 Layouting 2 kolom	231
Gambar 11.3.6 Hasil layouting 2 kolom.....	231
Gambar 11.3.7 Kode Streamlit Sidebar	232
Gambar 11.3.8 Halaman Streamlit diberikan sidebar	232
Gambar 11.3.9 Model Path	232
Gambar 11.3.10 Load model dengan cache.....	233
Gambar 11.3.11 Manajemen State dan Fungsi bantuan	234
Gambar 11.3.12 Tombol Reset Formulir.....	235
Gambar 11.3.13 Kode Formulir Streamlit	235
Gambar 11.3.14 Halaman Output Prediksi	236
Gambar 11.3.15 Halaman Input.....	237
Gambar 11.3.16 Halaman Input dan Output.....	237

DAFTAR TABEL

No table of figures entries found.



PERTEMUAN I

NUMPY DAN PANDAS

1.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar Numpy dan penggunaan *array* n-dimensi serta berbagai operasi manipulasi *array*
- B. Mahasiswa mengetahui berbagai manipulasi dan transformasi data pada pandas, termasuk pembuatan DataFrame
- C. Mahasiswa dapat menerapkan Numpy dan pandas untuk mengolah dan menganalisis data

1.2. DASAR TEORI

- **Numpy**

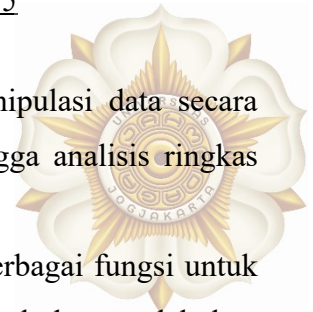
NumPy (Numerical Python) adalah pustaka Python yang digunakan untuk membangun dan mengelola *array* sebagai dasar komputasi numerik. NumPy dikembangkan pada tahun 2005 oleh Travis Oliphant sebagai proyek *open-source*. NumPy menawarkan kecepatan pemrosesan yang lebih tinggi, penggunaan memori yang efisien, serta dukungan terhadap struktur *array* berdimensi banyak. Kemampuan ini menjadikan NumPy penting dalam analisis numerik dan komputasi ilmiah.

Struktur utama NumPy adalah *ndarray*, yaitu *array* yang dapat berbentuk satu atau lebih dimensi, mulai dari 1D hingga 3D dan seterusnya. *Array* dapat dibentuk dari berbagai struktur sekuensial di Python maupun melalui fungsi internal untuk menghasilkan nilai tertentu.

Akses elemen pada *array* dilakukan melalui *indexing*, yaitu penunjukan posisi berdasarkan indeks yang dimulai dari nol. NumPy juga mendukung *slicing*, yakni pengambilan sebagian elemen berdasarkan rentang indeks tertentu, termasuk pada *array* multidimensi. Mekanisme ini memungkinkan manipulasi data secara efisien tanpa menyalin seluruh *array*.

- **Pandas**

Pandas adalah pustaka Python yang dirancang untuk mempermudah pengolahan dan analisis data terstruktur. Pustaka ini menyediakan dua struktur data utama, yaitu Series (data satu dimensi) dan DataFrame (data dua dimensi berbentuk tabel). Dengan indeks



yang fleksibel, Pandas memungkinkan pengguna melakukan manipulasi data secara efisien, mencakup pembersihan, transformasi, penggabungan, hingga analisis ringkas terhadap dataset.

Sebagai *library* utama dalam data analisis, Pandas menyediakan berbagai fungsi untuk memodifikasi struktur DataFrame, seperti menambah atau menghapus kolom, melakukan pengurutan, memfilter baris berdasarkan kondisi, serta melakukan *grouping* dan *aggregation* untuk merangkum data. Selain itu, Pandas juga mendukung operasi lanjutan seperti *merging*, *joining*, *concatenation*, dan *reshaping* melalui *pivoting* atau *melting*, yang menjadikannya sangat fleksibel untuk mengelola dataset dalam berbagai format.

Pandas dirancang dengan integrasi yang baik bersama NumPy, sehingga operasi numerik dan manipulasi *array* dapat dilakukan secara konsisten pada struktur DataFrame. Dengan performa yang efisien dan sintaks yang mudah dibaca, Pandas menjadi salah satu pustaka paling penting dalam ekosistem *data science* untuk penanganan data tabular.

1.3. ALAT DAN BAHAN

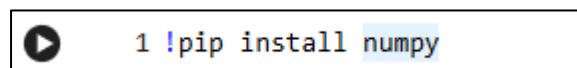
- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

1.4. LANGKAH PERCOBAAN

- **NumPy**

- 1) Install NumPy

NumPy telah terinstal di Google Colaboratory secara *default*. Jika belum terinstal, instalasi numpy dapat dilakukan dengan menggunakan PIP. Pada Jupyter Notebook termasuk Google Colab, instalasi NumPy dapat dilakukan seperti Gambar 1.4.1.

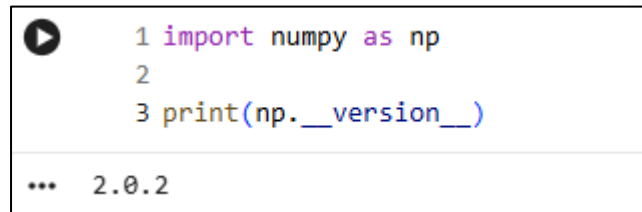


Gambar 1.4.1 Instalasi NumPy

- 2) Impor pustaka NumPy

Impor pustaka dengan *syntax* `import <pustaka>`. Nama alias juga dapat diberikan untuk mempersingkat pengetikan nama pemanggilan pustaka. Secara umum alias np

digunakan untuk menyingkat numpy. Melakukan impor dan membuat alias dapat dilakukan seperti Gambar 1.4.2. Pada waktu pembuatan modul ini, pustaka NumPy yang terinstal di Google Colaboratory adalah pustaka NumPy versi 2.0.2.



```

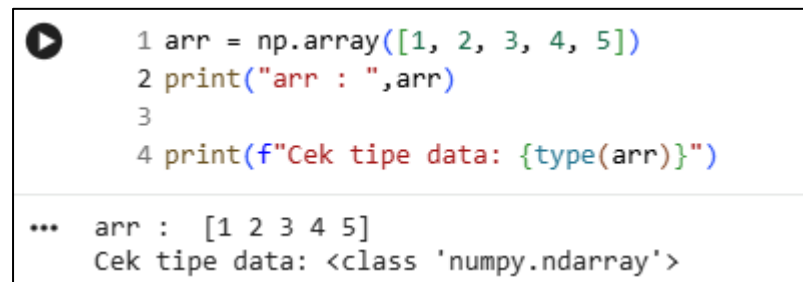
1 import numpy as np
2
3 print(np.__version__)
... 2.0.2

```

Gambar 1.4.2 Impor pustaka NumPy

3) Membuat *array* 1 dimensi

Membuat *array* dapat dilakukan dengan memanggil fungsi *array* seperti Gambar 1.4.3. Tipe data numpy berupa *ndarray* yang memiliki arti N-dimensional *array*.



```

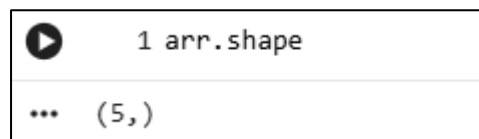
1 arr = np.array([1, 2, 3, 4, 5])
2 print("arr : ",arr)
3
4 print(f"Cek tipe data: {type(arr)}")
... arr : [1 2 3 4 5]
   Cek tipe data: <class 'numpy.ndarray'>

```

Gambar 1.4.3 Membuat *array* 1 dimensi

4) Melihat *shape* atau bentuk *array*

Untuk melihat bentuk *array* yang telah dibuat, dapat dilakukan dengan memanggil atribut *shape* pada objek *ndarray*. Lakukan seperti Gambar 1.4.4, hasil tersebut menginformasikan bahwa *array* terdiri dari 1 dimensi dan terdiri dari 5 elemen.



```

1 arr.shape
... (5,)

```

Gambar 1.4.4 Melihat bentuk *array*

5) Membuat *array* 2 dimensi

Membuat *array* 2 dimensi dapat dilakukan pada fungsi yang sama. Untuk membuat *array* 2 dimensi, masukkan objek *list* seperti pembuatan dimensi 1 ke dalam *list* sehingga membentuk *nested list* seperti pada Gambar 1.4.5. Jika di cek bentuk *array*-nya maka akan menampilkan (2, 3). Angka dua ini menunjukkan banyak elemen di

sumbu 0 atau di *list* terluar, sementara itu angka tiga menunjukkan banyak elemen di sumbu 1 atau *list* bagian dalam.



```
1 arr2d = np.array([[1,2,3],[4,5,6]])
2 print(arr2d)
3
4 print(f"Shape: {arr2d.shape}")
5 # Angka pertama = member di sumbu 0,
6 # Angka kedua =member di sumbu 1
```

```
... [[1 2 3]
      [4 5 6]]
      Shape: (2, 3)
```

Gambar 1.4.5 Membuat array 2 dimensi dan mengecek bentuk array

6) Membuat *array* 3 dimensi

Serupa dengan konsep pembuatan *array* 2 dimensi, *array* 3 dimensi akan memuat elemen 2 dimensi. Contoh pembuatan *array* 3 dimensi dapat dilihat di Gambar 1.4.1.



```

1 arr3d = np.array(
2     [
3         [
4             [1, 2, 3, 4], [5, 6, 7, 8]
5         ],
6         [[9,10 ,11, 12], [13, 14, 15, 16]],
7         [[16, 17, 18, 19], [20, 21, 22, 23]]
8     ]
9 )
10
11 print(arr3d)
12
13 print(f"Shape: {arr3d.shape}")
14 # Angka pertama = member di sumbu 0,
15 # Angka kedua = member di sumbu 1,
16 # Angka ketiga = member di sumbu 2

```

```

... [[[ 1  2  3  4]
      [ 5  6  7  8]]

      [[ 9 10 11 12]
      [13 14 15 16]]

      [[16 17 18 19]
      [20 21 22 23]]]
Shape: (3, 2, 4)

```

Gambar 1.4.6 Membuat array 3 dimensi

7) Membuat *array* dengan *String* sebagai member

```

1 arr_str = np.array(['hello', 'nama', 'saya'])
2 print(arr_str)

```

```

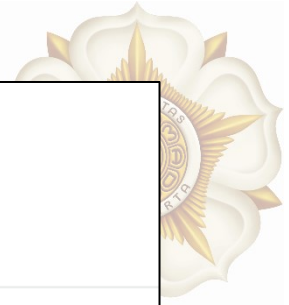
... ['hello' 'nama' 'saya']

```

Gambar 1.4.7 Array String

8) Melakukan *indexing* di *array* 1 dimensi

Indexing tidak jauh berbeda dengan fungsi *native indexing* untuk tipe data *list*. Mengakses elemen dengan indeks ke *n* dapat dilakukan dengan `objek_ndarray[n]` seperti Gambar 1.4.8 baris ke 3. Perlu diingat bahwa indeks selalu mulai dari 0.



```

1 arr = np.array([1,2,3,4,5])
2 # menampilkan elemen ke tiga
3 print(arr[2])
4 # hasil penjumlahan elemen 1 dengan elemen 5
5 print(arr[0] + arr[4])

... 3
    6

```

Gambar 1.4.8 Indexing array 1 dimensi

9) Melakukan *indexing* di array 2 dimensi

```

1 arr = np.array([[1, 2, 3], [4, 5, 6]])
2 # baris pertama, kolom ketiga. atau elemen 1 di sumbu 0, elemen 3 di sumbu 1
3 print(arr[0][2])
4 # baris kedua, kolom kedua. atau elemen 2 sumbu 0, elemen 2 sumbu 1
5 print(arr[1][1])

... 3
    5

```

Gambar 1.4.9 Indexing array 2 dimensi

10) Melakukan *indexing* di array 3 dimensi

```

1 arr = np.array([
2     [[1, 2, 3], [4, 5, 6]],
3     [[10, 11, 12], [13, 14, 14]]
4 ])
5
6 # menampilkan angka 11
7 #sumbu 0 : index 1
8 #sumbu 1 : index 0
9 #sumbu 2 : index 1
10 print(arr[1][0][1])
11
12 # menampilkan angka 14
13 #sumbu 0 : index 1
14 #sumbu 1 : index 1
15 #sumbu 2 : index 2
16 print(arr[1][1][2])

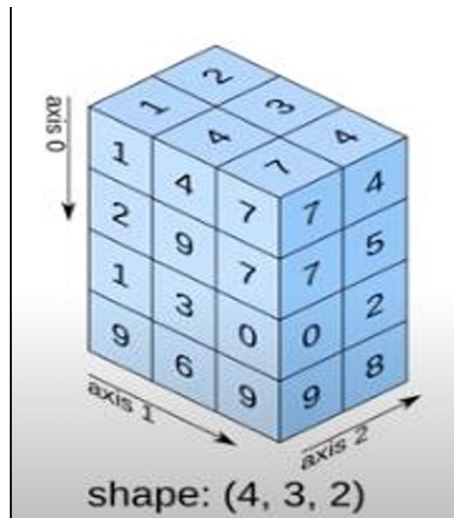
... 11
    14

```

Gambar 1.4.10 Indexing array 3 dimensi

11) Membuat *array* dari contoh

Diberikan sebuah *array* dengan bentuk seperti Gambar 1.4.11. Bagian yang tidak terlihat dianggap memiliki nilai 0.



Gambar 1.4.11 Contoh array 3 dimensi

Jika Anda diminta untuk membuatnya dalam bentuk kode, maka akan terlihat seperti Gambar 1.4.12.

```

1 implementasi_array_3d = np.array([
2     [[1, 2], [4, 3], [7, 4]],
3     [[2, 0], [9, 0], [7, 5]],
4     [[1, 0], [3, 0], [0, 2]],
5     [[9, 0], [6, 0], [9, 8]]
6 ])
7 print(implementasi_array_3d)
8
9 print("\nshape", implementasi_array_3d.shape)
10 # sumbu 0 : 4 elemen
11 # sumbu 1 : 3 elemen
12 # sumbu 2 : 2 elemen
13 # silahkan cek di slide halaman 5

... [[1 2]
      [4 3]
      [7 4]]

      [[2 0]
       [9 0]
       [7 5]]

      [[1 0]
       [3 0]
       [0 2]]

      [[9 0]
       [6 0]
       [9 8]]]

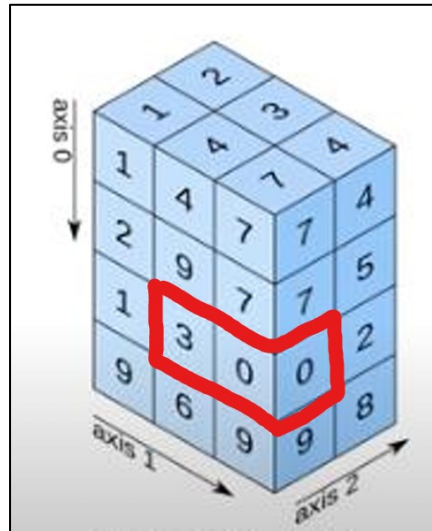
shape (4, 3, 2)

```

Gambar 1.4.12 Implementasi Array

12) Array slicing

NumPy menerapkan cara yang sama dengan cara *slicing* tipe data List untuk melakukan *slicing* tipe data *ndarray*. Sebagai contoh Anda ingin untuk mengambil Gambar 1.4.13 yang diberi tanda merah (3 dan 0), maka Anda harus bisa menentukan indeks lokasi target tersebut.



Gambar 1.4.13 Target slicing

Di sumbu 0, target tersebut berada di indeks ke 2. Di sumbu 1, target berada di indeks ke 1 dan 2 atau dapat disebut indeks ke 1 dan selanjutnya. Di sumbu 2, target hanya di indeks ke 0. Dengan demikian, penulisan kode *slicing* dapat dilakukan seperti Gambar 1.4.14.

```
1 target = implementasi_array_3d[2][1:][0]
2 print(target)

... [3 0]
```

Gambar 1.4.14 Contoh penerapan Slicing

13) Mengonversi tipe data

Melakukan konversi tipe data dapat dilakukan dengan fungsi **astype** dengan memasukkan target kelas.



```
1 arr = np.array(['1', '2', '3', '4', '5'])
2 print(arr)
3
4 arr_int = arr.astype(int)
5 print("Integer", arr_int)
6
7 arr_float = arr.astype(float)
8 print("Float", arr_float)

... ['1' '2' '3' '4' '5']
Integer [1 2 3 4 5]
Float [1. 2. 3. 4. 5.]
```

Gambar 1.4.15 Konversi tipe data di NumPy

14) *Reshape* ke dimensi lebih tinggi

Untuk mengubah bentuk *array* ke dimensi lebih tinggi, pastikan bahwa banyak elemen tetap harus sama. Sebagai contoh Gambar 1.4.16, banyak elemen *array* asli adalah 12, ketika ingin mengubah bentuk ke 2 dimensi, total banyak elemen tetap harus 12. Sebagai contoh perhitungannya, *reshaping* ke dimensi 2 dilakukan dengan banyak elemen di sumbu 0 adalah 3 dan di sumbu 1 adalah 4, total dari banyak elemen tersebut adalah 12 (cara perhitungan : $3 \times 4 = 12$). Hasil tersebut tetap sama dengan banyak elemen di *array* asli.



```
1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 print(arr)
3 print('Shape', arr.shape, '\n')
4
5 new_arr_2d = arr.reshape(3, 4)
6 # sumbu 0 : 3 member
7 # sumbu 1 : 4 member
8 print(new_arr_2d)
9 print('Shape', new_arr_2d.shape, '\n')
10
11 new_arr_3d = arr.reshape(3, 2, 2)
12 # sumbu 0 : 3 member
13 # sumbu 1 : 2 member
14 # sumbu 2 : 2 member
15 print(new_arr_3d)
16 print('Shape', new_arr_3d.shape)
17
```

```
... [ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape (12,)
```

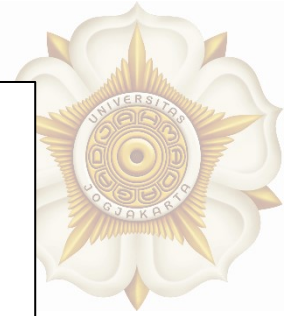
```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape (3, 4)
```

```
[[[ 1  2]
   [ 3  4]]
 [[ 5  6]
   [ 7  8]]
 [[ 9 10]
   [11 12]]]
Shape (3, 2, 2)
```

Gambar 1.4.16 Contoh Reshape ke dimensi lebih tinggi

15) *Reshape* ke dimensi 1

Untuk melakukan *reshape* ke dimensi yang lebih rendah, Anda perlu melakukan *reshape* ke dimensi 1 terlebih dahulu. Untuk mengubah bentuk ke dimensi 1 dapat dilakukan seperti Gambar 1.4.17. Setelah melakukan pengubahan bentuk ke dimensi 1, perubahan ke dimensi ke lebih tinggi dapat dilakukan.



```

1 #3d ke 1 d
2 arr = np.array([
3     [[1, 2, 3], [4, 5, 6]],
4     [[10, 11, 12], [13, 14, 15]]
5 ])
6 print(arr)
7 print('sesudah di reshape')
8 new_arr = arr.reshape(-1)
9
10 print(new_arr)

```

```

...  [[ 1  2  3]
      [ 4  5  6]]

      [[10 11 12]
       [13 14 15]]]
sesudah di reshape
[ 1  2  3  4  5  6 10 11 12 13 14 15]

```

Gambar 1.4.17 Reshape ke dimensi 1

16) Perbedaan fungsi *len* dan atribut *shape*

Telah kita mengetahui bahwa *shape* akan menampilkan bentuk *array*, sementara itu, *len* hanya mampu menghitung banyak elemen yang ada di sumbu 0. Masih menggunakan Gambar 1.4.11, berikut akan membandingkan hasil perbedaan *len* dan *shape*. Di hasil Gambar 1.4.18, terlihat bahwa *len* hanya memunculkan angka 4 karena di sumbu 0 hanya ada 4 elemen.

```

1 print(len(implementasi_array_3d))
2 print(implementasi_array_3d.shape)

```

```

...  4
      (4, 3, 2)

```

Gambar 1.4.18 Len vs Shape

17) Iterasi *ndarray*

```

1 for sumbu_0 in implementasi_array_3d:
2     for sumbu_1 in sumbu_0:
3         for sumbu_2 in sumbu_1:
4             print(sumbu_2, end=", ")
5

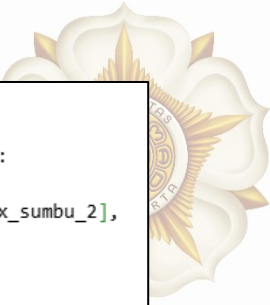
```

```

...  1, 2, 4, 3, 7, 4, 2, 0, 9, 0, 7, 5, 1, 0, 3, 0, 0, 2, 9, 0, 6, 0, 9, 8,

```

Gambar 1.4.19 Iterasi langsung



```

1 for idx_sumbu_0 in range(implementasi_array_3d.shape[0]):
2     for idx_sumbu_1 in range(implementasi_array_3d.shape[1]):
3         for idx_sumbu_2 in range(implementasi_array_3d.shape[2]):
4             print(
5                 implementasi_array_3d[idx_sumbu_0][idx_sumbu_1][idx_sumbu_2],
6                 end=", "
7             )
8
... 1, 2, 4, 3, 7, 4, 2, 0, 9, 0, 7, 5, 1, 0, 3, 0, 0, 2, 9, 0, 6, 0, 9, 8,

```

Gambar 1.4.20 Iterasi dengan indeks

18) Melakukan *join* pada *array* dimensi 1

Menggabungkan *array* dapat dilakukan dengan fungsi *concatenate*. Untuk menggabungkan *array* berdimensi 1 maka sumbu (*axis*) di set ke 0 seperti Gambar 1.4.21.

```

1 arr1 = np.array([1, 2, 3])
2 arr2 = np.array([4, 5, 6])
3 arr3 = np.concatenate((arr1, arr2), axis=0)
4 print(arr3)
... [1 2 3 4 5 6]

```

Gambar 1.4.21 Concatenate 2 array

19) Melakukan *join* pada *array* dimensi 2

Di dimensi 2, Anda bisa memilih untuk menggabungkan *array* ke sumbu 1 atau 2. Penggabungan *array* ke sumbu 1 akan terlihat seperti Gambar 1.4.22. Penggabungan *array* ke sumbu 0 akan terlihat seperti Gambar 1.4.23.

```

1 arr1 = np.array([[1, 2, 3], [4, 5, 6]])
2 arr2 = np.array([[11, 12, 13], [14, 15, 16]])
3
4 # kita menambah di sumbu 1, kolomnya
5 arr3 = np.concatenate((arr1, arr2), axis=1)
6 print(arr3)
... [[ 1  2  3 11 12 13]
     [ 4  5  6 14 15 16]]

```

Gambar 1.4.22 Menambahkan Array di sumbu 1



```

1 # kita menambah di sumbu 0, baris nya
2 arr3 = np.concatenate((arr1, arr2), axis=0)
3 print(arr3)

... [[ 1  2  3]
      [ 4  5  6]
      [11 12 13]
      [14 15 16]]

```

Gambar 1.4.23 Menambahkan Array di sumbu 0

20) Array Split

Melakukan *split* pada objek *ndarray* dapat dilakukan dengan fungsi **array_split** seperti Gambar 1.4.24.

```

1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 #dipisah jadi 5 bagian array
3 newarr = np.array_split(arr, 5)
4 print('sebelum')
5 print(arr)
6 print('sesudah')
7 print(newarr)
8 narr = newarr[2]
9 print(narr)

... sebelum
[ 1  2  3  4  5  6  7  8  9 10 11 12]
sesudah
[array([1, 2, 3]), array([4, 5, 6]), array([7, 8]), array([ 9, 10]), array([11, 12])]
[7 8]

```

Gambar 1.4.24 Array Split

21) Array Search

Melakukan pencarian dapat dilakukan dengan fungsi **where** dengan memasukkan kondisi seperti Gambar 1.4.25.

```

1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 #mencari elemen yg nilainya 3, dan mengembalikan index nya
3 x = np.where(arr == 3 )
4 print(x)

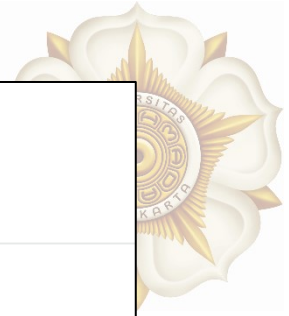
... (array([2]),)

```

Gambar 1.4.25 Mencari elemen dalam array

22) Sorting

Melakukan *sorting* dapat dilakukan dengan fungsi **sort**. Terdapat beberapa parameter yang bisa digunakan. Penjelasan lengkap mengenai parameter *sorting* dapat dibaca di dokumentasi NumPy. Sebagai contoh Gambar 1.4.26, *sorting* dilakukan secara *ascending*.



```
1 arr = np.array([11,21,3,4,19,1,21,12])
2 new_arr = np.sort(arr)
3 print(arr)
4 print(new_arr)

... [11 21  3  4 19  1 21 12]
     [ 1  3  4 11 12 19 21 21]
```

Gambar 1.4.26 Sorting NumPy

23) NumPy Random

Impor *package* random dari pustaka numpy untuk menggunakan numpy random. Gunakan fungsi **rand** untuk membuat nilai *random* dengan *shape* n. Untuk membuat nilai random dengan *shape* dimensi 1 dan 2 elemen, maka akan terlihat seperti Gambar 1.4.27.

```
1 from numpy import random
2 x = random.rand(2)
3 print(x)

... [0.27114082 0.92494933]
```

Gambar 1.4.27 Numpy Random

24) Operasi penjumlahan

NumPy memiliki operasi penjumlahan *array*. Operasi tersebut dapat diakses dengan fungsi **add** seperti Gambar 1.4.28.

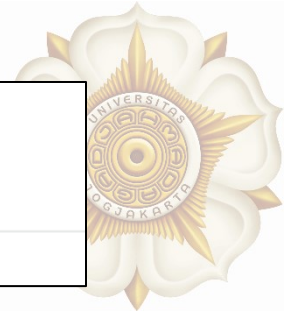
```
1 arr1 = np.array([1,2,3,4,5])
2 arr2 = np.array([10,20,30,40,50])
3 #elemen dari satu array ditambah dengan elemen di array lain
4 arr3 = np.add(arr1, arr2)
5 print(arr3)

... [11 22 33 44 55]
```

Gambar 1.4.28 Penjumlahan Array

25) Operasi *Summation* (Total jumlah)

NumPy juga memberikan operasi untuk melakukan kalkulasi total jumlah semua elemen.



```
1 arr1 = np.array([1,2,3,4,5])
2 arr3 = np.sum(arr1)
3 print(arr3)
```

... 15

Gambar 1.4.29 Summation

26) Product of elements

Untuk menghitung perkalian semua elemen, dapat dilakukan dengan menggunakan fungsi **prod**.

```
1 arr1 = np.array([1,2,3,4,5])
2 arr3 = np.prod(arr1)
3 print(arr3)
```

... 120

Gambar 1.4.30 Product of elements NumPy

• Dataframe Pandas

1) Instal Pandas

Pandas telah terinstal di Google Colaboratory secara *default*. Jika belum terinstal, instalasi pandas dapat dilakukan dengan menggunakan PIP. Pada Jupyter Notebook termasuk Google Colaboratory, instalasi Pandas dapat dilakukan seperti Gambar 1.4.31.

```
!pip install pandas
```

Gambar 1.4.31 Instalasi pandas

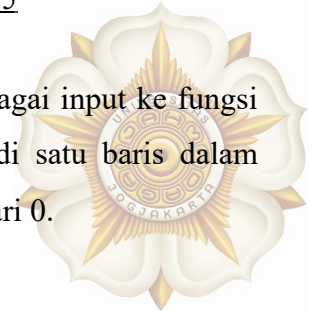
2) Import Library

Import library dengan *syntax* **import <pustaka>**. Nama alias juga dapat diberikan untuk mempersingkat pengetikan nama pemanggilan pustaka. Secara umum alias **pd** digunakan untuk menyingkat **pandas**. Pada percobaan ini, kita memerlukan *library* **pandas** dan **numpy**, sehingga kita akan mengimpor kedua *library* tersebut.

```
import pandas as pd
import numpy as np
```

Gambar 1.4.32 Import Library pandas

3) Membuat Dataframe



DataFrame dibuat dengan memberikan sebuah list Python sebagai input ke fungsi `pd.DataFrame()`. Setiap elemen dalam list otomatis menjadi satu baris dalam *DataFrame*, sementara indeks dihasilkan secara default mulai dari 0.

```
data = [2, 3, 5, 7, 1]
df = pd.DataFrame(data)
df
```

	0
0	2
1	3
2	5
3	7
4	1

Gambar 1.4.33 Membuat Dataframe

Kita juga dapat menambahkan parameter *columns* untuk menetapkan nama kolom dan parameter *index* untuk memberikan label indeks *custom*. Hasilnya, *DataFrame* tampil dalam format tabular seperti pada Gambar 1.4.34.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'], index=['satu', 'dua', 'tiga'])
df
```

	Nama	Umur
satu	Adi	34
dua	Maya	23
tiga	Joko	20

Gambar 1.4.34 Membuat Dataframe

4) Menambahkan Kolom

Penambahan kolom pada data *frame* dapat dilakukan dengan menambahkan `df['Nama Kolom']`.



```
data = {'Nama ': ['Jack', 'Joko'], 'Umur' : [20, 23]}
df = pd.DataFrame(data)
df['Alamat'] = ['Surabaya', 'Yogyakarta']
df
```

	Nama	Umur	Alamat
0	Jack	20	Surabaya
1	Joko	23	Yogyakarta

Gambar 1.4.35 Menambahkan Kolom

Penambahan kolom juga dapat dilakukan dengan fungsi `df.insert` seperti pada Gambar 1.4.36.

```
data = {'Nama ': ['Jack', 'Joko'], 'Umur' : [20, 23]}
df = pd.DataFrame(data)
print(df)
alamat = ['Medan', 'Jakarta']
df.insert(0, 'Alamat', alamat)
print(df)
```

	Nama	Umur
0	Jack	20
1	Joko	23

	Alamat	Nama	Umur
0	Medan	Jack	20
1	Jakarta	Joko	23

Gambar 1.4.36 Menambahkan Kolom

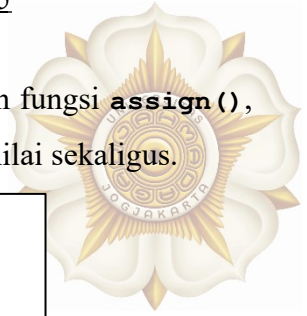
Kolom juga dapat ditambahkan pada posisi tertentu dengan menambahkan parameter `loc`, `column`, `value` pada fungsi `df.insert`.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
alamat = ['Medan', 'Jakarta', 'Solo']
df.insert(1, 'Alamat', alamat)
print(df)
```

	Nama	Alamat	Umur
0	Adi	Medan	34
1	Maya	Jakarta	23
2	Joko	Solo	20

Gambar 1.4.37 Menambahkan Kolom

5) Menambahkan Multi Kolom



Penambahan multi kolom dapat dilakukan dengan menggunakan fungsi `assign()`, yaitu dengan memberikan beberapa pasangan nama kolom dan nilai sekaligus.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
print(df)
alamat = ['Surabaya', 'Jakarta', 'Solo']
pekerjaan = ['Programmer', 'DB Admin', 'Web Developer']
df = df.assign(Job = pekerjaan, Alamat = alamat )
print('sesudah')
print(df)
```

	Nama	Umur
0	Adi	34
1	Maya	23
2	Joko	20

sesudah

	Nama	Umur	Job	Alamat
0	Adi	34	Programmer	Surabaya
1	Maya	23	DB Admin	Jakarta
2	Joko	20	Web Developer	Solo

Gambar 1.4.38 Menambahkan Multi Kolom

Penambahan multi kolom juga dapat dilakukan dengan membuat sebuah *dictionary* yang berisi pasangan *nama kolom* dan *nilai*, kemudian memasukkannya ke dalam *DataFrame* secara bersamaan.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
print(df)
alamat = ['Surabaya', 'Jakarta', 'Solo']
pekerjaan = ['Programmer', 'DB Admin', 'Web Developer']
d_kolom = {'Address': alamat, 'Job': pekerjaan}
df[['Address', 'Job']] = pd.DataFrame(d_kolom)
print(df)
```

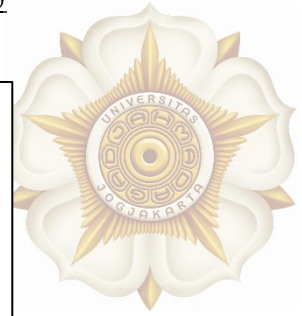
	Nama	Umur
0	Adi	34
1	Maya	23
2	Joko	20

	Nama	Umur	Address	Job
0	Adi	34	Surabaya	Programmer
1	Maya	23	Jakarta	DB Admin
2	Joko	20	Solo	Web Developer

Gambar 1.4.39 Menambahkan Multi Kolom

6) Menghapus Kolom

Penghapusan kolom dilakukan menggunakan fungsi `drop()` seperti pada Gambar 1.4.40. Parameter nama kolom dan *axis* digunakan untuk menentukan kolom yang akan dihapus. Parameter `inplace=True` membuat perubahan diterapkan langsung pada *DataFrame* tanpa membuat salinan baru.



```
data = {'Nama ': ['Jack', 'Joko'],
'Umur' : [20, 23],
'Alamat' : ['Surabaya', 'Jakarta'],
'Pekerjaan' : ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.drop('Pekerjaan', axis=1, inplace=True)
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Alamat
0	Jack	20	Surabaya
1	Joko	23	Jakarta

Gambar 1.4.40 Menghapus Kolom

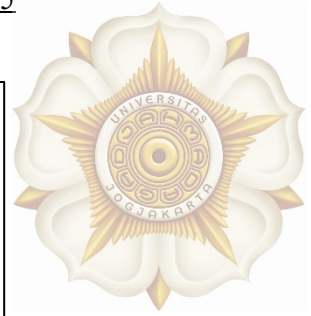
Penghapusan kolom juga dapat dilakukan dengan fungsi `df.pop()` (Gambar 1.4.41) dan juga perintah `del` (Gambar 1.4.42).

```
data = {'Nama ': ['Jack', 'Joko'],
'Umur' : [20, 23],
'Alamat' : ['Surabaya', 'Jakarta'],
'Pekerjaan' : ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.pop('Alamat')
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Pekerjaan
0	Jack	20	Programmer
1	Joko	23	DB Admin

Gambar 1.4.41 Menghapus Kolom dengan Fungsi pop



```
data = {'Nama': ['Jack', 'Joko'],
        'Umur': [20, 23],
        'Alamat': ['Surabaya', 'Jakarta'],
        'Pekerjaan': ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
del df['Alamat']
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Pekerjaan
0	Jack	20	Programmer
1	Joko	23	DB Admin

Gambar 1.4.42 Menghapus Kolom dengan Perintah del

7) Menghapus Multi Kolom

Penghapusan multi kolom dapat dilakukan dengan fungsi `df.drop()` seperti gambar berikut.

```
data = {'Nama': ['Jack', 'Joko'],
        'Umur': [20, 23],
        'Alamat': ['Surabaya', 'Jakarta'],
        'Pekerjaan': ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.drop(['Alamat', 'Pekerjaan'], axis=1, inplace=True)
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur
0	Jack	20
1	Joko	23

Gambar 1.4.43 Menghapus Multi Kolom

8) Sorting

Mengurutkan data berdasarkan indeks pada *dataframe* dapat dilakukan dengan fungsi `df.sort_index()` seperti pada Gambar 1.4.45.

```
data = {'Nama ': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data, index=[2, 3, 0, 1])
print(df)
```

	Nama	Umur	Alamat	Pekerjaan
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer

Gambar 1.4.44 Membuat Dataframe

```
sorted_df = df.sort_index()
sorted_df
```

	Nama	Umur	Alamat	Pekerjaan
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin

Gambar 1.4.45 Sorting Data

Parameter **axis=1** digunakan untuk mengurutkan kolom berdasarkan indeks kolom.

```
sorted_column = sorted_df.sort_index(axis=1)
sorted_column
```

	Alamat	Nama	Pekerjaan	Umur
0	Solo	Dwi	Web developer	34
1	Jakarta	Andi	Programmer	23
2	Surabaya	Jack	Programmer	20
3	Jakarta	Joko	DB Admin	23

Gambar 1.4.46 Sorting Data

9) Sorting Value

Untuk mengurutkan data berdasarkan nilai pada kolom tertentu, digunakan fungsi **sort_values** dengan parameter nama kolom seperti pada Gambar 1.4.47 dan Gambar 1.4.48.



```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat': ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan': ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data, index=[2, 3, 0, 1])
print(df)
df_sorted = df.sort_values(by='Nama')
print(df_sorted)
```

	Nama	Umur	Alamat	Pekerjaan
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer

	Nama	Umur	Alamat	Pekerjaan
1	Andi	23	Jakarta	Programmer
0	Dwi	34	Solo	Web developer
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin

Gambar 1.4.47 Sorting Data Berdasarkan Kolom Tertentu

```
df_sorted = df.sort_values(by=['Alamat', 'Nama'])
df_sorted
```

	Nama	Umur	Alamat	Pekerjaan
1	Andi	23	Jakarta	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
2	Jack	20	Surabaya	Programmer

Gambar 1.4.48 Sorting Data Berdasarkan Kolom Tertentu

10) Filtering

Filtering dataset dilakukan dengan memberikan kondisi logika pada *DataFrame* seperti pada Gambar 1.4.49.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data)
print(df)
filtered_df = df[df.Umur > 30]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin
2	Dwi	34	Solo	Web developer
3	Andi	23	Jakarta	Programmer

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.49 Filtering Data dengan Kondisi

Filtering juga dapat dilakukan menggunakan lebih dari satu kondisi seperti pada Gambar 1.4.50.

```
filtered_df = df[(df.Umur > 20) & (df.Alat== 'Jakarta')]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
1	Joko	23	Jakarta	DB Admin
3	Andi	23	Jakarta	Programmer

Gambar 1.4.50 Filtering Data dengan Kondisi

Filtering juga dapat dilakukan menggunakan fungsi `isin()`, yang mengecek apakah nilai pada kolom termasuk dalam sebuah list tertentu.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data)
nama1 = ['Dwi', 'Andi']
filtered_df = df[df.Nama.isin(nama1)]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer
3	Andi	23	Jakarta	Programmer

Gambar 1.4.51 Filtering Data dengan Fungsi isin()

```
filtered = df.query("Nama=='Joko' | Nama=='Andi'")
filtered
```

	Nama	Umur	Alamat	Pekerjaan
1	Joko	23	Jakarta	DB Admin
3	Andi	23	Jakarta	Programmer

Gambar 1.4.52 Filtering Data dengan Query

Filtering dapat dilakukan menggunakan fungsi **query()**, yang memungkinkan penulisan kondisi logis dalam bentuk ekspresi *string*.

```
df.loc[(df.Umur > 30)]
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.53 Filtering Data dengan Query

Filtering juga dapat dilakukan menggunakan fungsi **str.contains()**.

```
df[df.Pekerjaan.str.contains('Web')]
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.54 Filtering Data dengan fungsi str.contains()

11) Grouping

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
        'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
df
```

	Nama	Umur	Alamat	Pekerjaan	Gaji
0	Jack	20	Surabaya	Programmer	2000
1	Joko	23	Jakarta	DB Admin	3000
2	Dwi	34	Solo	Web developer	2400
3	Andi	23	Jakarta	Programmer	5000

Gambar 1.4.55 Membuat Dataframe

```
df.groupby('Pekerjaan').groups
```

```
{'DB Admin': [1], 'Programmer': [0, 3], 'Web developer': [2]}
```

Gambar 1.4.56 Menggabungkan Dataframe

Fungsi **groupby()** digunakan untuk mengelompokkan data berdasarkan nilai pada kolom *Pekerjaan*. Atribut **.groups** menampilkan indeks baris yang termasuk dalam setiap kelompok, sehingga terlihat pembagian data sesuai kategori pekerjaan.

```
df_group = df.groupby('Pekerjaan')
for pekerjaan, group in df_group:
    print(pekerjaan)
    print(group)
```

```
DB Admin
  Nama  Umur  Alamat  Pekerjaan  Gaji
1  Joko   23  Jakarta  DB Admin  3000
Programmer
  Nama  Umur  Alamat  Pekerjaan  Gaji
0  Jack   20  Surabaya  Programmer  2000
3  Andi   23  Jakarta  Programmer  5000
Web developer
  Nama  Umur  Alamat  Pekerjaan  Gaji
2  Dwi   34   Solo   Web developer  2400
```

Gambar 1.4.57 Menampilkan Data Menurut Pekerjaan

Objek hasil **groupby()** diiterasi untuk menampilkan setiap kelompok data secara terpisah. Variabel *pekerjaan* berisi nama kelompok, sedangkan *group* berisi *DataFrame* kecil yang memuat baris-baris sesuai kategori tersebut.



```
print(df_group.get_group('DB Admin'))
```

	Nama	Umur	Alamat	Pekerjaan	Gaji
1	Joko	23	Jakarta	DB Admin	3000

Gambar 1.4.58 Menampilkan Satu Kategori Tertentu

Fungsi `get_group()` digunakan untuk mengambil satu kelompok tertentu dari hasil `groupby()`

12) Aggregation

Operasi *aggregate* dilakukan dengan menerapkan fungsi seperti *mean*, *sum*, *min*, atau *max* pada data yang telah dikelompokkan menggunakan `groupby()`.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
        'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
print('sebelum di kelompokkan')
print(df)
df_group = df.groupby('Pekerjaan').mean(numeric_only=True)
print('setelah dikelompokkan')
df_group
```

```
sebelum di kelompokkan
  Nama  Umur  Alamat  Pekerjaan  Gaji
0  Jack   20  Surabaya  Programmer  2000
1  Joko   23   Jakarta    DB Admin  3000
2  Dwi   34    Solo  Web developer  2400
3  Andi   23   Jakarta  Programmer  5000
```

```
setelah dikelompokkan
      Umur  Gaji
Pekerjaan
DB Admin  23.0  3000.0
Programmer  21.5  3500.0
Web developer  34.0  2400.0
```

Gambar 1.4.59 Aggregation

Operasi *aggregate* dilakukan dengan mengelompokkan data berdasarkan kolom *Pekerjaan* menggunakan `groupby()`, kemudian menerapkan fungsi `mean()` untuk menghitung rata-rata pada kolom numerik seperti pada Gambar 1.4.60.

```
df_group = df.groupby('Pekerjaan').mean(numeric_only=True)
print('setelah dikelompokkan')
df_group
```

setelah dikelompokkan

	Umur	Gaji
Pekerjaan		
DB Admin	23.0	3000.0
Programmer	21.5	3500.0
Web developer	34.0	2400.0

Gambar 1.4.60 Aggregation

Contoh operasi *aggregate* lain dengan menerapkan beberapa fungsi secara sekaligus melalui `agg()`, setelah data dikelompokkan berdasarkan kolom *Pekerjaan*. Pada contoh ini, fungsi *mean*, *sum*, dan *std* digunakan untuk menghitung rata-rata, total, dan standar deviasi pada kolom Umur dan Gaji untuk setiap kelompok pekerjaan.

```
df_group = df.groupby('Pekerjaan').agg({
    'Umur': [np.mean, np.sum, np.std],
    'Gaji': [np.mean, np.sum, np.std]
})
print('setelah dikelompokkan')
df_group
```

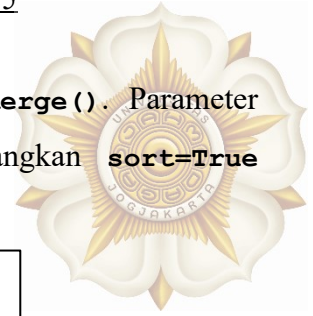
setelah dikelompokkan

/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`

	Umur			Gaji		
	mean	sum	std	mean	sum	std
Pekerjaan						
DB Admin	23.0	23	NaN	3000.0	3000	NaN
Programmer	21.5	43	2.12132	3500.0	7000	2121.320344
Web developer	34.0	34	NaN	2400.0	2400	NaN

Gambar 1.4.61 Aggregation

13) Merge



Penggabungan *dataframe* dilakukan menggunakan fungsi `merge()`. Parameter `on='Id'` menentukan *key column* yang digunakan, sedangkan `sort=True` mengurutkan hasil berdasarkan *key value* tersebut.

```
kiri = pd.DataFrame({'Id':[2, 3, 1],
'Nama' : ['Andi', 'Joko', 'Budi']})
kanan = pd.DataFrame({'Id':[1, 2, 3],
'Nama' : ['Mike', 'Jack', 'Jane']})
gabung = pd.merge(kiri, kanan, on='Id', sort=True)
gabung
```

	Id	Nama_x	Nama_y
0	1	Budi	Mike
1	2	Andi	Jack
2	3	Joko	Jane

Gambar 1.4.62 Merging Dataframe

```
karyawan = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat': ['Jakarta', 'Surabaya', 'Medan'],
'Id Pekerjaan' : [1,7,2]
})
pekerjaan = pd.DataFrame({
'Id Pekerjaan': [2,1,4,3],
'Nama': ['Programmer', 'Data Engineer', 'Manager', 'Web Developer'],
'Gaji' : [5000, 4000, 3000, 6000]
})
print(karyawan)
print(pekerjaan)
```

	Nama	Alamat	Id Pekerjaan
0	Budi	Jakarta	1
1	Joko	Surabaya	7
2	Maya	Medan	2

	Id Pekerjaan	Nama	Gaji
0	2	Programmer	5000
1	1	Data Engineer	4000
2	4	Manager	3000
3	3	Web Developer	6000

Gambar 1.4.63 Merging Dataframe

Penggabungan juga dapat dilakukan menggunakan `merge()` dengan parameter `how='left'`, sehingga semua baris dari *DataFrame* karyawan tetap ditampilkan, sementara data dari *DataFrame* pekerjaan hanya ditambahkan jika nilai Id Pekerjaan cocok.

```
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
                        how='left', sort=True)
gabung_merge
```

	Nama_x	Alamat	Id Pekerjaan		Nama_y	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0	
1	Maya	Medan	2	Programmer	5000.0	
2	Joko	Surabaya	7	NaN	NaN	

Gambar 1.4.64 Merging Dataframe

14) Join

Join dapat dilakukan dengan menggunakan **merge()** pada *key column* yang sama, di mana parameter **how='left'** memastikan seluruh data dari tabel kiri ditampilkan, sedangkan *suffixes* digunakan untuk membedakan nama kolom yang berasal dari kedua *DataFrame*.

```
#menggunakan suffixes
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
                        how='left', sort=True, suffixes=('_pegawai', '_pekerjaan'))
gabung_merge
```

	Nama_pegawai	Alamat	Id Pekerjaan		Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0	
1	Maya	Medan	2	Programmer	5000.0	
2	Joko	Surabaya	7	NaN	NaN	

Gambar 1.4.65 Joining Dataframe

Join juga dapat dilakukan dengan fungsi **join()**. *Join* bekerja berdasarkan indeks, sehingga hasilnya bisa berbeda dari **merge()** yang menggunakan *key column*. Parameter **how='left'** menjaga seluruh baris *DataFrame* kiri tetap tampil, sementara *rsuffix* digunakan untuk membedakan kolom yang duplikat dari *DataFrame* kanan.

```
#versi 1. hasil tidak sama dengan merge()
gabung_join = karyawan.join(pekerjaan, how='left', rsuffix='_pekerjaan')
gabung_join
```

	Nama	Alamat	Id Pekerjaan	Id Pekerjaan_pekerjaan	Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	2	Programmer	5000
1	Joko	Surabaya	7	1	Data Engineer	4000
2	Maya	Medan	2	4	Manager	3000

Gambar 1.4.66 Joining Dataframe

15) Concatenate

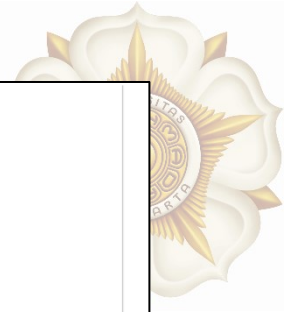
Concatenate dilakukan dengan menggunakan `pd.concat()` pada `axis=0`, sehingga *DataFrame* kedua ditambahkan sebagai baris baru di bawah *DataFrame* pertama. Parameter `ignore_index=True` digunakan agar indeks diurutkan ulang secara otomatis.

```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Alamat': ['Semarang', 'Yogya', 'Solo']
})
gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True)
gabung_concat
```

	Nama	Alamat
0	Jane	Semarang
1	mike	Yogya
2	dave	Solo
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.67 Concatenate

Dengan menambahkan parameter `axis=0` dan parameter `join='outer'` dapat menggabungkan semua kolom dari kedua *DataFrame*. Kolom yang tidak dimiliki oleh salah satu *DataFrame* akan berisi nilai *NaN*, sementara `ignore_index=True` mengatur ulang indeks agar berurutan.



```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Gaji': [5000, 4000, 3000]
})
gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True, join='outer')
gabung_concat
```

	Nama	Alamat	Gaji
0	Budi	Jakarta	NaN
1	Joko	Surabay	NaN
2	Maya	Medan	NaN
3	Jane	NaN	5000.0
4	mike	NaN	4000.0
5	dave	NaN	3000.0

Gambar 1.4.68 Concatenate

Jika menggunakan **axis=1**, penggabungan dilakukan secara horizontal sehingga kolom baru ditambahkan di samping *DataFrame* awal.

```
#gabung menjadi kolom baru. axis =1
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Pekerjaan': ['Programmer', 'Data Engineer', 'Manager', 'Web Developer'],
    'Gaji': [5000, 4000, 3000, 6000]
})
gabung_concat = pd.concat([kiri, kanan], axis=1, join='outer')
gabung_concat
```

	Nama	Alamat	Pekerjaan	Gaji
0	Budi	Jakarta	Programmer	5000
1	Joko	Surabay	Data Engineer	4000
2	Maya	Medan	Manager	3000
3	NaN	NaN	Web Developer	6000

Gambar 1.4.69 Concatenate

16) Reset Indeks



```
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Alamat': ['Semarang', 'Yogya', 'Solo']
})
gabung_append = pd.concat([kiri, kanan], ignore_index=True)
gabung_append
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.70 Reset Index

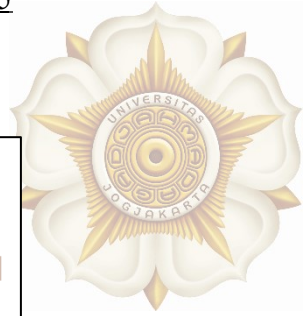
Reset *index* dilakukan dengan menggunakan `pd.concat()` dan menetapkan `ignore_index=True`, sehingga indeks lama dari kedua *DataFrame* diabaikan dan diganti dengan indeks baru yang berurutan setelah proses penggabungan.

```
new = gabungan_append.reset_index(drop=True)
new
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.71 Reset Index

Reset indeks juga dapat dilakukan dengan fungsi `reset_index(drop=True)`, yang menghapus indeks lama dan menggantinya dengan indeks baru yang berurutan. Parameter `drop=True` memastikan indeks sebelumnya tidak ikut disimpan sebagai kolom baru.



17) Set Indeks

```
kiri = pd.DataFrame({
    'ID' : ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})
print(kiri)
```

	ID	Nama	Alamat
0	001	Budi	Jakarta
1	002	Joko	Surabay
2	003	Maya	Medan

Gambar 1.4.72 Set Index

```
kiri.iloc[2]
```

2

ID 003

Nama Maya

Alamat Medan

dtype: object

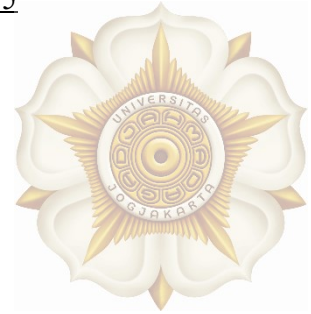
Gambar 1.4.73 Set Index

Set indeks dapat dilakukan dengan memilih baris tertentu sebagai referensi indeks baru. Pada Gambar 1.4.73, nilai pada baris ke-2 ditampilkan karena *DataFrame* sebelumnya telah atau akan diubah indeksnya menggunakan nilai kolom tertentu melalui `set_index()`, sehingga baris tersebut ditampilkan sesuai indeks barunya.

```
kiri.set_index('ID', inplace=True)
kiri
```

	Nama	Alamat
ID		
001	Budi	Jakarta
002	Joko	Surabay
003	Maya	Medan

Set indeks juga dapat dilakukan dengan fungsi `set_index()` yang menetapkan kolom *ID* sebagai indeks baru *DataFrame*.



```
kiri.loc['003']
```

	003
Nama	Maya
Alamat	Medan

```
dtype: object
```

Gambar 1.4.74 Set Index

`kiri.loc['003']` menampilkan data dengan indeks bernilai '003', sesuai indeks baru yang telah ditetapkan menggunakan kolom *ID*.

18) Pivot Tabel

```
karyawan = pd.DataFrame({
    'Emp_id' : ['001', '002', '003', '004', '005', '006'],
    'Nama': ['Budi', 'Joko', 'Maya', 'Andi', 'Dewi', 'Dian'],
    'Gender': ['Pria', 'Pria', 'Perempuan', 'Pria', 'Perempuan', 'Perempuan'],
    'Pekerjaan': ['Programmer', 'Data Engineer', 'Programmer', 'Manager',
    'Manager', 'Manager'],
    'Gaji': [5000, 4000, 5000, 6000, 8000, 6000]})
karyawan
```

	Emp_id	Nama	Gender	Pekerjaan	Gaji
0	001	Budi	Pria	Programmer	5000
1	002	Joko	Pria	Data Engineer	4000
2	003	Maya	Perempuan	Programmer	5000
3	004	Andi	Pria	Manager	6000
4	005	Dewi	Perempuan	Manager	8000
5	006	Dian	Perempuan	Manager	6000

Gambar 1.4.75 Membuat Dataframe

Pivot tabel dibuat menggunakan `pd.pivot_table()` dengan menetapkan kolom *Gender* dan *Pekerjaan* sebagai pembentuk kolom baru. Agregasi dilakukan melalui parameter `aggfunc`, untuk menjumlahkan nilai Gaji dan menghitung jumlah data berdasarkan Pekerjaan seperti pada Gambar 1.4.76

```

pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': 'sum', 'Pekerjaan': 'count'})
pivot_table

```

Gender	Perempuan	Pria
Pekerjaan	Manager	Programmer
Gaji	14000	5000
Pekerjaan	2	1

Gambar 1.4.76 Pivot Table

Menghitung sum, *mean*, median, dan lain-lain dapat dilakukan dengan `np.sum` seperti pada Gambar 1.4.77.

```
#cara kedua pakai numpy untuk menghitung sum, mean, dan lain lain
import numpy as np
pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': np.sum, 'Pekerjaan': 'count'})
pivot_table
```

/tmp/ipython-input-2104525659.py:3: FutureWarning: The provided callable <function sum at 0x7c2825b73100> is currently u
pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': np.sum, 'Pekerjaan': 'count'})

Gender	Perempuan	Pria			
Pekerjaan	Manager	Programmer	Data Engineer	Manager	Programmer
Gaji	14000	5000	4000	6000	5000
Pekerjaan	2	1	1	1	1

Gambar 1.4.77 Pivot Table

19) Melt

Melt digunakan untuk mengubah *DataFrame* dari format lebar (*wide*) menjadi format panjang (*long*).

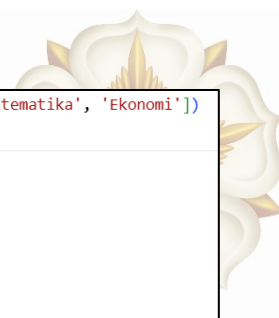
```

mahasiswa = pd.DataFrame({
    'Mhs_id': ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Bhs Inggris': [100, 90, 98],
    'Matematika': [90, 87, 72],
    'Ekonomi': [90, 80, 78]})
mahasiswa

```

	Mhs_id	Nama	Bhs Inggris	Matematika	Ekonomi
0	001	Budi	100	90	90
1	002	Joko	90	87	80
2	003	Maya	98	72	78

Gambar 1.4.78 Membuat Dataframe



```
melt_table = pd.melt(mahasiswa, id_vars=['Mhs_id', 'Nama'], value_vars=['Bhs Inggris', 'Matematika', 'Ekonomi'])
melt_table
```

	Mhs_id	Nama	variable	value
0	001	Budi	Bhs Inggris	100
1	002	Joko	Bhs Inggris	90
2	003	Maya	Bhs Inggris	98
3	001	Budi	Matematika	90
4	002	Joko	Matematika	87
5	003	Maya	Matematika	72
6	001	Budi	Ekonomi	90
7	002	Joko	Ekonomi	80
8	003	Maya	Ekonomi	78

Gambar 1.4.79 Melt Table

Kolom yang ditentukan pada **id_vars** tetap dipertahankan, sedangkan kolom pada **value_vars** diubah menjadi dua kolom baru, yaitu *variable* dan *value* sehingga tiap nilai mata pelajaran ditampilkan sebagai baris terpisah.

20) Lambda Function

Berikut penggunaan fungsi lambda dengan dua parameter untuk melakukan operasi penjumlahan.

```
x = lambda var1, var2 : var1 + var2
hasil = x(10, 12)
hasil
```

22

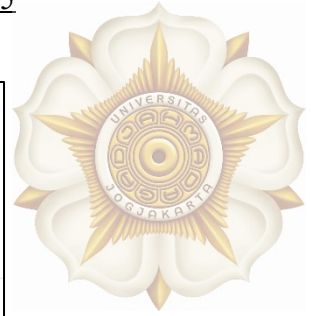
Gambar 1.4.80 Lambda Function

Berikut penggunaan fungsi lambda dengan satu parameter untuk melakukan operasi perkalian.

```
test = lambda var1 : var1 * 2
hasil = test(5)
hasil
```

10

Gambar 1.4.81 Lambda Function



```
mahasiswa = pd.DataFrame({
    'Mhs_id' : ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Matematika': [90, 47, 72]})
mahasiswa
```

	Mhs_id	Nama	Matematika
0	001	Budi	90
1	002	Joko	47
2	003	Maya	72

Gambar 1.4.82 Lambda Function

Berikut penggunaan fungsi lambda dengan kondisi bertingkat untuk menentukan nilai berdasarkan skor Matematika pada *DataFrame*.

```
mahasiswa['Grade'] = mahasiswa['Matematika'].apply(lambda nilai :
    'A' if nilai >=90 else 'B' if nilai >=70 and nilai <90 else ('C'))
mahasiswa
```

	Mhs_id	Nama	Matematika	Grade
0	001	Budi	90	A
1	002	Joko	47	C
2	003	Maya	72	B

Gambar 1.4.83 Lambda Function

21) Membaca File

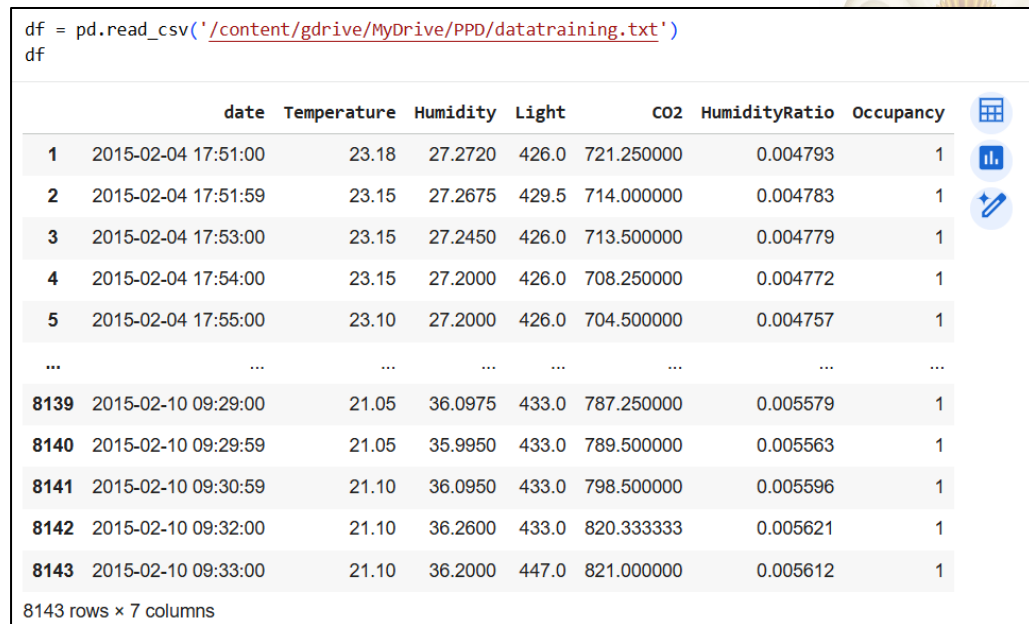
Membaca *file* dari Google Drive dapat dilakukan dengan menggunakan `drive.mount()`.

```
from google.colab import drive
drive.mount('/content/gdrive/')

Mounted at /content/gdrive/
```

Gambar 1.4.84 Menghubungkan Google Colaboratory dengan Google Drive

Lalu untuk membaca *file* dataset dari Google Drive, dapat dilakukan dengan menggunakan `pd.read_csv()` seperti pada Gambar 1.4.85.



```
df = pd.read_csv('/content/gdrive/MyDrive/PPD/datatraining.txt')
df
```

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.250000	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.000000	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.500000	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.250000	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.500000	0.004757	1
...	...	...	...	...	...	...	...
8139	2015-02-10 09:29:00	21.05	36.0975	433.0	787.250000	0.005579	1
8140	2015-02-10 09:29:59	21.05	35.9950	433.0	789.500000	0.005563	1
8141	2015-02-10 09:30:59	21.10	36.0950	433.0	798.500000	0.005596	1
8142	2015-02-10 09:32:00	21.10	36.2600	433.0	820.333333	0.005621	1
8143	2015-02-10 09:33:00	21.10	36.2000	447.0	821.000000	0.005612	1

8143 rows × 7 columns

Gambar 1.4.85 Membaca File dari Google Drive

1.5. TUGAS & ANALISIS

1. Buatlah *array* NumPy 2D berukuran 4×4 berisi angka acak (0–50), kemudian ubah bentuknya menjadi *array* 2×8 dan tampilkan hanya elemen-elemen yang bernilai lebih dari 25.
2. Buatlah *dataframe* untuk data karyawan yang berisi kolom Nama, Pekerjaan, dan Gaji. Lalu lakukan operasi **groupby()** pada kolom Pekerjaan dan hitung nilai maksimum, minimum, serta standar deviasi untuk kolom ‘Gaji’ menggunakan **agg()**.
3. Buat dua *DataFrame* untuk data karyawan dengan kolom Emp_id, Nama, dan Divisi, serta *DataFrame* gaji dengan kolom ID, Gaji, dan Status (masing-masing minimal tiga baris). Setelah itu, gabungkan kedua *DataFrame* tersebut menggunakan **merge()** dengan **left_on='Emp_id'** dan **right_on='ID'**, terapkan *left join*, dan gunakan *suffixes* untuk membedakan kolom yang sama.

1.6. REFERENSI

<https://numpy.org/>

<https://pandas.pydata.org/>

<https://www.python.org/doc/>



PERTEMUAN II

Exploratory Data Analysis dan Data Preprocessing

2.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar *Artificial Intelligence* (AI)
- B. Mahasiswa mengetahui dasar-dasar *Machine Learning* (ML)
- C. Mahasiswa dapat menerapkan langkah awal analisis data
- D. Mahasiswa mampu mengetahui dan menerapkan teknik-teknik dasar dalam *data preprocessing*

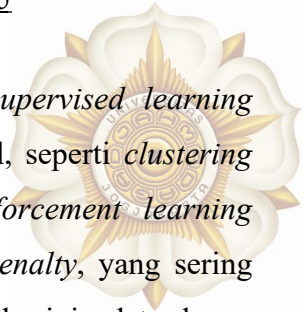
2.2. DASAR TEORI

- **Artificial Intelligence**

Artificial Intelligence (AI) merupakan bidang ilmu komputer yang berupaya membuat mesin mampu melakukan tugas-tugas yang biasanya membutuhkan kecerdasan manusia, seperti berpikir, belajar, mengenali pola, dan mengambil keputusan. AI tidak hanya terbatas pada sistem cerdas yang meniru manusia, tetapi juga mencakup algoritma dan pendekatan komputasional yang memungkinkan suatu sistem bekerja secara otonom dan adaptif. Perkembangan komputer modern, besarnya volume data, dan meningkatnya kemampuan komputasi membuat AI berkembang pesat di berbagai sektor seperti kesehatan, finansial, transportasi, hingga hiburan. Google AI Guides menjelaskan bahwa AI modern sangat bertumpu pada data, karena kemampuan kecerdasan sebuah sistem sangat bergantung pada kualitas data yang dipelajari dan pola-pola yang berhasil dipahami selama proses pemodelan.

- **Machine Learning**

Machine Learning adalah salah satu cabang AI yang menjadi fondasi bagi banyak inovasi teknologi saat ini. *Machine Learning* merupakan sebuah metode yang memungkinkan mesin meningkatkan performanya pada suatu tugas berdasarkan data. Inti dari ML adalah kemampuan sistem untuk mengenali pola dan membuat prediksi tanpa harus diprogram dengan aturan eksplisit. Secara umum, ML terbagi menjadi tiga, yaitu *supervised learning*, *unsupervised learning*, dan *reinforcement learning*. *Supervised learning* menggunakan data berlabel untuk melakukan prediksi atau klasifikasi, misalnya



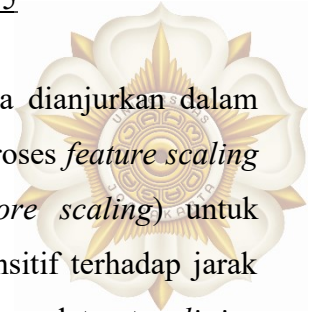
memprediksi harga rumah atau mendeteksi sentimen teks. *Unsupervised learning* berfungsi menemukan struktur tersembunyi dalam data tanpa label, seperti *clustering* pelanggan berdasarkan kebiasaan belanja. Sementara itu, *reinforcement learning* memungkinkan model belajar dari *feedback* berupa *reward* dan *penalty*, yang sering digunakan dalam robotika dan *game* AI. Dalam semua metode ini, data harus dipersiapkan dengan baik agar algoritma dapat belajar secara optimal. Kualitas model ML tidak hanya ditentukan oleh algoritma yang digunakan, tetapi juga oleh kualitas data dan *preprocessing* yang mendasarinya.

- **Exploratory Data Analysis**

Sebelum data dapat digunakan untuk membangun model ML, perlu dilakukan *Exploratory Data Analysis* (EDA). EDA merupakan proses penggalian informasi awal dari dataset untuk memahami karakteristik dan kondisi data secara menyeluruh. Gagasan EDA pertama kali diperkenalkan oleh John W. Tukey, yang menekankan pentingnya “mendengarkan apa yang data katakan” sebelum terburu-buru membuat asumsi atau membangun model. Dalam praktik modern, seperti yang dijelaskan dalam *Kaggle Learn* dan *IBM Data Analytics Guides*, EDA melibatkan berbagai aktivitas seperti melihat distribusi variabel melalui histogram atau *boxplot*, mencari korelasi antar fitur menggunakan *heatmap*, mendeteksi *outliers*, memeriksa tipe data, serta mengidentifikasi masalah seperti *missing values* atau duplikasi. Tahapan ini sangat penting karena memberikan gambaran awal tentang kualitas dataset, potensi masalah, dan tindakan apa saja yang perlu dilakukan selama *preprocessing*. Tanpa EDA, proses pemodelan bisa tidak akurat atau bias akibat ketidaktahuan terhadap struktur dan perilaku data.

- **Data Preprocessing**

Data preprocessing adalah proses sistematis untuk membersihkan, mengubah, dan menyiapkan data agar dapat digunakan secara optimal dalam algoritma *machine learning*. *Preprocessing* disebut sebagai tahapan kritis yang secara langsung memengaruhi performa dan generalisasi model. Tahapan pertama biasanya dimulai dari *data cleaning*, yaitu penanganan *missing values* menggunakan metode imputasi seperti *mean*, median, atau mode, serta menghapus atau memperbaiki data duplikat yang dapat mengganggu konsistensi analisis. Tahap berikutnya adalah data *transformation*, yang bertujuan mengubah data ke bentuk yang lebih representatif. Fitur kategorikal perlu di-*encode*



menggunakan *label encoding* atau *one-hot encoding*, sebagaimana dianjurkan dalam *Google Machine Learning Crash Course*. Selanjutnya, dilakukan proses *feature scaling* seperti normalisasi (*min-max scaling*) atau standardisasi (*z-score scaling*) untuk menyamakan rentang nilai antar fitur, terutama pada algoritma sensitif terhadap jarak seperti KNN dan SVM. Langkah penting lainnya adalah melakukan dataset *splitting* seperti *train-test split* (misalnya 80:20) untuk memastikan model dapat diuji secara objektif dan tidak *overfitting*. Seluruh proses ini umumnya dilakukan menggunakan pustaka numerik dan analitik seperti NumPy dan Pandas.

2.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

2.4. LANGKAH PERCOBAAN

- **Exploratory Data Analysis (EDA)**

1. Impor Pustaka

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

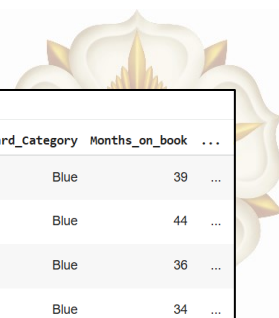
Gambar 2.4.1 Import Library

2. Load dataset

Link dataset: <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>

```
df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
data_science_practice/main/BankChurners.csv', delimiter=',')
```

3. Melihat lima baris pertama dengan `df.head()`



```
df_churning.head()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...
0	768805383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K	Blue	39	...
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue	44	...
2	713982108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K	Blue	36	...
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K	Blue	34	...
4	709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K	Blue	21	...

5 rows x 21 columns

Gambar 2.4.2 Lima Baris Pertama Dataset

4. Melihat lima baris terakhir dengan df.tail()

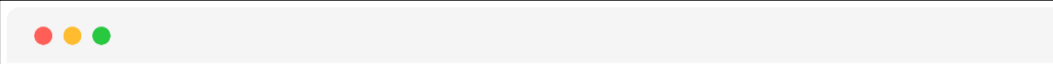
```
df_churning.tail()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...
10122	772366833	Existing Customer	50	M	2	Graduate	Single	\$40K - \$60K	Blue	40	...
10123	710638233	Attrited Customer	41	M	2	Unknown	Divorced	\$40K - \$60K	Blue	25	...
10124	716506083	Attrited Customer	44	F	1	High School	Married	Less than \$40K	Blue	36	...
10125	717406983	Attrited Customer	30	M	2	Graduate	Unknown	\$40K - \$60K	Blue	36	...
10126	714337233	Attrited Customer	43	F	2	Graduate	Married	Less than \$40K	Silver	25	...

5 rows x 21 columns

Gambar 2.4.3 Lima Baris Terakhir Dataset

5. Menyimpan nilai dari class menjadi 2 dataframe yang berbeda



```
existing_data = df_churning[(df_churning.Attrition_Flag == 'Existing Customer')]
attrited_data = df_churning[(df_churning.Attrition_Flag == 'Attrited Customer')]
```

Gambar 2.4.4 Pemisahan Dataset Bank Churner

Kode tersebut merupakan kode untuk memisahkan dataset berdasarkan nilai kolom `Attrition_Flag`, sehingga terbentuk dua subset yaitu *existing_data* yang berisi pelanggan yang masih aktif (*Existing Customer*) dan *attrited_data* yang berisi pelanggan yang berhenti atau *churn* (*Attrited Customer*).

6. Line chart



```
#sumbu x
x = np.array([2, 4, 6, 8])

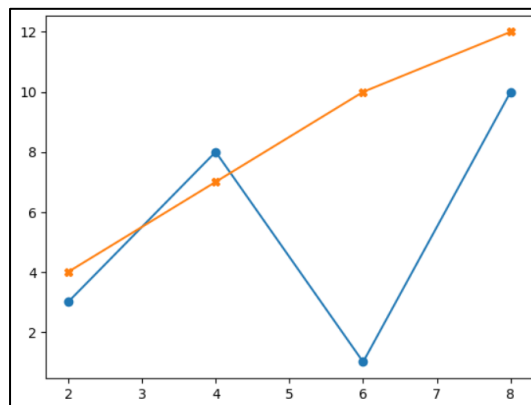
#sumbu y
y1 = np.array([3, 8, 1, 10])
y2 = np.array([4, 7, 10, 12])

plt.plot(x, y1, marker = 'o')
plt.plot(x, y2, marker = 'x')

plt.show()
```

Gambar 2.4.5 Pembuatan Line Chart dengan Data Dummy

Pada kode di atas, x dipakai sebagai sumbu horizontal, y1 dan y2 sebagai dua garis berbeda di sumbu vertikal. Parameter **marker='o'** dan **'x'** digunakan untuk memberi tanda di setiap titik data. **plt.show()** digunakan untuk menampilkan plot ke layar. Tujuan pemanggilan fungsi ini adalah menunjukkan cara membuat *line chart* untuk melihat tren atau perubahan nilai dari dua seri data sekaligus. *Line chart* yang ditampilkan dapat dilihat pada Gambar 2.4.6.



Gambar 2.4.6 Line Chart dari Data Dummy

Setelah memahami konsep *line chart* menggunakan data *dummy*, langkah berikutnya adalah menerapkannya pada dataset IoT untuk menampilkan perubahan nilai sensor terhadap waktu.

```
df_iot = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/datatraining.txt')
df_iot['date'] = pd.to_datetime(df_iot['date'])
df_iot.head()
```

Gambar 2.4.7 Load Dataset IoT

Setelah dataset siap, `df_iot.head()` dipanggil untuk menampilkan beberapa baris awal sebagai verifikasi bahwa data telah terbaca dengan benar.



	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

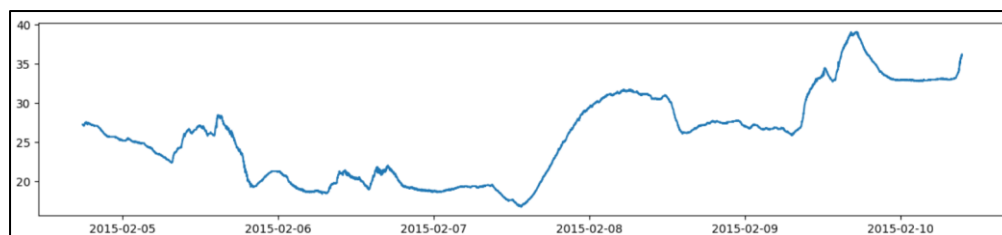
Gambar 2.4.8 Sampel Data IoT

Langkah berikutnya adalah menampilkan *line chart* untuk memvisualisasikan perubahan nilai sensor terhadap waktu.

```
plt.figure(figsize=(15, 3))
plt.plot(df_iot['date'], df_iot['Humidity'])
plt.show()
```

Gambar 2.4.9 Pembuatan Line Chart dengan Data IoT

Kode `plt.figure(figsize=(15, 3))` berfungsi untuk mengatur ukuran grafik agar panjang secara horizontal. Selanjutnya digunakan `plt.plot(df_iot['date'], df_iot['Humidity'])` untuk memetakan kolom *date* sebagai sumbu-x dan nilai *Humidity* sebagai sumbu-y. Grafik kemudian ditampilkan dengan `plt.show()`.



Gambar 2.4.10 Line Chart dengan Data IoT

7. Pie chart

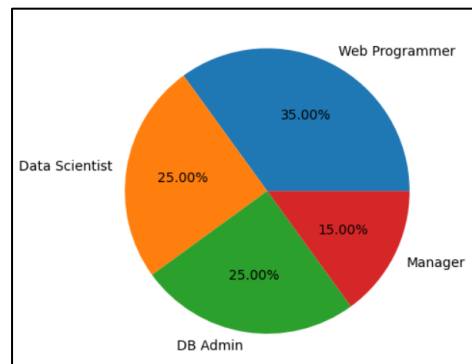


```
y = np.array([35, 25, 25, 15])
mylabels = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

plt.pie(y, labels = mylabels, autopct='%0.2f%%')
plt.show()
```

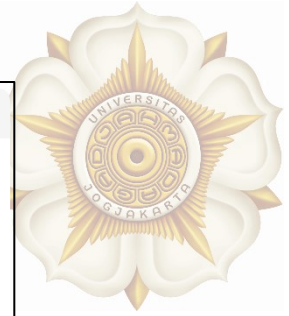
Gambar 2.4.11 Pembuatan Pie Chart dengan Data Dummy

Pembuatan *pie chart* dimulai dengan mendefinisikan *array* *y* yang berisi nilai-nilai numerik mewakili frekuensi tiap kategori, yaitu [35, 25, 25, 15]. Kemudian dibuat daftar **mylabels** yang merupakan nama kategori: “Web Programmer”, “Data Scientist”, “DB Admin”, dan “Manager”. Fungsi **plt.pie()** digunakan untuk menghasilkan *pie chart*, dengan parameter **labels=mylabels** untuk memberi nama pada setiap irisan grafik. Sedangkan **autopct='%0.2f%%'** digunakan untuk menampilkan persentase masing-masing irisan dengan format dua angka di belakang koma. Terakhir, **plt.show()** memunculkan visualisasi ke layar.



Gambar 2.4.12 Pie Chart dengan Data Dummy

Selanjutnya, pembuatan *pie chart* dilakukan menggunakan dataset sebelumnya yaitu *Bank Churner*. Visualisasi ini bertujuan untuk menampilkan proporsi kategori pada variabel **Marital_Status**.



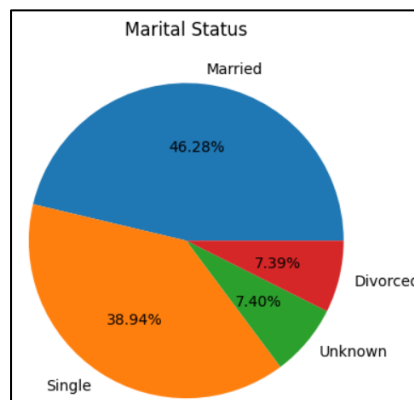
```
data = df_churning['Marital_Status'].value_counts()

# fungsi index untuk melihat label dari series
label = data.index

plt.pie(data, labels=label, autopct='%.2f%%')
plt.title('Marital Status')
plt.show()
```

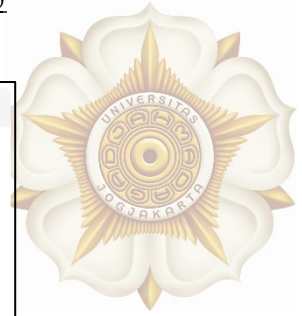
Gambar 2.4.13 Pembuatan Pie Chart dengan Data Bank Churner

Kode dimulai dengan memanggil `df_churning['Marital_Status'].value_counts()` untuk menghitung jumlah kemunculan setiap kategori dalam kolom tersebut. Nilai frekuensi yang dihasilkan kemudian disimpan dalam variabel `data`, sementara `data.index` digunakan untuk mengambil nama kategorinya dan disimpan ke dalam variabel `label`. Fungsi `plt.pie(data, labels=label, autopct='%.2f%%')` digunakan untuk menggambar *pie chart* dengan persentase setiap kategori ditampilkan secara otomatis. Selanjutnya, `plt.title('Marital Status')` digunakan untuk menambahkan judul pada grafik.



Gambar 2.4.14 Pie Chart Marital Status

8. Bar plot



```

y = np.array([35, 25, 25, 15])
x = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

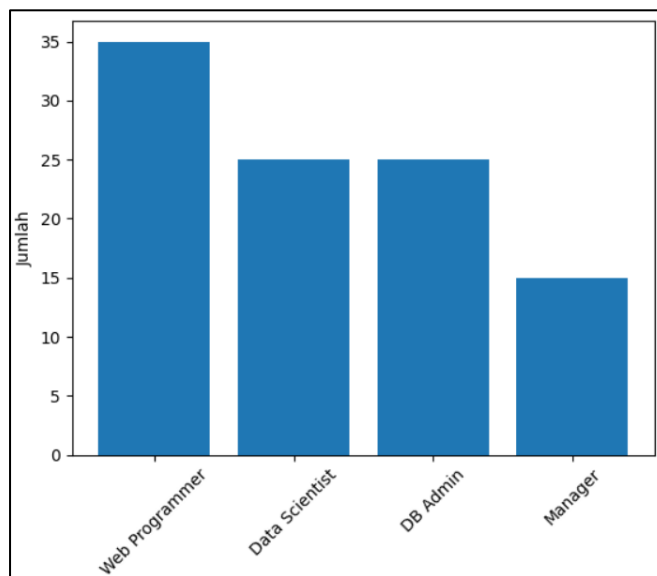
plt.bar(x,y)

plt.xticks(rotation=45)
plt.ylabel('Jumlah')
plt.show()

```

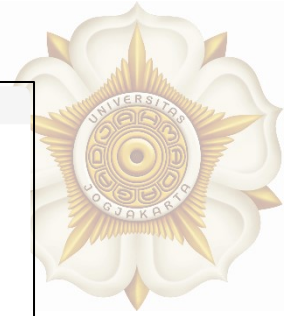
Gambar 2.4.15 Pembuatan Bar Plot dengan Data Dummy

Pembuatan bar plot dimulai dengan mendefinisikan *array* *y* yang berisi nilai jumlah pekerja pada beberapa jenis profesi, dan daftar *x* yang berisi nama kategori profesi seperti “Web Programmer”, “Data Scientist”, “DB Admin”, dan “Manager”. Selanjutnya, fungsi `plt.bar(x, y)` digunakan untuk menggambar batang pada sumbu *x* berdasarkan kategori, dengan tinggi batang mengikuti nilai yang diberikan pada variabel *y*.



Gambar 2.4.16 Bar Plot dengan Data Dummy

Langkah selanjutnya adalah membuat bar plot dengan dataset *Bank Churner* untuk membandingkan distribusi kategori **Marital_Status** antara dua kelompok nasabah, yaitu *existing customers* dan *attrited customers*.



```

y1 = existing_data['Marital_Status'].value_counts()
y2 = attrited_data['Marital_Status'].value_counts()

x1 = np.arange(len(y1.index))
x2 = np.arange(len(y2.index))
bar_width = 0.4

plt.bar(x1, y1, width=bar_width)
plt.bar(x2+bar_width, y2, width=bar_width)

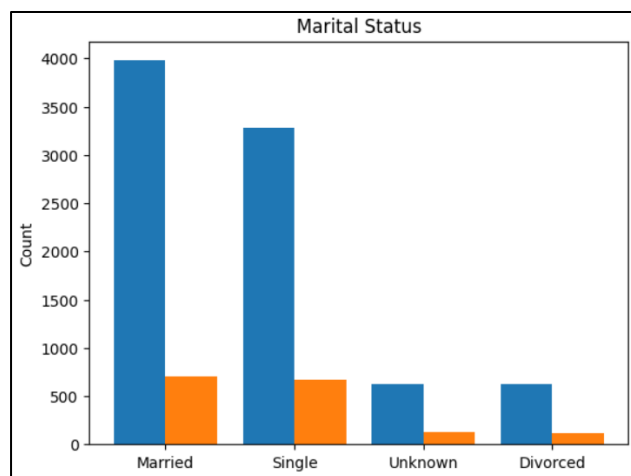
plt.title('Marital Status')
plt.ylabel('Count')
plt.xticks(x2 + bar_width / 2, labels=df_churning.Marital_Status.unique())

plt.show()

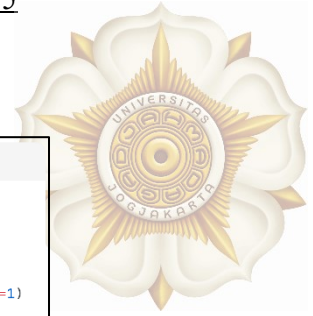
```

Gambar 2.4.17 Pembuatan Bar Plot Marital Status

Langkah pertama adalah menghitung jumlah masing-masing kategori marital status pada kedua subset menggunakan `value_counts()`, yang disimpan dalam variabel `y1` dan `y2`. Lalu membuat indeks posisi awal masing-masing kategori dengan `np.arange(len(y1.index))` dan `np.arange(len(y2.index))`. Variabel `bar_width = 0.4` digunakan untuk menentukan lebar batang. Fungsi `plt.bar(x1, y1, width=bar_width)` menggambar batang pertama untuk *existing customers*, sementara `plt.bar(x2 + bar_width, y2, width=bar_width)` menggambar batang kedua untuk *attrited customers*. Selanjutnya, judul grafik ditetapkan dengan `plt.title("Marital Status")`, dan sumbu-y diberi label "Count". `plt.xticks(x2 + bar_width / 2, labels = df_churning.Marital_Status.unique())`, digunakan untuk memastikan label kategori muncul tepat di tengah pasangan batang untuk setiap kelompok marital status.



Gambar 2.4.18 Bar Plot Marital Status



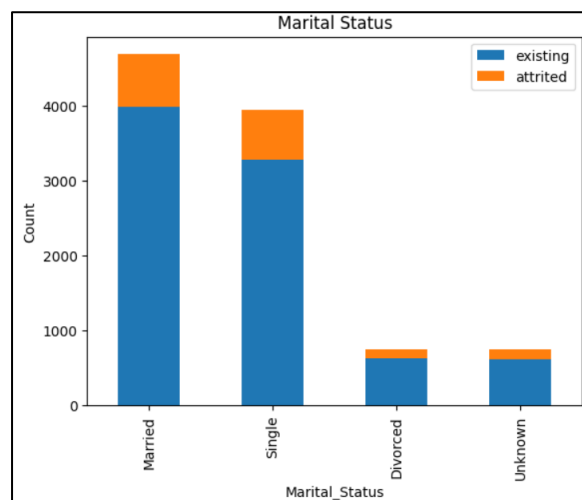
9. *Stacked* bar plot

```
# menggunakan dataset diatas
# contoh menggunakan stacked bar plot
data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', stacked=True) # bisa pakai cara 2
plt.title('Marital Status')
plt.ylabel('Count')
plt.show()
```

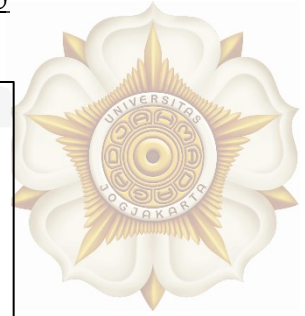
Gambar 2.4.19 Pembuatan *Stacked Bar Plot* Marital Status

Langkah selanjutnya yaitu membuat *stacked* bar plot untuk menganalisis komposisi kategori **Marital_Status** pada dua kelompok nasabah dalam dataset *Bank Churner*. Jumlah masing-masing kategori marital status dihitung menggunakan `value_counts()`, yang disimpan dalam variabel `data1` untuk *existing customers* dan `data2` untuk *attrited customers*, kemudian digabungkan menjadi satu *DataFrame* menggunakan `pd.concat()` dengan parameter `keys=['existing', 'attrited']` dan `axis=1`, sehingga muncul dua kolom baru yang merepresentasikan kedua kelompok tersebut. Visualisasi dibuat menggunakan `df_new.plot(kind='bar', stacked=True)`, yang menghasilkan diagram batang bertumpuk



Gambar 2.4.20 *Stacked Bar Plot* Marital Status

10. Histogram

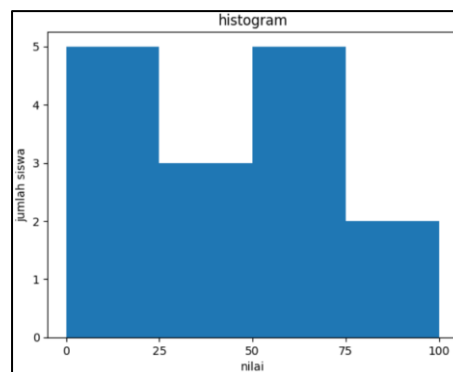


```
data = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
bins = [0,25,50,75,100]

plt.hist(data, bins = bins)
plt.title("histogram")
plt.xticks([0,25,50,75,100])
plt.xlabel('nilai')
plt.ylabel('jumlah siswa')
plt.show()
```

Gambar 2.4.21 Pembuatan Histogram dengan Data Dummy

Pembuatan histogram dimulai dengan pembuatan data *dummy* yang disimpan dalam *array*. Variabel *bins* didefinisikan digunakan untuk membagi data ke dalam empat interval atau kelas. Fungsi `plt.hist(data, bins=bins)` digunakan untuk menggambar histogram, di mana setiap batang menunjukkan jumlah data yang jatuh ke dalam masing-masing interval.



Gambar 2.4.22 Histogram dengan Data Dummy

Langkah selanjutnya adalah membuat histogram dengan dataset *Bank Churner*, khususnya pada variabel **Customer_Age**.

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

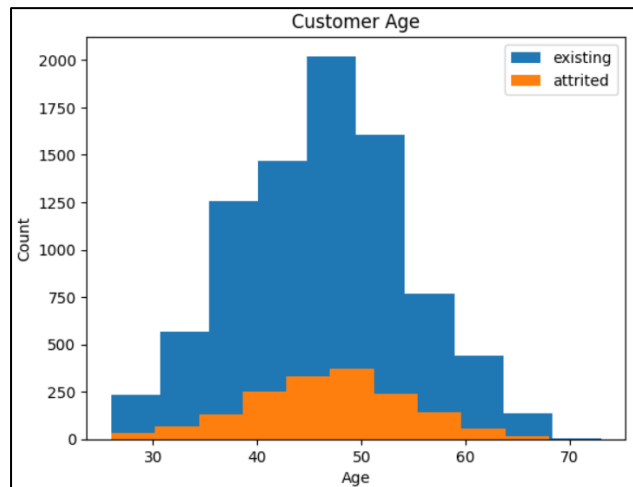
plt.hist(data1, label='existing')
plt.hist(data2, label='attrited')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend()
plt.show()
```

Gambar 2.4.23 Pembuatan Histogram Customer Age

Dua subset data *existing_data* dan *attrited_data* digunakan untuk memisahkan nasabah yang masih aktif dan nasabah yang sudah berhenti. Nilai usia dari masing-masing kelompok diambil menggunakan `existing_data['Customer_Age']` dan `attrited_data['Customer_Age']`, lalu disimpan dalam `data1` dan `data2`. Visualisasi dilakukan dengan memanggil `plt.hist(data1, label='existing')` dan `plt.hist(data2, label='attrited')`.



Gambar 2.4.24 Histogram Customer Age

X

11. Scatter plot

```

y1 = attrited_data['Months_on_book']
x1 = attrited_data['Customer_Age']

y2 = existing_data['Months_on_book']
x2 = existing_data['Customer_Age']

plt.scatter(x2, y2)
plt.scatter(x1, y1)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')

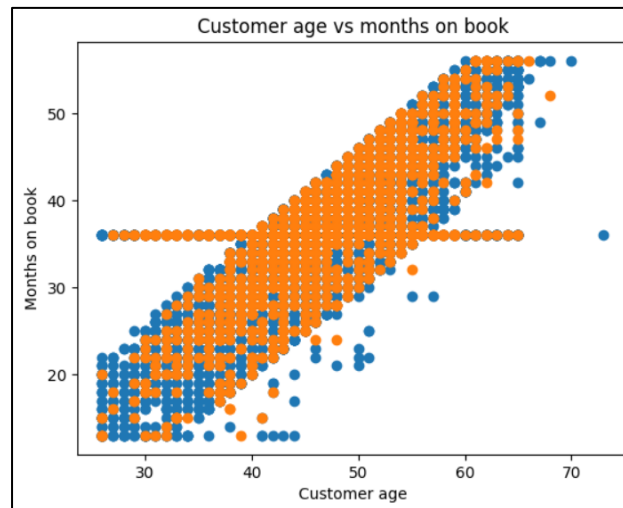
plt.show()

```

Gambar 2.4.25 Pembuatan Scatter Plot dengan `plt.scatter`

Pembuatan *scatter* plot dimulai dengan memisahkan kedua variabel untuk dua kelompok yaitu *attrited customers* (`x1` untuk usia dan `y1` untuk lama berlangganan) serta *existing customers* (`x2` dan `y2`). Dengan menggunakan `plt.scatter(x2, y2)`

dan `plt.scatter(x1, y1)`, kedua kelompok digambarkan dalam satu grafik *scatter*.



Gambar 2.4.26 Scatter Plot Customer Age vs Months on Book

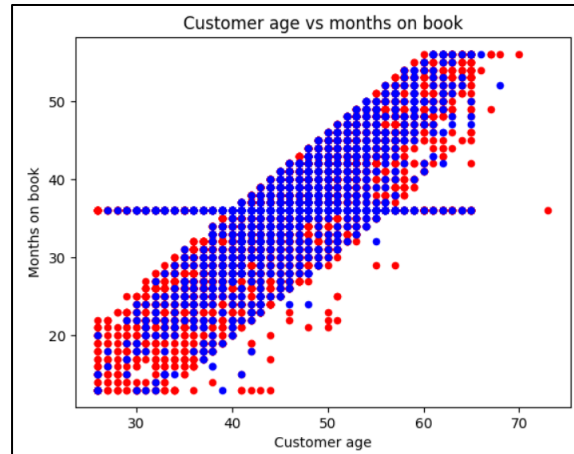
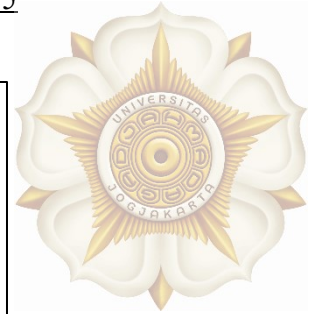
Versi lain pembuatan *scatter* plot menggunakan fungsi `.plot.scatter()`. Cara ini lebih ringkas dan terstruktur dibandingkan pemanggilan `plt.scatter()` secara manual.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.scatter dari object dataframe
ax=existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')
attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')
plt.show()
```

Gambar 2.4.27 Pembuatan Scatter Plot dengan fungsi `plot.scatter`

Visualisasi dimulai dengan memanggil `existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')`, yang menghasilkan scatter plot berwarna merah untuk nasabah yang masih aktif. Hasil plot ini disimpan dalam variabel `ax`, yang berfungsi sebagai *axes reference* untuk menumpuk plot berikutnya pada grafik yang sama. Baris selanjutnya memanggil `attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)`, yang menggambar titik-titik untuk nasabah churn dengan warna biru pada axis yang sama.



Gambar 2.4.28 Scatter Plot Customer Age vs Months on Book

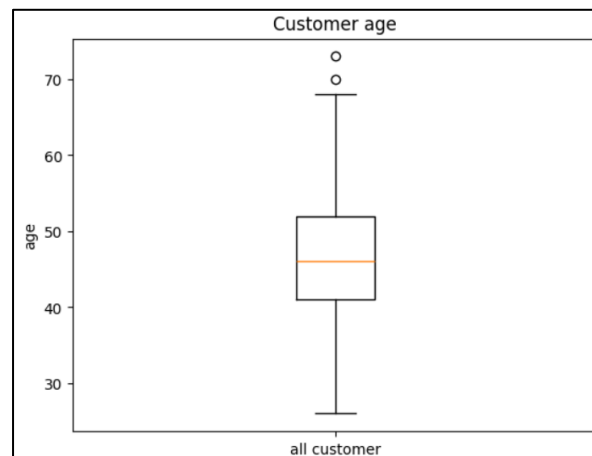
12. Box plot

```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi boxplot dari plt. single group.
plt.boxplot(df_churning['Customer_Age'])

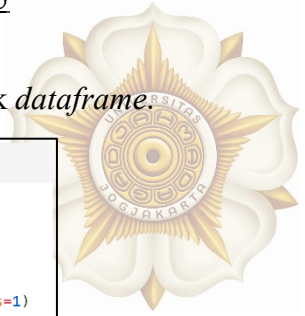
plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1], labels=['all customer'])
plt.show()
```

Gambar 2.4.29 Pembuatan Box Plot dengan plt.boxplot

Pembuatan *boxplot* dilakukan dengan memanggil `plt.boxplot(df_churning['Customer_Age'])`. Visualisasi yang dihasilkan memperlihatkan nilai median, sebaran kuartil, serta potensi *outlier* dalam satu kelompok data.



Gambar 2.4.30 Box Plot Customer Age



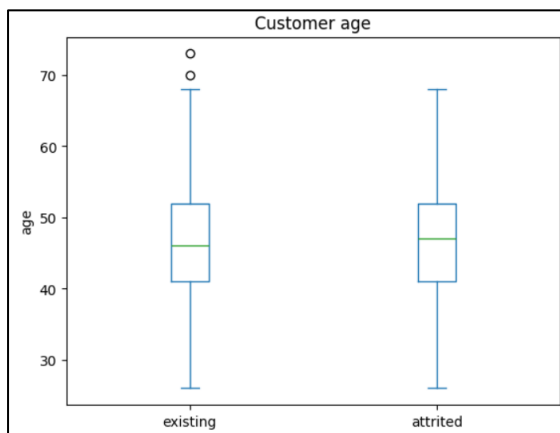
Pembuatan *boxplot* menggunakan fungsi `plot.box()` dari objek *dataframe*.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.box dari object dataframe
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

df_new.plot.box() #bisa pakai cara ini
#df_new.plot(kind='box') #bisa menggunakan cara ini juga

plt.title('Customer age')
plt.ylabel('age')
plt.show()
```

Gambar 2.4.31 Pembuatan Box Plot dengan fungsi `plot.box`



Gambar 2.4.32 Box Plot Customer Age

13. Violin plot

```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi violinplot dari plt. multiple group.

data1 = existing_data['Customer_Age'].values
data2 = attrited_data['Customer_Age'].values
#buat object dictionary dahulu
my_dictionary = {'existing': data1, 'attrited': data2}
#ambil value nya
plt.violinplot(list(my_dictionary.values()))

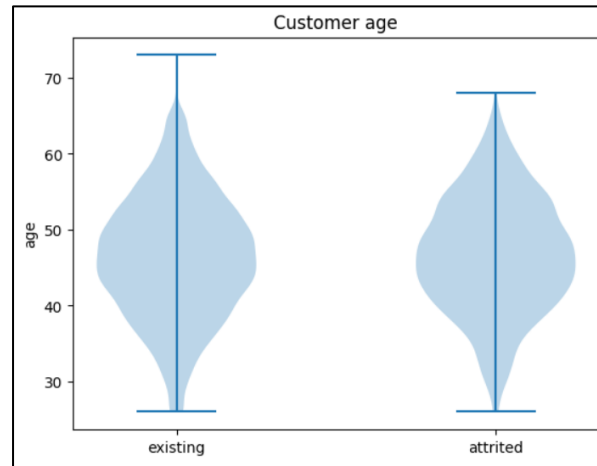
plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1,2], labels=list(my_dictionary.keys()))
plt.show()
```

Gambar 2.4.33 Pembuatan Violin Plot

Pembuatan violin plot dimulai dengan mengambil nilai `Customer_Age` dari kedua kelompok pelanggan, lalu menyusunnya dalam *dictionary*. Data tersebut kemudian divisualisasikan menggunakan fungsi utama `plt.violinplot()`, yang



menampilkan distribusi dan kepadatan usia untuk *existing* dan *attrited customers* dalam satu grafik.



Gambar 2.4.34 Violin Plot Customer Age

14. Sub plot

```
#versi 1
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1) #2 baris, 2 kolom, plot 1
plt.boxplot(existing_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['existing customer'])

plt.subplot(2, 2, 2)
plt.boxplot(attrited_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['attrited customer'])

plt.subplot(2, 2, 3)
existing_data['Customer_Age'].plot.hist()
plt.ylabel('Count')
plt.xlabel('Age of existing customer')
plt.ylim([0, 2100])

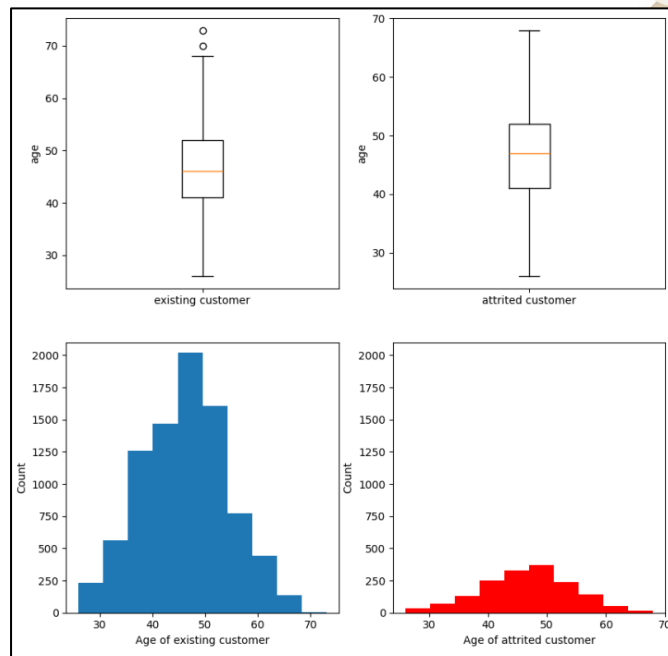
plt.subplot(2, 2, 4)
attrited_data['Customer_Age'].plot.hist(color='r')
plt.ylabel('Count')
plt.xlabel('Age of attrited customer')
plt.ylim([0, 2100])

plt.show()
```

Gambar 2.4.35 Pembuatan Sub Plot

Kode di atas membuat empat visualisasi sekaligus dalam satu *figure* menggunakan `plt.subplot()`, yaitu dua *boxplot* (untuk *existing* dan *attrited customers*) serta dua histogram usia untuk kedua kelompok tersebut. Tujuan penggunaan *subplot* adalah

menampilkan perbandingan distribusi usia antar kelompok dalam satu tampilan sehingga memudahkan analisis visual secara menyeluruh.



Gambar 2.4.36 Sub Plot

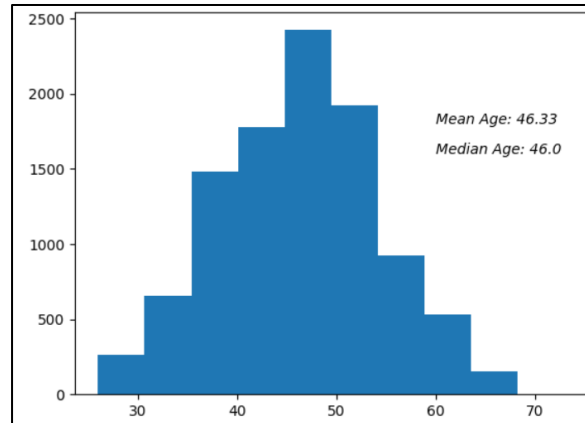
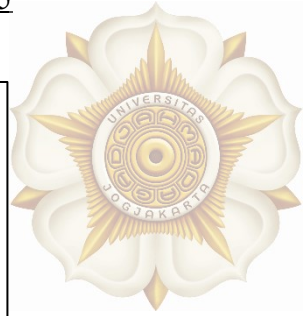
15. Annotation

```
# Show the distribution of a data and its measure of central tendency
data = df_churning['Customer_Age']
plt.hist(data)

plt.text(60, 1600, "Median Age: " + str(round(data.median(),2)),
        style = 'italic', fontsize=10)
plt.text(60, 1800, "Mean Age: " + str(round(data.mean(),2)),
        style = 'italic', fontsize=10)
plt.show()
```

Gambar 2.4.37 Pembuatan Annotation

Kode di atas menampilkan histogram usia pelanggan (Customer_Age) dan menambahkan informasi statistik langsung pada grafik menggunakan `plt.text()`. Dua anotasi yang ditampilkan yaitu *Median Age* dan *Mean Age*.



Gambar 2.4.38 Annotation pada Histogram

16. Axis

```
#x = np.array([1, 2, 3, 4])
x = np.array(["01/02/2020", "01/03/2020", "01/04/2020", "01/05/2020"])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([13, 4, 10, 12])

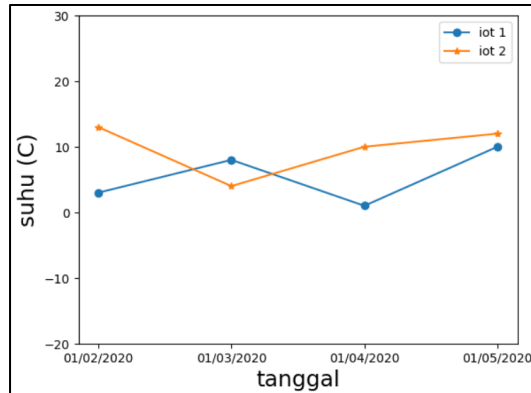
plt.plot(x,y1, marker='o', label='iot 1')
plt.plot(x,y2, marker='*', label='iot 2')
plt.legend()

# set label
plt.ylabel('suhu (c)', fontsize=18) # buat label untuk sumbu y
plt.xlabel('tanggal', fontsize=18) # buat label untuk sumbu x

#set axis range
plt.ylim([-20,30])
plt.show()
```

Gambar 2.4.39 Pengaturan Axis

Kode di atas menampilkan dua *line chart* yang menunjukkan data suhu pada beberapa hari. Fungsi `plt.plot()` digunakan untuk menggambar dua garis berbeda dengan *marker* serta label yang dibedakan. Kemudian `plt.legend()` ditambahkan untuk menampilkan keterangan setiap garis. Setelah itu, `plt.ylabel()` dan `plt.xlabel()` digunakan untuk memberikan label pada sumbu-y dan sumbu-x agar grafik lebih informatif. `plt.ylim([-20, 30])` digunakan untuk mengatur rentang nilai pada sumbu-y, agar tampilan grafik menjadi lebih mudah dibaca.



Gambar 2.4.40 Line Chart

17. Legend

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

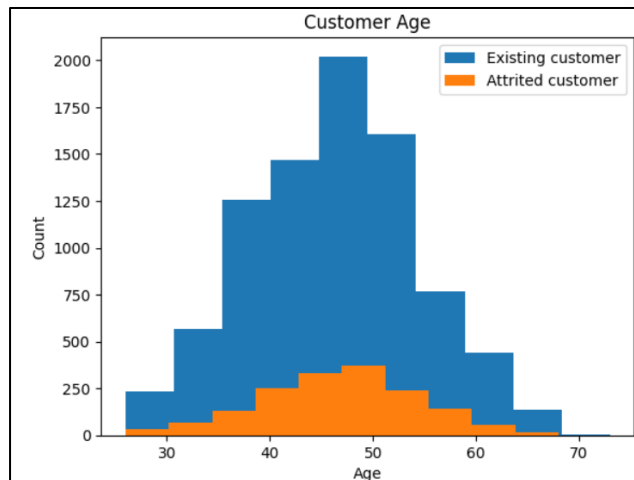
plt.hist(data1, label='Existing customer') # pertama, set label dulu
plt.hist(data2, label='Attrited customer')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend() # kedua, panggil fungsi legend
plt.show()
```

Gambar 2.4.41 Pembuatan Legend

Kode di atas menampilkan dua histogram usia (*Customer_Age*) untuk *existing* dan *attrited customers* dalam satu grafik. Label untuk masing-masing kelompok ditentukan terlebih dahulu melalui parameter label pada fungsi `plt.hist()`. Setelah kedua histogram didefinisikan, `plt.legend()` dipanggil untuk menampilkan kotak *legend* yang menunjukkan warna dari masing-masing kelompok.



Gambar 2.4.42 Histogram dengan Legend

• Data Preprocessing

1. Impor pustaka

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

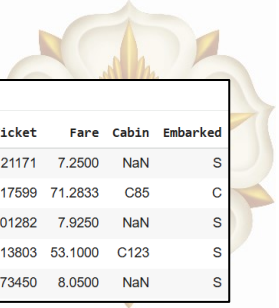
Gambar 2.4.43 Import Library

2. Load dataset

Link: <https://www.kaggle.com/datasets/yasserh/titanic-dataset>

```
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
data_science_practice/main/train.csv')
```

3. Melihat lima baris data pertama dengan df.head()



```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Gambar 2.4.44 Lima Baris Pertama Dataset

4. Melihat lima baris data terakhir dengan df.tail()

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Gambar 2.4.45 Lima Baris Terakhir Dataset

5. Melihat bentuk dataset dengan df.shape

```
# Melihat bentuk dataset. Baris, kolom
df.shape

(891, 12)
```

Gambar 2.4.46 Melihat Bentuk Dataset dengan df.shape

Output (891, 12) menunjukkan bahwa dataset memiliki 891 baris data dan 12 kolom fitur.

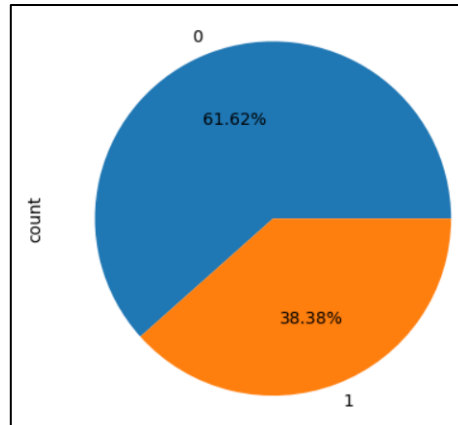
6. Visualisasi distribusi data



```
data = df['Survived'].value_counts()
data.plot(kind='pie', autopct='%0.2f%%')
plt.show()
```

Gambar 2.4.47 Membuat Pie Chart

Kode di atas membuat *pie chart* untuk menampilkan proporsi penumpang yang selamat dan tidak selamat berdasarkan nilai *Survived* dalam dataset. Hasilnya dapat dilihat pada Gambar 2.4.48

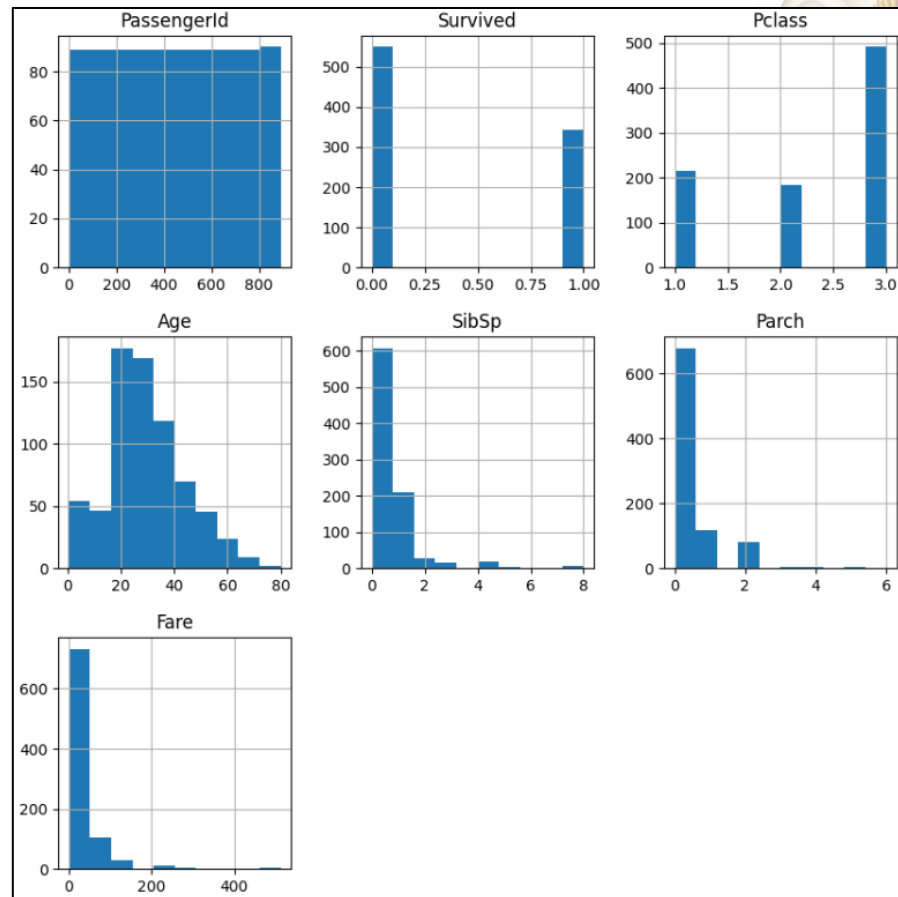


Gambar 2.4.48 Pie Chart Distribusi Data

```
df.hist(figsize=(10,10))
plt.show()
# menggunakan histogram, untuk melihat distribusi nya
```

Gambar 2.4.49 Membuat Histogram

Kode di atas menampilkan histogram untuk seluruh kolom numerik dalam dataset guna melihat distribusi datanya. Hasilnya dapat dilihat pada Gambar 2.4.50.



Gambar 2.4.50 Histogram Kolom Numerik

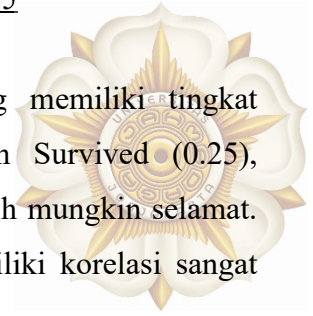
7. Analisis korelasi antar variabel

```
df.corr(numeric_only = True)
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

Gambar 2.4.51 Korelasi Antar Variabel

Kode di atas menghitung korelasi antar variabel numerik dalam dataset, sehingga kita bisa melihat hubungan mana yang paling kuat atau lemah antar fitur. Hasilnya, menunjukkan bahwa Pclass memiliki korelasi negatif cukup kuat dengan Survived



(-0.33), artinya penumpang kelas lebih rendah cenderung memiliki tingkat keselamatan lebih rendah. Fare berkorelasi positif dengan Survived (0.25), menunjukkan bahwa penumpang dengan tiket lebih mahal lebih mungkin selamat. Sementara variabel lain seperti Age, SibSp, dan Parch memiliki korelasi sangat lemah terhadap Survived.

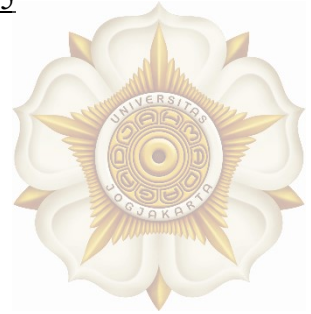
8. Descriptive statistics

df.describe()							
	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Gambar 2.4.52 Deskripsi Statistik

Hasil `df.describe()` menampilkan ringkasan statistik untuk fitur numerik. Dari output di atas, terlihat bahwa rata-rata usia penumpang adalah 29.7 tahun, dengan rentang dari 0.42 hingga 80 tahun. Harga tiket (Fare) memiliki variasi besar, mulai dari 0 hingga 512, menunjukkan ketimpangan biaya perjalanan. Variabel Pclass dan Survived juga terlihat sebagai fitur kategorikal numerik dengan nilai 0–1 dan 1–3. Statistik ini membantu kita untuk memahami sebaran, pusat data, serta potensi *outlier* sebelum dianalisis lebih lanjut.

9. Mengecek *missing values* dan *duplicates*



```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Gambar 2.4.53 Cek Missing Value

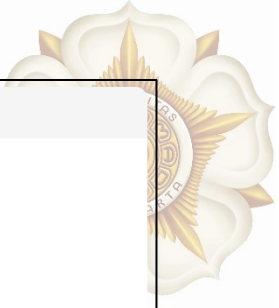
Pengecekan *missing values* di atas menunjukkan bahwa sebagian besar kolom tidak memiliki data kosong, namun terdapat 177 *missing* pada kolom *Age*, 687 *missing* pada *Cabin*, dan 2 *missing* pada *Embarked*. Kolom *Cabin*, *Age* dan *Embarked* perlu di-*impute* agar dataset tetap dapat digunakan.

```
# Duplicate check
df.duplicated().sum()
df.drop_duplicates(inplace=True)
```

Gambar 2.4.54 Cek Data Duplikat

Kode di atas digunakan untuk mengecek apakah terdapat baris duplikat dalam dataset. Jika ditemukan duplikasi, `df.drop_duplicates(inplace=True)` akan menghapus baris-baris tersebut.

10. Imputation



```
#imputation. kita isi nilai kosong
# kolom numerik
df['Age'].fillna(df['Age'].median(), inplace=True)
#df['Age'].fillna(df['Age'].mean(), inplace=True)

# kolom kategori
df['Cabin'].fillna(df['Cabin'].value_counts().index[0], inplace=True)
df['Embarked'].fillna(df['Embarked'].value_counts().index[0], inplace=True)
df.isnull().sum()
```

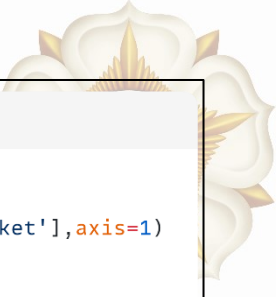
Gambar 2.4.55 Imputasi Missing Value

Kolom numerik seperti Age diisi menggunakan nilai median agar distribusinya tetap stabil, sedangkan kolom kategorikal seperti Cabin dan Embarked diisi menggunakan modus. Setelah dilakukan *imputation*, kita perlu memeriksa apakah masih ada *missing values* pada dataset.

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0
dtype:	int64

Gambar 2.4.56 Missing Value

11. Memeriksa *categorical attribute*



```
# get X and y
df_X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
df_y = df['Survived']

#check categorical attributes
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

Gambar 2.4.57 Memeriksa Atribut Kategorikal

Kode di atas memisahkan fitur (X) dan target (y) dengan menghapus kolom yang tidak diperlukan dari X, seperti PassengerId, Name, Survived, Cabin, dan Ticket. Selanjutnya, `df_X.select_dtypes(include=['object', 'bool']).columns` digunakan untuk mengecek atribut kategorikal dalam X. Hasilnya dapat dilihat pada Gambar Gambar 2.4.58.

```
Index(['Sex', 'Embarked'], dtype='object')
```

Gambar 2.4.58 Atribut Kategorikal

Output tersebut menunjukkan bahwa kolom kategorikal dalam fitur X adalah Sex dan Embarked, sehingga kedua kolom ini perlu dilakukan *encoding* sebelum *modelling*.

12. One hot encoding

```
# One Hot Encode
df_onehot = pd.get_dummies(df_X, columns=['Sex', 'Embarked'], drop_first=True)
df_onehot.head()
```

Gambar 2.4.59 One Hot Encoding

Kode di atas digunakan untuk *one-hot encoding* pada kolom kategorikal Sex dan Embarked menggunakan `pd.get_dummies()`. Parameter `drop_first=True` digunakan untuk menghindari *dummy trap* dengan menghapus salah satu kategori. Hasilnya, fitur kategorikal diubah menjadi kolom biner.



	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	True	False	True
1	1	38.0	1	0	71.2833	False	False	False
2	3	26.0	0	0	7.9250	False	False	True
3	1	35.0	1	0	53.1000	False	False	True
4	3	35.0	0	0	8.0500	True	False	True

Gambar 2.4.60 Hasil One Hot Encoding

```

# ohe = OneHotEncoder(sparse_output=False, drop='first')
ohe = OneHotEncoder(sparse_output=False)

ohe.fit(df_X[['Sex', 'Embarked']])

# for col in column_categorical:
df_ohe = ohe.transform(df_X[['Sex', 'Embarked']])

# ohe.categories_
df_ohe

```

Gambar 2.4.61 One Hot Encoding dengan OneHotEncoder

Kode di atas menggunakan OneHotEncoder dari sklearn untuk melakukan one-hot encoding pada kolom Sex dan Embarked. Encoder di-fit pada data kategorikal, lalu ditransformasi menjadi representasi numerik biner. Hasil **df_ohe** berisi matriks encoded yang siap digabungkan ke fitur model.

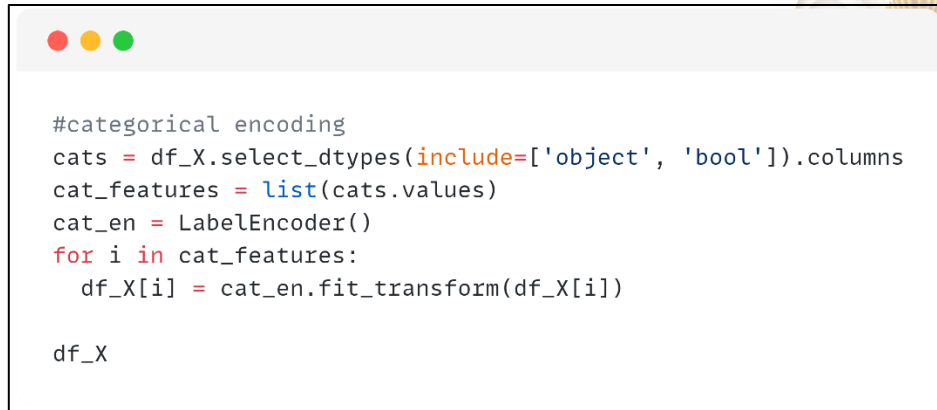
```

array([[0., 1., 0., 0., 1.],
       [1., 0., 1., 0., 0.],
       [1., 0., 0., 0., 1.],
       ...,
       [1., 0., 0., 0., 1.],
       [0., 1., 1., 0., 0.],
       [0., 1., 0., 1., 0.]])

```

Gambar 2.4.62 Hasil One Hot Encoding

13. Label encoding



```
#categorical encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
cat_en = LabelEncoder()
for i in cat_features:
    df_X[i] = cat_en.fit_transform(df_X[i])

df_X
```

Gambar 2.4.63 Label Encoding

Kode di atas digunakan untuk melakukan *label encoding* semua kolom kategorikal dalam X. Setiap kolom diubah menjadi nilai numerik menggunakan LabelEncoder, sehingga fitur kategorikal dapat digunakan oleh algoritma *machine learning* yang hanya menerima *input* numerik.

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	1	22.0	1	0	7.2500	2
1	1	0	38.0	1	0	71.2833	0
2	3	0	26.0	0	0	7.9250	2
3	1	0	35.0	1	0	53.1000	2
4	3	1	35.0	0	0	8.0500	2
...	...	...	...	...	...	...	...
886	2	1	27.0	0	0	13.0000	2
887	1	0	19.0	0	0	30.0000	2
888	3	0	28.0	1	2	23.4500	2
889	1	1	26.0	0	0	30.0000	0
890	3	1	32.0	0	0	7.7500	1

891 rows × 7 columns

Gambar 2.4.64 Hasil Label Encoding

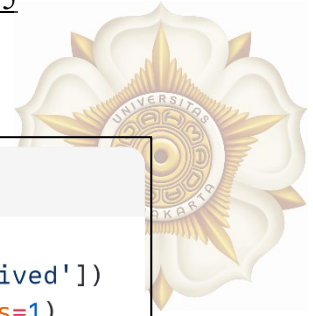
```
#label encoding for y
le = LabelEncoder()
le.fit(df_y)
df_y= le.fit_transform(df_y)
df_y
```

Gambar 2.4.65 Label Encoding untuk y

Kode di atas melakukan *label encoding* pada target y, mengubah nilai kategorikal seperti 0/1 atau label teks menjadi angka yang dapat digunakan oleh model *machine learning*.

[illegible]

Gambar 2.4.66 Hasil Label Encoding



14. Mengecek korelasi atribut

```
df_y_new = pd.DataFrame(df_y, columns=['Survived'])
df_gabung = pd.concat([df_X, df_y_new], axis=1)
df_gabung.corr()
```

Gambar 2.4.67 Cek Korelasi Atribut

Kode di atas menggabungkan kembali fitur (X) dan target (y) menjadi satu *DataFrame* lalu menghitung korelasi antar semua variabel untuk melihat hubungan fitur terhadap *Survived*.

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
Pclass	1.000000	0.131900	-0.339898	0.083081	0.018443	-0.549500	0.162098	-0.338481
Sex	0.131900	1.000000	0.081163	-0.114631	-0.245489	-0.182333	0.108262	-0.543351
Age	-0.339898	0.081163	1.000000	-0.233296	-0.172482	0.096688	-0.018754	-0.064910
SibSp	0.083081	-0.114631	-0.233296	1.000000	0.414838	0.159651	0.068230	-0.035322
Parch	0.018443	-0.245489	-0.172482	0.414838	1.000000	0.216225	0.039798	0.081629
Fare	-0.549500	-0.182333	0.096688	0.159651	0.216225	1.000000	-0.224719	0.257307
Embarked	0.162098	0.108262	-0.018754	0.068230	0.039798	-0.224719	1.000000	-0.167675
Survived	-0.338481	-0.543351	-0.064910	-0.035322	0.081629	0.257307	-0.167675	1.000000

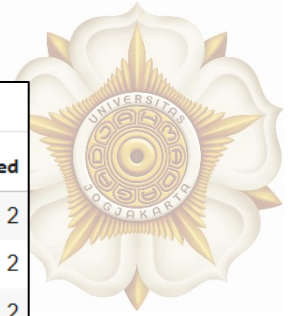
Gambar 2.4.68 Korelasi Antar Variabel

15. Membagi dataset

```
#hold out, dibagi menjadi training dan testing set
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.3, random_state=42)
X_train
```

Gambar 2.4.69 Pembagian Dataset

Kode di atas membagi dataset menjadi *training set* (70%) dan *testing set* (30%) menggunakan *train_test_split*. *Training* dipakai untuk melatih model, sedangkan *testing* untuk mengevaluasi performanya agar tidak *overfitting*.



X_train								
	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	
445	1	1	4.0	0	2	81.8583	2	
650	3	1	28.0	0	0	7.8958	2	
172	3	0	1.0	1	1	11.1333	2	
450	2	1	36.0	1	2	27.7500	2	
314	2	1	43.0	1	1	26.2500	2	
...	...	...	...	...	...	...	...	...
106	3	0	21.0	0	0	7.6500	2	
270	1	1	28.0	0	0	31.0000	2	
860	3	1	41.0	2	0	14.1083	2	
435	1	0	14.0	1	2	120.0000	2	
102	1	1	21.0	0	1	77.2875	2	
623 rows × 7 columns								

Gambar 2.4.70 Sampel Data Training

16. Scaling

```
#scaling
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train
```

Gambar 2.4.71 Scaling dengan StandardScaler

Kode di atas merupakan kode untuk melakukan *standardization* menggunakan StandardScaler. Hasilnya dapat dilihat pada Gambar 2.4.72.



```
X_train
array([[ -1.63788124,  0.72077194, -1.91971935, ...,  1.99885349,
         0.98099823,  0.57000481],
       [ 0.80326712,  0.72077194, -0.0772525 , ..., -0.47932706,
        -0.46963364,  0.57000481],
       [ 0.80326712, -1.38740139, -2.15002771, ...,  0.75976322,
        -0.40613632,  0.57000481],
       ...,
       [ 0.80326712,  0.72077194,  0.92075038, ..., -0.47932706,
        -0.34778742,  0.57000481],
       [-1.63788124, -1.38740139, -1.15202483, ...,  1.99885349,
        1.72907416,  0.57000481],
       [-1.63788124,  0.72077194, -0.61463866, ...,  0.75976322,
        0.8913508 ,  0.57000481]])
```

Gambar 2.4.72 Sampel Data Training Setelah Scaling

```
#scaling
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train
```

Gambar 2.4.73 Scaling dengan MinMaxScaler

Kode di atas merupakan kode *normalization* menggunakan MinMaxScaler. Hasilnya dapat dilihat pada Gambar 2.4.74.

```
array([[0.      , 1.      , 0.04498618, ..., 0.33333333, 0.15977676,
        1.      ],
       [1.      , 1.      , 0.34656949, ..., 0.      , 0.01541158,
        1.      ],
       [1.      , 0.      , 0.00728826, ..., 0.16666667, 0.02173075,
        1.      ],
       ...,
       [1.      , 1.      , 0.50992712, ..., 0.      , 0.02753757,
        1.      ],
       [0.      , 0.      , 0.17064589, ..., 0.33333333, 0.2342244 ,
        1.      ],
       [0.      , 1.      , 0.25860769, ..., 0.16666667, 0.15085515,
        1.      ]])
```

Gambar 2.4.74 Sampel Data Training Setelah Scaling

2.5. TUGAS & ANALISIS

- **Exploratory Data Analysis**

Dataset: <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>



- 1) Silahkan membaca dataset diatas menggunakan *dataframe*!
- 2) Jelaskan apa tujuan dari penggunaan dataset ini!
- 3) Definisikan atribut mana yang menjadi input dan atribut mana yang menjadi output/class/label!
- 4) Berikan penjelasan singkat untuk setiap atribut/*variable* dari dataset tersebut

- **Data Preprocessing**

Dataset: <https://www.kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data>

- 1) Silahkan membaca dataset diatas menggunakan *dataframe*!
- 2) Buatlah visualisasi histogram untuk beberapa variabel numerik, lalu jelaskan pola distribusi yang terlihat!
- 3) Hitung dan visualisasikan korelasi antar variabel, kemudian jelaskan hubungan yang ditemukan!
- 4) Lakukan *exploratory data analysis* (EDA) dengan membuat *descriptive statistics* serta melakukan pengecekan *missing value* dan duplikasi. Jelaskan tindakan apa yang perlu dilakukan jika keduanya ditemukan!
- 5) Lakukan *preprocessing* lanjutan berupa *encoding* variabel kategorikal (*one-hot & label encoding*), melakukan *train-test split* (80:20), serta menerapkan *standardization* atau *normalization* pada fitur numerik. Jelaskan alasan penggunaan masing-masing teknik!

2.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.



PERTEMUAN III

Classification

3.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar beberapa algoritma klasifikasi, seperti *Logistic Regression*, KNN, *Decision Tree*, dan *Random Forest*
- B. Mahasiswa mengetahui fungsi dan peran metrik evaluasi klasifikasi, seperti *accuracy*, *precision*, dan *recall* dalam menilai performa model
- C. Mahasiswa dapat menerapkan berbagai algoritma klasifikasi untuk membangun model
- D. Mahasiswa mampu melakukan evaluasi model dengan menyusun tabel performa berisi *accuracy*, *precision*, dan *recall* dari setiap model

3.2. DASAR TEORI

- **Classification**

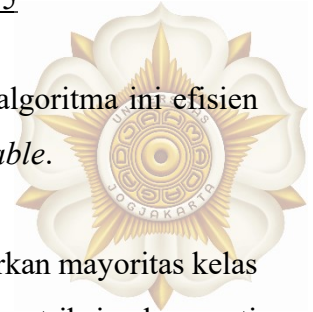
Klasifikasi merupakan salah satu teknik *supervised learning* yang bertujuan untuk memprediksi label atau kategori dari sebuah sampel berdasarkan fitur yang dimiliki. Model klasifikasi belajar dari data berlabel, kemudian menghasilkan suatu fungsi yang mampu menentukan kelas dari data baru yang belum pernah dilihat. Klasifikasi bekerja dengan cara mencari pola dari data historis untuk menentukan keputusan kelas yang paling mendekati. Klasifikasi umum digunakan dalam aplikasi seperti deteksi spam, diagnosis medis, hingga rekomendasi konten.

- **Algoritma Classification**

Ada banyak metode yang bisa dipakai untuk melakukan klasifikasi. Beberapa algoritma di bawah ini termasuk yang paling sering digunakan dan dijelaskan dalam dokumentasi Scikit-Learn dan TensorFlow karena sederhana, efektif, dan cocok untuk berbagai jenis dataset.

1. *Logistic Regression*

Logistic Regression merupakan algoritma klasifikasi berbasis regresi linier yang memodelkan probabilitas suatu sampel termasuk ke dalam kelas tertentu menggunakan fungsi *sigmoid*. *Output* model berada dalam rentang 0–1, yang



kemudian dikonversi menjadi label kelas. Meskipun sederhana, algoritma ini efisien dan bekerja baik pada data yang memiliki hubungan linear *separable*.

2. *K-Nearest Neighbors (KNN)*

K-Nearest Neighbors mengklasifikasikan suatu data baru berdasarkan mayoritas kelas dari k tetangga terdekatnya. Kedekatan dihitung menggunakan metrik jarak seperti *Euclidean distance*. Algoritma ini bersifat *non-parametric* dan sangat bergantung pada struktur distribusi data serta nilai k yang dipilih.

3. *Decision Tree*

Decision Tree bekerja dengan cara membagi dataset secara rekursif berdasarkan fitur yang memberikan informasi terbaik. Proses pemilihan fitur dilakukan menggunakan ukuran seperti *Gini impurity* atau *entropy/information gain*. Kelebihannya adalah interpretasi yang mudah, tetapi mudah mengalami *overfitting* tanpa pengaturan parameter yang tepat

4. *Random Forest*

Random Forest adalah algoritma *ensemble* yang menggabungkan banyak *Decision Tree* melalui teknik *bagging*. Setiap pohon dilatih pada subset data yang berbeda, dan hasil akhirnya ditentukan melalui voting. *Random Forest* membuat model lebih stabil, mengurangi *overfitting*, dan meningkatkan akurasi dibanding satu pohon tunggal.

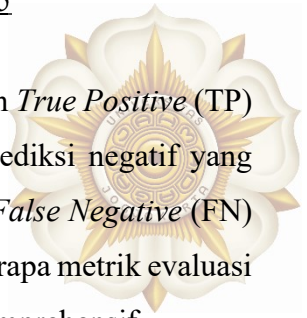
5. *Adaboost (Adaptive Boosting)*

AdaBoost bekerja dengan menggabungkan beberapa *weak learner* (biasanya *Decision Tree* kecil) secara berurutan. Pada setiap iterasi, bobot sampel diperbarui sehingga model berikutnya lebih fokus pada data yang sebelumnya salah diklasifikasikan. Dengan cara ini, *AdaBoost* mampu meningkatkan performa model secara signifikan terutama pada dataset yang kompleks.

• **Metrik Evaluasi Classification**

Dalam *machine learning* khususnya pada model klasifikasi, diperlukan metrik evaluasi untuk menilai seberapa baik model mampu memprediksi label dengan benar. Evaluasi ini dilakukan dengan membandingkan hasil prediksi model terhadap nilai aktual melalui sebuah struktur yang disebut *confusion matrix*.

1. *Confusion Matrix*



Confusion matrix terdiri dari empat komponen penting, antara lain *True Positive* (TP) yaitu prediksi positif yang benar, *True Negative* (TN) yaitu prediksi negatif yang benar, *False Positive* (FP) yaitu prediksi positif yang salah, dan *False Negative* (FN) yaitu prediksi negatif yang salah. Berdasarkan nilai-nilai ini, beberapa metrik evaluasi dapat dihitung untuk memahami performa model secara lebih komprehensif.

2. *Accuracy, Precision, Recall, F1 Score*

Accuracy merupakan metrik paling dasar yang menunjukkan proporsi prediksi yang benar dari seluruh prediksi yang dibuat oleh model. Rumusnya didefinisikan sebagai berikut:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision mengukur tingkat ketepatan model dalam melakukan prediksi positif, yaitu berapa banyak prediksi positif yang benar-benar positif. *Precision* dihitung dengan rumus berikut:

$$Precision = \frac{TP}{TP + FP}$$

Sementara itu, *Recall* mengukur kemampuan model dalam menemukan seluruh sampel yang benar-benar positif. *Recall* dihitung menggunakan rumus:

$$Recall = \frac{TP}{TP + FN}$$

Karena *precision* dan *recall* memiliki fokus yang berbeda, F1-Score digunakan sebagai metrik yang menyeimbangkan keduanya. Metrik ini sangat berguna ketika dataset bersifat *imbalanced* dan diperlukan keseimbangan antara kemampuan model menghindari *false positives* serta *false negatives*. F1-score merupakan *harmonic mean* dari *precision* dan *recall*, dengan rumus:

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**



- Python3
- Google Colaboratory

3.4. LANGKAH PERCOBAAN

[Isi dengan instruksi langkah demi langkah yang jelas disertai penjelasan singkat.

Tambahkan gambar ilustrasi jika diperlukan]

1. Impor Pustaka

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from imblearn.metrics import sensitivity_specificity_support
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
```

Gambar 3.4.1 Import Pustaka

2. Load dataset

Dataset dapat diunduh pada link berikut:

<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

```
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/refs/heads/main/healthcare-dataset-stroke-data.csv')
```

3. Exploratory Data Analysis (EDA)

a. Melihat lima baris pertama dan terakhir pada dataset dengan df.head() dan df.tail().

df.head()												
	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 3.4.2 Lima Baris Pertama Dataset

```
df.tail()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

Gambar 3.4.3 Lima Baris Terakhir Dataset

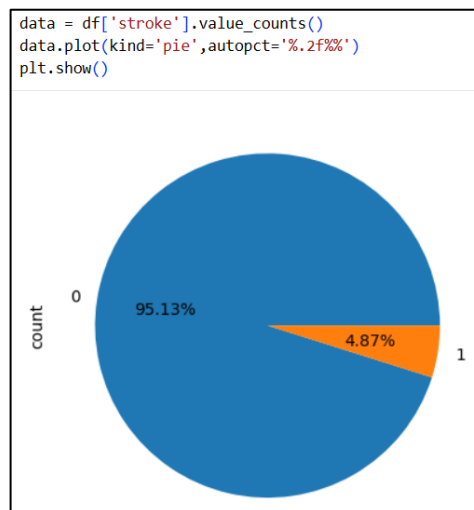
b. Melihat deskripsi statistik dengan df.describe()

```
df.describe()
```

	id	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	4909.000000	5110.000000
mean	36517.829354	43.226614	0.097456	0.054012	106.147677	28.893237	0.048728
std	21161.721625	22.612647	0.296607	0.226063	45.283560	7.854067	0.215320
min	67.000000	0.080000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	17741.250000	25.000000	0.000000	0.000000	77.245000	23.500000	0.000000
50%	36932.000000	45.000000	0.000000	0.000000	91.885000	28.100000	0.000000
75%	54682.000000	61.000000	0.000000	0.000000	114.090000	33.100000	0.000000
max	72940.000000	82.000000	1.000000	1.000000	271.740000	97.600000	1.000000

Gambar 3.4.4 Hasil Deskripsi Statistik Dataset

c. Melihat presentase target kelas



Gambar 3.4.5 Presentase Target Kelas

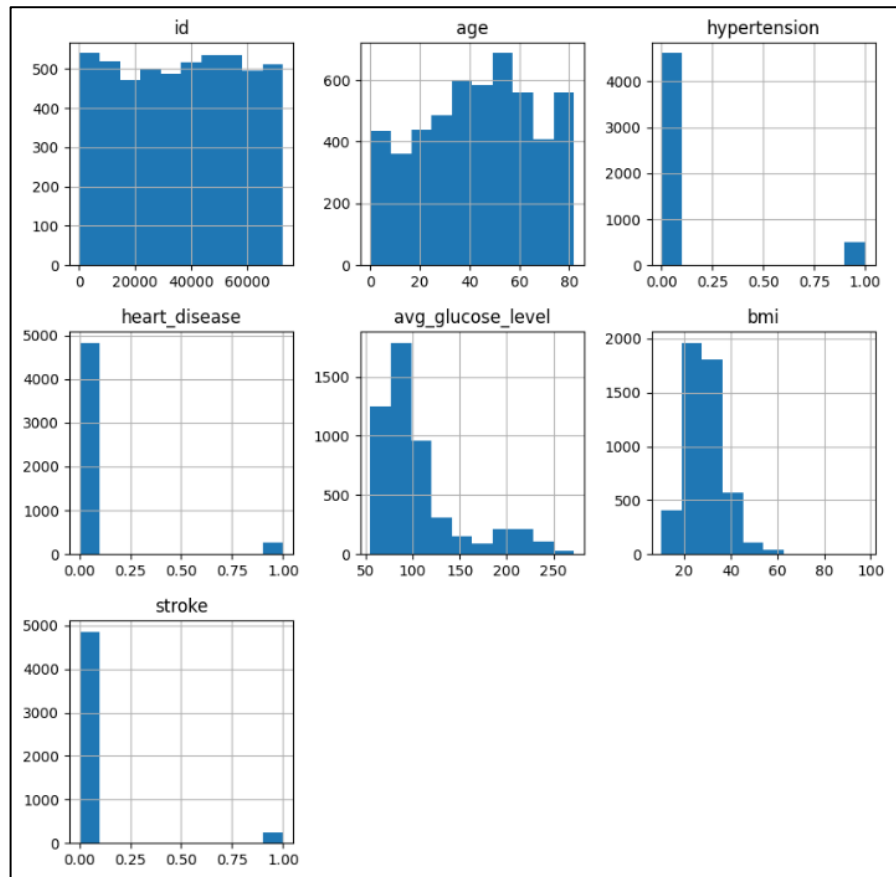
Pie chart tersebut menunjukkan distribusi variabel stroke, di mana kelas tidak mengalami stroke (0) mendominasi dengan proporsi sekitar 95.13%, sedangkan kelas mengalami stroke (1) hanya sebesar 4.87%. Hal ini menandakan bahwa dataset bersifat

imbalanced, karena jumlah sampel pada kelas positif jauh lebih sedikit dibanding kelas negatif.

d. Melihat histogram

```
df.hist(figsize=(10,10))
plt.show()
```

Gambar 3.4.6 Kode untuk Menampilkan Histogram



Gambar 3.4.7 Histogram Seluruh Variabel Numerik

e. Mengecek *missing value* dengan `df.isnull().sum()`



df.isnull().sum()	
	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	201
smoking_status	0
stroke	0
dtype: int64	

Gambar 3.4.8 Pengecekan Missing Value

Hasil pengecekan *missing value* di atas menunjukkan bahwa hanya variabel *bmi* memiliki nilai kosong sebanyak 201 baris, sementara kolom lainnya tidak memiliki *missing value*.

f. Mengecek atribut kategorikal

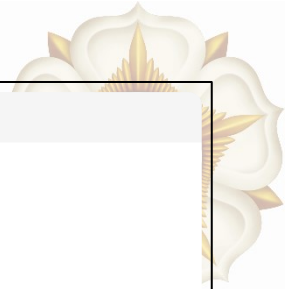
```
df_x = df.drop(['id', 'stroke'],axis=1)
# definisikan kolom yg jadi input
df_y = df[['stroke']]
# definisikan kolom yg jadi output
cats = df_x.select_dtypes(include=['object', 'bool']).columns
print(cats)

Index(['gender', 'ever_married', 'work_type', 'Residence_type',
       'smoking_status'],
      dtype='object')
```

Gambar 3.4.9 Pengecekan Atribut Kategorikal

Hasil pengecekan atribut kategorikal menunjukkan bahwa variabel kategorikal yang perlu dilakukan *encoding* yaitu *gender*, *ever_married*, *work_type*, *Residence_type*, dan *smoking_status*.

4. Data Preprocessing



```
# data preprocessing dimulai
# membuat X and y. X untuk input variable, y untuk target class
df_X = df.drop(['id', 'stroke'],axis=1)
df_y = df[['stroke']]

#label encoding for y.
#merubah nilai yg ada di y menjadi 0 atau 1.
#sebenarnya ini tidak diperlukan karena nilai y di dataset sudah 0 atau 1
le = LabelEncoder()
df_y= le.fit_transform(df_y['stroke'])

#imputation
df_X['bmi'].fillna(df_X['bmi'].median(), inplace=True)

#categorical encoding
#merubah categorical value menjadi numerical value
#bisa pakai label encoding, ordinal atau one hot encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    df_X[i] = le.fit_transform(df_X[i])

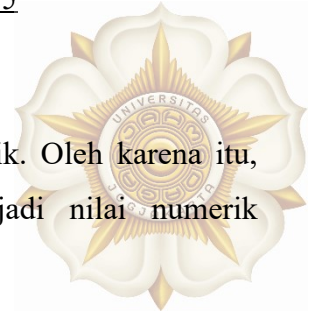
#menyimpan X dan y menjadi numpy arrays
X = df_X.astype(float).values
y = df_y.astype(float)

#hold-out method
#dibagi menjadi training dan testing set
#70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#scaling
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Gambar 3.4.10 Kode untuk Melakukan Data Preprocessing

- a. Membuat variable `df_X` untuk input variable dan `df_y` untuk target class
Pada tahap awal, dataset dipisahkan menjadi dua, yaitu variabel fitur dan variabel target. Fitur (X) merupakan atribut independen yang digunakan sebagai inputan model, sedangkan target (y) adalah variabel dependen yang ingin diprediksi.
- b. *Label encoding* untuk y
Proses *label encoding* dilakukan untuk memastikan variabel y berada dalam format numerik.
- c. *Imputation* dengan nilai tengah (median)
Nilai hilang pada fitur numerik diatasi menggunakan metode pengisian median, karena median lebih stabil terhadap keberadaan *outlier*.



d. *Categorical encoding*

Model *machine learning* hanya dapat memproses data numerik. Oleh karena itu, semua fitur bertipe *object* atau *boolean* dikonversi menjadi nilai numerik menggunakan Label Encoding.

e. Menyimpan x dan y menjadi *numpy arrays*

Agar kompatibel dengan fungsi-fungsi pada pustaka Scikit-Learn, variabel fitur dan target dikonversi menjadi array NumPy bertipe float.

f. Membagi data *training* dan *testing*

Dataset dibagi menjadi data *training* dan data *testing* dengan komposisi 70% data *training* dan 30% data *testing*. Pembagian ini dilakukan agar model dapat dievaluasi menggunakan data yang tidak pernah digunakan selama proses pelatihan.

g. Normalisasi data (*standardization*)

Standardisasi dilakukan menggunakan StandardScaler, yang mengubah distribusi setiap fitur sehingga memiliki nilai rata-rata 0 dan standar deviasi 1. Proses fit hanya dilakukan pada data *training* untuk mencegah data *leakage*, kemudian digunakan untuk mentransformasi data *training* dan data *testing*.

Tahapan *preprocessing* di atas memastikan bahwa dataset berada dalam kondisi optimal sebelum digunakan untuk membangun model klasifikasi. Langkah berikutnya, dilakukan proses pemeriksaan hasil *preprocessing* untuk memastikan seluruh transformasi sudah sesuai.

a. Variabel input X

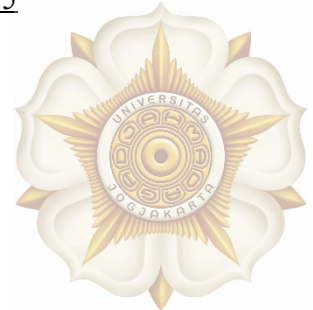
Seluruh nilai kategorik telah dikonversi menjadi numerik.

x									
array([[1. , 67. , 0. , ..., 228.69, 36.6 , 1.],									
[0. , 61. , 0. , ..., 202.21, 28.1 , 2.],									
[1. , 80. , 0. , ..., 105.92, 32.5 , 2.],									
...,									
[0. , 35. , 0. , ..., 82.99, 30.6 , 2.],									
[1. , 51. , 0. , ..., 166.29, 25.6 , 1.],									
[0. , 44. , 0. , ..., 85.28, 26.2 , 0.]])									

Gambar 3.4.11 Variabel X Setelah Dikonversi Menjadi Numerik

b. Variabel target y

Variabel target atau label berupa nilai biner (0 dan 1) hasil dari proses *label encoding*.



```
y
array([1., 1., 1., ..., 0., 0., 0.])
```

Gambar 3.4.12 Variabel *y* Setelah Proses Label Encodingc. Data *training* yang telah dinormalisasi (X_train)

X_train menampilkan nilai fitur yang telah distandardisasi, sehingga setiap kolom memiliki rata-rata 0 dan standar deviasi 1.

```
X_train
array([[ 1.18418048, -1.7467638, -0.31719928, ..., -0.340693,
        -1.64591683, -1.29622579],
       [ 1.18418048, -0.63635252, -0.31719928, ...,  2.26654137,
        -0.78843307,  1.52066342],
       [ 1.18418048,  0.02989425,  3.15259225, ..., -0.32155489,
        -0.30772248,  0.58170035],
       ...,
       [-0.84446587, -1.87290652, -0.31719928, ..., -0.18803315,
        -1.43804198, -1.29622579],
       [ 1.18418048,  1.62888649, -0.31719928, ...,  2.01062472,
        0.27692553, -0.35726272],
       [-0.84446587,  0.11872715, -0.31719928, ..., -0.12416526,
        2.78441591,  1.52066342]])
```

Gambar 3.4.13 Hasil Transformasi pada Data Training

d. Data testing yang telah dinormalisasi (X_test)

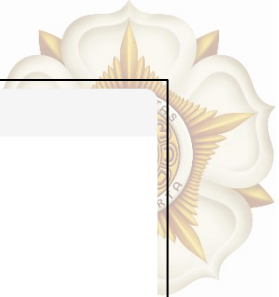
X_test menunjukkan hasil transformasi fitur pada data testing menggunakan *scaler* yang sama seperti data *training*, sehingga menghasilkan distribusi nilai yang serupa dengan angka-angka pada skala terstandarisasi.

```
X_test
array([[ 1.18418048, -0.54751962, -0.31719928, ..., -0.90971812,
        -0.76244872, -1.29622579],
       [ 1.18418048, -0.14777156, -0.31719928, ..., -0.89992653,
        -0.07386327,  0.58170035],
       [-0.84446587, -1.569098, -0.31719928, ..., -0.69675096,
        -0.82740961, -1.29622579],
       ...,
       [ 1.18418048, -0.05893866, -0.31719928, ..., -0.26569829,
        -0.21677723,  0.58170035],
       [-0.84446587,  0.60730811, -0.31719928, ..., -0.80846414,
        -0.63252693, -1.29622579],
       [-0.84446587,  0.74055746, -0.31719928, ..., -0.72746095,
        -0.46362862,  0.58170035]])
```

Gambar 3.4.14 Hasil Transformasi pada Data Testing

5. Modelling

a. Logistic Regression



```

model=LogisticRegression()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

# y_pred jika di print out keluar vector prediksi nya
#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

print('F1 ',f1_score(y_test, y_pred, average='macro'))

```

Gambar 3.4.15 Kode untuk Logistic Regression

- Melatih model

Pada tahap ini, objek **LogisticRegression()** dibuat sebagai model klasifikasi. Model kemudian dilatih menggunakan data *training* (*X_train* dan *y_train*) agar dapat mempelajari hubungan antara fitur dan label target.

- Membuat prediksi

Model yang telah dilatih digunakan untuk memprediksi kelas dari data testing (*X_test*). Hasil prediksi disimpan pada variabel *y_pred*, yang berupa vektor berisi nilai kelas hasil prediksi.

- Mengukur performa model

Performa model diukur dengan metrik *accuracy*, *precision*, *recall*, dan *confusion matrix*. Penggunaan parameter **average='macro'** memastikan bahwa setiap kelas memiliki kontribusi yang sama dalam perhitungan *precision* dan *recall*.

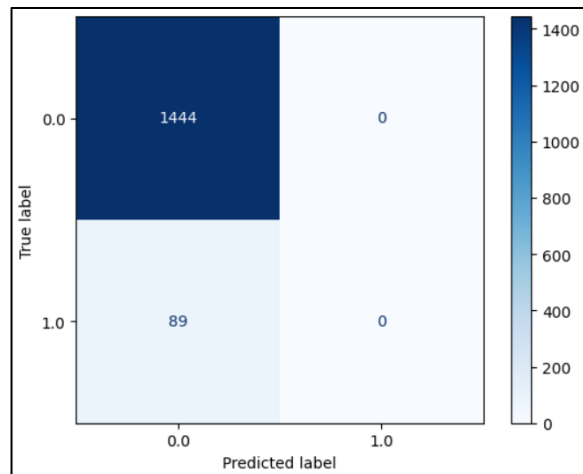
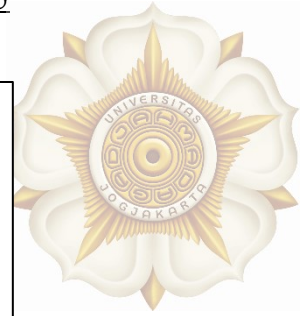
```

Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]

```

Gambar 3.4.16 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.17 Confusion Matrix

Confusion matrix divisualisasikan menggunakan warna (heatmap) untuk memudahkan interpretasi. Visualisasi ini membantu melihat pola kesalahan model, misalnya apakah model cenderung salah pada kelas tertentu.

- Menghitung *F1 score*

F1-score merupakan kombinasi harmonis dari *precision* dan *recall*, sehingga memberikan gambaran performa yang lebih seimbang terutama pada dataset dengan distribusi kelas yang tidak merata.

F1 0.485052065838092

Gambar 3.4.18 Output F1 Score

- Model *coefficient*

```
model.coef_
array([[ -0.01878829,  1.55249921,  0.10370412,  0.07921647, -0.18371621,
        -0.06959651,  0.06033662,  0.19742689, -0.03156246,  0.02218138]])
```

Gambar 3.4.19 Pengecekan Nilai Koefisien dari Model Logistic Rgression

Output `model.coef_` menunjukkan nilai koefisien yang diperoleh model *Logistic Regression* untuk setiap fitur setelah proses pelatihan. Nilai positif menunjukkan bahwa fitur tersebut cenderung meningkatkan peluang prediksi kelas 1, sedangkan nilai negatif menurunkan peluang tersebut

- Model *intercept*



```
model.intercept_  
array([-4.04531553])
```

Gambar 3.4.20 Pengecekan Nilai Konstanta pada Model Logistic Regression

Output `model.intercept_` merupakan nilai konstanta dalam fungsi keputusan model. *Intercept* bernilai *negative* menunjukkan bahwa, ketika seluruh fitur bernilai nol (setelah melalui proses normalisasi), model memiliki kecenderungan awal untuk memprediksi kelas 0.

b. *K-Nearest Neighbors*



```
model=KNeighborsClassifier(n_neighbors=10)  
model.fit(X_train, y_train)  
  
# membuat prediksi  
y_pred = model.predict(X_test)  
  
#menghitung performa model, dengan accuracy dll  
print('Accuracy ',accuracy_score(y_test, y_pred))  
print('Precision ',precision_score(y_test, y_pred, average='macro'))  
print('Recall ',recall_score(y_test, y_pred, average='macro'))  
print('Confusion matrix ', confusion_matrix(y_test, y_pred))  
  
# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)  
cm = confusion_matrix(y_test, y_pred)  
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))  
  
# plt.show()  
disp.plot(cmap=plt.cm.Blues)
```

Gambar 3.4.21 Kode untuk Melatih Model KNN

- Melatih model

Model KNN dibentuk dengan menetapkan jumlah tetangga (*k*) sebanyak 10, yang berarti keputusan klasifikasi akan ditentukan berdasarkan 10 sampel terdekat pada fitur. Data training (*X_train* dan *y_train*) digunakan untuk menyimpan representasi data sehingga model dapat menghitung jarak antar sampel pada tahap prediksi.

- Membuat prediksi

Model kemudian digunakan untuk memprediksi kelas dari data testing (*X_test*). Proses prediksi dilakukan dengan menghitung jarak antara sampel testing dan seluruh sampel *training*, kemudian memilih kelas mayoritas dari 10 tetangga terdekat.

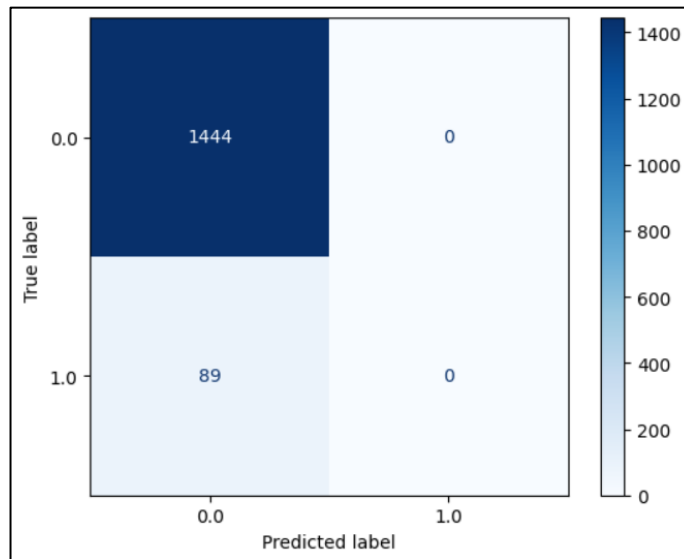


- Mengukur performa model

```
Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]
```

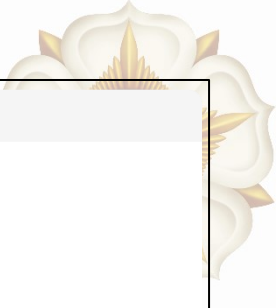
Gambar 3.4.22 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.23 Confusion Matrix

c. *Decision Tree*



```

model=DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.24 Kode untuk Melatih Model Decision Tree

- Melatih model

Model *Decision Tree* dibentuk menggunakan kriteria pemilihan *split entropy*, yang mengukur tingkat ketidakaturan data pada setiap *node*. Semakin rendah nilai *entropy*, semakin baik kualitas pemisahan yang dilakukan. Data *training* (X_{train} dan y_{train}) digunakan untuk membangun struktur pohon keputusan yang optimal berdasarkan pola data.

- Membuat prediksi

Model yang telah terbentuk kemudian digunakan untuk memprediksi kelas dari data testing (X_{test}). Prediksi dilakukan dengan menelusuri jalur pada pohon keputusan, mulai dari *root* hingga mencapai *node* daun yang menentukan kelas akhir.

- Mengukur performa model

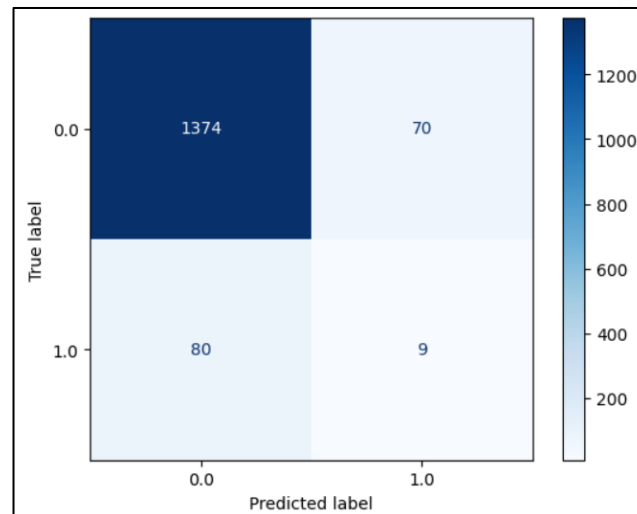
```

Accuracy 0.9021526418786693
Precision 0.5294517089478175
Recall 0.5263235706059946
Confusion matrix [[1374  70]
 [ 80   9]]

```

Gambar 3.4.25 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.26 Confusion Matrix

d. Random Forest

```

model=RandomForestClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

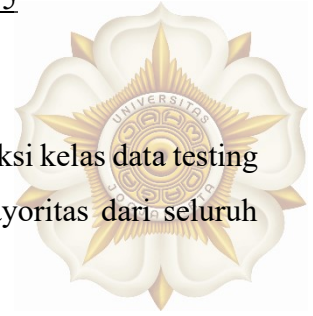
# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.27 Kode untuk Melatih Model Random Forest

- Melatih model

Model *Random Forest* dibentuk dengan menggunakan parameter *default*, yang secara otomatis membuat sejumlah pohon keputusan dan melatihnya pada subset acak dari dataset. Algoritma ini bekerja berdasarkan prinsip *bagging*, yaitu menggabungkan hasil prediksi dari banyak pohon untuk mengurangi varians dan meningkatkan akurasi. Data *training* (X_{train} dan y_{train}) digunakan untuk membangun *forest* tersebut.



- Membuat prediksi

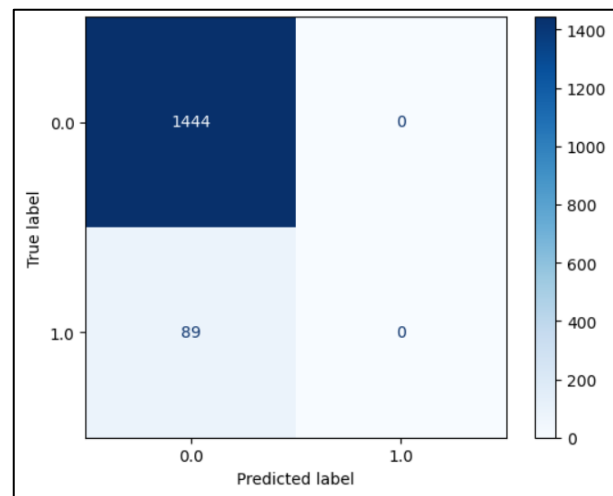
Model yang telah dilatih kemudian digunakan untuk memprediksi kelas data testing (X_{test}). Prediksi akhir ditentukan melalui proses voting mayoritas dari seluruh pohon yang ada dalam *ensemble*.

- Mengukur performa model

```
Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]
```

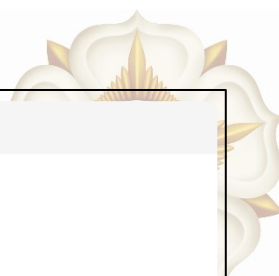
Gambar 3.4.28 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.29 Confusion Matrix

e. Ada Boost



```

model=AdaBoostClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.30 Kode untuk Melatih Model Ada Boost

- Melatih model

Model *AdaBoost* dibentuk menggunakan parameter *default*, di mana algoritma akan melatih beberapa *weak learners* (biasanya *decision tree* berukuran kecil) secara berurutan. Setiap *weak learner* diberikan bobot berdasarkan tingkat kesalahannya, sehingga model berikutnya lebih fokus pada sampel-sampel yang sulit diprediksi. Proses ini meningkatkan akurasi keseluruhan model. Data latih (*X_train* dan *y_train*) digunakan sebagai dasar pembelajaran.

- Melakukan prediksi

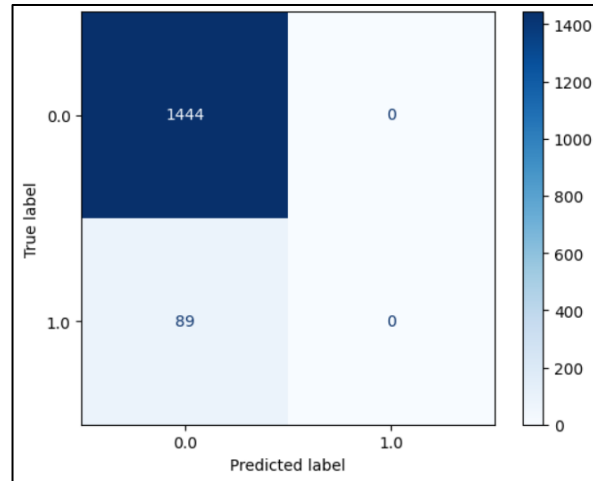
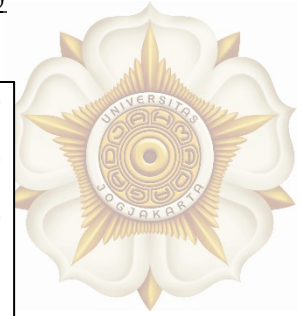
Setelah proses pelatihan selesai, model digunakan untuk memprediksi kelas data testing (*X_test*). Prediksi akhir merupakan kombinasi berbobot dari seluruh *weak learners* yang telah dilatih sebelumnya.

- Mengukur performa model

Accuracy	0.9419439008480104
Precision	0.4709719504240052
Recall	0.5
Confusion matrix	[[1444 0]
	[89 0]]

Gambar 3.4.31 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.32 Confusion Matrix

3.5. TUGAS & ANALISIS

Dataset: <https://www.kaggle.com/datasets/bhavikjikadara/loan-status-prediction>

1. Jelaskan apa tujuan penggunaan dataset ini?
2. Definisikan atribut yang menjadi input dan output nya?
3. Silahkan membuat model klasifikasi *logistic regression*, KNN, *Decision Tree*, dan *Random Forest* untuk kasus dataset diatas!
4. Buatlah tabel yang menjelaskan performa dari model *machine learning* untuk kasus dataset diatas! Kolom pertama "model", kolom selanjutnya "*accuracy, precision, recall*"
5. Jelaskan dari hasil eksperimen di atas, model mana yang paling baik? Jelaskan alasan anda!

3.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.



PERTEMUAN IV

Regression

4.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar regresi
- B. Mahasiswa mengetahui prinsip kerja berbagai algoritma regresi, khususnya *Linear Regression*, *Decision Tree Regression*, dan *Random Forest Regression*
- C. Mahasiswa dapat menerapkan berbagai algoritma regresi untuk membangun model
- D. Mahasiswa mampu mengevaluasi performa model regresi menggunakan metrik RMSE, R^2 , dan koefisien korelasi (R)

4.2. DASAR TEORI

- **Regresi**

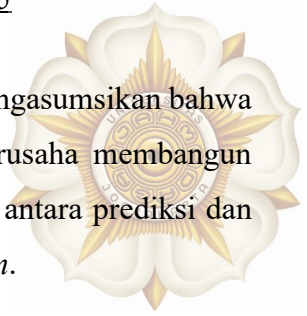
Regresi adalah metode statistik dalam *machine learning* yang digunakan untuk memodelkan hubungan antara satu atau lebih variabel independen (fitur) dengan variabel dependen (target) yang bersifat numerik atau kontinu. Tujuan utama regresi adalah untuk memahami pola hubungan antara fitur dan target, memperkirakan nilai masa depan berdasarkan data sebelumnya, dan mengidentifikasi seberapa besar pengaruh masing-masing fitur terhadap *output*.

Dalam *machine learning*, regresi termasuk dalam *supervised learning*, karena model dilatih menggunakan pasangan *input-output* sehingga bisa belajar memprediksi nilai target yang belum diketahui. Contoh penggunaan regresi antara lain yaitu prediksi harga rumah, estimasi permintaan barang, prediksi curah hujan, dan evaluasi risiko. Regresi bekerja dengan cara menemukan fungsi atau model matematis yang paling sesuai dengan pola data. Model tersebut kemudian digunakan untuk memprediksi nilai baru berdasarkan *input* baru.

- **Algoritma Regresi**

Algoritma regresi yang umum digunakan adalah *Linear Regression*, *Decision Tree Regression*, dan *Random Forest Regression*. Masing-masing algoritma memiliki karakteristik dan cara kerja yang berbeda sesuai jenis data dan pola hubungan antar variabel.

1. *Linear Regression*



Linear Regression adalah algoritma regresi paling dasar yang mengasumsikan bahwa hubungan antara fitur dan target bersifat linier. Model ini berusaha membangun sebuah garis atau *hyperplane* terbaik yang meminimalkan *error* antara prediksi dan nilai sebenarnya. Berikut merupakan persamaan *linear regression*.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Dimana:

b_0 = intercept (nilai dasar)

b_n = koefisien regresi

x_n = nilai fitur

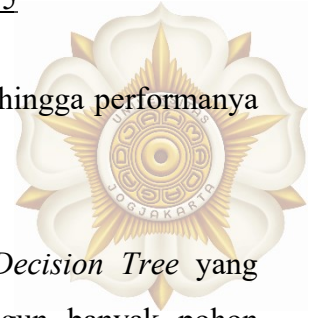
y = target/prediksi

Model *linear regression* mencari koefisien terbaik menggunakan teknik *Least Squares*, yaitu metode yang meminimalkan jumlah kuadrat *error*. Kelebihan *linear regression* yaitu sangat cepat dan efisien untuk dataset besar, mudah dipahami dan dijelaskan (*interpretable*), cocok untuk hubungan yang mendekati linear. Sedangkan kekurangannya adalah kurang cocok untuk pola non-linear dan sensitif terhadap *outlier*.

2. *Decision Tree Regression*

Decision Tree Regression merupakan salah satu algoritma regresi yang bekerja dengan membangun struktur pohon keputusan sebagai representasi model. Algoritma ini membagi data menjadi beberapa subset berdasarkan titik split yang dianggap paling optimal dalam mengurangi *error*. Setiap pembagian *node* dilakukan secara rekursif hingga mencapai *leafnode* yang berisi nilai prediksi. Pemilihan split biasanya dilakukan berdasarkan pengukuran *error* seperti *Mean Squared Error* (MSE) sehingga pemisahan data menghasilkan kelompok dengan variasi nilai target yang lebih homogen.

Decision Tree Regression sangat efektif untuk memodelkan hubungan non-linear karena struktur pohon memungkinkan model membuat batas keputusan yang kompleks. Selain itu, *decision tree* tidak memerlukan normalisasi atau *scaling* data dan dapat menangani fitur numerik maupun kategorikal. Meskipun demikian, algoritma ini memiliki kelemahan utama berupa kecenderungan *overfitting*, yaitu



ketika model terlalu menyesuaikan diri dengan data *training* sehingga performanya menurun saat diuji pada data baru.

3. *Random Forest Regression*

Random Forest Regression merupakan pengembangan dari *Decision Tree* yang menggunakan pendekatan *ensemble learning*, yaitu membangun banyak pohon keputusan dan menggabungkan prediksinya. Setiap pohon dibangun menggunakan sampel data yang dipilih secara acak melalui *bootstrap sampling*, serta subset fitur yang juga dipilih secara acak pada setiap pemisahan *node*. Proses ini menciptakan berbagai variasi pohon yang kemudian memberikan hasil prediksi lebih stabil dan tidak mudah *overfitting* dibanding *tree* tunggal.

Prediksi akhir pada *Random Forest* biasanya diperoleh dari rata-rata hasil prediksi semua pohon, sehingga model mampu menangkap pola kompleks tanpa mengorbankan akurasi. Selain itu, *Random Forest* menyediakan informasi mengenai *feature importance*, yang menunjukkan fitur mana yang paling berpengaruh dalam proses prediksi. Hal ini membuat *Random Forest* menjadi salah satu algoritma regresi yang umum digunakan, terutama pada dataset yang besar dan memiliki banyak fitur.

Meski demikian, model ini memerlukan sumber daya komputasi yang lebih besar.

• **Metrik Evaluasi Regresi**

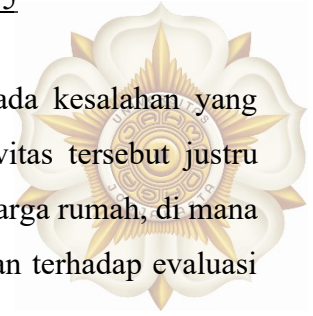
Evaluasi model regresi dilakukan menggunakan metrik khusus yang mampu mengukur kesalahan prediksi nilai numerik. Untuk menilai performa model regresi, digunakan dua metrik utama.

1. RMSE

RMSE mengukur rata-rata jarak antara nilai prediksi dan nilai sebenarnya dengan memberikan penalti lebih besar terhadap kesalahan yang besar. RMSE merupakan akar dari *Mean Squared Error*, sehingga unitnya sama dengan unit target dan lebih mudah diinterpretasikan dalam masalah nyata. Nilai RMSE yang lebih kecil menandakan bahwa model menghasilkan prediksi yang lebih akurat.

$$RMSE = \sqrt{\frac{1}{n} \sum (y_{pred} - y_{true})^2}$$

RMSE yang memiliki nilai rendah menunjukkan bahwa model mampu menghasilkan prediksi yang lebih mendekati nilai sebenarnya, sehingga tingkat akurasinya semakin



baik. Karena RMSE memberikan penalti yang lebih besar pada kesalahan yang ekstrem, metrik ini menjadi sensitif terhadap *outlier*. Sensitivitas tersebut justru menjadikan RMSE cocok digunakan pada data kontinu seperti harga rumah, di mana perbedaan nilai yang besar dapat memberikan dampak signifikan terhadap evaluasi performa model.

2. *Coefficient of Determination* (R^2),

Coefficient of Determination atau R^2 menunjukkan seberapa besar proporsi variansi target yang dapat dijelaskan oleh model. Nilai R^2 berada pada rentang 0 hingga 1, di mana nilai mendekati 1 menandakan bahwa model memiliki kemampuan prediksi yang baik dan dapat menangkap pola hubungan dengan baik. Berbeda dengan metrik klasifikasi seperti akurasi atau *precision*, RMSE dan R^2 merupakan metrik yang dirancang khusus untuk kasus regresi, karena mampu menilai kualitas prediksi numerik secara tepat.

$$0 \leq R^2 \leq 1$$

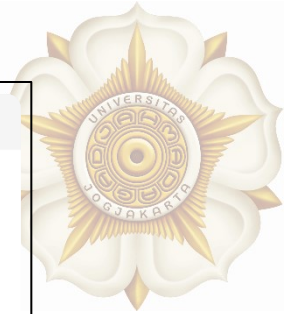
Nilai R^2 yang semakin mendekati 1 menunjukkan bahwa model mampu menjelaskan proporsi variansi target dengan lebih baik, sehingga performanya dapat dikatakan semakin sesuai dengan pola data sebenarnya. Sebaliknya, jika nilai R^2 rendah, berarti model kurang mampu menangkap hubungan antara fitur dan target. Karena sifatnya yang mengukur seberapa besar variasi data yang dapat dijelaskan oleh model, R^2 sangat berguna dalam menilai tingkat kecocokan model regresi, terutama ketika ingin memahami seberapa representatif model tersebut terhadap pola keseluruhan dalam dataset.

4.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

4.4. LANGKAH PERCOBAAN

1. Impor Pustaka



```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

Gambar 4.4.1 Import Library

2. Load dataset

Dataset dapat diunduh pada link berikut:

<https://www.kaggle.com/datasets/shivachandel/kc-house-data>

```
df=pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/
kc_house_data.csv')
```

3. Exploratory Data Analysis (EDA)

a. Melihat 5 baris pertama dan terakhir pada dataset dengan `df.head()` dan `df.tail()`.

df.head()																
	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	0
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	1991
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	0
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	910	1965	0
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	0	1987	0

5 rows × 21 columns

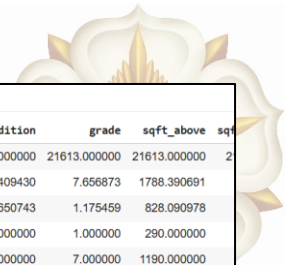
Gambar 4.4.2 Lima Baris Pertama Dataset

df.tail()																
	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0	0	...	8	1530	0	2009	
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	0	0	...	8	2310	0	2014	
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0	0	...	7	1020	0	2009	
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0	0	...	8	1600	0	2004	
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076	2.0	0	0	...	7	1020	0	2008	

5 rows × 21 columns

Gambar 4.4.3 Lima Baris Terakhir Dataset

b. Melihat deskripsi statistik dengan `df.describe()`.



```
df.describe()
```

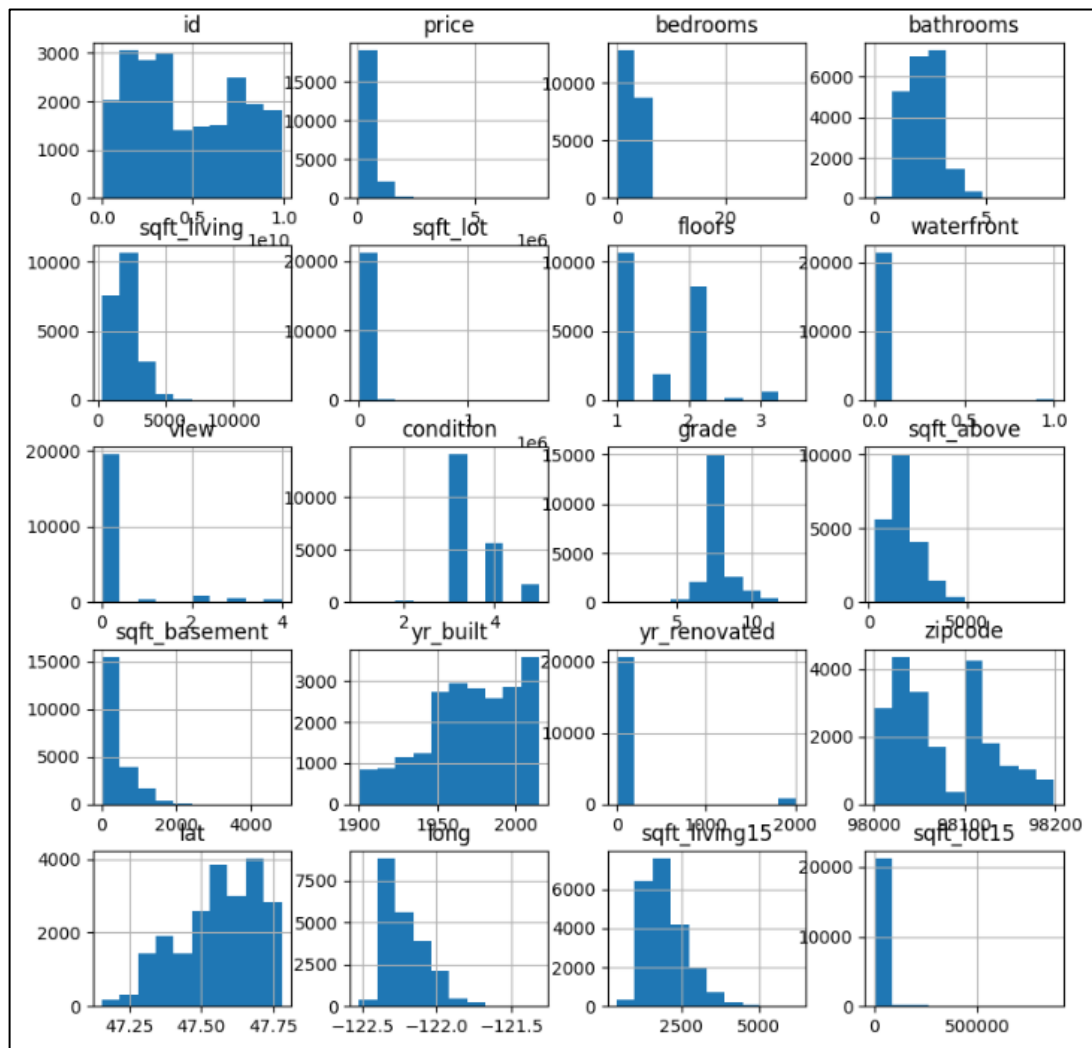
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+04	1.494309	0.007542	0.234303	3.409430	7.656873	1788.390691	1788.390691
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.766318	0.650743	1.175459	828.090978	828.090978
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	1.000000	290.000000	290.000000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	3.000000	7.000000	1190.000000	1190.000000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	1560.000000	1560.000000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000000	4.000000	8.000000	2210.000000	2210.000000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000000	5.000000	13.000000	9410.000000	9410.000000

Gambar 4.4.4 Hasil Deskripsi Statistik Dataset

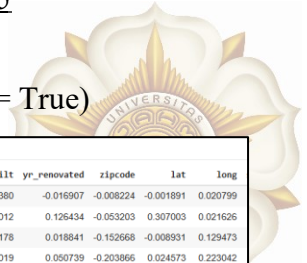
c. Melihat histogram

```
df.hist(figsize=(10,10))
plt.show()
```

Gambar 4.4.5 Kode untuk Menampilkan Histogram



Gambar 4.4.6 Histogram Seluruh Variabel Numerik



d. Mengecek korelasi atribut numeric dengan `df.corr(numeric_only = True)`

df.corr(numeric_only = True)																		
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long
id	1.000000	-0.016762	0.001286	0.005160	-0.012258	-0.132109	0.018525	-0.002721	0.011592	-0.023783	0.008130	-0.010842	-0.005151	0.021380	-0.016907	-0.008224	-0.001891	0.020799
price	-0.016762	1.000000	0.308350	0.525138	0.702035	0.089661	0.256794	0.266369	0.397293	0.036362	0.667434	0.605567	0.323816	0.054012	0.126434	-0.053203	0.307003	0.021626
bedrooms	0.001286	0.308350	1.000000	0.515884	0.576671	0.031703	0.175429	-0.005582	0.079532	0.028472	0.356967	0.477600	0.303093	0.154178	0.018841	-0.152668	-0.008931	0.129473
bathrooms	0.005160	0.525138	0.515884	1.000000	0.754665	0.087740	0.500653	0.063744	0.187737	-0.124982	0.664983	0.685342	0.283770	0.506019	0.050739	-0.203866	0.024573	0.223042
sqft_living	-0.012258	0.702035	0.576671	0.754665	1.000000	0.172826	0.353949	0.103818	0.284611	-0.058753	0.762704	0.876597	0.435043	0.318049	0.055363	-0.199430	0.052529	0.240223
sqft_lot	-0.132109	0.089661	0.031703	0.087740	0.172826	1.000000	-0.005201	0.021604	0.074710	-0.008958	0.113621	0.183512	0.015286	0.053080	0.007644	-0.129574	-0.085683	0.229521
floors	0.018525	0.256794	0.175429	0.500653	0.353949	-0.005201	1.000000	0.023698	0.029444	-0.263768	0.458183	0.523885	-0.245705	0.489319	0.006338	-0.059121	0.049614	0.125419
waterfront	-0.002721	0.266369	-0.005582	0.063744	0.103818	0.021604	0.023698	1.000000	0.401857	0.016653	0.082775	0.072075	0.080588	-0.026161	0.092885	0.030285	-0.014274	-0.041910
view	0.011592	0.397293	0.079532	0.187737	0.284611	0.074710	0.029444	0.401857	1.000000	0.045990	0.251321	0.167649	0.276947	-0.053440	0.103917	0.084827	0.006157	-0.078400
condition	-0.023783	0.036362	0.028472	-0.124982	-0.058753	-0.008958	-0.263768	0.016653	0.045990	1.000000	-0.144674	-0.158214	0.174105	-0.361417	-0.060618	0.003026	-0.014941	-0.106500
grade	0.008130	0.667434	0.356967	0.664983	0.762704	0.113621	0.458183	0.082775	0.251321	-0.144674	1.000000	0.755923	0.168392	0.446963	0.014414	-0.184862	0.114084	0.198372
sqft_above	-0.010842	0.605567	0.477600	0.685342	0.876597	0.183512	0.523885	0.072075	0.167649	-0.158214	0.755923	1.000000	-0.051943	0.423898	0.023285	-0.261190	-0.000816	0.343803
sqft_basement	-0.005151	0.323816	0.303093	0.283770	0.435043	0.015286	-0.245705	0.080588	0.276947	0.174105	0.168392	-0.051943	1.000000	-0.133124	0.071323	0.074845	0.110538	-0.144765
yr_built	0.021380	0.054012	0.154178	0.506019	0.318049	0.053080	0.489319	-0.026161	-0.053440	-0.361417	0.446963	0.423898	-0.133124	1.000000	-0.224874	-0.346869	-0.148122	0.409356
yr_renovated	-0.016907	0.126434	0.018841	0.050739	0.055363	0.007644	0.006338	0.092885	0.103917	-0.060618	0.014414	0.023285	0.071323	-0.224874	1.000000	0.064357	0.029398	-0.068372
zipcode	-0.008224	-0.053203	-0.152668	-0.203866	-0.199430	-0.129574	-0.059121	0.030285	0.084827	0.003026	-0.184862	-0.261190	0.074845	-0.346869	0.064357	1.000000	0.267048	-0.564072
lat	-0.001891	0.307003	-0.008931	0.024573	0.052529	-0.085683	0.049614	-0.014274	0.006157	-0.014941	0.114084	-0.000816	0.110538	-0.148122	0.029398	0.267048	1.000000	-0.135512
long	0.020799	0.021626	0.129473	0.223042	0.240223	0.229521	0.125419	-0.041910	-0.078400	-0.106500	0.198372	0.343803	-0.144765	0.409356	-0.068372	-0.564072	-0.135512	1.000000
sqft_living15	-0.002901	0.585379	0.391638	0.568634	0.756420	0.144608	0.279885	0.086463	0.280439	-0.092824	0.713202	0.731870	0.200355	0.326229	-0.002673	-0.279033	0.048858	0.334605
sqft_lot15	-0.136798	0.082447	0.029244	0.087175	0.183286	0.718557	-0.011269	0.030703	0.072575	-0.003406	0.119248	0.194050	0.017276	0.070958	0.007854	-0.147221	-0.086419	0.254451

Gambar 4.4.7 Korelasi Atribut Numerik

e. Mengecek missing value dengan `df.isnull().sum()`

df.isnull().sum()	
	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
zipcode	0
lat	0
long	0
sqft_living15	0
sqft_lot15	0
dtype:	int64

Gambar 4.4.8 Pengecekan Missing Value



f. Mengecek atribut kategorikal

```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)

Index([], dtype='object')
```

Gambar 4.4.9 Pengecekan Atribut Kategorikal

4. Modelling

a. Linear Regression

```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
reg = LinearRegression()
reg.fit(X_train, y_train)

print('coef of determination training ', reg.score(X_train, y_train))
print('coef of determination testing ', reg.score(X_test, y_test))

print('coefficient')
print(reg.coef_)
print('intercept')
print(reg.intercept_)

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[0:10])
```

Gambar 4.4.10 Kode untuk Membuat Model Linear Regression

- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik



Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*

Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Selanjutnya, objek model *Linear Regression* dibuat dengan `reg = LinearRegression()`. Model kemudian dilatih menggunakan data *training* melalui perintah `reg.fit(X_train, y_train)`, di mana model mempelajari hubungan linier antara fitur dan target.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Melihat parameter model

Model *Linear Regression* menghasilkan dua parameter, yaitu koefisien dan *intercept*. Koefisien menunjukkan pengaruh masing-masing fitur terhadap nilai prediksi, sedangkan *intercept* merupakan nilai dasar ketika semua fitur bernilai nol. Kedua parameter ini ditampilkan dalam kode melalui `reg.coef_` dan `reg.intercept_`.

- Membuat prediksi model

Tahap selanjutnya adalah membuat prediksi pada data *testing* menggunakan `reg.predict(X_test)`. Hasil prediksi kemudian disimpan dalam variabel `y_pred`, kemudian ditampilkan untuk melihat 10 prediksi pertama. Untuk memastikan hasilnya masuk akal, nilai prediksi tersebut dibandingkan dengan 10 nilai sebenarnya dari `y_test`. Langkah ini memberikan gambaran awal mengenai akurasi model sebelum dilakukan evaluasi lebih lanjut menggunakan metrik seperti RMSE atau R^2 .



```

coef of determination training 0.6995155846436756
coef of determination testing 0.6994627057969904
coefficient
[ -3.43081477e+04  4.03129700e+04  1.12001375e+02  9.91841247e-02
  5.27154218e+03  5.43877177e+05  5.50830616e+04  2.31460673e+04
  9.49081794e+04  7.22190668e+01  3.97823082e+01 -2.59441847e+03
  2.19209734e+01 -5.56358731e+02  5.95216324e+05 -1.96904658e+05
  1.62077488e+01 -3.30430480e-01]
intercept
6641646.708095346
prediction
[ 458597.0676416  748993.75994823 1243303.75799061 1665116.95095454
  737302.05741732 283239.58524967 831732.8758231 495383.02095346
  385779.81919014 474179.42285135]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]

```

Gambar 4.4.11 Hasil Prediksi Model Linear Regression

Tahap selanjutnya adalah evaluasi model dengan *metriks* RMSE dan R^2 untuk mengukur kualitas prediksi dari model regresi.

```

mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

print('rmse : ', rmse)
print('r2 : ', r2)

```

Gambar 4.4.12 Kode Evaluasi Model Menggunakan RMSE dan R^2

Pertama, nilai *mean squared error* (MSE) dihitung menggunakan fungsi `mean_squared_error(y_test, y_pred)`. Nilai MSE menggambarkan rata-rata kuadrat selisih antara nilai prediksi dengan nilai sebenarnya. Semakin kecil nilai MSE, semakin baik performa model. Selanjutnya, nilai *Root Mean Squared Error* (RMSE) diperoleh dengan mengambil akar kuadrat dari MSE menggunakan `np.sqrt(mse)`. RMSE memberikan interpretasi kesalahan dalam satuan yang sama dengan target, sehingga lebih mudah untuk dibandingkan.

Kemudian, performa model juga dievaluasi menggunakan R^2 (*coefficient of determination*) melalui fungsi `r2_score(y_test, y_pred)`. Nilai R^2 menunjukkan seberapa besar variansi data yang dapat dijelaskan oleh model. Nilai yang mendekati 1 berarti model memiliki kemampuan prediksi yang sangat baik, sedangkan nilai mendekati 0 menandakan model kurang mampu menjelaskan pola hubungan dalam data.

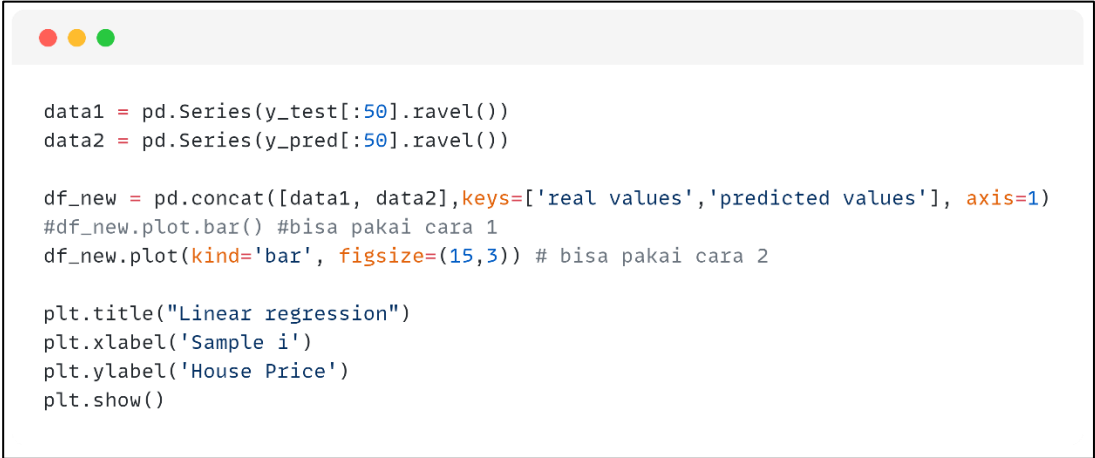


```
rmse : 208296.72772118862
r2 : 0.6994627057969904
```

Gambar 4.4.13 Hasil Evaluasi Menggunakan Metrik RMSE dan R^2

Berdasarkan hasil evaluasi, model menghasilkan nilai RMSE sebesar 208296.72, yang berarti rata-rata kesalahan prediksi model berada di kisaran angka tersebut terhadap nilai harga rumah sebenarnya. Sementara itu, nilai R^2 sebesar 0.699 menunjukkan bahwa model mampu menjelaskan sekitar 69.9% variansi dalam data harga rumah. Nilai ini mengindikasikan bahwa model *Linear Regression* memiliki kemampuan yang cukup baik dalam memodelkan hubungan antara fitur dan harga rumah, meskipun masih terdapat sekitar 30% variansi yang belum dapat dijelaskan oleh model.

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset testing.



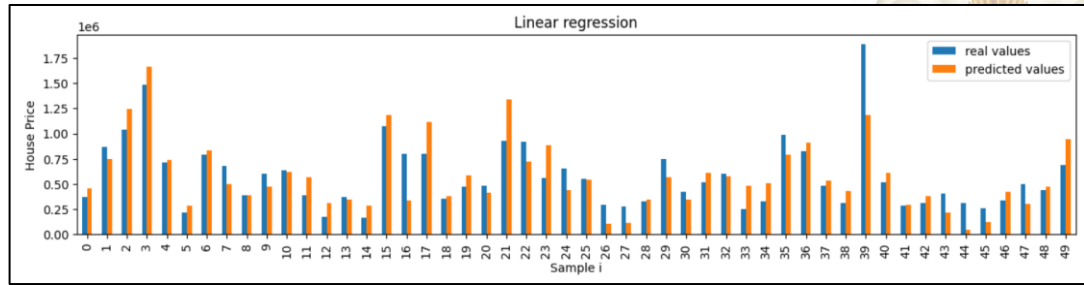
```
data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("Linear regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()
```

Gambar 4.4.14 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

- Pertama, diambil 50 sampel awal dari `y_test` dan `y_pred`, kemudian masing-masing diubah menjadi objek `Series`.
- Kedua `Series` ini digabungkan menjadi sebuah `DataFrame` bernama `df_new`, dengan dua kolom yang diberi label *real values* dan *predicted values*.
- Setelah data digabung, grafik batang (*bar chart*) dibuat menggunakan perintah `df_new.plot(kind='bar', figsize=(15,3))`. Visualisasi ini berfungsi sebagai alat untuk mengevaluasi apakah pola prediksi model mendekati pola sebenarnya.



Gambar 4.4.15 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

b. Decision Tree Regression

```

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
dt = DecisionTreeRegressor(max_depth=10)
dt.fit(X_train, y_train)

print('coef of determination training ', dt.score(X_train, y_train))
print('coef of determination testing ', dt.score(X_test, y_test))

print('prediction')
y_pred = dt.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

```

Gambar 4.4.16 Kode untuk Membuat Model Decision Tree Regression

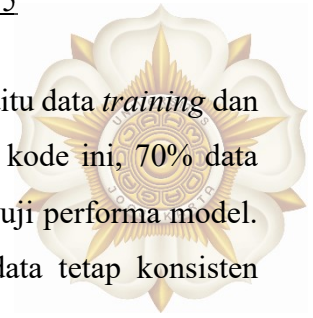
- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik

Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*



Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Model *Decision Tree Regression* dibuat dengan objek `DecisionTreeRegressor(max_depth=10)` yang berfungsi sebagai representasi pohon keputusan untuk tugas regresi. Parameter `max_depth=10` digunakan untuk membatasi kedalaman maksimum pohon sehingga model tidak terlalu kompleks dan dapat mengurangi risiko *overfitting*. Setelah objek model dibuat, tahap *training* dilakukan dengan memanggil `dt.fit(X_train, y_train)` sehingga model mempelajari pola hubungan antara fitur pada data latih dan nilai targetnya.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Membuat prediksi model

Setelah model selesai dilatih menggunakan data *training*, langkah berikutnya adalah membuat prediksi terhadap data yang belum pernah dilihat model, yaitu data *testing*. Proses ini dilakukan menggunakan perintah `dt.predict(X_test)`, yang menghasilkan serangkaian nilai hasil prediksi dan disimpan dalam variabel `y_pred`.

```
coef of determination training  0.9185274272472612
coef of determination testing  0.7689346283378805
prediction
[ 372512.5      837693.06862745 1087247.96153846 1984617.64705882
 656314.08658009 237079.87179487  849106.21276596 517991.26363636
 366270.875    519039.40166667]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]
```

Gambar 4.4.17 Hasil Prediksi Model Decision Tree Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik seperti RMSE dan R^2 . Berikut hasil evaluasi model menggunakan algoritma *Decision Tree Regression*.



```
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

print('rmse : ', rmse)
print('r2 : ', r2)
```

Gambar 4.4.18 Kode Evaluasi Model Menggunakan RMSE dan R2

```
rmse : 182642.01673989513
r2 : 0.7689346283378805
```

Gambar 4.4.19 Hasil Evaluasi Model

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset *testing*.

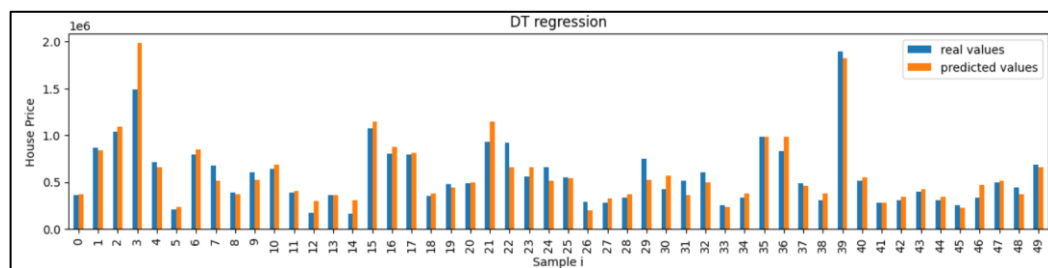
```
data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

df_new = pd.concat([data1, data2],keys=['real values','predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("DT regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()
```

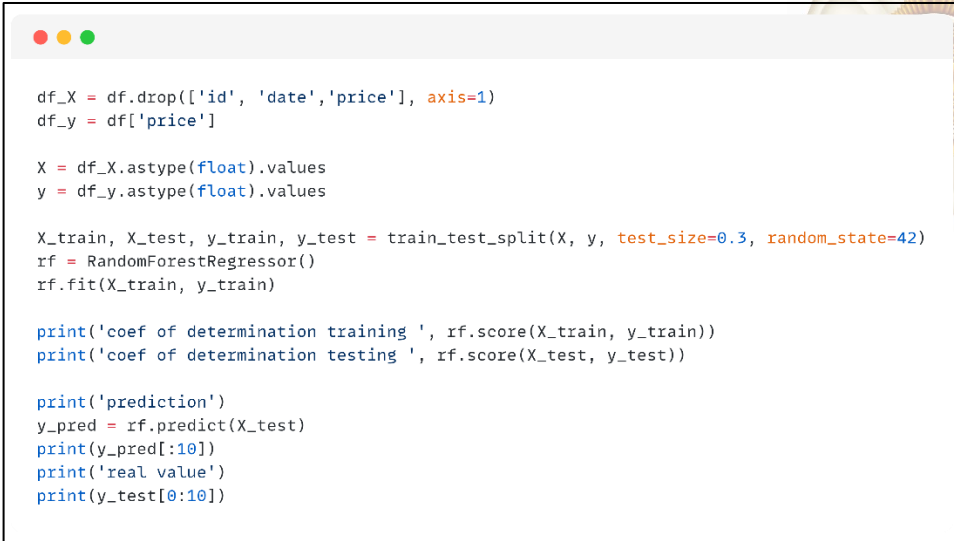
Gambar 4.4.20 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

Berikut hasil perbandingan antara nilai harga rumah sebenarnya dan nilai hasil prediksi model *Decision Tree Regression* untuk 50 sampel pertama dari data testing.



Gambar 4.4.21 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

c. Random Forest Regression



```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

print('coef of determination training ', rf.score(X_train, y_train))
print('coef of determination testing ', rf.score(X_test, y_test))

print('prediction')
y_pred = rf.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])
```

Gambar 4.4.22 Kode untuk Membuat Model Random Forest Regression

- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik

Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*

Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Tahap pemodelan *Random Forest Regression* dimulai dengan membentuk objek model melalui `RandomForestRegressor()`, yang secara *default* akan membuat sekumpulan pohon keputusan sebagai dasar prediksi. Model ini kemudian dilatih menggunakan data *training* dengan memanggil `rf.fit(X_train, y_train)`,



sehingga setiap pohon dalam *ensemble* mempelajari pola hubungan antara fitur dan harga rumah dari subset data yang berbeda.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Membuat prediksi model

Tahap berikutnya adalah membuat prediksi menggunakan `rf.predict(X_test)`, yang menghasilkan nilai estimasi harga rumah dengan data testing.

```
coef of determination training 0.9822001440062388
coef of determination testing 0.8596089494795431
prediction
[ 379509.      885196.      1130435.43333333 2179107.
  703632.1    249551.38    838150.5      630879.5
  404713.92    541167.67      ]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]
```

Gambar 4.4.23 Hasil Prediksi Model Random Forest Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik seperti RMSE dan R^2 . Berikut hasil evaluasi model menggunakan algoritma *Decision Tree Regression*.

```
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

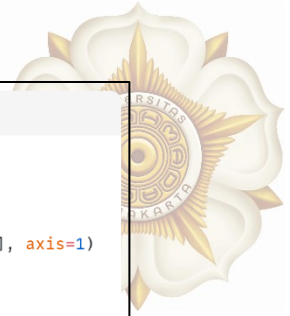
print('rmse : ', rmse)
print('r2 : ', r2)
```

Gambar 4.4.24 Kode Evaluasi Model Menggunakan RMSE dan R^2

```
rmse : 142364.91502742676
r2 : 0.8596089494795431
```

Gambar 4.4.25 Hasil Evaluasi Model

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset *testing*.



```

data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

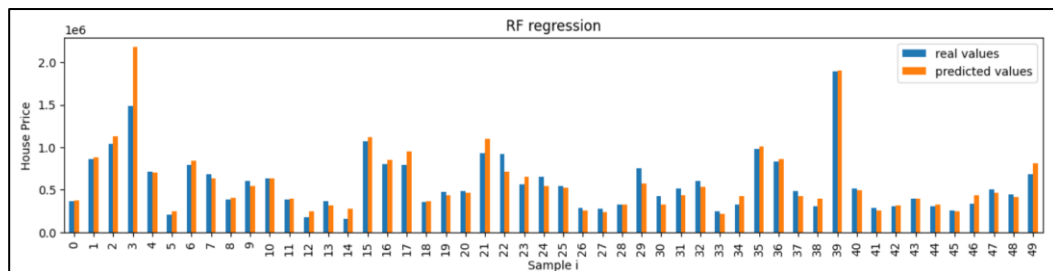
df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("RF regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()

```

Gambar 4.4.26 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

Berikut hasil perbandingan antara nilai harga rumah sebenarnya dan nilai hasil prediksi model *Random Forest Regression* untuk 50 sampel pertama dari data testing.



Gambar 4.4.27 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

4.5. TUGAS & ANALISIS

Dataset: <https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/data>

1. Jelaskan apa tujuan penggunaan dataset ini dan definisikan atribut yang menjadi input dan *output*-nya!
2. Sebutkan variabel yang paling mempengaruhi *output* atau mempunyai nilai korelasi tinggi!
3. Silahkan membuat model regresi dengan *Linear Regression*, *Decision Tree Regression*, *Random Forest Regression* dari dataset diatas!
4. Buatlah tabel yang menjelaskan performa dari model *machine learning* untuk kasus dataset diatas! Kolom pertama "model", kolom selanjutnya "RMSE, R2, R".
5. Jelaskan dari hasil eksperimen di atas, model mana yang paling baik? Jelaskan alasan anda!

4.6. REFERENSI

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.





PERTEMUAN V

FORECASTING

5.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa dapat menerapkan metode forecasting berbasis *machine learning*, meliputi Neural Network, K-Nearest Neighbors (KNN), Decision Tree, Support Vector Regression (SVR), dan Random Forest, untuk memprediksi nilai periode berikutnya berdasarkan data historis
- B. Mahasiswa dapat melakukan analisis dan perbandingan performa model menggunakan metrik evaluasi *Root Mean Squared Error* (RMSE) dan *Pearson Correlation Coefficient*
- C. Mahasiswa dapat menentukan model forecasting yang paling akurat berdasarkan hasil evaluasi menggunakan RMSE dan *Pearson Correlation Coefficient*

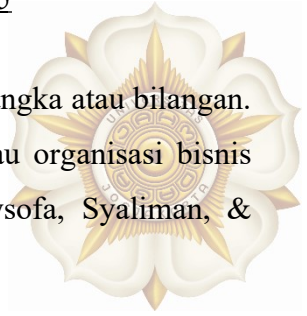
5.2 DASAR TEORI

- **Supervised Learning**

Supervised learning merupakan salah satu teknik *machine learning* yang menggunakan dataset (*data training*) berlabel (*labeled data*) untuk melatih mesin, sehingga mampu mengidentifikasi label input berdasarkan fitur yang dimiliki dan melakukan prediksi atau klasifikasi. Algoritma yang termasuk dalam teknik *supervised learning* antara lain Decision Tree, K-Nearest Neighbor (KNN), Naïve Bayes, Regresi, dan Support Vector Machine (SVM) (Retnoningsih & Pramudita, 2020). Pada algoritma *Supervised Learning*, sistem diberikan training data set berupa informasi masukan dan keluaran yang diinginkan, sehingga sistem akan mempelajari berdasarkan data yang telah ada. Sistem akan mencari pola dari data set, kemudian pola itu akan dijadikan sebagai acuan untuk kumpulan data berikutnya (Santoso, Abijono, & Anggreini, 2021).

- **Forecasting**

Forecasting adalah proses memprediksi suatu keadaan di masa kini dan masa depan dengan menguji keadaan di masa lalu. Metode ini banyak digunakan untuk melakukan prediksi dengan berbagai algoritma (Vimala & Nugroho, 2022). Fungsi ramalan adalah membantu pengambilan keputusan berdasarkan pertimbangan terhadap apa yang akan terjadi saat keputusan tersebut dilaksanakan. Ramalan dapat bersifat kualitatif, yaitu tidak berbentuk angka, seperti perkiraan bahwa cuaca besok akan cerah. Selain itu, ramalan



juga bisa bersifat kuantitatif, yang berarti dinyatakan dalam bentuk angka atau bilangan. Peramalan merupakan bagian penting dalam setiap perusahaan atau organisasi bisnis karena berperan dalam pengambilan keputusan manajemen (Maysofa, Syaliman, & Sapriadi, 2023).

- **RMSE**

Root Mean Squared Error (RMSE) merupakan salah satu metrik evaluasi yang digunakan untuk mengukur tingkat kesalahan antara nilai aktual dan nilai hasil prediksi. RMSE dihitung dengan cara mengambil akar kuadrat dari rata-rata selisih kuadrat antara nilai aktual dan nilai prediksi. Semakin kecil nilai RMSE, semakin baik performa model karena menunjukkan bahwa model memiliki error yang rendah. RMSE banyak digunakan dalam peramalan (forecasting) karena sensitif terhadap error yang besar, sehingga mampu memberikan gambaran akurasi model secara lebih representatif (Chai & Draxler, 2014).

- **Pearson Correlation Coefficient**

Pearson Correlation Coefficient adalah ukuran statistik yang digunakan untuk menentukan tingkat hubungan linear antara dua variabel, yaitu nilai aktual dan nilai prediksi. Nilai koefisien berkisar antara -1 hingga 1 , di mana nilai mendekati 1 menunjukkan korelasi positif yang sangat kuat, nilai mendekati -1 menunjukkan korelasi negatif yang kuat, dan nilai mendekati 0 menunjukkan tidak adanya hubungan linear. Dalam konteks forecasting, Pearson Correlation digunakan untuk menilai seberapa baik pola prediksi mengikuti pola aktual. Semakin tinggi nilai korelasi, semakin baik model dalam menangkap pola data (Benesty et al., 2009).

5.3 ALAT DAN BAHAN

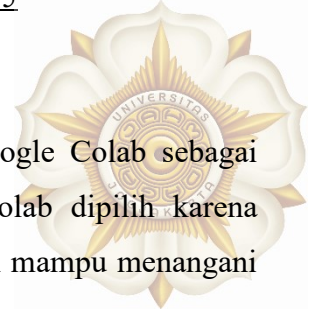
A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

5.4 LANGKAH PERCOBAAN



1. Persiapan IDE

Mahasiswa menyiapkan lingkungan kerja dengan membuka Google Colab sebagai platform utama untuk menjalankan eksperimen *forecasting*. Colab dipilih karena mendukung seluruh library machine learning yang diperlukan dan mampu menangani komputasi tanpa membebani perangkat mahasiswa.

2. Import Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh *library* yang diperlukan untuk melakukan preprocessing, visualisasi, dan proses *forecasting*. *Library* utama yang digunakan meliputi:

1. pandas dan numpy untuk membaca, memanipulasi, dan mengelola data.
2. matplotlib dan seaborn untuk membuat visualisasi data, seperti tren deret waktu dan grafik hasil prediksi.
3. MeanSquaredError / RMSE dan *Pearson correlation* untuk evaluasi performa model.
4. MLPRegressor, KNeighborsRegressor, DecisionTreeRegressor, dan RandomForestRegressor untuk melakukan pemodelan forecasting dengan berbagai algoritma *supervised learning*.
5. StandardScaler atau *library* pendukung lain apabila dibutuhkan untuk normalisasi atau transformasi data.

```
[ ] # Import Library

import numpy as np
import pandas as pd
from math import sqrt
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
import math
from sklearn.metrics import mean_squared_error
from numpy import array
from sklearn.neural_network import MLPRegressor
from sklearn.neighbors import KNeighborsRegressor
from scipy.stats import pearsonr
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVC
from sklearn.svm import SVR
from xgboost import XGBClassifier

from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import mean_squared_log_error
from numpy import array
from scipy.stats import kurtosis, skew
```

Gambar 5.4.1 Impor Pustaka



3. Import Dataset

Mahasiswa membuat dataset <https://www.kaggle.com/datasets/lakshmi25npathi/bike-sharing-dataset> yang dapat diakses melalui Bike Sharing Dataset. Dataset ini berisi data peminjaman sepeda per hari, termasuk tanggal, kondisi cuaca, dan total jumlah peminjaman. Pada tahap ini mahasiswa:

1. Mengimpor dataset dan menampilkan 5 baris pertama untuk memastikan dataset terbaca dengan benar.

```
# Read Dataset
import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/bikesharing_day.csv')
df.head()
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	1/1/2011	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	1/2/2011	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	1/3/2011	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	1/4/2011	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	1/5/2011	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600

Gambar 5.4.2 Pembacaan Dataset

2. Melakukan konversi kolom tanggal ke format datetime

```
# Change Data Format
df_ori = df
df_ori['date'] = pd.to_datetime(df_ori['dteday'])
df_ori['cnt'].iloc[:10]
```

	cnt
0	985
1	801
2	1349
3	1562
4	1600

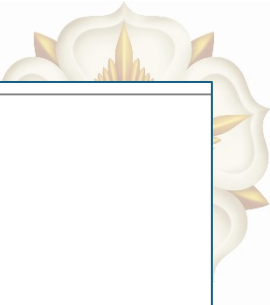
Gambar 5.4.3 Konversi tanggal ke format datetime

4. Membentuk Date Time Series

Data time series membutuhkan pembentukan input dan output secara berurutan. Oleh karena itu, mahasiswa membuat:

1. Input (X): nilai cnt selama 7 hari sebelumnya,
2. Output (y): jumlah peminjaman sepeda pada hari ke-8.

Pembentukan ini dilakukan dengan teknik *sliding window*, yaitu menggeser jendela data dari hari ke hari untuk menghasilkan pasangan input–output.



```
[ ] # Function Definition Sliding Window

def split_sequences(sequences, n_steps_in, n_steps_out):
    X, y = list(), list()
    for i in range(len(sequences)):
        # find the end of this pattern
        end_ix = i + n_steps_in
        out_end_ix = end_ix + n_steps_out

        # check if we are beyond the dataset
        if out_end_ix > len(sequences):
            break

        # gather input and output parts of the pattern
        seq_x, seq_y = sequences[i:end_ix, :-1], sequences[out_end_ix - 1, -1]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)
```

Gambar 5.4.4 Fungsi sliding window

5. Menambahkan Fitur Statistik

Agar model memiliki informasi tambahan, mahasiswa melakukan feature engineering dengan menambahkan fitur-fitur statistik pada setiap jendela data, seperti:

1. Nilai minimum
2. Nilai maksimum
3. Selisih antara max-min
4. Rata-rata
5. Standar deviasi
6. Median
7. Kurtosis
8. Skewness



```
[ ] # Function Definition Statistic Feature

def stats_features(input_data):
    inp = list()
    for i in range(len(input_data)):
        inp2=list()
        inp2=input_data[i]
        min=float(np.min(inp2))
        max=float(np.max(inp2))
        diff=(max-min)
        std=float(np.std(inp2))
        mean=float(np.mean(inp2))
        median=float(np.median(inp2))
        kurt=float(kurtosis(inp2))
        sk=float(skew(inp2))
        inp2=np.append(inp2,min)
        inp2=np.append(inp2,max)
        inp2=np.append(inp2,diff)
        inp2=np.append(inp2,std)
        inp2=np.append(inp2,mean)
        inp2=np.append(inp2,median)
        inp2=np.append(inp2,kurt)
        inp2=np.append(inp2,sk)

        #print(list(inp2))
        inp=np.append(inp,inp2)

    inp=inp.reshape(len(input_data),-1)

    #print(inp)
    return inp
```

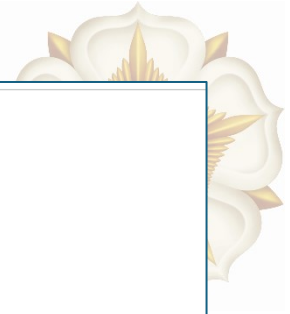
Gambar 5.4.5 Fungsi statistik

Fitur-fitur ini memberi gambaran kondisi distribusi data dalam jendela waktu tersebut, sehingga diharapkan dapat meningkatkan performa model.

6. Pemrosesan Data

Pada tahap ini dilakukan proses pengolahan data agar dapat digunakan sebagai input bagi model-model machine learning. Seluruh proses preprocessing dilaksanakan dalam satu rangkaian, meliputi:

1. Pembentukan Data Time Series Menggunakan *Sliding Window*
2. Pemisahan Data Training dan Testing



```
# Preprocessing

df_X = df_ori[['cnt', 'cnt']]
in_seq = df_X.astype(float).values
#out_seq = df_y.astype(float).values

#in_seq1 = in_seq.reshape(in_seq.shape[0], in_seq.shape[1])
#out_seq = out_seq.reshape((len(out_seq), 1))

#from numpy import hstack
#dataset = hstack((in_seq1, out_seq))

n_steps_in, n_steps_out = 7, 1
X, y = split_sequences(in_seq, n_steps_in, n_steps_out)

n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle = False)
X_train = stats_features(X_train)
X_test = stats_features(X_test)
```

Gambar 5.4.6 Preprocessing Dataset

```
# Output from Data Transformation with Sliding Window Technique

in_seq

array([[ 985.,  985.],
       [ 801.,  801.],
       [1349., 1349.],
       ...,
       [1341., 1341.],
       [1796., 1796.],
       [2729., 2729.]])

# Input Features

X

... array([[ 985.,  801., 1349., ..., 1600., 1606., 1510.],
          [ 801., 1349., 1562., ..., 1606., 1510.,  959.],
          [1349., 1562., 1600., ..., 1510.,  959.,  822.],
          ...,
          [1749., 1787.,  920., ...,  441., 2114., 3095.],
          [1787.,  920., 1013., ..., 2114., 3095., 1341.],
          [ 920., 1013.,  441., ..., 3095., 1341., 1796.]])

# Data Shape

X.shape

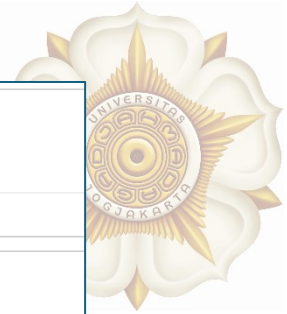
(724, 7)

# Array Input (Sample)

X [0]

array([ 985.,  801., 1349., 1562., 1600., 1606., 1510.]])
```

Gambar 5.4.7 Preprocessing Dataset



```
# Prediction Target

y[0]

np.float64(959.0)
```

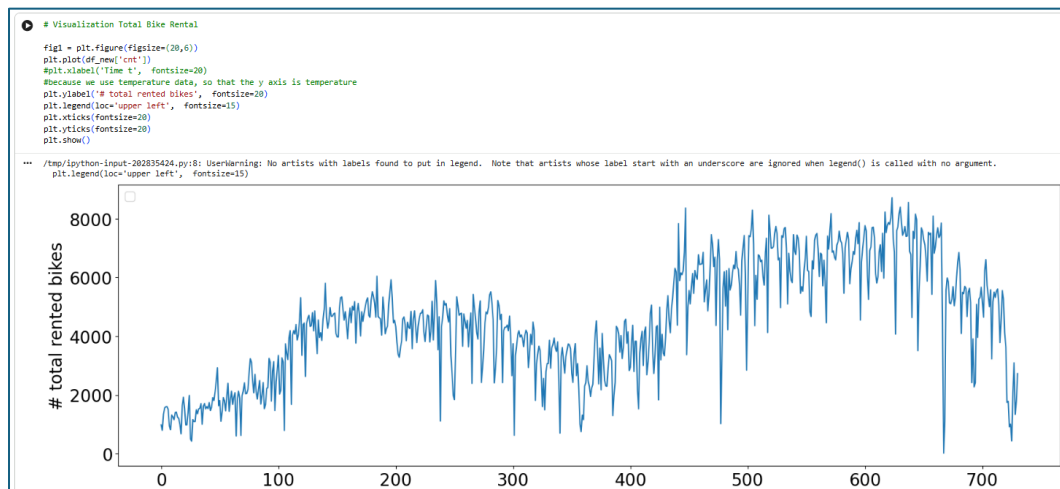
```
# Dataframe from Time Series Analysis

df_new=df_ori[['date','cnt']]
df_new.set_index('date')
```

date	cnt
2011-01-01	985
2011-01-02	801
2011-01-03	1349
2011-01-04	1562
2011-01-05	1600
...	...
2012-12-27	2114
2012-12-28	3095
2012-12-29	1341
2012-12-30	1796
2012-12-31	2729

731 rows x 1 columns

Gambar 5.4.8 Preprocessing Dataset

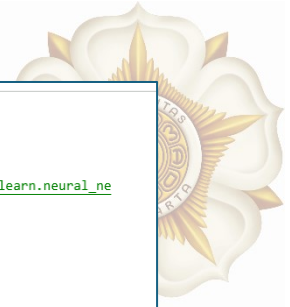


Gambar 5.4.9 Visualisasi Dataset

7. Pelatihan Model Machine Learning

Mahasiswa melatih empat model *supervised learning* untuk *forecasting*:

1. Multilayer Perceptron (MLP/Neural Network)



```
# Neural Network Model

def mlp (X_train, X_test, y_train, y_test):
    # mlp = multilayer perceptron / neural network for regression.
    # to setup parameter, please refer to = https://scikit-learn.org/stable/modules/generated/sklearn.neural\_ne
    mlp_model = MLPRegressor(random_state=42)
    mlp_model = MLPRegressor(hidden_layer_sizes=(100,100, ), max_iter=1000, random_state=42)
    # the model learning from training data
    mlp_model.fit(X_train, y_train)
    # get the prediction output
    y_pred = mlp_model.predict(X_test)
    y_pred = np.round(y_pred, 0)

    # get the root mean square error between prediction and real test data
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    # get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    # returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.10 Fungsi Multilayer Perceptron

2. K-Nearest Neighbors (KNN)

```
# K-Nearest Neighbors Model

def knn (X_train, X_test, y_train, y_test):
    #k nearest neighbor for regression.
    #to setup the parameter please refer to : https://scikit-learn.org/stable/modules/generated/sklearn.neighb
    knn_model = KNeighborsRegressor()
    #the model is learning from training data
    knn_model.fit(X_train, y_train)
    #get the prediction output
    y_pred = knn_model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    #get the root mean square error between prediction and real test data
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

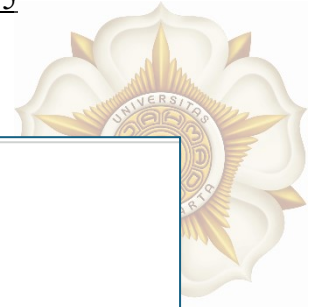
Gambar 5.4.11 Fungsi KNN

3. Decision Tree (DT)

```
# Decision Tree Model

def dt (X_train, X_test, y_train, y_test):
    model= DecisionTreeRegressor(random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.12 Fungsi Decision Tree



4. Random Forest (RF)

```
# Random Forest Model

def rf (X_train, X_test, y_train, y_test):
    model=RandomForestRegressor(random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.13 Fungsi Random Forest

Setiap model dilatih menggunakan data training, kemudian menghasilkan prediksi berdasarkan data testing. Tujuan langkah ini adalah membandingkan performa antar model.

8. Melakukan Prediksi

Setelah model dilatih, masing-masing model menghasilkan nilai prediksi jumlah peminjaman sepeda untuk periode berikutnya berdasarkan input data uji. Prediksi ini kemudian dibandingkan dengan data aktual.

```
# Call the Machine Learning Model Function

#calling the function mlp.
#returning rmse, pearson correlation, and prediction output
rmse_mlp, corr_mlp, y_pred_mlp = mlp(X_train, X_test, y_train, y_test)

#calling the function knn
#returning rmse, pearson correlation, and prediction output
rmse_knn, corr_knn, y_pred_knn = knn(X_train, X_test, y_train, y_test)
rmse_dt, corr_dt, y_pred_dt = dt(X_train, X_test, y_train, y_test)

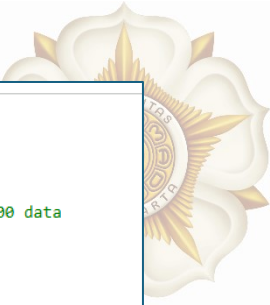
#rmse_svm, corr_svm, y_pred_svm = svm(X_train, X_test, y_train, y_test)
rmse_rf, corr_rf, y_pred_rf = rf(X_train, X_test, y_train, y_test)
```

Gambar 5.4.14 Lakukan prediksi

9. Visualisasi Hasil

Untuk melihat seberapa dekat pola prediksi dengan pola data asli mahasiswa dapat membuat visualisasi perbandingan antara:

1. Nilai aktual (y_{test}), dan
2. Prediksi model (terutama MLP sebagai model utama)



```
# Visualization

fig1 = plt.figure(figsize=(20,6))
# plotting the result
# because the total number of test is data is high, so we just print the first 100 data
# y_test[0:100] get the first 100 data from y_test or real test data.
# y_pred_mlp is the prediction result from MLP
# y_pred_knn is the prediction result from KNN

plt.plot(y_test, label='Real data')
# plt.plot(y_pred_knn, label='KNN')
# plt.plot(y_pred_dt, label='DT')
# plt.plot(y_pred_rf, label='RF')
plt.plot(y_pred_mlp, label='MLP')

#title
# pyplot.title('First 100 Test Data')
# the x axis is timestamp, with interval 1 day
plt.xlabel('Time t', fontsize=20)
# because we use total rented bikes, so that the y axis is rented bikes
plt.ylabel('rented bikes', fontsize=20)
plt.legend(loc='upper left', fontsize=15)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.show()
```

Gambar 5.4.15 Visualisasi hasil prediksi

10. Evaluasi Model

Evaluasi dilakukan menggunakan dua metrik utama:

1. *Root Mean Squared Error (RMSE)*
 - Mengukur seberapa besar kesalahan prediksi
 - Semakin kecil RMSE → model makin baik.
2. *Pearson Correlation Coefficient*
 - Mengukur hubungan linear antara nilai prediksi dan nilai aktual
 - Semakin mendekati 1 → prediksi semakin mengikuti pola aktual.

Mahasiswa diminta mencatat nilai RMSE dan Pearson dari seluruh model untuk dianalisis.



```
# Model Evaluation

# print out the RMSE , pearson correlation coefficient for MLP and KNN
# low RMSE is better
# high pearson correlation coefficient is better.
print('_____')
print('MLP')
print('RMSE : %.3f' %rmse_mlp)
print('Pearson correlation coefficient: %.3f' %corr_mlp)
print('_____')
print('KNN')
print('RMSE : %.3f' %rmse_knn)
print('Pearson correlation coefficient: %.3f' %corr_knn)
print('_____')
print('DT')
print('RMSE : %.3f' %rmse_dt)
print('Pearson correlation coefficient: %.3f' %corr_dt)
print('_____')
print('RF')
print('RMSE : %.3f' %rmse_rf)
print('Pearson correlation coefficient: %.3f' %corr_rf)
```

Gambar 5.4.16 Model Evaluation

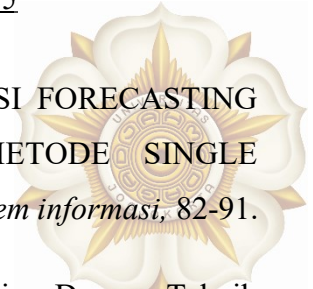
5.5 TUGAS DAN ANALISIS

1. Dengan menggunakan dataset ini <https://www.kaggle.com/datasets/timoboz/tesla-stock-data-from-2010-to-2020/data>
2. Buatlah model forecasting untuk memprediksi target (close) 2 hari kedepan, dengan menggunakan data close 7 hari sebelumnya!
3. Buatlah model forecasting untuk memprediksi target (close) 2 hari kedepan, dengan menggunakan data close dan open 7 hari sebelumnya!

5.6 REFERENSI

Benesty, J., Chen, J., Huang, Y., & Cohen, I. (2009). Pearson Correlation Coefficient. Dalam *Noise Reduction in Speech Processing* (pp. 1–4). Springer. https://doi.org/10.1007/978-3-642-00296-0_5

Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3), 1247–1250. <https://doi.org/10.5194/gmd-7-1247-2014>



Maysofa, R., Syaliman, I., & Sapriadi, D. (2023). IMPLEMENTASI FORECASTING PADA PENJUALAN INAURA HAIR CARE DENGAN METODE SINGLE EXPONENTIAL SMOOTHING. *Jurnal Testing dan Implementasi Sistem informasi*, 82-91.

Retnoningsih, E., & Pramudita, A. (2020). Mengenal Machine Learning Dengan Teknik Supervised dan Unsupervised Learning Menggunakan Python. *BINA INSANI ICT JOURNAL*, 56-165.

Santoso, A., Abijono, H., & Anggreini, D. (2021). ALGORITMA SUPERVISED LEARNING DAN UNSUPERVISED LEARNING DALAM PENGOLAHAN DATA. *Jurnal Teknologi Terapan*, 315-318.

Vimala, T., & Nugroho, S. (2022). FORECASTING PENJUALAN OBAT MENGGUNAKAN METODE SINGLE, DOUBLE, DAN TRIPLE EXPONENTIAL SMOOTHING (STUDI KASUS : APOTEK MANDIRI MEDIKA). *Jurnal Penerapan Teknologi Informasi dan Komunikasi*, 90-99.



PERTEMUAN VI

CLUSTERING

6.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep dasar *unsupervised learning* dan perannya dalam *machine learning*.
- B. Mahasiswa mampu menerapkan berbagai teknik *clustering* seperti K-Means, K-Medoids, dan DBSCAN untuk mengelompokkan data berdasarkan karakteristik tertentu.
- C. Mahasiswa dapat membandingkan kualitas hasil clustering menggunakan metrik evaluasi seperti *Inertia* dan *Silhouette Score*.
- D. Mahasiswa memahami konsep *Association Rule* dan mampu menerapkannya untuk menemukan pola hubungan antar-item dalam dataset.

6.2 DASAR TEORI

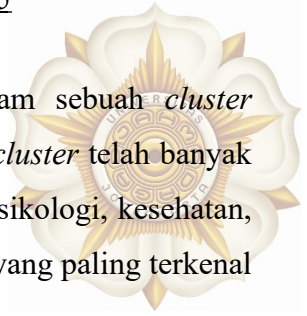
A. Unsupervised Learning

Unsupervised Learning merupakan salah satu pembelajaran mesin dimana model harus menemukan pola dari data tanpa bantuan label eksternal. Pembelajaran ini mengaplikasikan algoritma untuk menganalisis dan menemukan pola dalam data tanpa intervensi atau bantuan manusia. Fokusnya adalah eksplorasi data dan pengelompokan menggunakan algoritma seperti K-Means, *Artificial Neural Network* (ANN), dan Gaussian Mixture Model (GMM) yang menawarkan fleksibilitas yang lebih besar meski dengan akurasi yang biasanya lebih rendah (Nurhalizah & Ardianto, 2024).

Algoritma ini digunakan dalam pendeteksian pola dan pemodelan deskriptif untuk membentuk dasar algoritma untuk menemukan model yang tepat. Algoritma ini digunakan untuk clustering dan aturan asosiasi. Keuntungan *unsupervised learning* adalah algoritma lebih fleksibel untuk mencari pola yang mungkin belum diketahui sebelumnya karena tidak bergantung pada label (Wijoyo, et al., 2024).

B. Clustering

Clustering atau klasterisasi merupakan suatu teknik atau metode untuk mengelompokkan data. *Clustering* adalah salah satu alat yang penting dalam pengolahan data statistik untuk melakukan analisis data. Analisis *cluster* merupakan seperangkat



metode yang digunakan untuk mengelompokkan objek ke dalam sebuah *cluster* berdasarkan informasi yang ditemukan pada data. Saat ini analisis *cluster* telah banyak digunakan di berbagai bidang ilmu pengetahuan seperti ekonomi, psikologi, kesehatan, social masyarakat dan kependudukan. Salah satu metode clustering yang paling terkenal adalah K-Means (Afidaha & Masrukan, 2023).

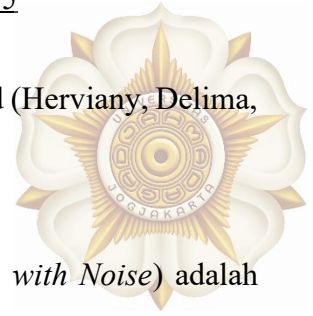
C. K-Means

Algoritma K-Means merupakan algoritma non hirarki yang berasal dari metode data clustering. Algoritma K-Means dimulai dengan pembentukan partisi klaster di awal kemudian secara interaktif partisi cluster ini diperbaiki hingga tidak terjadi perubahan yang signifikan pada partisi cluster. K-Means ini mempartisi data ke dalam kelompok sehingga data berkarakteristik sama dimasukkan kedalam satu kelompok yang sama dan data yang berkarakteristik berbeda dikelompokkan kedalam kelompok yang lain. Prinsip utama dari teknik ini adalah menyusun K buah partisi/pusat massa (centroid)/rata-rata (mean) dari sekumpulan data. Adapun tujuan dari pengelompokan data ini adalah untuk meminimalkan fungsi objektif yang diset dalam proses pengelompokan yang pada umumnya berusaha meminimalkan variasi dalam suatu kelompok dan memaksimalkan variasi antar kelompok (Sulistiyawati & Supriyanto, 2021).

D. K-Medoids

Algoritma K-Medoids berperan untuk menemukan Medoids dalam suatu cluster yang menjadi titik pusat cluster. K-Medoids lebih kuat daripada K-Means seperti pada K-Medoids kami mendapatkan K selaku objek representatif diminimalkan hasil ketidaksamaan objek data, sedangkan K-Means menggunakan hasil kuadrat jarak Euclidean pada data benda. Dan metrik jarak ini mengurangi data yang noise dan outliers. Ini adalah teknik berbasis objek yang representatif. Dalam metode ini kita memilih objek sebenarnya untuk merepresentasikan cluster daripada mengambil nilai rata-rata objek dalam cluster sebagai titik referensi.

Algoritma K-Medoids secara komputasi lebih sulit daripada K-Means karena menghitung medoid menggunakan frekuensi kejadian. K-Medoids memiliki potensi penting karakteristik pusat mana yang berada di antara data tunjuk sendiri. Algoritma baru diusulkan untuk pengelompokan K-Medoids yang berjalan seperti K-Means



algoritma dan menguji beberapa metode untuk memilih awal medoid (Herviany, Delima, Nurhidayah, & Kasini, 2021).

E. DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) adalah algoritma *clustering* berbasis kepadatan yang mengelompokkan titik data berdasarkan seberapa padat area di sekitarnya. Algoritma ini menggunakan dua parameter, yaitu epsilon (ϵ) sebagai radius pencarian dan minPts sebagai jumlah minimum titik agar sebuah titik dianggap core point. DBSCAN unggul dalam mendeteksi cluster dengan bentuk yang tidak beraturan dan dapat mengidentifikasi noise atau outlier tanpa membutuhkan jumlah cluster di awal (Ester et al., 1996; Han, Kamber, & Pei, 2011).

F. Association Rule

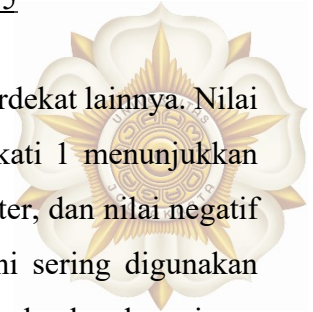
Association Rule Mining merupakan teknik dalam *data mining* untuk menemukan pola atau hubungan antar-item dalam dataset transaksi. Aturan asosiasi biasanya direpresentasikan dalam bentuk $A \rightarrow B$, yang berarti bahwa kemunculan item A sering diikuti oleh kemunculan item B. Penilaian kualitas aturan dilakukan menggunakan *support*, *confidence*, dan *lift*, yang masing-masing mengukur frekuensi, probabilitas keterkaitan, serta kekuatan hubungan antar-item dalam dataset. Metode ini umum digunakan dalam *market basket analysis* untuk mengungkap pola perilaku konsumen (Agrawal, Imieliński, & Swami, 1993; Han, Kamber, & Pei, 2011).

G. Inertia

Inertia adalah metrik evaluasi pada algoritma K-Means yang mengukur tingkat kompaknya sebuah cluster. Nilai inertia dihitung sebagai jumlah kuadrat jarak seluruh data terhadap centroid cluster-nya. Nilai inertia yang lebih rendah menunjukkan cluster yang lebih kompak dan representatif. Metrik ini sangat sering digunakan dalam *Elbow Method* untuk menentukan jumlah cluster optimal, meskipun hanya relevan untuk metode clustering berbasis centroid seperti K-Means (Bishop, 2006; scikit-learn documentation).

H. Silhouette Score

Silhouette Score merupakan metode evaluasi yang mengukur seberapa baik suatu data ditempatkan pada cluster tertentu dengan mempertimbangkan kedekatan data



tersebut terhadap cluster-nya sendiri dan jaraknya terhadap cluster terdekat lainnya. Nilai silhouette berada pada rentang -1 hingga 1 , di mana nilai mendekati 1 menunjukkan pemisahan cluster yang baik, nilai 0 menandakan overlap antar-cluster, dan nilai negatif menunjukkan kemungkinan pengelompokan yang salah. Metrik ini sering digunakan untuk membandingkan beberapa algoritma clustering dalam sebuah eksperimen (Rousseeuw, 1987; Tan, Steinbach, & Kumar, 2019).

6.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

6.4 LANGKAH PERCOBAAN

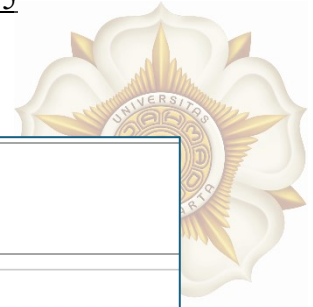
1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Colab dipilih karena mendukung berbagai *library machine learning* yang diperlukan, seperti *pandas*, *numpy*, *scikit-learn*, *matplotlib*, hingga *sklearn-extra* untuk K-Medoids. Selain itu, Colab menyediakan GPU/CPU gratis sehingga proses komputasi *clustering* dapat berjalan lebih cepat tanpa membebani perangkat mahasiswa.

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh library yang dibutuhkan untuk melakukan preprocessing, visualisasi, dan proses clustering. Library utama meliputi:

1. *pandas* dan *numpy* untuk manipulasi data
2. *StandardScaler* untuk standarisasi
3. *K-Means* dan *K-Medoids* untuk algoritma clustering
4. *matplotlib* dan *seaborn* untuk visualisasi



5. silhouette_score untuk evaluasi hasil clustering, dsb

```
!pip install kneed
!pip install scikit-learn-extra
!pip install numpy==1.26.4

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
from sklearn.metrics import silhouette_score
from kneed import KneeLocator, DataGenerator
from sklearn.metrics import silhouette_samples, silhouette_score
import plotly.express as px
from sklearn.preprocessing import LabelEncoder
```

Gambar 6.4.1 Impor Pustaka

3. Read dan Load Dataset Information

Mahasiswa kemudian memuat dataset Credit Card Dataset for Clustering ke dalam DataFrame. Dataset ini berisi data perilaku transaksi pelanggan kartu kredit, seperti jumlah saldo, total pembayaran, jumlah pembelian, penarikan cash advance, dan pembayaran minimum. Pada tahap ini mahasiswa:

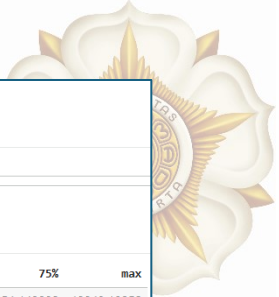
1. Mengimpor dataset dari sumber online atau drive,
2. Menampilkan beberapa baris awal untuk memastikan dataset terbaca dengan benar

```
# load dataset
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/CC%20GENERAL.csv', delimiter=',')
df.head()
```

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY	ONEOFF_PURCH
0	C10001	40.900749	0.818182	95.40	0.00	95.4	0.000000	0.166667	
1	C10002	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	0.000000	
2	C10003	2495.148862	1.000000	773.17	773.17	0.0	0.000000	1.000000	
3	C10004	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.083333	
4	C10005	817.714335	1.000000	16.00	16.00	0.0	0.000000	0.083333	

Gambar 6.4.2 Membaca dataset

3. Melihat struktur dataset melalui fungsi info() dan describe() untuk memahami jenis data dan distribusinya.



```
# size dataset
df.shape
(8950, 18)

# ringkasan statistik
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
BALANCE	8950.0	1564.474828	2081.531879	0.000000	128.281915	873.385231	2054.140036	19043.13856
BALANCE_FREQUENCY	8950.0	0.877271	0.236904	0.000000	0.888889	1.000000	1.000000	1.000000
PURCHASES	8950.0	1003.204834	2136.634782	0.000000	39.635000	361.280000	1110.130000	49039.57000
ONEOFF_PURCHASES	8950.0	592.437371	1659.887917	0.000000	0.000000	38.000000	577.405000	40761.25000
INSTALLMENTS_PURCHASES	8950.0	411.067645	904.338115	0.000000	0.000000	89.000000	468.637500	22500.00000
CASH_ADVANCE	8950.0	978.871112	2097.163877	0.000000	0.000000	0.000000	1113.821139	47137.21176

Gambar 6.4.3 Melihat struktur Dataset

```
# korelasi antar kolom dataset
df.corr(numeric_only=True)
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES
BALANCE	1.000000	0.322412	0.181261	0.164350	
BALANCE_FREQUENCY	0.322412	1.000000	0.133674	0.104323	
PURCHASES	0.181261	0.133674	1.000000	0.916845	
ONEOFF_PURCHASES	0.164350	0.104323	0.916845	1.000000	
INSTALLMENTS_PURCHASES	0.126469	0.124292	0.679896	0.330622	
CASH_ADVANCE	0.496692	0.099388	-0.051474	-0.031326	
PURCHASES_FREQUENCY	-0.077944	0.229715	0.393017	0.264937	
ONEOFF_PURCHASES_FREQUENCY	0.073166	0.202415	0.498430	0.524891	

Gambar 6.4.4 Melihat korelasi antar kolom

4. Data Cleaning

Dataset kemudian diperiksa untuk memastikan tidak ada data hilang atau nilai yang tidak sesuai. Mahasiswa:

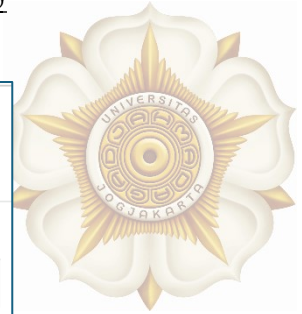
1. Menghapus kolom CUST_ID karena bersifat identitas dan tidak relevan bagi proses clustering

```
df_new = df.drop('CUST_ID', axis = 1)
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	

Gambar 6.4.5 Menghapus kolom CUST_ID

2. Mengisi (*impute*) nilai kosong pada fitur seperti MINIMUM_PAYMENTS dan CREDIT_LIMIT menggunakan nilai median agar tidak mengganggu hasil clustering



```
# check missing values
```

```
df_new.isnull().sum()
```

...	0
BALANCE	0
BALANCE_FREQUENCY	0
PURCHASES	0
ONEOFF_PURCHASES	0

Gambar 6.4.6 Menghitung jumlah kolom null

```
# imputasi
```

```
df_new['MINIMUM_PAYMENTS'].fillna(df_new['MINIMUM_PAYMENTS'].median(), inplace=True)
```

```
df_new['CREDIT_LIMIT'].fillna(df_new['CREDIT_LIMIT'].median(), inplace=True)
```

/tmp/ipython-input-1602066474.py:3: FutureWarning:

A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate result is a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'.

Gambar 6.4.7 Imputasi Data

3. Melakukan pengecekan ulang untuk memastikan seluruh nilai hilang telah teratasi

```
# cek hasil imputasi
```

```
df_new.isnull().sum()
```

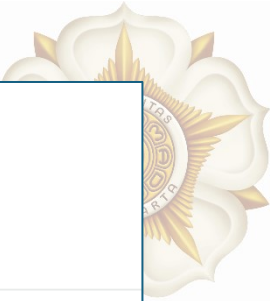
...	0
BALANCE	0
BALANCE_FREQUENCY	0
PURCHASES	0
ONEOFF_PURCHASES	0
INSTALLMENTS_PURCHASES	0

Gambar 6.4.8 Cek hasil Imputasi Data

Pembersihan data penting dilakukan karena nilai kosong dapat menyebabkan algoritma clustering gagal konvergen atau menghasilkan cluster yang tidak akurat.

5. Standarisasi Data

Sebelum melakukan *clustering*, mahasiswa wajib melakukan standarisasi menggunakan *StandardScaler*. Hal ini dilakukan karena sebagian fitur seperti “BALANCE” atau “PURCHASES” memiliki skala yang jauh lebih besar dibanding fitur lain. Dengan standarisasi, setiap fitur akan memiliki distribusi dengan mean = 0 dan standar deviasi = 1 sehingga algoritma clustering tidak bias terhadap fitur berskala besar.



```
# scaling

X = df_new.astype(float).values
scaler = StandardScaler().fit(X)
X_new= scaler.transform(X)
X_new

array([[ -0.73198937, -0.24943448, -0.42489974, ..., -0.3024    ,
        -0.52555097,  0.36067954],
       [ 0.78696085,  0.13432467, -0.46955188, ...,  0.09749953,
        0.2342269 ,  0.36067954],
       [ 0.44713513,  0.51808382, -0.10766823, ..., -0.0932934 ,
        -0.52555097,  0.36067954],
       ...,
       [-0.7403981 , -0.18547673, -0.40196519, ..., -0.32687479,
        0.32919999, -4.12276757],
       [-0.74517423, -0.18547673, -0.46955188, ..., -0.33830497,
        0.32919999, -4.12276757],
       [-0.57257511, -0.88903307,  0.04214581, ..., -0.3243581 ,
        -0.52555097, -4.12276757]])
```

Gambar 6.4.9 Data Scaling

6. Menentukan Jumlah Cluster (Elbow Method & Silhoutte Score)

Untuk menentukan jumlah cluster optimal, mahasiswa melakukan dua tahap evaluasi:

1. Elbow Method

Mahasiswa menjalankan K dengan jumlah cluster 1 hingga 10 dan mencatat nilai inertia. Nilai inertia kemudian divisualisasikan untuk menemukan titik “siku” yang menunjukkan penurunan inertia tidak lagi signifikan.

a. K-Means

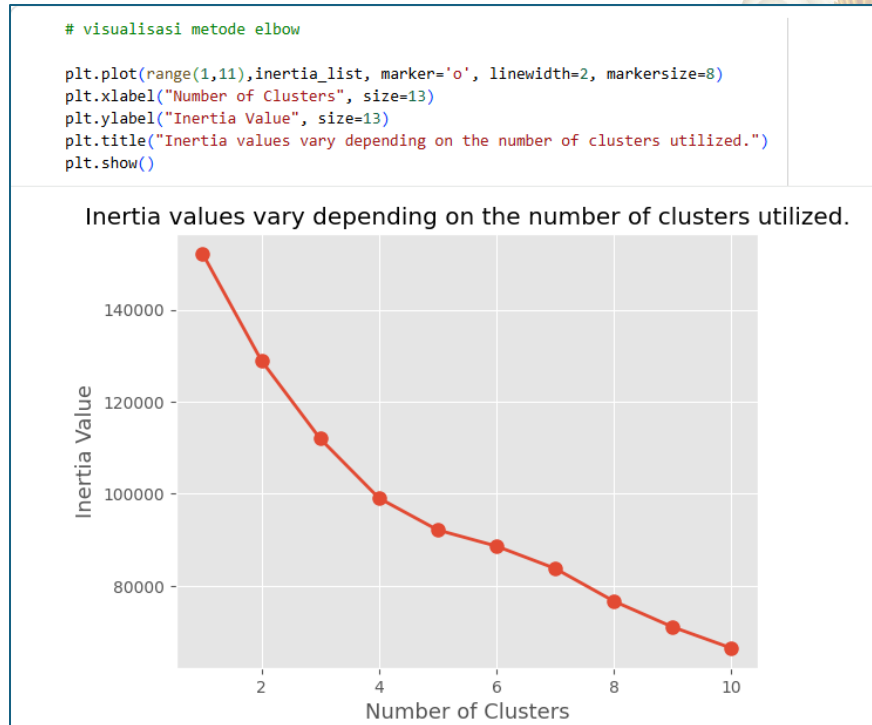
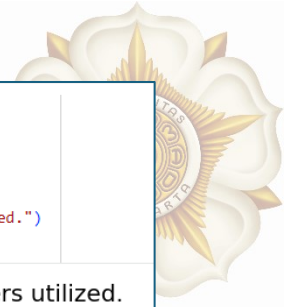
```
from sklearn.cluster import KMeans

# metode elbow untuk menentukan jumlah k

inertia_list = []
for num_clusters in range(1, 11):
    kmeans_model = KMeans(n_clusters=num_clusters)
    kmeans_model.fit(X_new)
    inertia_list.append(kmeans_model.inertia_)
    print("For n_clusters = {}, inertia value is {}".format(num_clusters, kmeans_model.inertia_))

... For n_clusters = 1, inertia value is 152150.00000000004)
For n_clusters = 2, inertia value is 128936.94462472606)
For n_clusters = 3, inertia value is 111973.97335474695)
For n_clusters = 4, inertia value is 99062.37980412853)
For n_clusters = 5, inertia value is 92131.46554452999)
For n_clusters = 6, inertia value is 88621.00610679774)
For n_clusters = 7, inertia value is 83766.7010851524)
For n_clusters = 8, inertia value is 76657.09288658875)
For n_clusters = 9, inertia value is 71051.78747876118)
For n_clusters = 10, inertia value is 66459.87103455774)
```

Gambar 6.4.10 Elbow Method



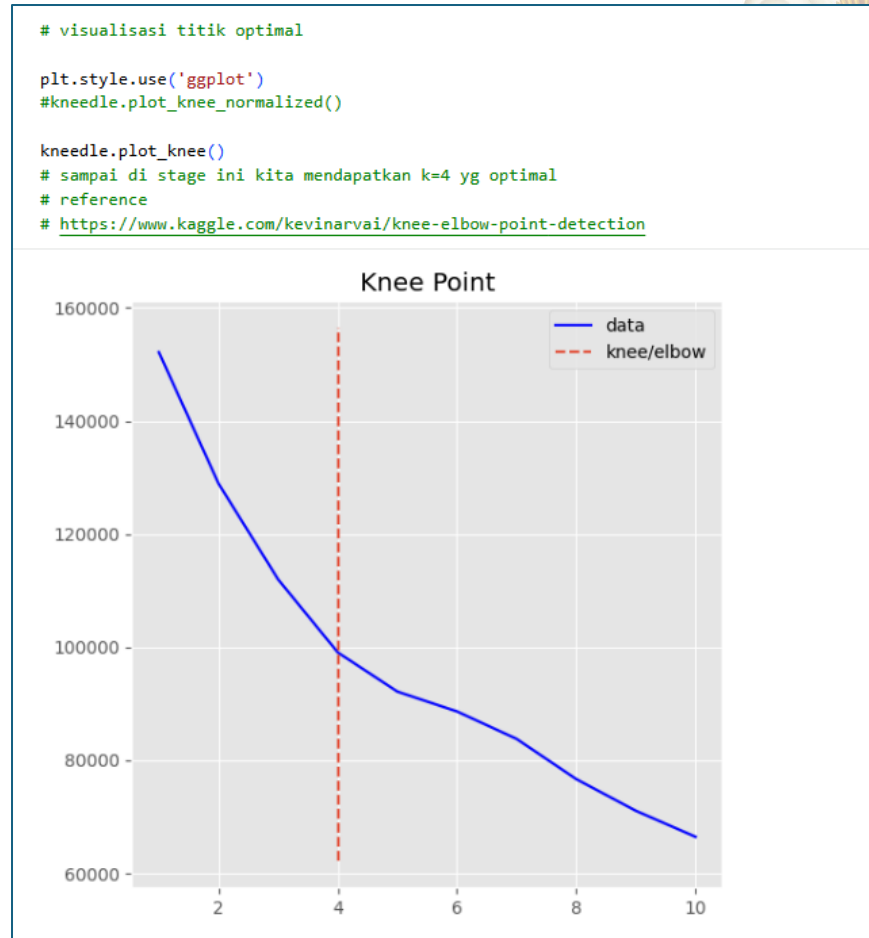
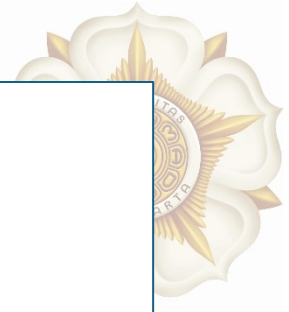
Gambar 6.4.11 Visualisasi Elbow Method

```
# menentukan titik optimal menggunakan kneed

kneedle = KneedleLocator(range(1,11),inertia_list, S=1.0, curve='convex', direction='decreasing')
print(round(kneedle.knee, 3))
print(round(kneedle.elbow, 3))
```

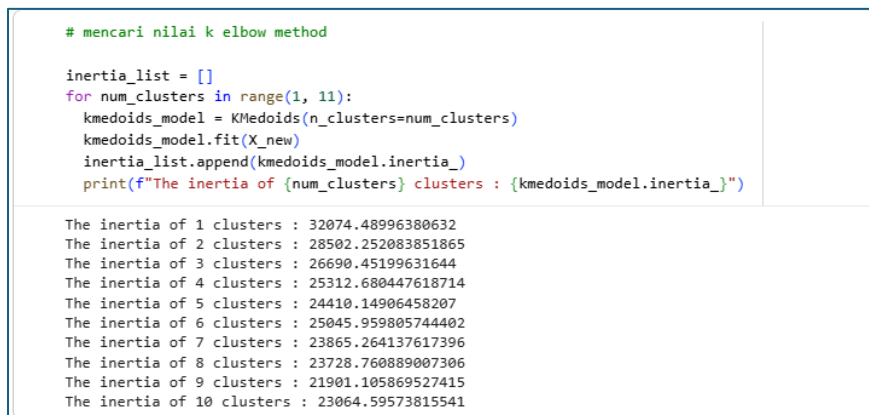
4
4

Gambar 6.4.12 Menentukan titik optimal menggunakan kneed

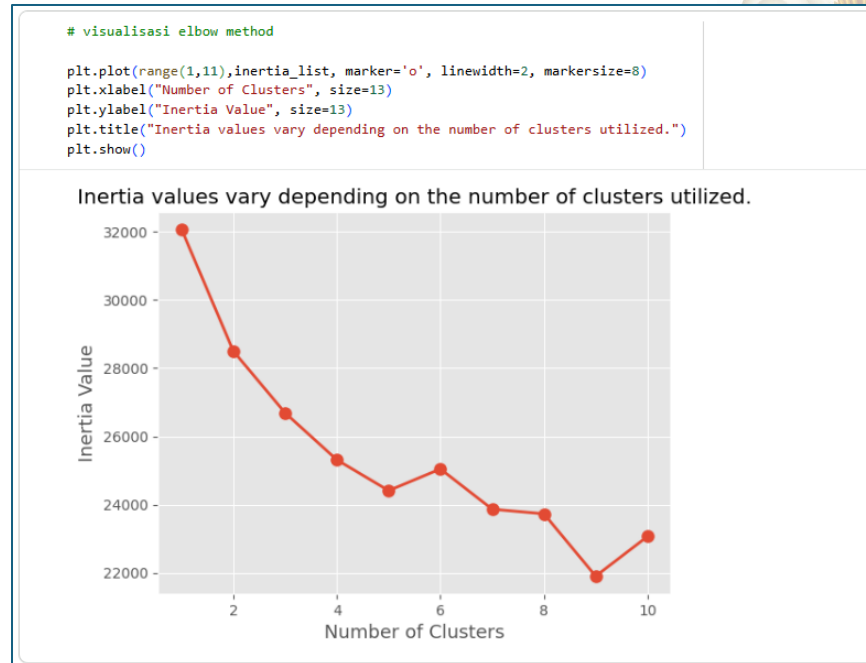
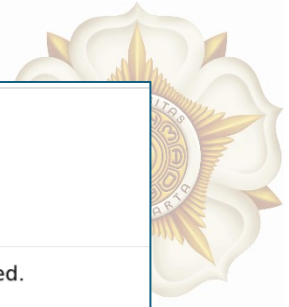


Gambar 6.4.13 Visualisasi Knee Point

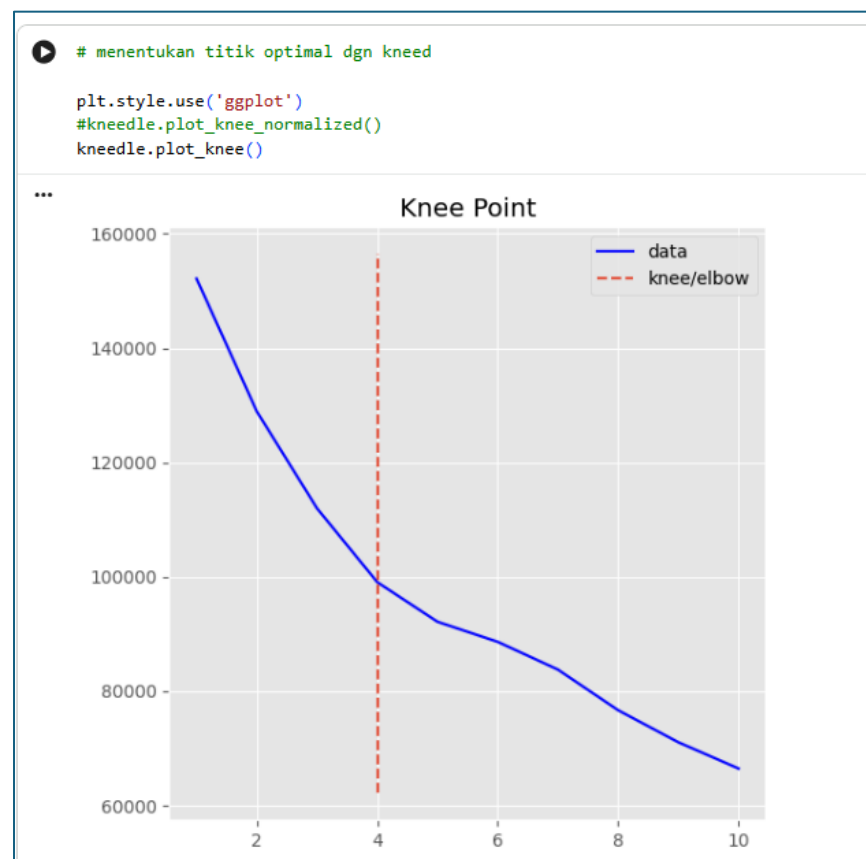
b. K-Medoids



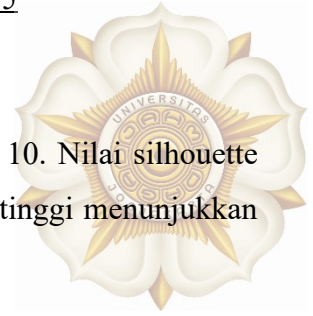
Gambar 6.4.14 Mencari nilai k elbow method



Gambar 6.4.15 Visualisasi elbow method



Gambar 6.4.16 Visualisasi Knee Point



2. Silhouette Score

Mahasiswa menghitung nilai silhouette untuk cluster 2 hingga 10. Nilai silhouette membantu menilai kualitas pemisahan cluster, di mana nilai tertinggi menunjukkan cluster yang paling terpisah dengan baik.

a. K-Means

```
# menentukan silhouette score

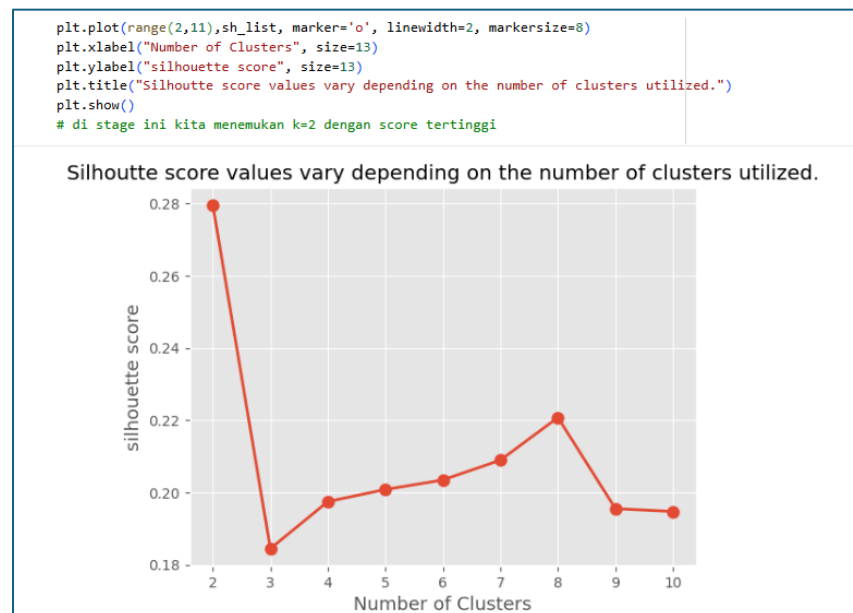
from sklearn.metrics import silhouette_samples, silhouette_score

sh_list = []
for num_clusters in range(2, 11):
    kmeans = KMeans(n_clusters=num_clusters)
    cluster_labels = kmeans.fit_predict(X_new)

    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))

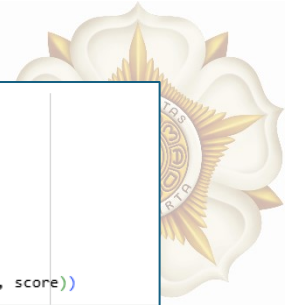
For n_clusters = 2, silhouette score is 0.27953987242960154)
For n_clusters = 3, silhouette score is 0.18447069544775743)
For n_clusters = 4, silhouette score is 0.19749974009976254)
For n_clusters = 5, silhouette score is 0.20083224723760784)
For n_clusters = 6, silhouette score is 0.2034742960849018)
For n_clusters = 7, silhouette score is 0.20896235614091754)
For n_clusters = 8, silhouette score is 0.22075625282182323)
For n_clusters = 9, silhouette score is 0.195520101529126)
For n_clusters = 10, silhouette score is 0.1947488460098652)
```

Gambar 6.4.17 Menentukan silhouette score



Gambar 6.4.18 Visualisasi Solhouette score

b. K-Medoids

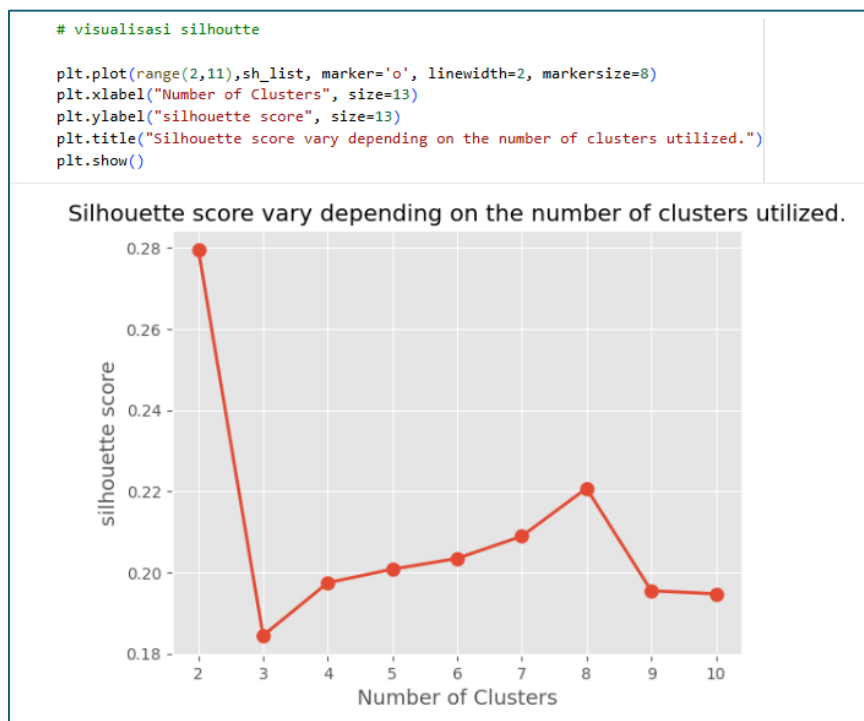


```
from sklearn.metrics import silhouette_samples, silhouette_score
sh_list = []

for num_clusters in range(2, 11):
    kmedoids = KMedoids(n_clusters=num_clusters)
    cluster_labels = kmedoids.fit_predict(X_new)
    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))

For n_clusters = 2, silhouette score is 0.1945165537378692)
For n_clusters = 3, silhouette score is 0.16024040458444103)
For n_clusters = 4, silhouette score is 0.13738603460433335)
For n_clusters = 5, silhouette score is 0.14859330798827058)
For n_clusters = 6, silhouette score is 0.07497117420304265)
For n_clusters = 7, silhouette score is 0.051193591100429474)
For n_clusters = 8, silhouette score is 0.03912406632314492)
For n_clusters = 9, silhouette score is 0.08491760468793152)
For n_clusters = 10, silhouette score is 0.03469231089500277)
```

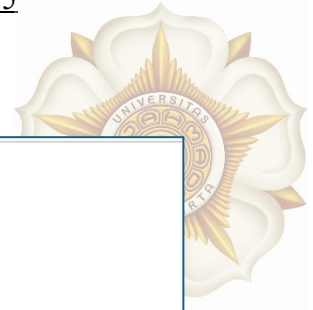
Gambar 6.4.19 Kmedoid



Gambar 6.4.20 Visualisasi Kmedoid

7. Menerapkan Algoritma K-Means

Setelah jumlah cluster ditentukan, mahasiswa menjalankan algoritma K-Means pada data yang telah distandarisasi. Tahap ini meliputi:



1. Melatih model K-Means sesuai jumlah cluster optimal

```
# algoritma kmeans menggunakan nilai silhoutte

k_means = KMeans(n_clusters = 2, random_state = 42)
k_means.fit(X_new)
labels = k_means.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURC
0	40.900749	0.818182	95.40	0.00	
1	3202.467416	0.909091	0.00	0.00	
2	2495.148862	1.000000	773.17	773.17	
3	1666.670542	0.636364	1499.00	1499.00	
4	817.714335	1.000000	16.00	16.00	

Gambar 6.4.21 Training K-Mean sesuai kluster optimal

2. Menghasilkan label cluster untuk setiap data dan menambahkan label cluster ke DataFrame

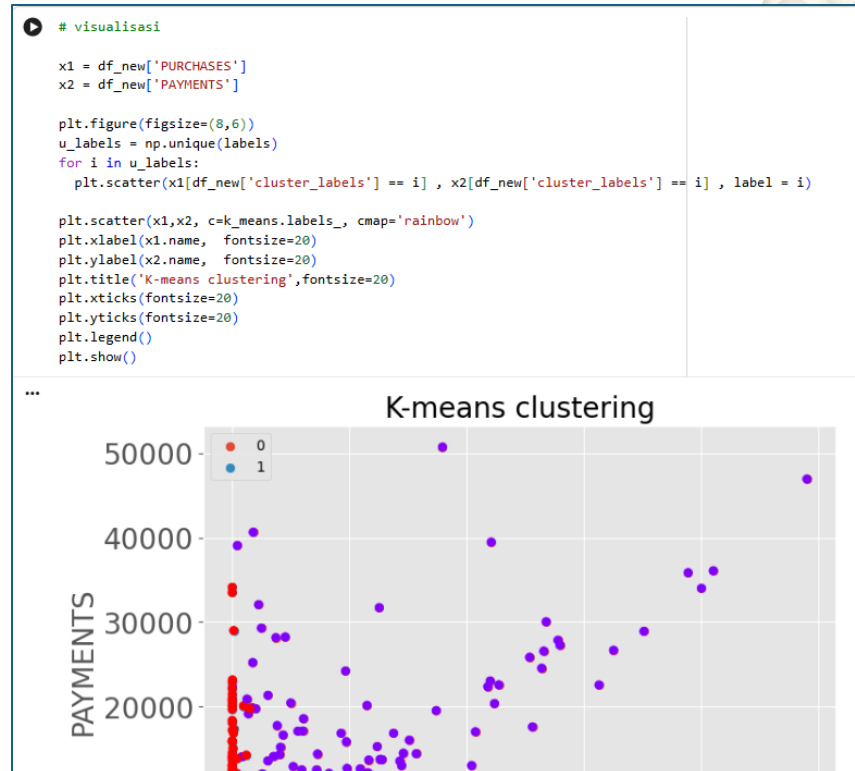
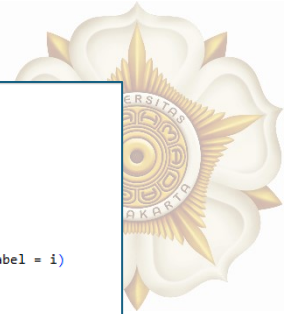
```
# cek centroids

centroids = k_means.cluster_centers_
centroids
```

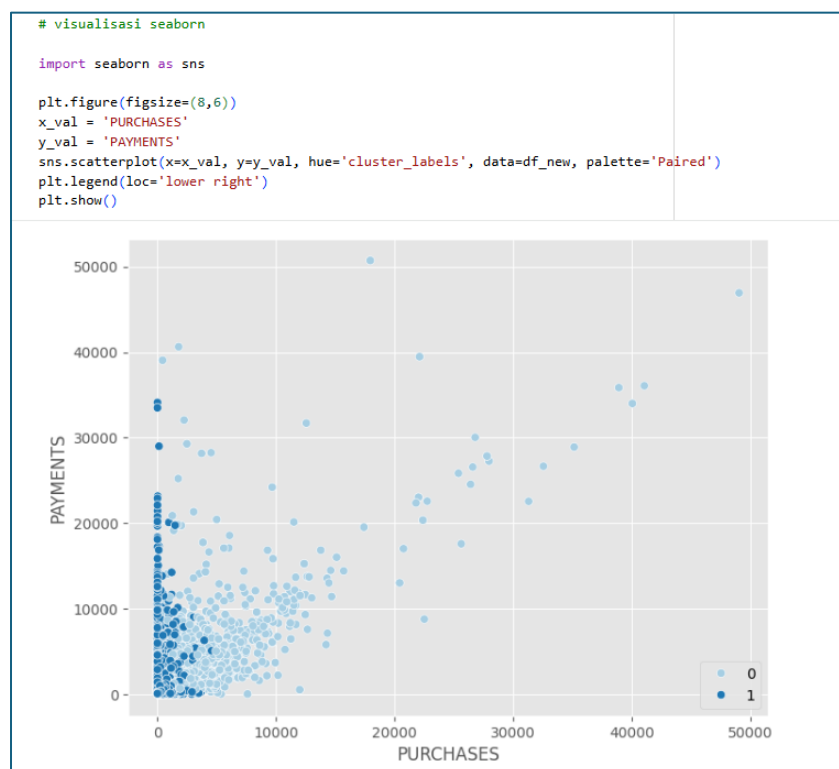
```
array([[ 0.28073957,  0.43247256,  1.10795859,  0.89576982,  0.97358346,
        -0.10307996,  1.1156293 ,  1.29514356,  0.87789229, -0.26387424,
        -0.14839813,  1.26760651,  0.74056992,  0.64489838,  0.15037657,
         0.44987573,  0.27971163],
       [-0.07339984, -0.1130707 , -0.28967769, -0.23420057, -0.25454508,
         0.02695043, -0.29168321, -0.33861752, -0.22952646,  0.06899037,
         0.03879895, -0.33141791, -0.19362328, -0.16860979, -0.03931621,
        -0.11762079, -0.07313109]])
```

Gambar 6.4.22 Cek Centroids

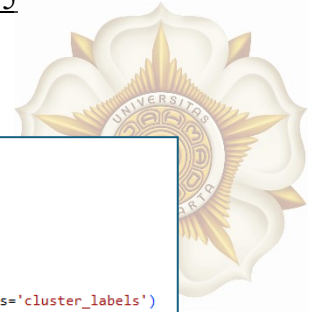
3. Melakukan visualisasi 2D/3D menggunakan fitur-fitur seperti PURCHASES, PAYMENTS, dan BALANCE.



Gambar 6.4.23 Visualisasi K Means Clustering



Gambar 6.4.24 Visualisasi klustering



```
# visualisasi plotly

import plotly.express as px

x_val = 'PURCHASES'
y_val = 'PAYMENTS'
z_val = 'BALANCE'
fig = px.scatter_3d(df_new, x=x_val, y=y_val, z=z_val, color='cluster_labels', labels='cluster_labels')
fig.show()
```

Gambar 6.4.25 Visualisasi Scatter

8. Menerapkan Algoritma K-Medoids

Mahasiswa kemudian mengulangi proses di atas menggunakan K-Medoids, algoritma berbasis medoid yang lebih tahan terhadap outlier dibandingkan K-Means. Langkahnya meliputi:

1. Evaluasi jumlah cluster menggunakan inertia

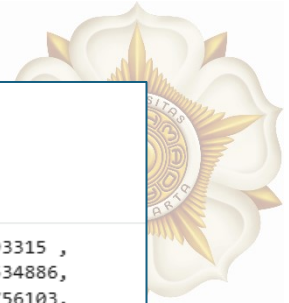
```
# algoritma kmedoids menggunakan elbow

k_medoids = KMedoids(n_clusters = 4, random_state = 42)
k_medoids.fit(X_new)
labels = k_medoids.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE
0	40.900749	0.818182	95.40	0.00	95.4	0.0
1	3202.467416	0.909091	0.00	0.00	0.0	6442.9
2	2495.148862	1.000000	773.17	773.17	0.0	0.0
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.7
4	817.714335	1.000000	16.00	16.00	0.0	0.0

Gambar 6.4.26 Evaluasi jumlah cluster menggunakan inertia

2. Pelatihan model K-Medoids dan menambahkan label cluster ke DataFrame



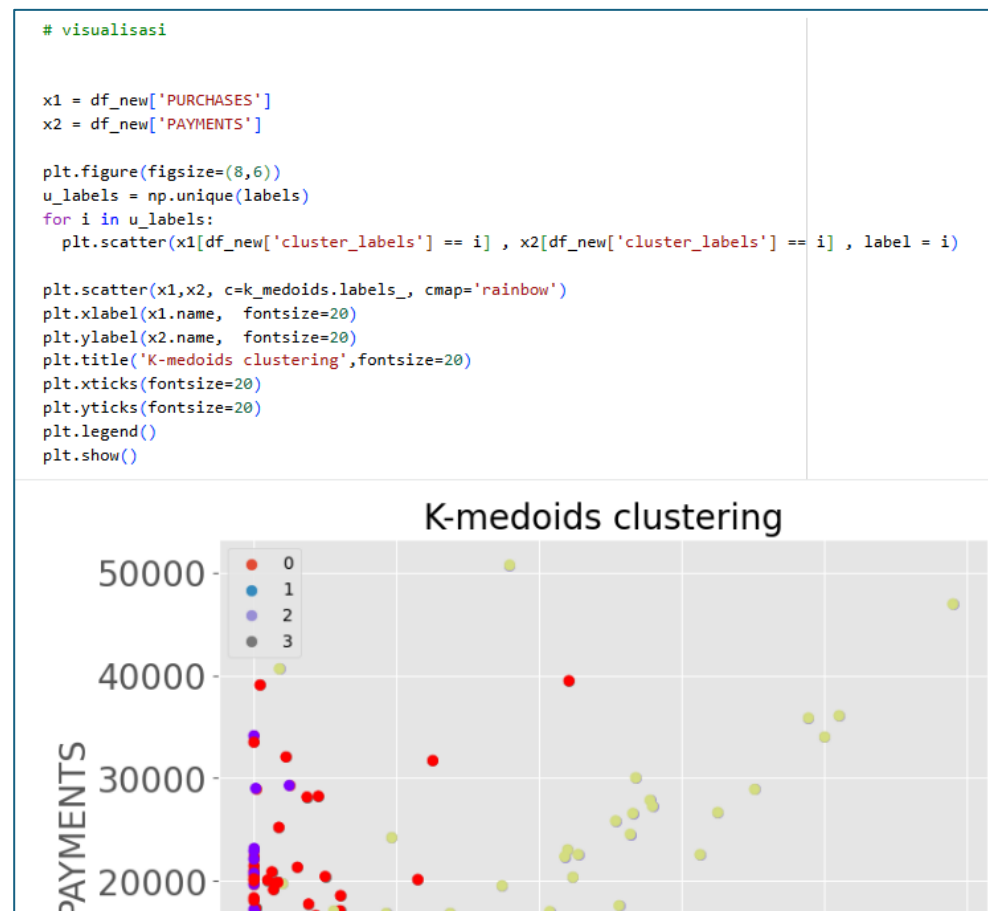
```
# cek centroids

centroids = k_medoids.cluster_centers_
centroids

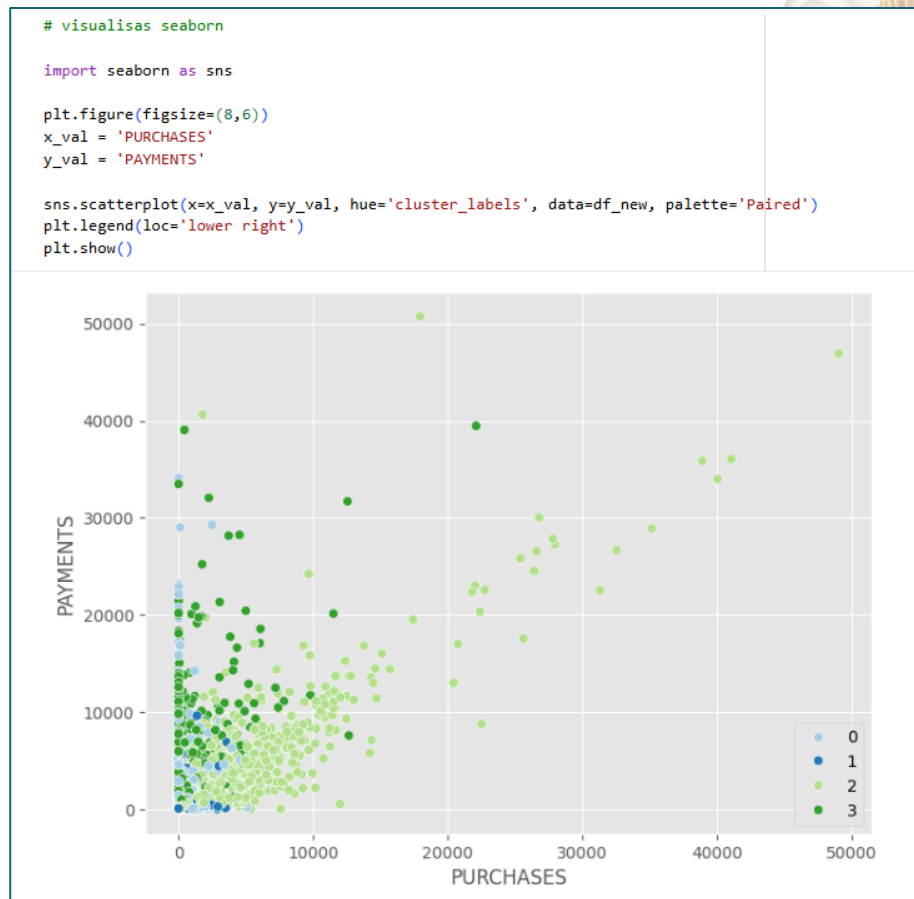
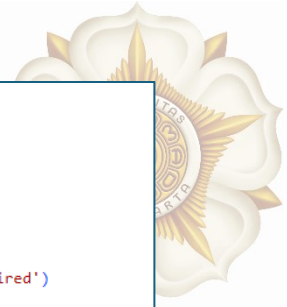
array([[ -0.73017008, -1.78447531, -0.15174467,  0.01660604, -0.3893315 ,
        -0.46678555, -0.39122513, -0.39931927, -0.49762862, -0.67534886,
        -0.47606982, -0.39063931, -0.20456068, -0.35069111, -0.32756103,
         0.50015018,  0.36067954],
       [ -0.04321221,  0.51808382, -0.41993839, -0.29307072, -0.45457623,
        -0.44947139, -1.01412545, -0.39931927, -0.91699519, -0.25891333,
        -0.18299798, -0.51133325, -0.41069279, -0.47801089, -0.17593333,
        -0.52555097,  0.36067954],
       [ -0.65121444,  0.51808382,  0.24224266,  0.03901242,  0.50039651,
        -0.46678555,  1.06221062,  0.15936716,  1.38951716, -0.67534886,
        -0.47606982,  0.37375564, -0.27327139,  0.0131038 , -0.2796588 ,
         0.04428414,  0.36067954],
       [ 1.16410803,  0.51808382, -0.40096824, -0.32982224, -0.3423 ,
         0.80421891, -0.18359002, -0.39931927, -0.2879466 ,  1.8232743 ,
         0.69621752, -0.350408 ,  0.41383563, -0.14385479,  0.17482124,
        -0.52555097,  0.36067954]])
```

Gambar 6.4.27 Pelatihan model K-Medoids

3. Visualisasi hasil clustering untuk membandingkan pola dengan K-Means



Gambar 6.4.28 Visualisasi K-Medoids clustering

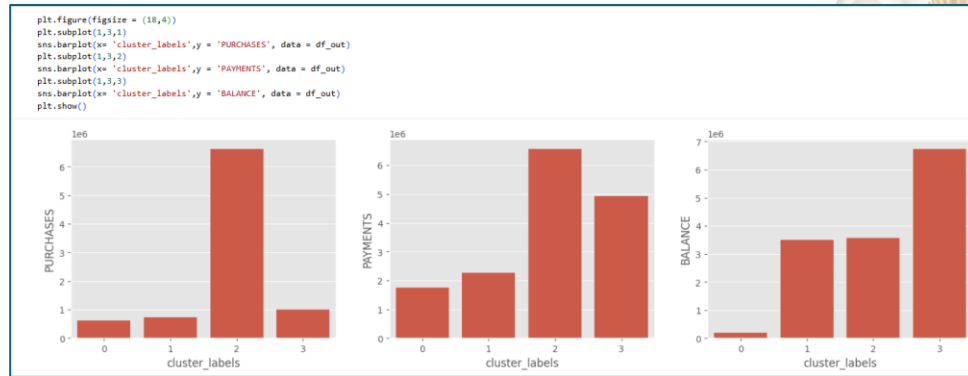


Gambar 6.4.29 Visualisasi Kmedoid Scatter

9. Analisis dan Interpretasi Cluster

Setelah kedua algoritma dijalankan, mahasiswa melakukan analisis pada setiap cluster dengan melihat fitur-fitur seperti:

1. rata-rata saldo
2. total pembelian
3. total pembayaran
4. jumlah transaksi, dan lainnya.



Gambar 6.4.30 Interpretasi Cluster

Mahasiswa kemudian membuat ringkasan atau profil cluster yang menjelaskan karakteristik setiap kelompok, misalnya:

1. cluster dengan pelanggan bertransaksi tinggi
2. cluster dengan pembayaran minimum rendah
3. cluster dengan penggunaan kartu kredit intensif

Analisis ini bertujuan membantu mahasiswa memahami bagaimana algoritma clustering digunakan untuk segmentasi pelanggan.



Gambar 6.4.31 Analisis Cluster

6.5 TUGAS DAN ANALISIS

1. Dataset Preparation

- Download dataset : <https://www.kaggle.com/datasets/arjunbhasin2013/ccdata>
- Upload di google drive atau github masing-masing
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dataset ini



2. Menggunakan dataset lain, silahkan mengerjakan clustering menggunakan K-Means dan K-Medoids. Dataset Air Traffic Passengers Statistics:

https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkr-6vcg/about_data

6.6 REFERENSI

Afidaha, M. W., & Masrukan, M. H. (2023). *Penerapan Metode Clustering dengan Algoritma K-Means untuk Pengelompokan Data Migrasi Penduduk Tiap Kecamatan di Kabupaten Rembang*. PRISMA, Prosiding Seminar Nasional Matematika (pp. 729-738). Semarang: Jurusan Matematika, Universitas Negeri Semarang.

Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, 207–216.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 226–231.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.

Herviany, N., Delima, R., Nurhidayah, S., & Kasini, A. (2021). *Perbandingan Algoritma K-Means dan K-Medoids untuk Pengelompokan Daerah Rawan Tanah Longsor di Provinsi Jawa Barat*. MALCOM: Indonesian Journal of Machine Learning and Computer Science, 34-40.

Nurhalizah, S., & Ardianto, D. (2024). *Penerapan Unsupervised Learning dalam Analisis Pola Data*. Jurnal Sains Komputer dan Informatika.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.



Sulistiyawati, A., & Supriyanto, E. (2021). *Pengelompokan Data Menggunakan Metode K-Means untuk Analisis Cluster*. Jurnal Matematika dan Statistika.

Tan, P.-N., Steinbach, M., & Kumar, V. (2019). *Introduction to Data Mining* (2nd ed.). Pearson.

Wijoyo, S., et al. (2024). *Unsupervised Learning and Its Application in Pattern Discovery*. Jurnal Teknologi Informasi dan Sains Data.

scikit-learn. (2024). *Clustering Performance Evaluation – Inertia & Silhouette Score*. Diakses dari dokumentasi resmi scikit-learn.

PERTEMUAN VII

IMBALANCED DATASET



7.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep imbalanced dataset serta permasalahan yang muncul akibat ketidakseimbangan kelas dalam model machine learning
- B. Mahasiswa mampu menerapkan teknik balancing data seperti SMOTE dan ADASYN untuk menyeimbangkan distribusi kelas pada dataset
- C. Mahasiswa memahami konsep dan metode feature selection (filter, wrapper, embedded) untuk memilih fitur yang paling relevan

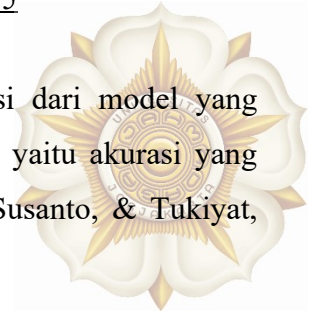
7.2 DASAR TEORI

A. Imbalanced Class

Imbalance class adalah kondisi di mana suatu himpunan data terdapat satu kelas yang memiliki jumlah yang jauh lebih kecil dibanding kelas lainnya. *Imbalance class* menyebabkan kelas minoritas diprediksi secara salah menjadi kelas mayoritas. Selain itu, *imbalance class* mengakibatkan tingkat sensitivitas rendah pada data dengan dua kategori atau binomial. Masalah lain yang dapat muncul yaitu banyaknya data kosong yang dapat menyebabkan beberapa algoritma dalam metode klasifikasi tidak dapat dijalankan. Hal ini mengakibatkan tingkat akurasi rendah dan kualitas data yang buruk saat data diolah. Data dengan *imbalance class* dan banyaknya *missing value* perlu diberikan perlakuan tertentu agar diperoleh metode klasifikasi dengan ukuran ketepatan prediksi yang tinggi. *Imbalance class* dapat diatasi dengan memanipulasi data, memodifikasi algoritma, atau gabungan keduanya (Istiana & Mustafiril, 2023).

B. Confusion Matrix

Confusion Matrix adalah suatu metode yang digunakan untuk melakukan perhitungan akurasi pada konsep data *mining* (Normawati & Prayogi, 2021). Matriks ini diimplementasikan dengan sebuah tabel yang digunakan untuk mengevaluasi kinerja algoritma klasifikasi dalam *machine learning*. Tabel ini membandingkan hasil prediksi model dengan label sebenarnya dari data, sehingga kita bisa melihat sejauh mana model melakukan prediksi dengan benar dan salah. Evaluasi performansi dilakukan untuk



menguji hasil dari klasifikasi dengan mengukur nilai performansi dari model yang telah dibuat. Parameter pengujian yang digunakan untuk evaluasi yaitu akurasi yang perhitungannya diperoleh dari tabel *confusion matrix* (Septiana, Susanto, & Tukiyyat, 2021).

C. Feature Selection

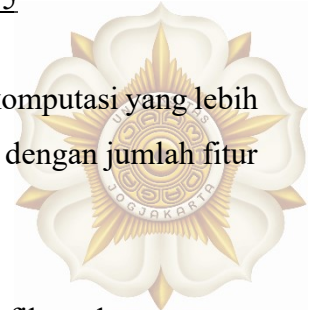
Feature selection merupakan sebuah proses identifikasi subset fitur yang paling relevan pada kumpulan fitur yang berguna untuk meningkatkan kinerja algoritma (Pratama & Kusnawi, 2023). Salah satu faktor penting untuk meningkatkan nilai akurasi klasifikasi adalah seleksi fitur. Apabila dataset berisi fitur yang banyak, maka dimensi ruang menjadi besardantingkat akurasi klasifikasi menjadi rendah. Seleksi fitur digunakan untuk mengurangi jumlah set fitur yang besar menjadi subset fitur yang relatif lebih kecil agar nilai akurasi klasifikasi meningkat. Seleksi fitur diperlukan untuk mendapatkan fitur-fitur yang relevan yang selanjutnya akan digunakan dalam proses konstruksi model (Septiana, Susanto, & Tukiyyat, 2021).

D. Filter Method

Filter method merupakan pendekatan feature selection yang mengevaluasi relevansi setiap fitur berdasarkan sifat statistiknya terhadap variabel target tanpa melibatkan model pembelajaran (Guyon & Elisseeff, 2003). Metode ini umumnya menggunakan ukuran seperti korelasi antara fitur dan target untuk menilai kekuatan hubungan, di mana fitur dengan nilai korelasi lebih tinggi dianggap lebih informatif. Selain itu, uji statistik seperti ANOVA (Analysis of Variance) juga sering digunakan untuk mengukur sejauh mana suatu fitur mampu membedakan nilai mean antar-kelompok berdasarkan label, sehingga fitur dengan nilai F-test tinggi dinilai lebih relevan. Dengan sifatnya yang sederhana dan efisien, filter method menjadi langkah awal yang penting dalam pemilihan fitur sebelum menerapkan metode seleksi yang lebih kompleks.

E. Wrapper Method

Wrapper method merupakan pendekatan feature selection yang mengevaluasi subset fitur dengan menggunakan model prediktif secara langsung. Teknik ini mencoba berbagai kombinasi fitur—baik dengan forward selection, backward elimination, atau recursive feature elimination—dan memilih subset yang menghasilkan performa model terbaik. Karena melibatkan proses pelatihan model berulang kali, wrapper method lebih



akurat dibandingkan filter method, tetapi membutuhkan waktu dan komputasi yang lebih besar. Namun, metode ini tetap sangat efektif terutama pada dataset dengan jumlah fitur yang tidak terlalu besar. (Kohavi & John, 1997).

F. Embedded Method

Embedded method merupakan gabungan keunggulan dari filter dan wrapper method. Pada pendekatan ini, proses pemilihan fitur dilakukan bersamaan dengan proses pelatihan model, sehingga lebih efisien dibandingkan wrapper namun lebih akurat dibandingkan filter. Contoh metode embedded yang paling umum adalah LASSO (Least Absolute Shrinkage and Selection Operator) yang melakukan regularisasi L1 untuk mendorong beberapa koefisien fitur menjadi nol, serta Tree-based models seperti Random Forest dan Gradient Boosting yang menggunakan feature importance dalam proses pemilihan fitur. Embedded method menjadi favorit karena keseimbangan antara akurasi dan efisiensi komputasi. (Tibshirani, 1996).

G. SMOTE

SMOTE merupakan metode *oversampling* yang paling populer digunakan. SMOTE dilakukan dengan menambah data sintesis pada kelas minoritas. Data sintesis merupakan data baru yang dibangkitkan. SMOTE dilakukan dengan menambah jumlah data pada kelas minoritas dengan cara membangkitkan data baru berdasarkan K tetangga terdekat. Data pada kelas minoritas dilakukan *oversampling* dengan mengambil data pada kelas minoritas dan menambah sampel sintesis di sepanjang garis yang menghubungkan salah satu atau semua K tetangga terdekat data kelas minoritas tersebut. Jumlah tetangga K dipilih secara acak (Wijayanti, Kencana, & Sumarjaya, 2021).

H. ADASYN

Adaptive Synthetic Sampling Approach (ADASYN) merupakan teknik untuk mengatasi masalah data *imbalance* dengan melakukan *oversampling* pada kelas minoritas yaitu dengan menggunakan bobot distribusi untuk data pada kelas minoritas berdasarkan pada tingkat kesulitan pembelajaran model. Data sintesis dihasilkan dari data minoritas yang sulit untuk dipahami dibandingkan dengan data minoritas yang lebih mudah untuk dipahami (Ramadhanti, Santoso, & Widiharih, 2022).



7.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

7.4 LANGKAH PERCOBAAN

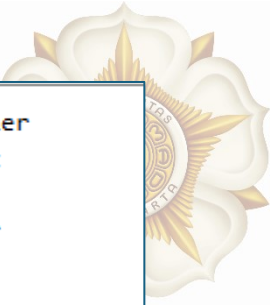
1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Google Colab dipilih karena telah mendukung berbagai library machine learning yang dibutuhkan, seperti pandas, numpy, matplotlib, seaborn, dan scikit-learn. Selain itu, Colab menyediakan resource komputasi berbasis CPU atau GPU secara gratis sehingga proses pengolahan data, pemodelan, serta penerapan teknik feature selection dapat berjalan dengan lebih efisien tanpa membebani perangkat mahasiswa.

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh library yang dibutuhkan untuk melakukan preprocessing, visualisasi, dan proses feature selection, meliputi:

1. Pandas dan Numpy untuk manipulasi data
2. Matplotlib dan Seaborn untuk visualisasi, serta scikit-learn yang menyediakan berbagai fungsi untuk pemilihan fitur seperti SelectKBest, RFE, dan SelectFromModel.
3. Scikit-learn juga digunakan untuk membangun model klasifikasi (seperti Decision Tree dan SVM) serta evaluasi performa menggunakan metrik seperti accuracy, precision, recall, dan confusion matrix.



```

from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import RandomOverSampler
from scipy.stats import norm
from imblearn.over_sampling import ADASYN
from imblearn.over_sampling import SMOTE

import seaborn as sns
import matplotlib.pyplot as plt
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

Gambar 7.4.1 Impor Pustaka

3. Import dan Load Dataset Information

Mahasiswa kemudian memuat Stroke Prediction Dataset ke dalam DataFrame. Dataset ini berisi informasi klinis dan demografis pasien yang dapat digunakan untuk memprediksi potensi terjadinya stroke. Fitur-fitur di dalam dataset mencakup usia, jenis kelamin, riwayat hipertensi, riwayat penyakit jantung, status merokok, indeks massa tubuh (BMI), serta nilai glukosa darah. Pada tahap ini mahasiswa melakukan beberapa langkah penting, yaitu:

1. Mengimpor dataset dari sumber online atau Google Drive menuju DataFrame pandas.
2. Menampilkan beberapa baris awal dataset untuk memastikan data berhasil dimuat dan tidak terjadi kesalahan pembacaan.

```

# mount drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

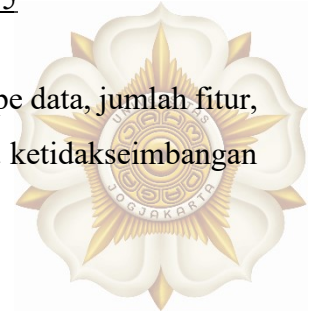
# read file
df = pd.read_csv('/content/drive/MyDrive/P Penambangan Data Sem 6/healthcare-dataset-stroke-data-latihan.csv')

# baris data dari atas
df.head()

```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 7.4.2 Pembacaan Dataset



3. Menganalisis struktur dataset menggunakan untuk memahami tipe data, jumlah fitur, nilai statistik dasar, serta memeriksa adanya missing value atau ketidakseimbangan kelas pada target (stroke).

```
# cek missing values
df.isnull().sum()

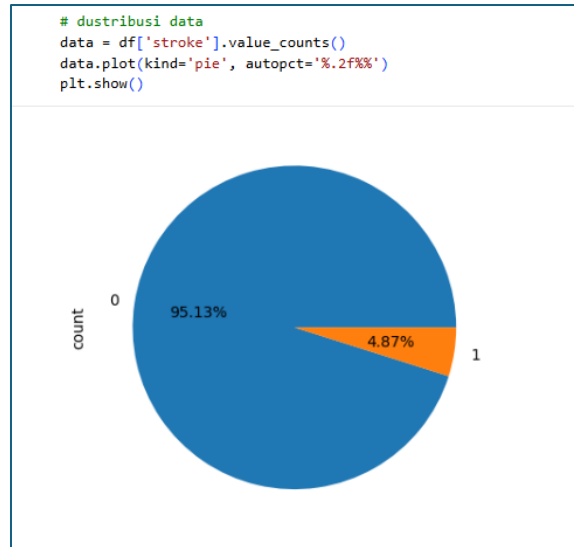
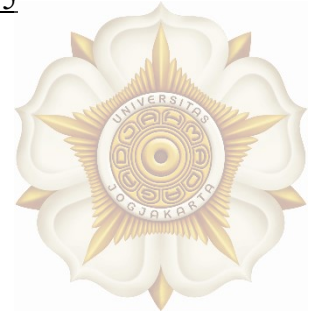
***
id      0
gender  0
age      0
hypertension  0
heart_disease  0
ever_married  0
work_type  0
Residence_type  0
avg_glucose_level  0
bmi      201
smoking_status  0
stroke    0

dtype: int64
```

Gambar 7.4.3 Mengecek Null Value

4. Exploratory Data Analysis

Mahasiswa memvisualisasikan distribusi variabel target (stroke) menggunakan grafik pie untuk mengetahui apakah dataset memiliki ketidakseimbangan kelas. Visualisasi ini penting untuk menentukan apakah teknik balancing seperti SMOTE atau ADASYN perlu digunakan pada tahap selanjutnya.



Gambar 7.4.4 Distribusi data

5. Data Cleaning

Pada tahap ini dilakukan pemeriksaan kualitas data berdasarkan hasil isnull(). Pada fitur bmi ditemukan 201 missing value. Karena BMI merupakan variabel numerik dan memiliki variasi yang cukup stabil, missing value diatasi menggunakan teknik imputasi dengan mengisi nilai kosong menggunakan nilai rata-rata (mean) dari fitur tersebut.

```
# imputasi missing value

df['bmi'].fillna(df['bmi'].mean(), inplace=True)
```

Gambar 7.4.5 Data Cleaning



```
# cek missing values after imputation
df.isnull().sum()

***
0
id      0
gender  0
age     0
hypertension  0
heart_disease  0
ever_married  0
work_type  0
Residence_type  0
avg_glucose_level  0
bmi      0
smoking_status  0
stroke   0

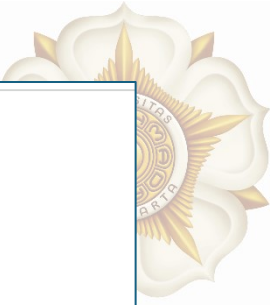
dtype: int64
```

Gambar 7.4.6 Mengecek jumlah data null

6. Random Under Sampling

Setelah data cleaning, dataset dipisahkan menjadi fitur (df_X) dan target (df_y). Kolom bmi yang memiliki nilai kosong diisi dengan rata-rata, dan variabel kategori seperti gender, ever_married, work_type, Residence_type, dan smoking_status dikodekan agar dapat digunakan dalam pemodelan. Semua fitur kemudian dikonversi ke tipe numerik.

Dataset dibagi menjadi data train dan test dengan rasio 75:25. Karena jumlah sampel kelas mayoritas jauh lebih banyak dibanding kelas minoritas, data train diseimbangkan menggunakan RandomUnderSampler, sehingga jumlah sampel pada kedua kelas menjadi sama. Distribusi kelas sebelum dan sesudah proses ini ditampilkan untuk memastikan kesetimbangan data.



```

from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

under_sampling = RandomUnderSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

le = LabelEncoder()
df_y = le.fit_transform(df_y)

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

X_resampled, y_resampled = under_sampling.fit_resample(X_train, y_train)

print('sebelum undersampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah undersampling')
print(pd.DataFrame(y_resampled).value_counts())

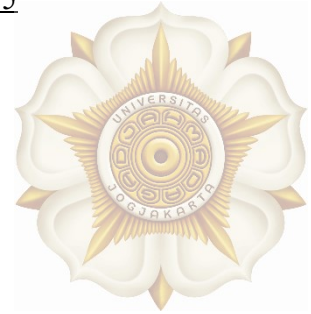
```

```

... sebelum undersampling
0
0    3663
1     169
Name: count, dtype: int64
setelah undersampling
0
0     169
1     169
Name: count, dtype: int64

```

Gambar 7.4.7 Random Sampling

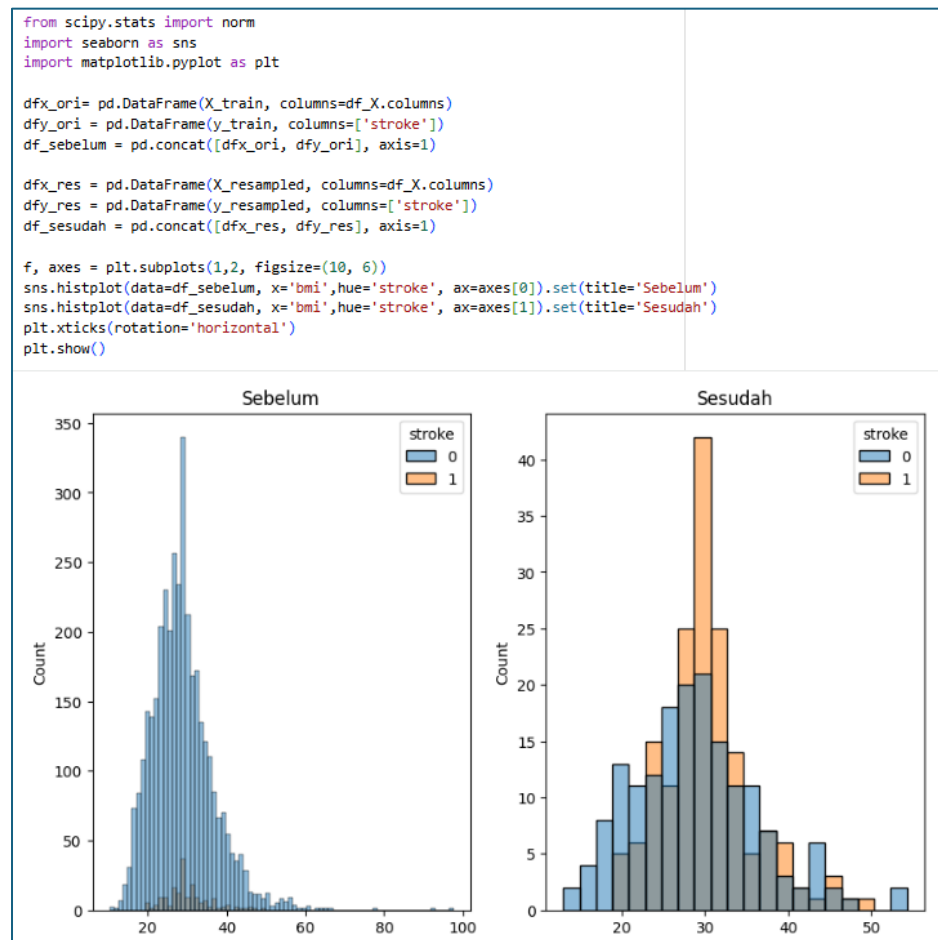


```
# menampilkan kolom stroke
df_y = df[['stroke']]
df_y
```

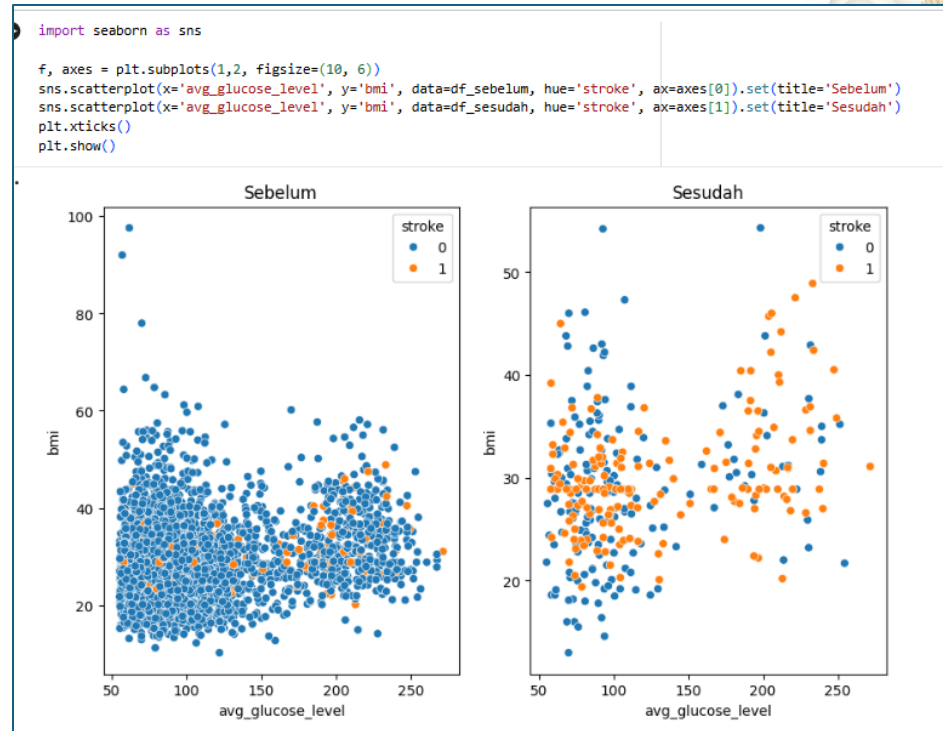
	stroke
0	1
1	1
2	1
3	1
4	1
...	...
5105	0
5106	0
5107	0
5108	0
5109	0

5110 rows × 1 columns

Gambar 7.4.8 Melihat data



Gambar 7.4.9 Perbandingan sebelum random sampling dan sesudah



Gambar 7.4.10 Visualisasi Scatter

7. Random Over Sampling

Selain undersampling, salah satu teknik untuk menangani imbalanced data adalah oversampling. Setelah data cleaning dan pemisahan fitur serta target, RandomOverSampler digunakan untuk menambah jumlah sampel pada kelas minoritas sehingga kedua kelas memiliki jumlah yang seimbang. Distribusi kelas sebelum dan sesudah oversampling ditampilkan untuk memastikan data train menjadi seimbang dan siap digunakan dalam pemodelan.



```

from imblearn.over_sampling import RandomOverSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

over_sampling = RandomOverSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values

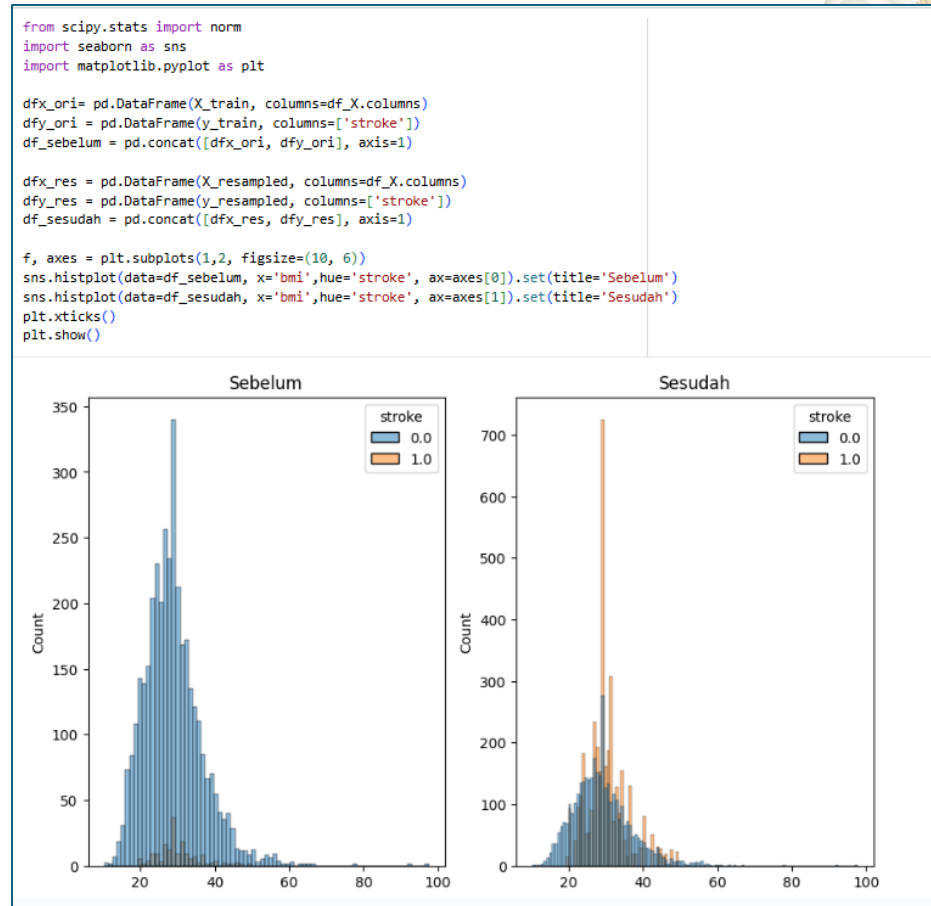
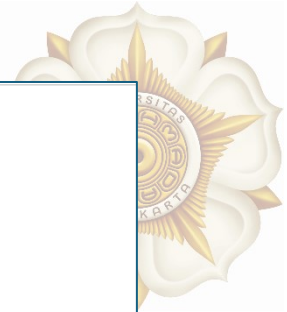
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

X_resampled, y_resampled = over_sampling.fit_resample(X_train, y_train)

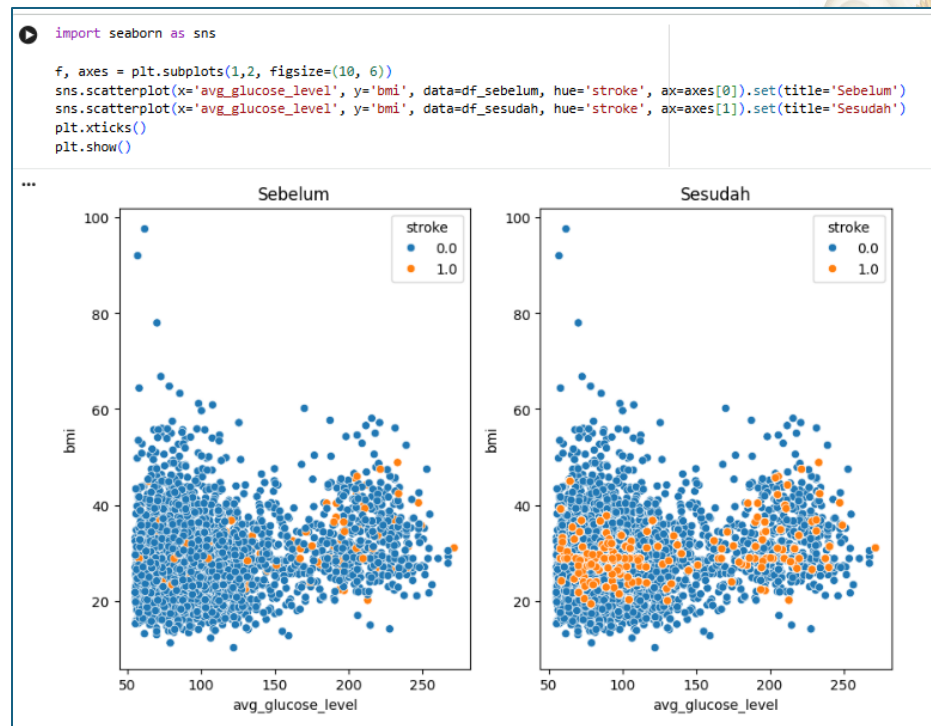
print('sebelum random oversampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah random oversampling')
print(pd.DataFrame(y_resampled).value_counts())

*** sebelum random oversampling
0
0.0    3663
1.0     169
Name: count, dtype: int64
setelah random oversampling
0
0.0    3663
1.0    3663
Name: count, dtype: int64
    
```

Gambar 7.4.11 Random over sampling



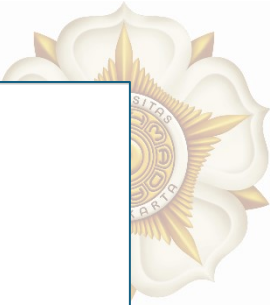
Gambar 7.4.12 Sebelum vs Sesudah over sampling



Gambar 7.4.13 Visualisasi scatter

8. SMOTE

Selain undersampling dan oversampling biasa, teknik lain untuk menangani imbalanced data adalah SMOTE (Synthetic Minority Over-sampling Technique). Setelah data cleaning dan pemisahan fitur serta target, SMOTE digunakan untuk membuat sampel sintetis pada kelas minoritas, sehingga jumlah sampel pada kedua kelas menjadi seimbang tanpa menghapus data kelas mayoritas. Distribusi kelas sebelum dan sesudah SMOTE ditampilkan untuk memastikan data train seimbang dan siap digunakan untuk pelatihan model.



```

from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
smote = SMOTE(random_state=0)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

print('Sebelum SMOTE')
print(pd.DataFrame(y_train).value_counts())
print('Setelah SMOTE')
print(pd.DataFrame(y_resampled).value_counts())

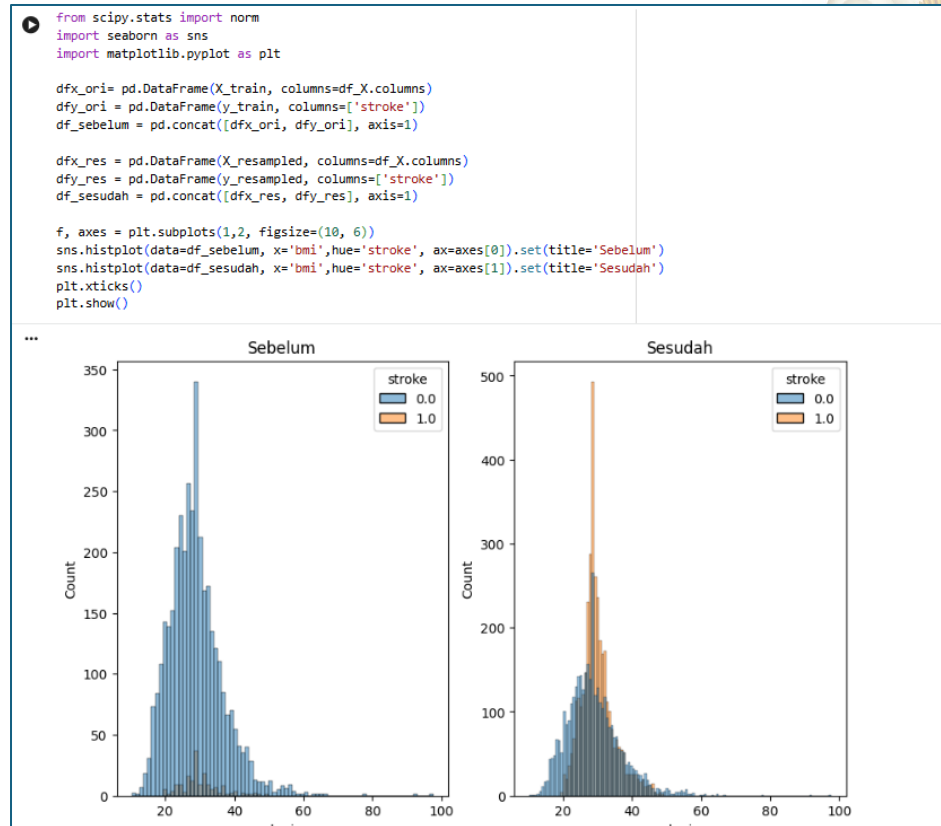
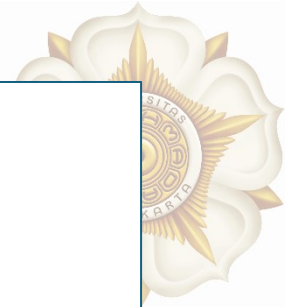
```

```

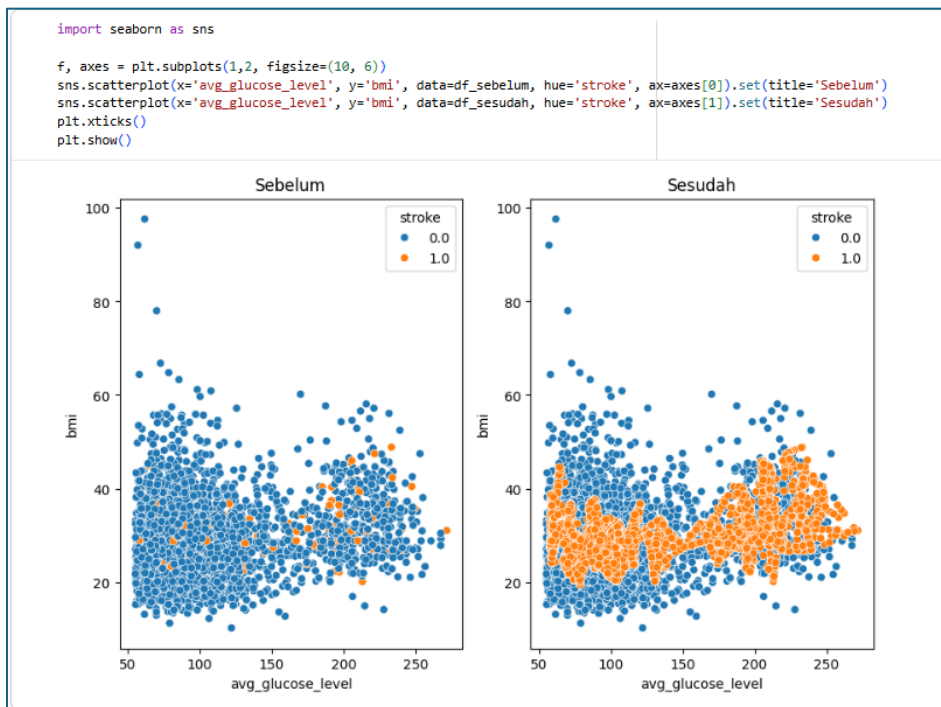
*** Sebelum SMOTE
0
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah SMOTE
0
0.0    3663
1.0    3663
Name: count, dtype: int64

```

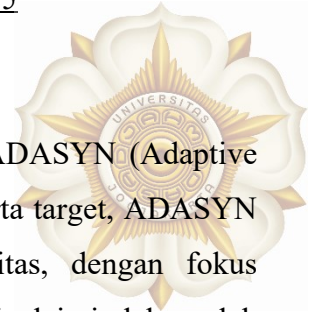
Gambar 7.4.14 SMOTE



Gambar 7.4.15 Sebelum vs Sesudah SMOTE



Gambar 7.4.16 Visualisasi Scatter



9. ADASYN

Selain SMOTE, salah satu teknik penyeimbangan kelas adalah ADASYN (Adaptive Synthetic Sampling). Setelah data cleaning dan pemisahan fitur serta target, ADASYN digunakan untuk membuat sampel sintetis pada kelas minoritas, dengan fokus menambahkan lebih banyak sampel di sekitar titik-titik yang sulit dipelajari oleh model. Teknik ini membantu model belajar lebih baik dari daerah kelas minoritas yang kompleks. Distribusi kelas sebelum dan sesudah ADASYN ditampilkan untuk memastikan data train seimbang dan siap digunakan untuk pelatihan model.

```

from imblearn.over_sampling import ADASYN
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
adasyn = ADASYN(random_state=0)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)

print('Sebelum ADASYN')
print(pd.DataFrame(y_train).value_counts())
print('Setelah ADASYN')
print(pd.DataFrame(y_resampled).value_counts())

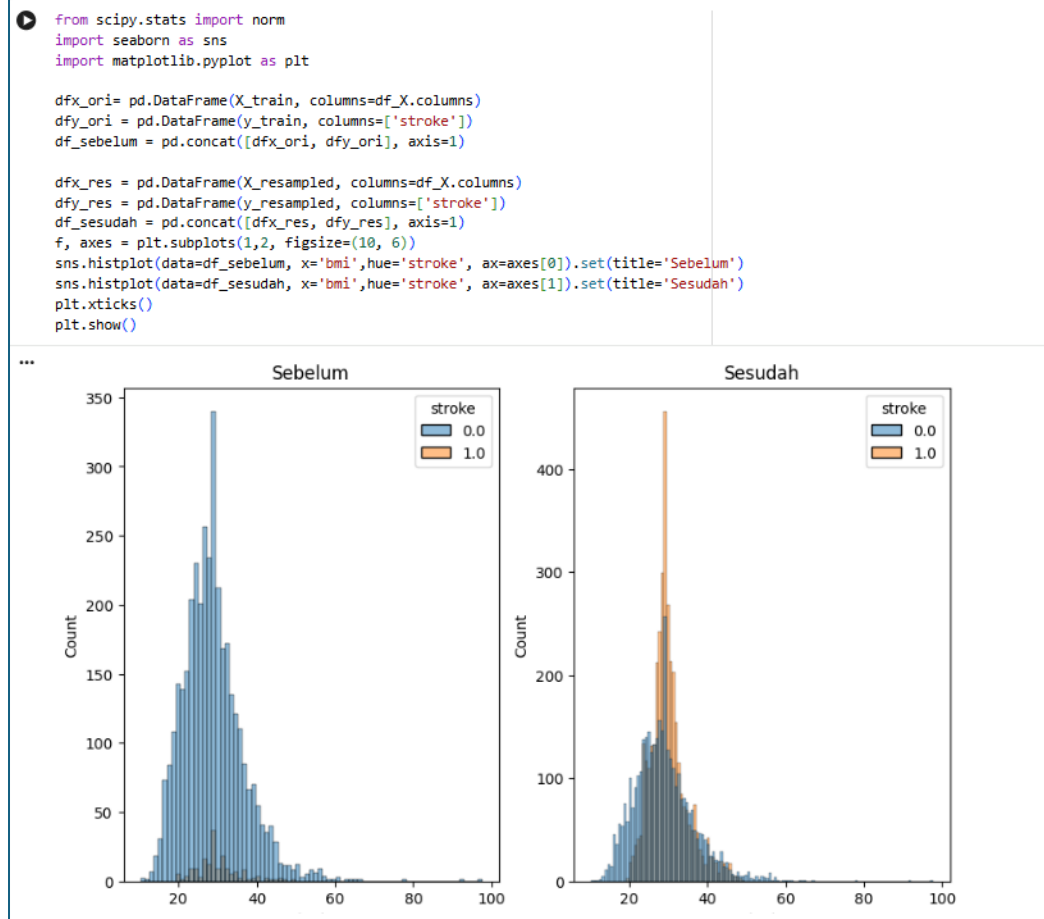
```

```

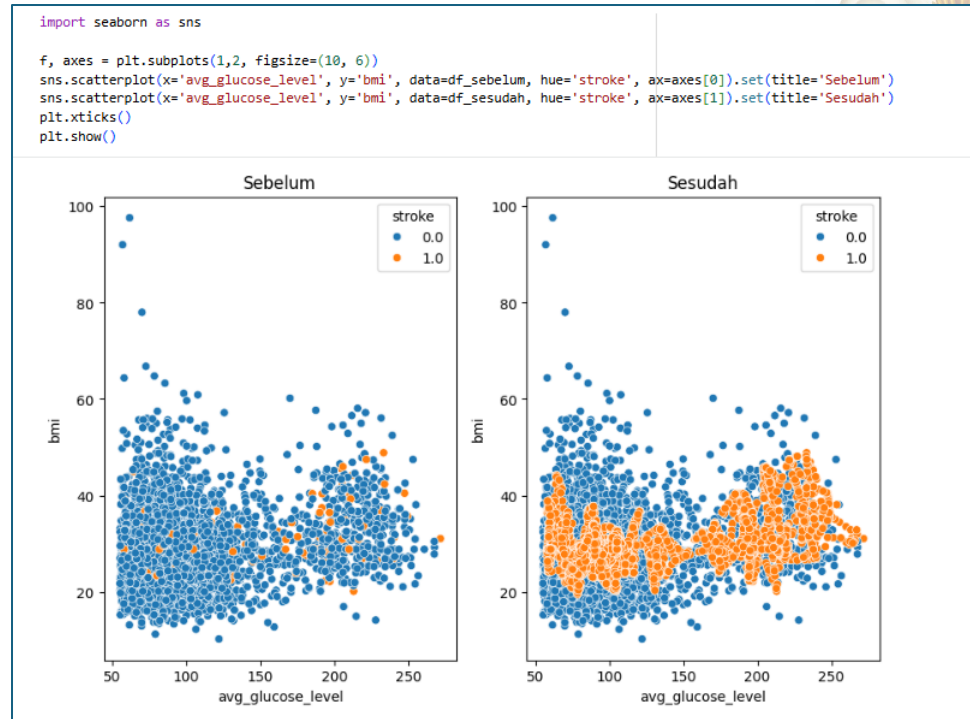
... Sebelum ADASYN
0
0.0 3663
1.0 169
Name: count, dtype: int64
Setelah ADASYN
0
1.0 3676
0.0 3663
Name: count, dtype: int64

```

Gambar 7.4.17 ADASYN



Gambar 7.4.18 Sebelum dan Sesudah Adasyn



Gambar 7.4.19 Visualisasi scatter

7.5 TUGAS DAN ANALISIS

1. Tugas 1

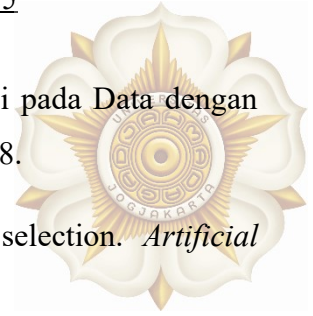
- Download dataset <https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>
- Upload dataset ke gdrive atau github masing-masing
- Tampilkan dataset dalam dataframe
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dari dataset ini

2. Tugas 2

- Dengan menggunakan dataset lain, kombinasikan SMOTE atau ADASYN dengan classifier.
- Tampilkan performa model (accuracy, precision, recall) sebelum dan sesudah data balancing pada training set nya

7.6 REFERENSI

Machine Learning Research, 3, 1157–1182.



Istiana, A., & Mustafiril, M. (2023). Perbandingan Metode Klasifikasi pada Data dengan Imbalance Class dan Missing Value. *JURNAL INFORMATIKA*, 101-108.

Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.

Normawati, & Prayogi, T. (2021). Implementasi Naïve Bayes Classifier Dan Confusion Matrix Pada Analisis Sentimen Berbasis Teks Pada Twitter. *J-SAKTI (Jurnal Sains Komputer & Informatika)*, 697-711.

Pratama, R., & Kusnawi, K. (2023). Komparasi Algoritma Supervised Learning dan Feature Selection pada Klasifikasi Penyakit Gagal Jantung. *Indonesian Journal of Computer Science*, 3722-3733.

Ramadhanti, A. P., Santoso, H. B., & Widiharhi, T. (2022). PERBANDINGAN SMOTE DAN ADASYN PADA DATA IMBALANCE UNTUK KLASIFIKASI RUMAH TANGGA MISKIN KABUPATEN TEMANGGUNG DENGAN ALGORITMA K-NEAREST NEIGHBOR. *JURNAL GAUSSIAN*, 499 - 505.

Septiana, R., Susanto, A., & Tukiya. (2021). Analisis Sentimen Vaksinasi Covid-19 Pada Twitter Menggunakan Naive Bayes Classifier Dengan Feature Selection Chi-Squared Statistic Dan Particle Swarm Optimization. *Jurnal Sistem Komputer dan Kecerdasan Buatan*, 49-56.

Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.

Wijayanti, N. L. P., Kencana, I. W., & Sumarjaya, I. M. (2021). SMOTE: POTENSI DAN KEKURANGANNYA PADA SURVEI. *E-Jurnal Matematika*, 235-240.

PERTEMUAN VIII

NEURAL NETWORK



8.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep Multilayer Perceptron (MLP) dan mampu menerapkannya pada teknik klasifikasi dan regresi
- B. Mahasiswa mampu menganalisis hasil klasifikasi menggunakan MLP dengan memanfaatkan metrik evaluasi dan pengujian model.
- C. Mahasiswa mampu menganalisis hasil regresi menggunakan MLP dengan membandingkan hasil prediksi dengan data asli.

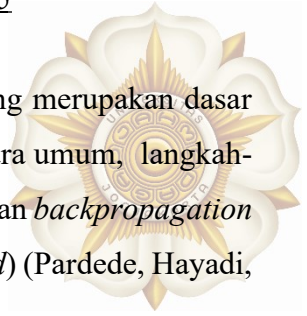
8.2 DASAR TEORI

A. Neural Network

Neural network atau jaringan syaraf tiruan adalah jaringan atau jaring laba-laba neuron, merupakan tiruan buatan dari otak manusia yang menghubungkan lapisan input ke lapisan *output*. *Neural network* adalah Teknik pemodelan yang sangat canggih dan dapat memodelkan fungsi yang sangat kompleks. Menggunakan jaringan syaraf tiruan, dimungkinkan untuk memprediksi harapan masa depan, mengklasifikasikan nilai data ke dalam kelompok yang memungkinkan keputusan dibuat menggunakan mesin yang cerdas secara artifisial. Seperti manusia, jaringan saraf tiruan dilatih oleh supervisor sesuai dengan paradigma pembelajaran yang diawasi. *Neural network* adalah bagian dari kecerdasan buatan dan pembelajaran mesin. Pembelajaran mendalam menjadi lebih maju dengan bertambahnya data. Oleh karena itu, jaringan saraf menyesuaikan kinerjanya menjadi lebih besar saat mereka tumbuh lebih besar dan menangani arus informasi yang sangat besar (Hukubun, 2022).

B. Multilayer Perceptron

Multi Layer Perceptron (MLP) salah satu algoritma *deep learning*, sudah banyak digunakan dalam penelitian masalah klasifikasi karena memiliki kelebihan dibandingkan metode klasifikasi konvensional lain. MLP memiliki kelebihan seperti mampu mengadaptasikan dirinya dengan data, dapat memperkirakan hubungan antar keanggotaan kelas dengan atribut dari objek, lebih reliabel terhadap *noise*



dalam data, dan mampu menghitung nilai probabilitas posterior, yang merupakan dasar untuk membangun aturan klasifikasi dan analisis statistika. Secara umum, langkah-langkah dalam pelatihan algoritma *multilayer perceptron* menggunakan *backpropagation* untuk mengubah nilai bobot-bobotnya dalam arah mundur (*backward*) (Pardede, Hayadi, & Iskandar, 2022).

C. Backpropagation Learning

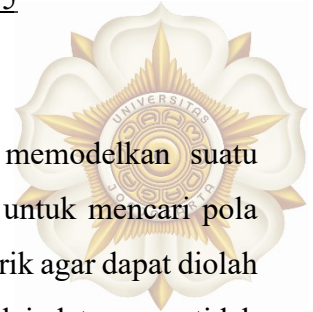
Backpropagation adalah algoritma pembelajaran yang digunakan untuk mengoptimalkan bobot (weights) neuron dalam jaringan syaraf tiruan. Prinsipnya adalah menghitung error antara output jaringan dan target, kemudian menyebarkan error tersebut ke seluruh lapisan untuk memperbaiki bobot secara bertahap (Goodfellow, Bengio, & Courville, 2016). Dengan metode ini, jaringan dapat belajar dari kesalahan dan meningkatkan performa prediksi baik pada klasifikasi maupun regresi (Haykin, 2009).

D. Activation Function

Fungsi aktivasi adalah komponen penting dalam neural network yang menentukan output dari sebuah neuron berdasarkan input yang diterima. Fungsi ini memungkinkan jaringan untuk mempelajari hubungan non-linear pada data, sehingga MLP dapat melakukan klasifikasi atau regresi yang kompleks (Goodfellow, Bengio, & Courville, 2016). Tanpa fungsi aktivasi, jaringan hanya akan menjadi kombinasi linear dari input (Haykin, 2009). Beberapa fungsi aktivasi populer adalah Sigmoid, Tanh, ReLU, dan Softmax (Nielsen, 2015).

E. Klasifikasi

Klasifikasi adalah salah satu teknik dalam *supervised learning* yang bertujuan untuk mengelompokkan data ke dalam kategori berdasarkan label yang telah ditentukan. Dalam klasifikasi, model dilatih menggunakan dataset berlabel untuk mengenali pola dan menentukan kelas suatu data baru yang belum diketahui. Beberapa algoritma yang umum digunakan dalam klasifikasi meliputi Support Vector Machines (SVM), Normal Bayesian Classifier (NBC), K-Nearest Neighbor (KNN), Trees Gradient Boosted (GBT), Random Trees (RT), dan Artificial Neural Networks (ANN). Algoritma-algoritma ini membantu dalam memprediksi kelas suatu data berdasarkan fitur yang dimilikinya, memungkinkan aplikasi luas dalam berbagai bidang, termasuk deteksi penyakit, analisis gambar, dan sistem rekomendasi (Roihan, Sunarya, & Rafika, 2020).



F. Regresi

Linear regression merupakan salah satu algoritma yang memodelkan suatu persamaan untuk menghitung estimasi. Pada metode ini bertujuan untuk mencari pola pada nilai numerik, sehingga data yang dibutuhkan berupa data numerik agar dapat diolah dengan model algoritma ini. Linear regression memprediksi nilai data yang tidak diketahui dengan menggunakan nilai data lain yang terkait dan diketahui. Secara matematis memodelkan variabel yang tidak diketahui atau tergantung dan variabel yang dikenal atau independen sebagai persamaan linier (Mardiani, et al., 2023)

8.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

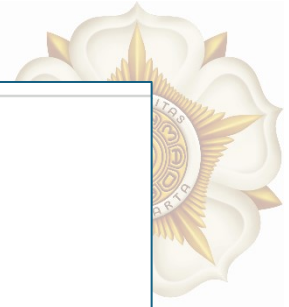
8.4 LANGKAH PERCOBAAN

1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Google Colab dipilih karena telah mendukung berbagai library machine learning yang dibutuhkan, Colab juga menyediakan resource komputasi berbasis CPU atau GPU secara gratis sehingga proses pelatihan model MLP dapat berjalan lebih optimal tanpa membutuhkan perangkat keras khusus

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor library yang diperlukan untuk manipulasi data, preprocessing, visualisasi, dan pembangunan model neural network. Library yang digunakan antara lain: pandas, numpy, matplotlib, seaborn, scikit-learn (MLPClassifier dan MLPRegressor), serta imbalanced-learn untuk metrik tambahan pada data tidak seimbang.



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from imblearn.metrics import sensitivity_specificity_support
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OrdinalEncoder
from sklearn.neural_network import MLPClassifier

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Gambar 8.4.1 Impor Pustaka

3. Import dan Load Dataset Information

Dataset diambil dari sumber online (Kaggle/GitHub) dan dibaca menggunakan pandas.

Untuk percobaan:

1. Klasifikasi: dataset Bank Customer Churn digunakan untuk memprediksi pelanggan yang akan churn.
2. Regresi: dataset KC House Data digunakan untuk memprediksi harga rumah.

4. Exploratory Data Analysis

Pada tahap ini, mahasiswa melakukan eksplorasi awal terhadap dataset untuk memahami karakteristik data sebelum digunakan dalam model MLP.

5. Data Cleaning

Mahasiswa memeriksa keberadaan nilai hilang (missing values) dan memproses variabel kategorikal menggunakan teknik encoding (Label Encoding atau Ordinal Encoding). Fitur numerik kemudian dinormalisasi dengan StandardScaler untuk meningkatkan stabilitas pelatihan model MLP.

6. MLP untuk Klasifikasi

Mahasiswa membangun model MLPClassifier untuk memprediksi kategori target pada dataset churn. Proses mencakup:

1. Melihat beberapa baris data awal dan akhir head()

```
# read file
df = pd.read_csv('/content/drive/MyDrive/Penambangan Data Sem 6/BankChurners.csv')

# baris data dari atas
df.head()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category
0	768805383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K
2	713982108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K
4	709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K

5 rows x 23 columns

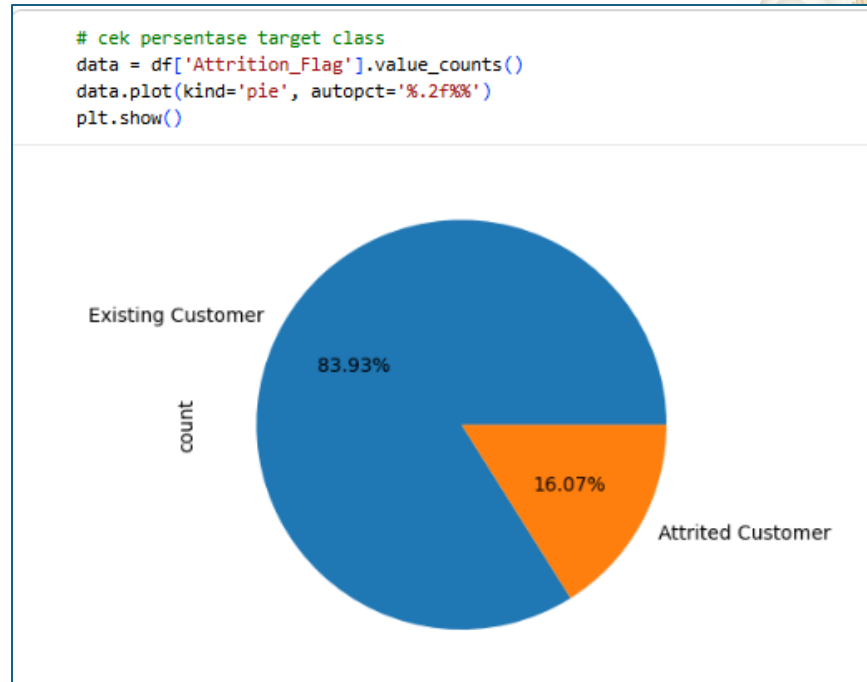
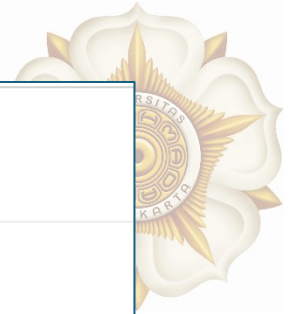
Gambar 8.4.2 Membaca dataset

2. Menganalisis distribusi variabel menggunakan histogram



Gambar 8.4.3 Menganalisis dataset

3. Mengecek proporsi target class (khusus dataset klasifikasi)



Gambar 8.4.4 Mengecek proporsi data

4. Mengecek missing value

```
# cek missing value
df.isnull().sum()
```

	0
CLIENTNUM	0
Attrition_Flag	0
Customer_Age	0
Gender	0

Gambar 8.4.5 Mengecek nilai null

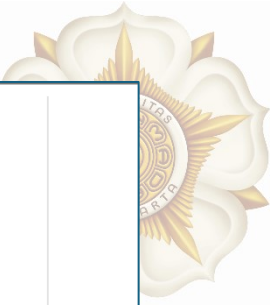
5. Mengidentifikasi atribut kategorikal dan numerik

```
# cek kategorikal atribut
df_X = df.drop(columns=['CLIENTNUM', 'Attrition_Flag'], axis=1)
df_y = df['Attrition_Flag']
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

```
Index(['Gender', 'Education_Level', 'Marital_Status', 'Income_Category',
      'Card_Category'],
      dtype='object')
```

Gambar 8.4.6 Mengidentifikasi atribut kategorikal dan numerik

6. Membagi data menjadi train dan test



```

# get X and y
df_X = df.drop(['CLIENTNUM', 'Attrition_Flag'],axis=1)
df_y = df[['Attrition_Flag']]

# label encoding for y.
# merubah nilai yg ada di y menjadi 0 atau 1.
# sebenarnya ini tidak diperlukan karena nilai y di dataset sudah 0 atau 1
le = LabelEncoder()
df_y = le.fit_transform(df_y[['Attrition_Flag']])

# categorical encoding
# merubah categorical value menjadi numerical value
# bisa pakai label encoding, ordinal atau one hot encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    le.fit(df_X[i])
    df_X[i] = le.transform(df_X[i])

# menyimpan X dan y menjadi numpy arrays
X = df_X.astype(float).values
y = df_y.astype(float)

# hold-out method, dibagi menjadi training dan testing set. 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# scaling
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

# mulai learning
model=MLPClassifier(max_iter=500)

# model=MLPClassifier(max_iter=1000, random_state=42, activation='logistic')
model.fit(X_train, y_train)

```

MLPClassifier

MLPClassifier(max_iter=500)

Gambar 8.4.7 Pembagian data training dan testing

```

# koef model
model.coefs_

```

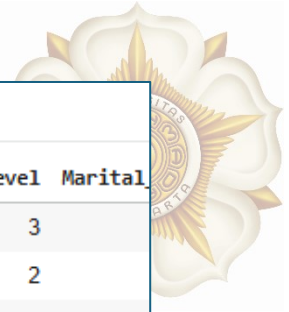
...

```

[ 0.25021812],
[-0.24980602],
[ 0.30144528],
[-0.17717679],
[-0.37201294],
[-0.2334015 ],

```

Gambar 8.4.8 Koefisien model



df_X					
	Customer_Age	Gender	Dependent_count	Education_Level	Marital
0	45	1	3	3	
1	49	0	5	2	
2	51	1	3	2	
3	40	0	4	3	
4	40	1	3	5	
...	...	...	...	...	...
10122	50	1	2	2	
10123	41	1	2	6	
10124	44	0	1	3	
10125	30	1	2	2	
10126	43	0	2	2	

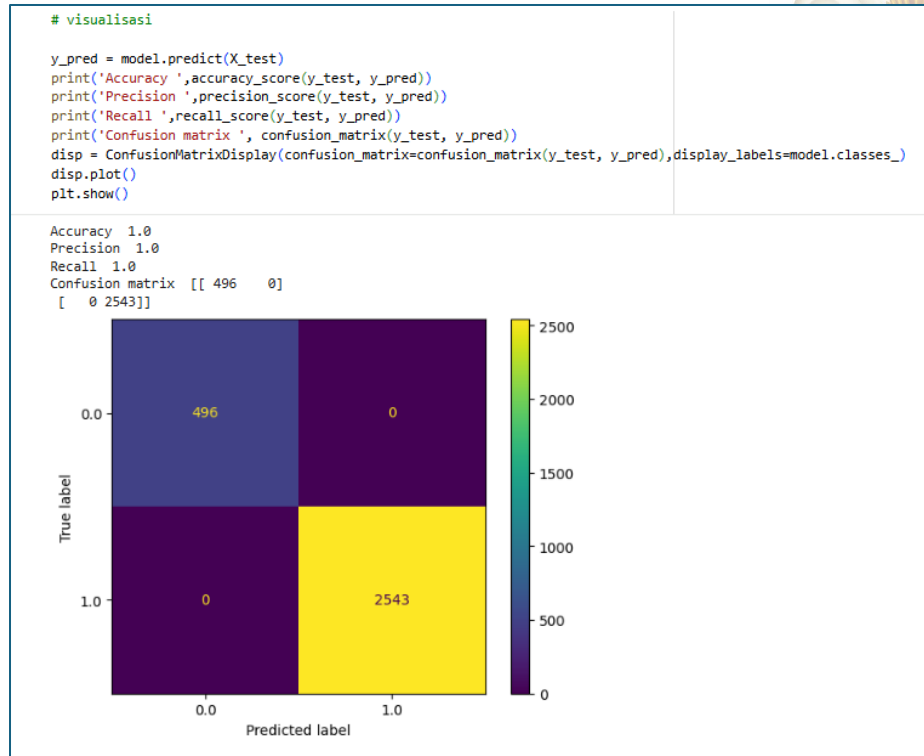
10127 rows × 21 columns

Gambar 8.4.9 Visualisasi Data

X_train[0]	
array([0.33519339, 1.05692234, -0.25437357, 0.4990858, -0.61710507, -0.56580813, -0.25591165, 0.25586758, -0.51407227, -0.33752656, -2.22835505, -0.05917368, 0.98686487, -0.14826286, -0.57038549, -0.90317904, -1.65490765, -1.12681489, -0.11093226, -0.43609419, 0.43609247])	

Gambar 8.4.10 Visualisasi Data

7. Mengevaluasi performa model menggunakan metrik seperti accuracy, precision, recall, serta confusion matrix



Gambar 8.4.11 Visualisasi Confusion Matrix

8. Melatih model dengan MLPClassifier

```
data = [{'Customer_Age':68, 'Gender':'F', 'Dependent_count':5, 'Education_Level':'Graduate',
'Marital_Status':'Single', 'Income_Category':'less than $40K', 'Card_Category':'Blue',
'Months_on_book':44, 'Total_Relationship_Count':6, 'Months_Inactive_12_mon':1,
'Contacts_Count_12_mon':2, 'Credit_Limit':8700, 'Total_Revolving_Bal':800, 'Avg_Open_To_Buy':7392,
'Total_Amt_Chng_Q4_Q1':1.541, 'Total_Trans_Amt':1200, 'Total_Trans_Ct':33, 'Total_Ct_Chng_Q4_Q1':3.714,
'Avg_Utilization_Ratio':0.10},
{'Customer_Age':50, 'Gender':'M', 'Dependent_count':2, 'Education_Level':'Graduate',
'Marital_Status':'Single', 'Income_Category':'$40K - $60K', 'Card_Category':'Blue',
'Months_on_book':38, 'Total_Relationship_Count':3, 'Months_Inactive_12_mon':2,
'Contacts_Count_12_mon':0, 'Credit_Limit':8077, 'Total_Revolving_Bal':1965, 'Avg_Open_To_Buy':6112,
'Total_Amt_Chng_Q4_Q1':0.637, 'Total_Trans_Amt':1329, 'Total_Trans_Ct':26, 'Total_Ct_Chng_Q4_Q1':0.444,
'Avg_Utilization_Ratio':0.243}]

df_input = pd.DataFrame(data)

# categorical encoding
# Use the original df_X columns to ensure correct feature order and presence
original_cols = df.drop(['CLIENTNUM', 'Attrition_Flag'], axis=1).columns
df_input = df_input.reindex(columns=original_cols, fill_value=0) # Reindex to match original columns, fill missing with 0 if necessary

cats = df_input.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)

# Create a dictionary to store the fitted LabelEncoders for each categorical feature
label_encoders = {}

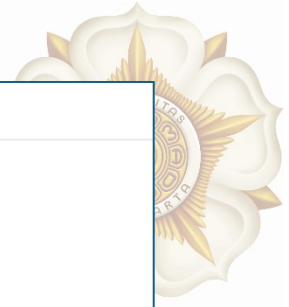
for i in cat_features:
    # Initialize a new LabelEncoder for each feature
    le = LabelEncoder()
    all_unique_values = pd.concat([df_X[i], df_input[i]]).astype(str).unique()
    le.fit(all_unique_values)
    label_encoders[i] = le
    df_input[i] = label_encoders[i].transform(df_input[i].astype(str))

# scaling
df_input = scaler.transform(df_input.astype(float).values)

result= model.predict(df_input)
for i in result:
    int_result = int(i)
    if (int_result == 0):
        decision = 'Existing Customer'
    elif (int_result==1):
        decision = 'Attrited Customer'
    else:
        decision = 'Not defined'
    print('Result prediction is ', decision)
```

Result prediction is Attrited Customer
Result prediction is Attrited Customer

Gambar 8.4.12 Melatih model MLP



df_input
array([[2.69329634e+00, 3.05998799e+00, 2.08111844e+00, 2.13632131e+00, 3.46696655e+00, 2.75905345e+00, 5.59066952e+00, 1.00082113e+00, 1.41213614e+00, -1.32245379e+00, -4.09541072e-01, 9.83216719e-03, -4.44011649e-01, -6.39776613e-03, 3.62111505e+00, -9.40555602e-01, -1.35950899e+00, 1.26325766e+01, -6.32215193e-01, -4.36229815e-01, -2.30899076e+00], [4.59304076e-01, 5.06305363e+00, -2.54373565e-01, 2.13632131e+00, 3.46696655e+00, -1.89575277e+00, 5.59066952e+00, 2.55867580e-01, -5.14072268e-01, -3.37526559e-01, -2.22835505e+00, -5.91736804e-02, 9.86864872e-01, -1.48262862e-01, -5.70385493e-01, -9.03179036e-01, -1.65490765e+00, -1.12681489e+00, -1.10932261e-01, -4.36229815e-01, -2.30899076e+00]])
cat_features
['Gender', 'Education_Level', 'Marital_Status', 'Income_Category', 'Card_Category']
result
array([1., 1.])

Gambar 8.4.13 Melihat hasil prediksi

7. MLP untuk Regresi

Mahasiswa membangun model **MLPRegressor** untuk memprediksi nilai kontinu (harga rumah). Langkah yang dilakukan:

1. Melihat beberapa baris data awal dan akhir head()

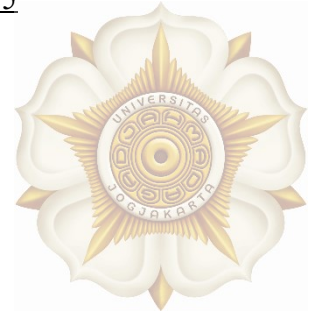
```
# read dataset
df = pd.read_csv('/content/drive/MyDrive/Penambangan Data Sem 6/kc_house_data.csv')
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sq
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180.0	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170.0	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770.0	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050.0	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680.0	

5 rows × 21 columns

Gambar 8.4.14 Membaca dataset

2. Mengecek missing value

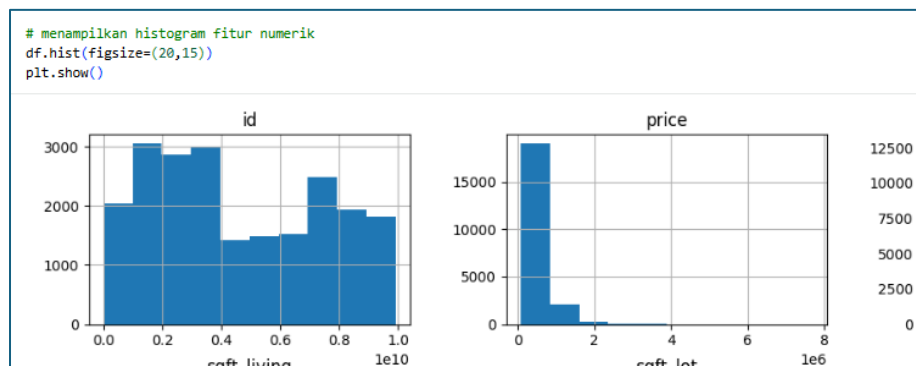


```
# cek missing values
df.isnull().sum()
```

	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0

Gambar 8.4.15 Mengecek missing value

3. Menampilkan histogram fitur numerik



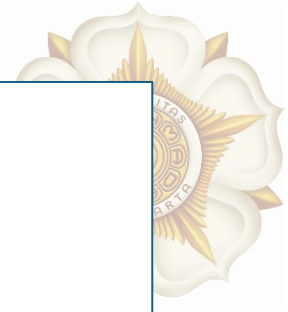
Gambar 8.4.16 Histogram fitur numerik

4. Mengukur performa model menggunakan R^2 (coefficient of determination)

```
# cek kategori atribut
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df[['price']]
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

Index([], dtype='object')

Gambar 8.4.17 Mengecek Performa



```
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
import numpy as np

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(df_X)
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

reg = MLPRegressor()
reg.fit(X_train, y_train)

print('coef of determination training ', reg.score(X_train, y_train))
print('coef of determination testing ', reg.score(X_test, y_test))

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

coef of determination training  0.563641874546569
coef of determination testing  0.5484535001698863
prediction
[ 509811.91473618  733841.13263335 1094460.32414506 1358233.15019149
  632004.49719819  401595.71074248  638014.39483258  492009.77523119
  438309.65054247  315041.81873972]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
warnings.warn(
```

Gambar 8.4.18 Mengecek performa

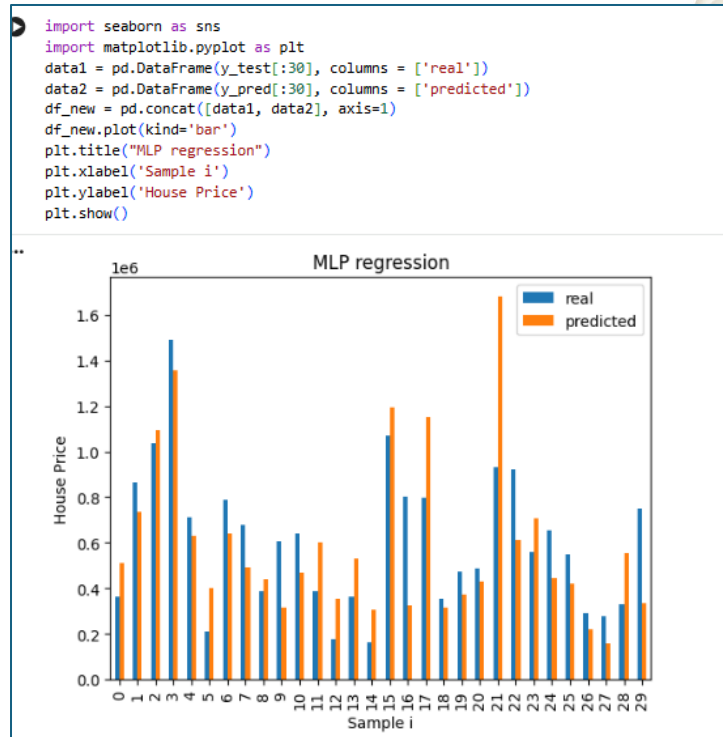
5. Mengukur performa model menggunakan R^2 (coefficient of determination) dan RMSE

```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print('rmse : ', rmse)
print('r2 : ', r2)

rmse : 255319.8034520965
r2 : 0.5484535001698863
```

Gambar 8.4.19 Evaluasi model

6. Membuat visualisasi perbandingan prediksi vs nilai asli



Gambar 8.4.20 Visualisasi hasil prediksi

8.5 TUGAS DAN ANALISIS

1. Tugas 1

- Download dataset: [Bank Customer Churn Dataset](#) dan [kc_house_data](#)
- Jelaskan tujuan dataset ini
- Jelaskan feature dataset ini

2. Tugas 2

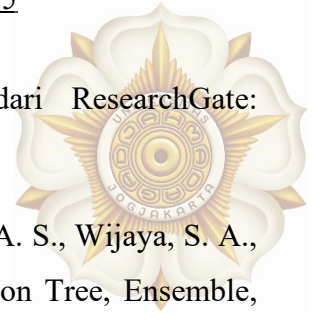
- Silahkan mencari dataset untuk klasifikasi, dan buat model MLP untuk klasifikasi
- Silahkan mencari dataset untuk regresi, dan buat model MLP untuk regresi
- Tampilkan masing-masing performa model klasifikasi dan regresi

8.6 REFERENSI

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Haykin, S. (2009). *Neural networks and learning machines* (3rd ed.). Pearson.

Nielsen, M. A. (2015). *Neural networks and deep learning: A textbook*. Determination Press.



Hukubun, A. J. (2022). *Neural Network*. Diambil kembali dari ResearchGate: https://www.researchgate.net/publication/365581579_Neural_Network

Mardiani, E., Rahmansyah, N., Ningsih, S., Lantana, D. A., Wirawan, A. S., Wijaya, S. A., & Putri, D. N. (2023). Komparasi Metode Knn, Naive Bayes, Decision Tree, Ensemble, Linear Regression Terhadap Analisis Performa Pelajar SMA. *INNOVATIVE: Journal Of Social Science Research*, 13880-13892.

Pardede, D., Hayadi, B. H., & Iskandar. (2022). KAJIAN LITERATUR MULTI LAYER PERCEPTRON: SEBERAPA BAIK PERFORMA ALGORITMA INI. *Journal of ICT Aplication and System*, 23-34.

Roihan, A., Sunarya, P. A., & Rafika, A. S. (2020). Pemanfaatan Machine Learning dalam Berbagai Bidang: Review paper. *IJCIT ((Indonesian Journal on Computer and Information Technology))*, 75-82.

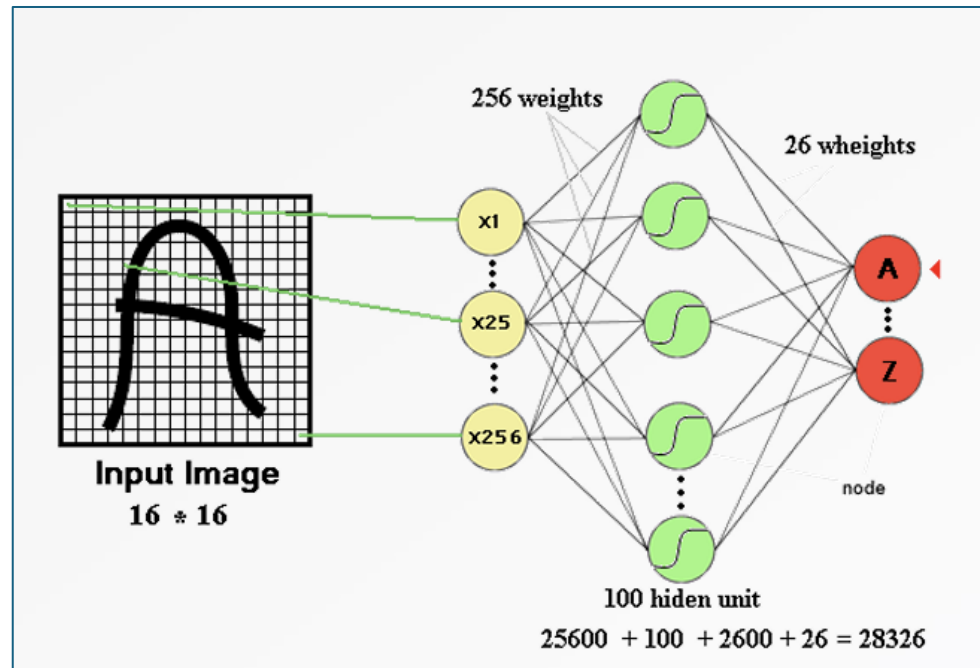
WORK



seperti MLP (Multi-Layer PERceptron) memiliki kelemahan jika langsung digunakan pada citra mentah. Hal ini terjadi oleh sebab berikut:

1. Jumlah Parameter Ekstrem

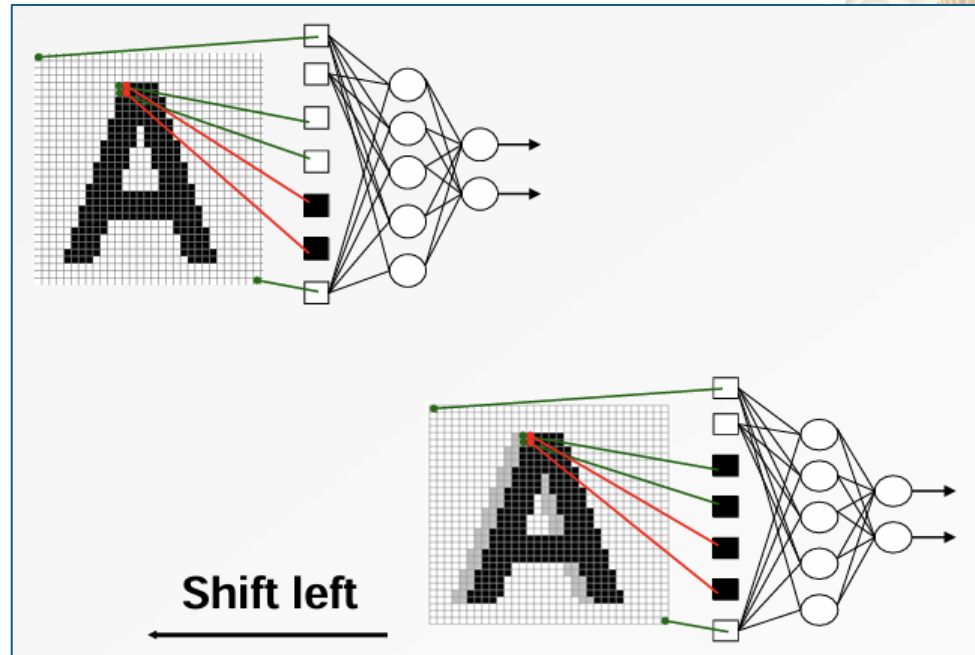
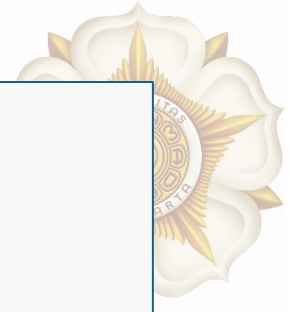
Citra kecil berukuran 16 x 16 saja jika dihubungkan ke hidden layer bisa menghasilkan puluhan ribu bobot (weights), yang menyulitkan komputasi.



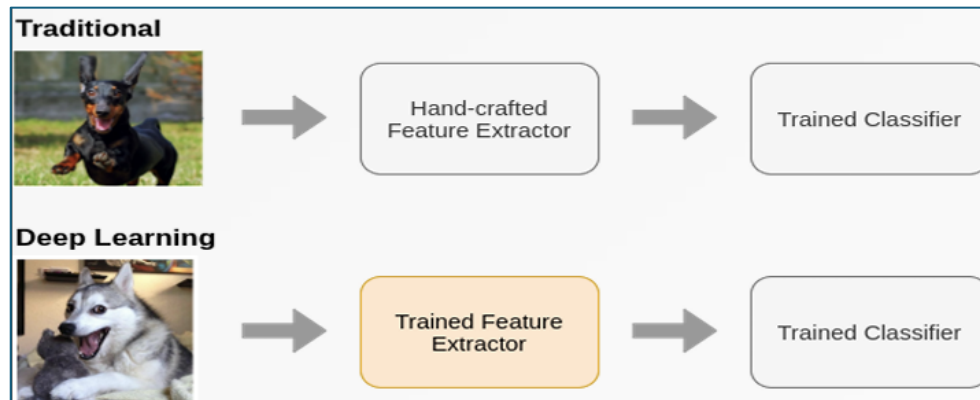
Gambar 0.2 Hidden Layer MLP

2. Hilangnya Invariansi Spasial

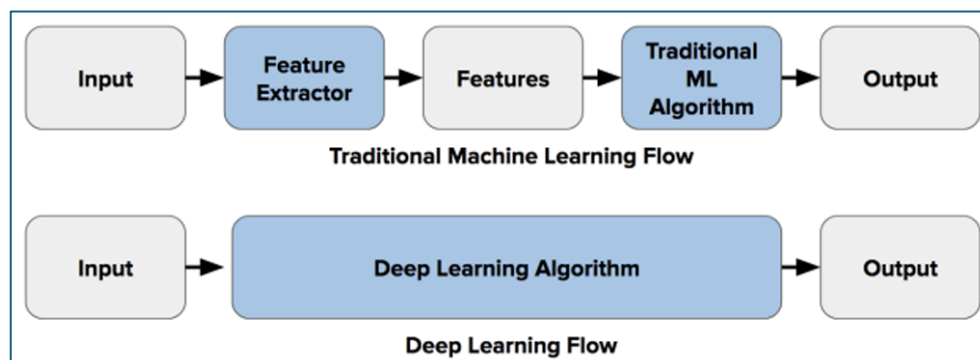
MLP tidak tahan terhadap pergeseran (shift), penskalaan (scaling), atau distorsi citra. Jika objek bergeser sedikit saja, MLP mungkin gagal mengenalinya.



Gambar 0.3 Pergeseran Citra pada MLP



Gambar 0.4 Ilustrasi Alur Tradisional vs Deep Learning



Gambar 0.5 Ilustrasi Alur Tradisional vs Deep Learning



- **Arsitektur Convolutional Neural Network (CNN)**

CNN adalah jaringan saraf *feed-forward* yang dirancang untuk mengekstrak pola visual langsung dari piksel dengan *preprocessing* minimal. Alur utama CNN terdiri dari:

1. Convolution Layer

Lapisan ini adalah inti dari CNN. Lapisan ini memiliki sejumlah Filter (kernel) yang “berjalan” di atas citra input untuk mengekstrak fitur. Operasi ini disebut konvolusi, yaitu perkalian titik (*dot product*) antara nilai filter dan bagian citra. Setiap filter akan menghasilkan *Feature Map* yang mendeteksi pola tertentu (misal: garis vertikal, lengkungan).

2. ReLU (Rectified Linear Unit)

Setelah konvolusi, fungsi aktivasi ReLU diterapkan untuk memperkenalkan non-linearitas. Rumusnya adalah $\Phi(x) = \max(x, 0)$, yang mengganti semua nilai negatif piksel menjadi nol.

3. Pooling Layer (Subsampling)

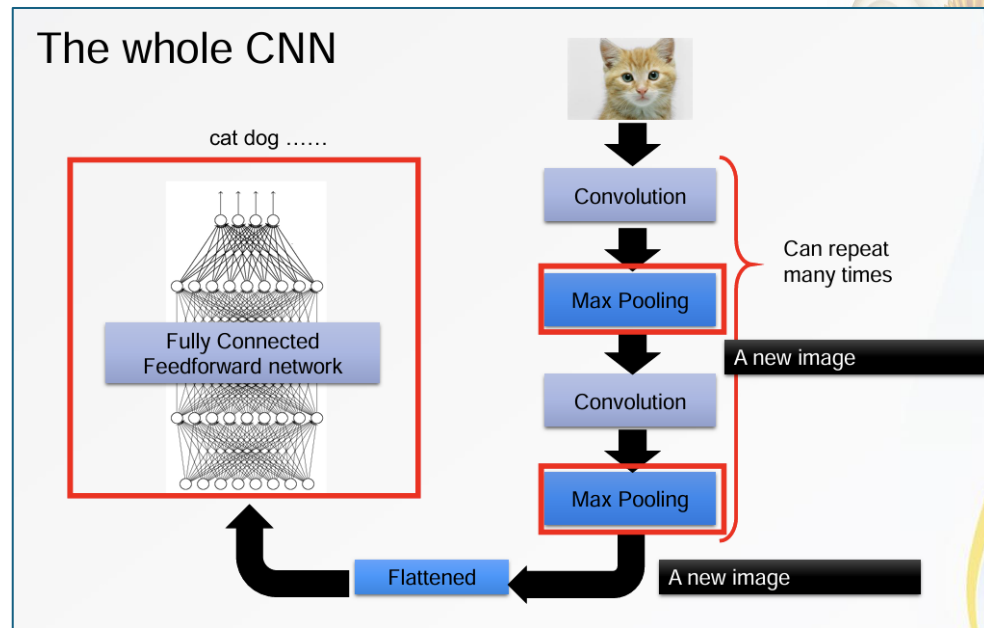
Lapisan ini berfungsi mengurangi dimensi citra (*downsampling*) agar parameter komputasi lebih sedikit dan model lebih kebal terhadap pergeseran objek 11. Teknik paling umum adalah Max Pooling, yang mengambil nilai terbesar dari area tertentu (misal 2 x 2).

4. Flattening

Mengubah matriks fitur multi-dimensi (hasil pooling terakhir) menjadi vektor satu dimensi (vektor baris panjang) agar bisa dimasukkan ke jaringan saraf biasa.

5. Fully Connected Layer (Dense).

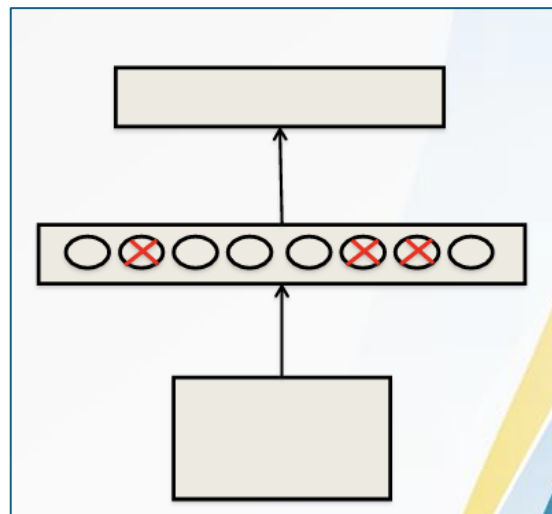
Bagian akhir dari CNN yang bertindak sebagai classifier (seperti MLP biasa). Lapisan ini menghubungkan semua fitur yang telah diekstrak ke kelas output (misalnya: Kucing, Anjing, Burung).



Gambar 0.6 Alur CNN secara keseluruhan

- **Drop Out**

Dropout adalah teknik regularisasi di mana neuron dimatikan secara acak (misalnya dengan probabilitas 0.5) selama proses pelatihan. Ini sangat efektif untuk mengurangi *overfitting* pada jaringan saraf yang dalam.



Gambar 0.7 Dropout

9.3. ALAT DAN BAHAN



- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colab
- **Pustaka Python**
 - tensorflow
 - matplotlib

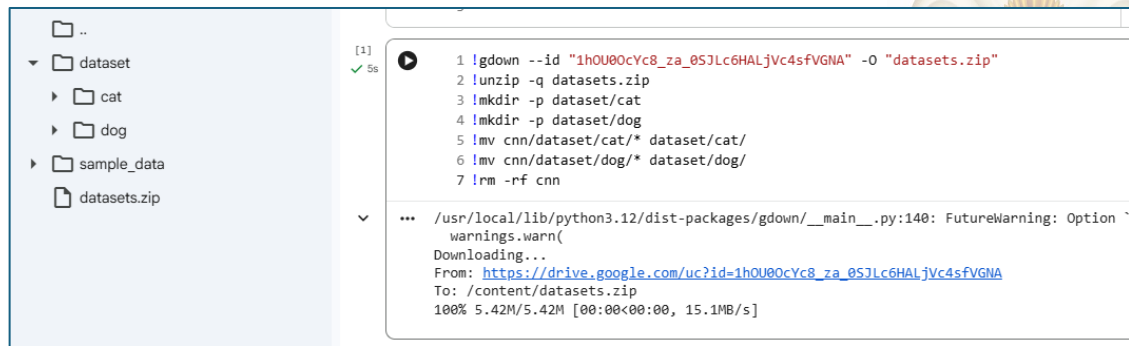
9.4. LANGKAH PERCOBAAN

Percobaan kali ini akan mencoba membuat model untuk klasifikasi gambar Dog vs Cat. Modul praktikum ini akan menggunakan sumber daya Google Colab untuk memanfaatkan GPU free tier. Berikut adalah langkah-langkah pembuatan model klasifikasi Dog vs Cat:

1. Unduh datasets

Unduh dataset Dog vs Cat dengan menuliskan perintah berikut di Google Colab. Jika sel berhasil dijalankan, folder dataset akan terbuat dan terlihat seperti Gambar 0.1.

```
!gdown --id "1hOU0OcYc8_za_0SJLc6HALjVc4sfVGNA" -O "datasets.zip"
!unzip -q datasets.zip
!mkdir -p dataset/cat
!mkdir -p dataset/dog
!mv cnn/dataset/cat/* dataset/cat/
!mv cnn/dataset/dog/* dataset/dog/
!rm -rf cnn
```



Gambar 0.1 Unduh Dataset Dog vs Cat

Jika Google Colab tidak bisa mengunduh dataset tersebut, gunakan link dibawah ini untuk mengunduh secara manual.

https://drive.google.com/file/d/1hOU0OcYc8_zs_0SJLc6HALjVc4sfVGNA

Setelah mengunduh secara manual, unggah file ke runtime agar terlihat seperti Gambar 0.1. Struktur folder harus terdiri dari dataset yang memiliki dua folder, yaitu cat dan dog. Setiap folder cat dan dog memiliki foto yang merepresentasikan folder tersebut.

2. Impor pustaka

Impor pustaka tensorflow, matplotlib, numpy, dan beberapa modul dari tensorflow keras. Secara default, Google Colab telah menyediakan pustaka tersebut, sehingga tidak perlu mengunduh pustaka. Tambahkan kode impor pustaka seperti Gambar 0.2.

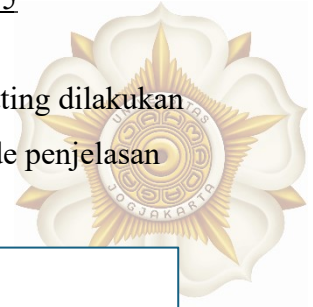
```

1 import os
2
3 import tensorflow as tf
4 import matplotlib.pyplot as plt
5 import os
6 import random
7 import numpy as np
8
9 from tensorflow.keras import Sequential
10 from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Dropout, Flatten
  
```

Gambar 0.2 Impor Pustaka

3. Pembacaan dan pembagian dataset

Setelah melakukan impor dan mengunduh dataset, tahap selanjutnya adalah membuat pembacaan dataset dan melakukan data splitting untuk membagi data menjadi data training dan data validasi. Dataset ini memiliki ukuran piksel 224 x 224 x 3, sehingga



target size disesuaikan dengan ukuran dataset yaitu 224 x 224. Splitting dilakukan dengan kombinasi 80% untuk training dan 20% untuk validasi. Kode penjelasan tersebut akan diimplementasikan menjadi Gambar 0.3.

```
1 data_path = "dataset/"
2 base_dir = os.path.join(data_path)
3
4 IMAGE_SIZE = 224 #model input size
5 BATCH_SIZE = 64
6
7 datagen = tf.keras.preprocessing.image.ImageDataGenerator(
8     rescale = 1./255,
9     validation_split=0.2)
10
11 train_generator = datagen.flow_from_directory(
12     base_dir,
13     target_size=(IMAGE_SIZE, IMAGE_SIZE),
14     batch_size = BATCH_SIZE,
15     subset='training')
16
17 val_generator = datagen.flow_from_directory(
18     base_dir,
19     target_size=(IMAGE_SIZE, IMAGE_SIZE),
20     batch_size = BATCH_SIZE,
21     subset='validation')

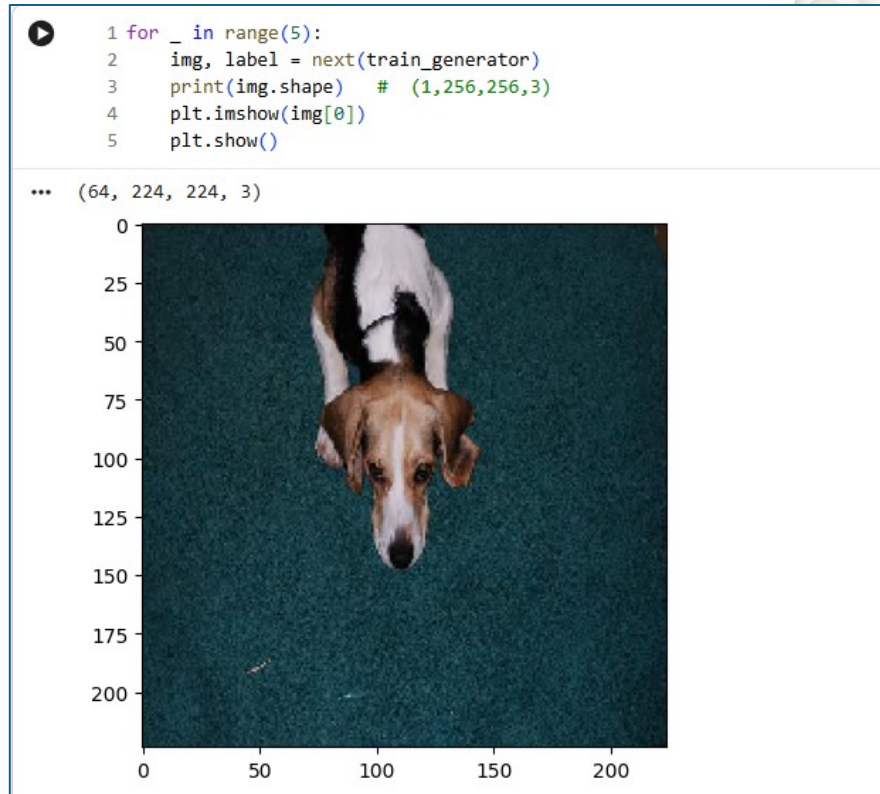
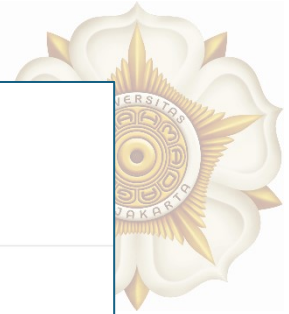
```

... Found 80 images belonging to 2 classes.
Found 20 images belonging to 2 classes.

Gambar 0.3 Data Preparation dan Splitting

4. Uji coba pembacaan data

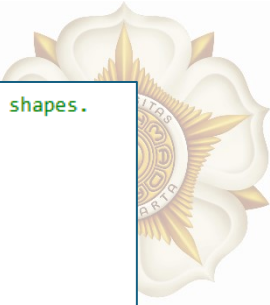
Uji coba pembacaan data guna memastikan bahwa data yang dimuat adalah data yang benar dan sesuai. Jika benar maka akan tertampil seperti Gambar 0.4.



Gambar 0.4 Pengujian pembacaan data

5. Cek kewarasan

Untuk memastikan bahwa data yang dimuat adalah benar, pengecekan dengan membaca langsung data training atau validasi dapat dilakukan. Lakukan pendefinisian ulang terhadap IMG_SHAPE dengan nilai yang dikeluarkan oleh bentuk dari data langsung tersebut. Dengan demikian, kita yakin bahwa informasi data yang akan diberikan ke model CNN adalah benar. Berikut adalah cara mengambil informasi untuk membuat informasi IMG_SHAPE.



```

1 # This loop gets one batch of images and labels to inspect their shapes.
2 # The 'break' ensures it only runs for the first batch.
3 for image_batch, label_batch in train_generator:
4     break
5
6 print(f"Image batch shape: {image_batch.shape}")
7 print(f"Label batch shape: {label_batch.shape}")
8
9 IMG_SHAPE = (IMAGE_SIZE, IMAGE_SIZE, 3)
10
11 num_classes = 2

```

Image batch shape: (16, 224, 224, 3)
Label batch shape: (16, 2)

Gambar 0.5 Pendefinisian IMG_SHAPE dan kelas

6. Model CNN

Buat model CNN dengan Keras API dari tensorflow. Model ini akan membuat 2 lapis blok konvolusi. Seperti yang telah dipelajari di atas pada topik 0, implementasi teori tersebut akan terlihat seperti Gambar 0.6 Model CNN sederhana.

```

1 model = Sequential()
2
3 #-----
4 # Conv Block 1: 32 Filters, MaxPool.
5 #-----
6 model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=IMG_SHAPE))
7 model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
8 model.add(MaxPooling2D(pool_size=(2, 2)))
9 model.add(Dropout(0.1))
10 #-----
11 # Conv Block 2: 64 Filters, MaxPool.
12 #-----
13 model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
14 model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
15 model.add(MaxPooling2D(pool_size=(2, 2)))
16
17 # Flatten the convolutional features.
18 #-----
19 model.add(Flatten())
20 model.add(Dense(512, activation='relu'))
21 model.add(Dense(num_classes, activation='softmax'))
22
23 model.compile(optimizer='rmsprop',
24               loss='categorical_crossentropy',
25               metrics=['accuracy'],
26               )
27
28 model.summary()

```

Gambar 0.6 Model CNN sederhana

Bagian akhir Gambar 0.6 menjalankan fungsi summary. Fungsi tersebut akan menampilkan ringkasan model yang telah dibuat. Hasil ringkasan akan terlihat seperti Gambar 0.6. Model ini memiliki 102,827,554 parameter yang akan dilatih.

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
    Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
conv2d_1 (Conv2D)	(None, 224, 224, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
dropout (Dropout)	(None, 112, 112, 32)	0
conv2d_2 (Conv2D)	(None, 112, 112, 64)	18,496
conv2d_3 (Conv2D)	(None, 112, 112, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
flatten (Flatten)	(None, 200704)	0
dense (Dense)	(None, 512)	102,760,960
dense_1 (Dense)	(None, 2)	1,026

Total params: 102,827,554 (392.26 MB)
 Trainable params: 102,827,554 (392.26 MB)
 Non-trainable params: 0 (0.00 B)

Gambar 0.7 Ringkasan model CNN sederhana

7. Latih model

Latih model CNN dengan data yang telah disiapkan seperti Gambar 0.6. Sebagai uji coba (smoke test) dan mempersingkat waktu, model akan dilatih sebanyak 20 epochs. Simpan hasil evaluasi seperti baris 5 Gambar 0.8.

```

[8] 1 epochs = 20
    2 history = model.fit(train_generator,
    3     epochs=epochs,
    4     validation_data=val_generator)
    5 loss, acc = model.evaluate(val_generator, verbose=1)

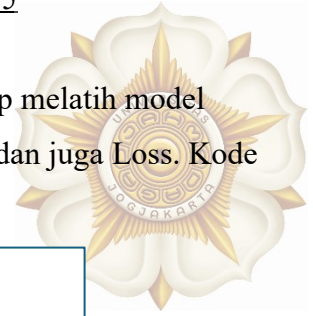
```

... /usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your self._warn_if_super_not_called()

Epoch 1/20	34s 23s/step - accuracy: 0.3875 - loss: 37.3456 - val_accuracy: 0.5000 - val_loss: 1.1778
Epoch 2/20	1s 430ms/step - accuracy: 0.5208 - loss: 1.0483 - val_accuracy: 0.5000 - val_loss: 0.6916
Epoch 3/20	1s 440ms/step - accuracy: 0.5208 - loss: 0.6975 - val_accuracy: 0.5000 - val_loss: 0.6750
Epoch 4/20	1s 159ms/step - accuracy: 0.4896 - loss: 0.7161 - val_accuracy: 0.5000 - val_loss: 0.7077
Epoch 5/20	1s 155ms/step - accuracy: 0.5104 - loss: 0.7154 - val_accuracy: 0.5000 - val_loss: 0.6926
Epoch 6/20	1s 149ms/step - accuracy: 0.5083 - loss: 0.6928 - val_accuracy: 0.5000 - val_loss: 0.6899
Epoch 7/20	1s 154ms/step - accuracy: 0.4698 - loss: 0.6865 - val_accuracy: 0.5000 - val_loss: 0.6866
Epoch 8/20	1s 146ms/step - accuracy: 0.4563 - loss: 0.6927 - val_accuracy: 0.5000 - val_loss: 0.6847
Epoch 9/20	1s 147ms/step - accuracy: 0.5000 - loss: 0.6818 - val_accuracy: 0.6500 - val_loss: 0.6891
Epoch 10/20	1s 425ms/step - accuracy: 0.6958 - loss: 0.6711 - val_accuracy: 0.5000 - val_loss: 0.6758

Gambar 0.8 Latih model CNN sederhana

8. Evaluasi

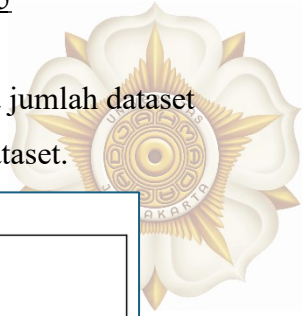


Plot hasil evaluasi yang telah disimpan ke dalam variabel pada tahap melatih model untuk melakukan evaluasi. Evaluasi ini akan menampilkan akurasi dan juga Loss. Kode untuk membuat plotting dapat dilihat di Gambar 0.9.

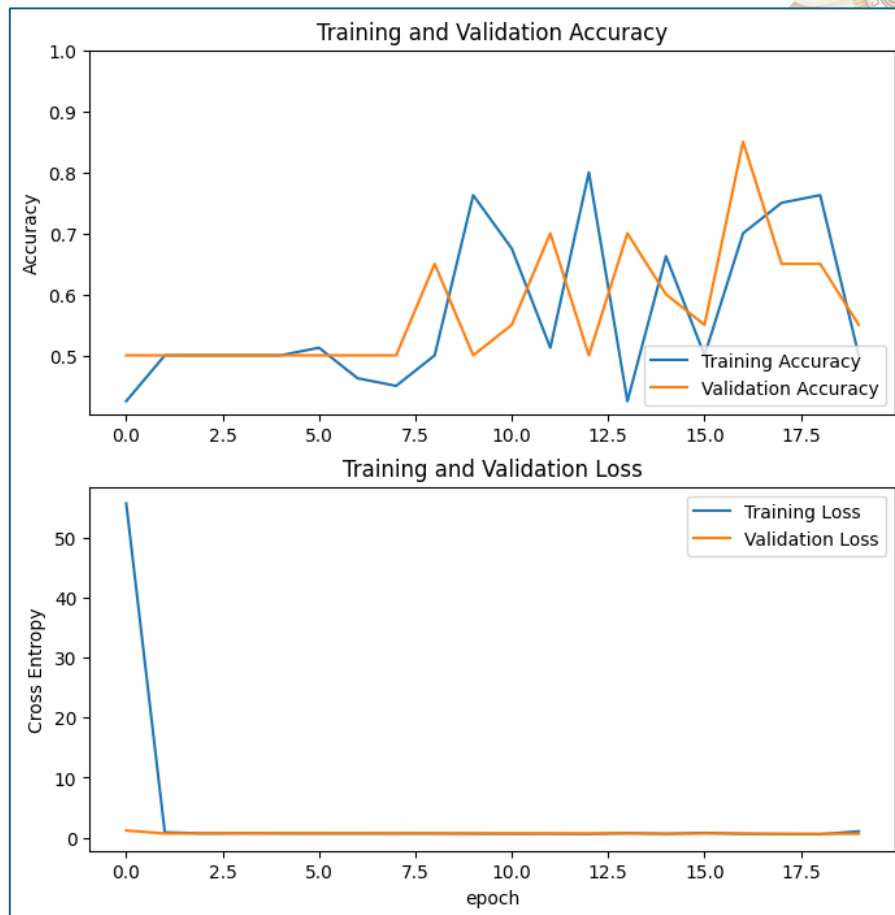
```
1 acc = history.history['accuracy']
2 val_acc = history.history['val_accuracy']
3 loss = history.history['loss']
4 val_loss = history.history['val_loss']
5
6 plt.figure(figsize=(8, 8))
7 plt.subplot(2, 1, 1)
8 plt.plot(acc, label='Training Accuracy')
9 plt.plot(val_acc, label='Validation Accuracy')
10 plt.legend(loc='lower right')
11 plt.ylabel('Accuracy')
12 plt.ylim([min(plt.ylim()),1])
13 plt.title('Training and Validation Accuracy')
14
15 plt.subplot(2, 1, 2)
16 plt.plot(loss, label='Training Loss')
17 plt.plot(val_loss, label='Validation Loss')
18 plt.legend(loc='upper right')
19 plt.ylabel('Cross Entropy')
20 plt.title('Training and Validation Loss')
21 plt.xlabel('epoch')
22 plt.show()
```

Gambar 0.9 Kode plot evaluasi

Sebagai contoh dari hasil uji coba pelatihan model, didapatkan bahwa model ini tidak mengalami overfitting. Kesimpulan tersebut dapat diambil salah satu caranya dengan cara visual terhadap Gambar 0.10, yaitu akurasi pada tahap validasi tidak tertinggal jauh dari tahap training. Pada kasus ini, validasi justru lebih tinggi dan memiliki nilai



akurasi yang rendah. Hal tersebut mungkin terjadi karena sedikitnya jumlah dataset yang tersedia. Untuk meningkatkan dapat dilakukan penambahan dataset.



Gambar 0.10 Plot evaluasi

9.5. TUGAS & ANALISIS

1. Carilah dataset berisi gambar, boleh berupa klasifikasi binary maupun multi-class! Jumlah dataset tidak harus besar.
2. Bangun sebuah model klasifikasi citra menggunakan CNN untuk melakukan training dan melakukan prediksi pada dataset tersebut!
3. Tampilkan hasil prediksi berupa metrik evaluasi serta visualisasi grafik loss dan akurasi selama proses pelatihan!

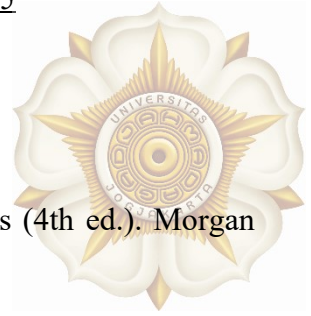
9.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.





PERTEMUAN X

MODEL DEPLOYMENT (FLASK)

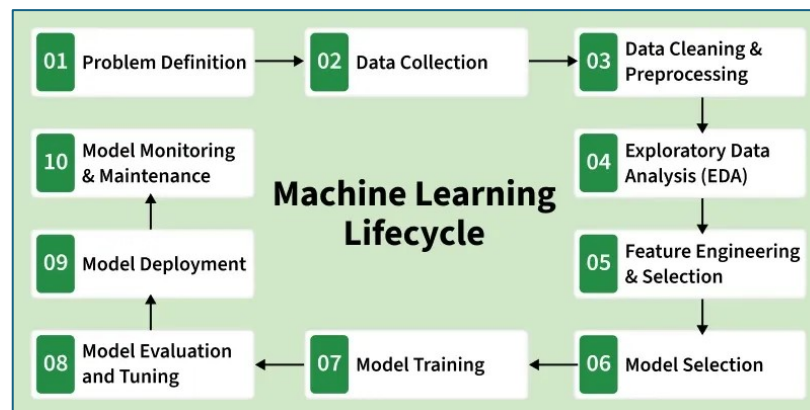
10.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar model deployment dalam siklus hidup data mining
- B. Mahasiswa mampu melakukan serialization (penyimpanan) dan deserialization (pemuatan) model machine learning menggunakan pustaka pickle atau joblib
- C. Mahasiswa dapat membangun layanan web (Web Service) sederhana menggunakan framework Flask.
- D. Mahasiswa dapat mengintegrasikan model machine learning ke dalam API sehingga dapat menerima input data baru dan memberikan hasil prediksi.

10.2. DASAR TEORI

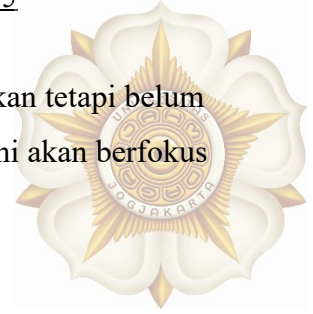
- **Tahapan Pengembangan Machine Learning**

Pengembangan model machine learning bukanlah proses satu kali jalan, melainkan sebuah siklus (cycle) yang berkelanjutan. Secara umum, tahapan ini, yang sering disebut sebagai *ML Pipeline*. Secara umum dapat terlihat seperti pada Gambar 0.1.



Gambar 0.1 Machine Learning Lifecycle (Sumber: www.geeksforgeeks.org)

Pada pertemuan sebelumnya kita telah melakukan berbagai tahapan pengembangan mulai dari *Problem Definition* hingga *Model Evaluation and Tuning*. Pada tahapan



sebelumnya kita telah mendapatkan sebuah model yang bisa digunakan tetapi belum bisa dimanfaatkan oleh orang lain. Oleh karena itu, pertemuan kali ini akan berfokus pada tahap kesembilan yaitu Model Deployment.

- **Model Deployment**

Dalam data mining, membuat model yang akurat hanyalah sebagian dari pekerjaan.

Agar model tersebut berguna, model harus di-deploy atau diterapkan ke dalam produksi sehingga dapat digunakan oleh pengguna akhir atau sistem lain. Deployment mengubah model statis menjadi layanan aktif.

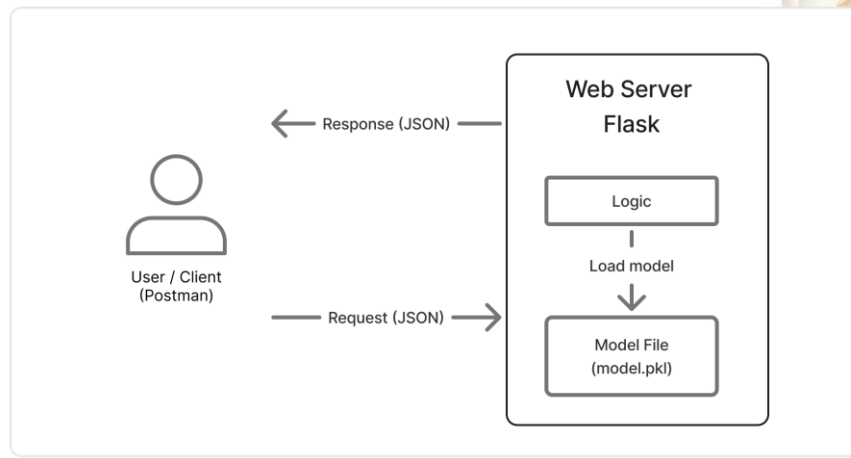
Skema deployment model bermacam-macam. Terdapat skema deployment model secara tertanam di dalam aplikasi (embedded), skema client-server, dan lain-lain. Dari banyaknya pilihan skema deployment yang dapat dilakukan, modul ini akan menggunakan skema client-server. Penggunaan aplikasi pada skema ini akan melalui perantara RESTful API.

Skema alur data pada client-server pada :

1. Flask bertindak sebagai Web Service. Ia menyediakan URL (alamat) yang bisa diakses. Flask menjembatani komunikasi antara dunia luar (internet/user) dengan logika internal Python dan Model Machine Learning. Tanpa Flask, model kita hanya bisa dijalankan di dalam komputer lokal pengembangnya saja.
2. API Endpoint: Data diterima oleh "pintu gerbang" server yang kita buat dengan Flask (misalnya di alamat /predict).
3. Web Server (Flask): Aplikasi Flask menerima data tersebut, lalu memproses format datanya agar sesuai dengan input yang dibutuhkan model.
4. Model Loading: Flask memanggil file model (.pkl) yang sudah dilatih sebelumnya. Model ini bertindak sebagai "otak" yang melakukan perhitungan matematika untuk menghasilkan prediksi.
5. Response: Hasil prediksi dikirimkan kembali oleh Flask kepada Client dalam format JSON (JavaScript Object Notation), sehingga mudah dibaca oleh manusia maupun program lain.



Penjelasan tersebut dapat diilustrasikan seperti pada Gambar 0.2.



Gambar 0.2 Skema alur data client-server

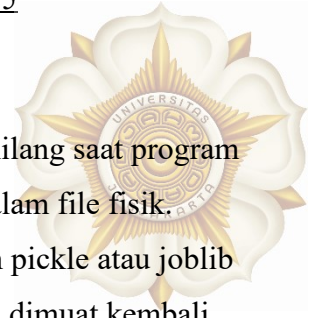
Flask bertindak sebagai Web Service. Ia menyediakan URL (alamat) yang bisa diakses. Flask menjembatani komunikasi antara dunia luar (internet/user) dengan logika internal Python dan Model Machine Learning. Tanpa Flask, model kita hanya bisa dijalankan di dalam komputer lokal pengembangnya saja.

- **Flask Framework**

Flask adalah micro-web framework yang ditulis dalam bahasa Python. Disebut "micro" karena tidak memerlukan alat atau pustaka tertentu (seperti ORM) secara bawaan. Flask sangat populer untuk membuat API (Application Programming Interface) bagi model Machine Learning karena ringan, cepat, dan mudah dipelajari. Selain Flask ada berbagai framework lain yang dapat digunakan seperti: Django, Fastapi, Tornado, ataupun Pyramid.



Gambar 0.3 Logo Flask



- **Model Serialization**

Model machine learning yang telah dilatih di memori (RAM) akan hilang saat program ditutup. Untuk itu, kita perlu menyimpan objek model tersebut ke dalam file fisik.

Proses ini disebut Serialization. Di Python, kita sering menggunakan pickle atau joblib (dari Scikit-Learn) untuk menyimpan model ke format .pkl agar bisa dimuat kembali (deserialization) oleh server Flask tanpa perlu melatih ulang model setiap kali ada permintaan.

10.3. ALAT DAN BAHAN

- **Perangkat Keras**

- Komputer/Laptop

- **Perangkat Lunak**

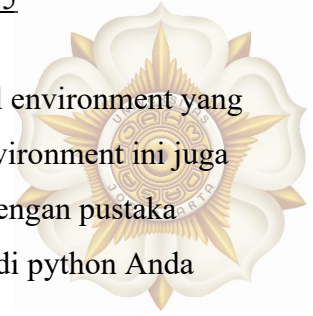
- Python3
- Text Editor / IDE. Catatan: Tidak disarankan menggunakan Jupyter Notebook untuk menjalankan server Flask.
- Postman (Opsional, untuk testing API)

- **Pustaka Python**

- tensorflow
- flask
- scikit-learn
- pandas
- joblib atau pickle

10.4. LANGKAH PERCOBAAN

Pada pembuatan modul praktikum, environment yang digunakan adalah sistem operasi Linux Ubuntu di dalam WSL, Python 3.12, dan memiliki akses GPU. Oleh karena ini beberapa perintah dapat disesuaikan dengan target environment deployment masing-masing.



Dalam membuat proyek python alangkah baiknya menggunakan virtual environment yang sesuai dengan environment produksi yang ada di server. Pembuatan environment ini juga dapat membantu dalam manajemen pustaka agar tidak terjadi konflik dengan pustaka global lainnya. Untuk mempelajari lebih lanjut mengenai environment di python Anda dapat membaca referensi berikut:

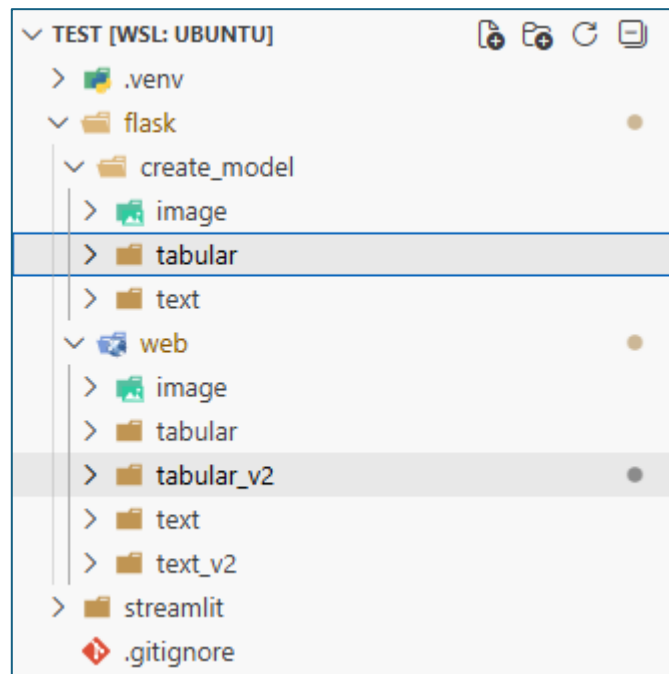
- (vanilla) <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>
- (UV Astral) <https://docs.astral.sh/uv/>
- (Miniconda) <https://www.anaconda.com/docs/getting-started/miniconda/main>

File dataset yang diperlukan dalam praktikum ini dapat Anda unduh pada link berikut:

<https://drive.google.com/file/d/1R5vXII015dJ0s3HYbZMvvBdcM3E6VVh2/view?usp=sharing>

Praktikum ini akan membuat aplikasi prediksi klasifikasi penyakit hati. Silakan salin folder berikut ke direktori proyek kalian:

- flask/create_model/tabular
- flask/web/tabular_v2



Gambar 0.1 Folder yang perlu disalin



Setelah menyalin folder tersebut, lakukan langkah-langkah berikut:

1. Unduh pustaka

Unduh pustaka flask, numpy, scikit-learn, pandas, dan juga tensorflow. Pada perintah berikut, pustaka tensorflow yang dipilih adalah tensorflow yang memiliki layer ke hardware GPU komputer. Sebelum Anda menyalin kode dibawah ini dan menjalankannya secara langsung, pastikan Anda telah membaca instalasi tensorflow yang sesuai dengan environment Anda di link berikut:

<https://www.tensorflow.org/install/pip#windows-wsl2>

Berikut kode instalasi PIP jika menggunakan WSL dan memiliki GPU:

```
pip install flask numpy scikit-learn pandas "tensorflow[and-gpu]" joblib
```

2. Pembuatan model dan penyimpanan model

Seperti pada praktikum sebelumnya, dalam membuat model diperlukan data preprocessing, feature engineering, dan lain-lain. Pembuatan model kali ini akan menggunakan model vanilla Decision Tree untuk klasifikasi. Cuplikan kode dibawah ini digunakan untuk menyiapkan data ke model Decision Tree.

```
1 #read the dataset
2 pd.options.display.max_columns=None
3 df_ori = pd.read_csv('hcvdat0.csv')
4 df_ori = df_ori.dropna()
5 #get the X and y
6 df_X = df_ori.drop(['Unnamed: 0', 'Category'],axis=1)
7 df_y = df_ori['Category']
8 #relabelling the class
9 df_y=df_y.replace('0=Blood Donor',0)
10 df_y=df_y.replace('0s=suspect Blood Donor',0)
11 df_y=df_y.replace('1=Hepatitis',1)
12 df_y=df_y.replace('2=Fibrosis',2)
13 df_y=df_y.replace('3=Cirrhosis',3)
14
15 #get the numerical attributes
16 df_X_num = df_X.drop(['Sex'], axis=1)
17 #convert single categorical column into numeric, label encoding
18 le = preprocessing.LabelEncoder()
19 df_X['Sex'] = le.fit_transform(df_X['Sex'])
20
21 #combine numerical attributes with a newly converted attribute (categorical to numeric, Sex column)
22 df_X_new = pd.concat([df_X['Sex'], df_X_num], axis=1)
```

Gambar 0.2 Preprocessing Dataset

Buat model Decision Tree seperti pada Gambar 0.3 Decision Tree Model.



```
1 def dt(X, y):
2     model = DecisionTreeClassifier(random_state=0)
3     model = model.fit(X, y)
```

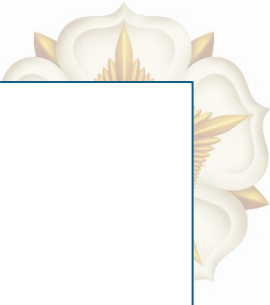
Gambar 0.3 Decision Tree Model

Untuk menyimpan hasil training model tersebut, pustaka joblib dapat digunakan. Lakukan import pustaka joblib dan simpan model seperti Gambar 0.4. Hasil model akan tersimpan pada file “dt_model.model” yang terdefinisikan di baris ke 7.

```
1 import joblib
2
3 def dt(X, y):
4     model = DecisionTreeClassifier(random_state=0)
5     model = model.fit(X, y)
6     filename = 'dt_model.model'
7     joblib.dump(model, filename)
```

Gambar 0.4 Menyimpan model hasil training

Gabungkan keseluruhan cuplikan kode tersebut menjadi satu sehingga script python akan membaca dataset, melakukan preprocessing data, training data, dan juga menyimpan hasil model. Kode utuh akan terlihat seperti Gambar 0.5.



```

1 import pandas as pd
2 from sklearn import preprocessing
3 from sklearn.tree import DecisionTreeClassifier
4 import joblib
5
6 def dt(X, y):
7     model = DecisionTreeClassifier(random_state=0)
8     model = model.fit(X, y)
9     filename = 'dt_model.model'
10    joblib.dump(model, filename)
11
12 if __name__ == '__main__':
13     #read the dataset
14     pd.options.display.max_columns=None
15     df_ori = pd.read_csv('hcvdat0.csv')
16     df_ori = df_ori.dropna()
17     #get the X and y
18     df_X = df_ori.drop(['Unnamed: 0', 'Category'], axis=1)
19     df_y = df_ori['Category']
20     #relabelling the class
21     df_y=df_y.replace('0=Blood Donor',0)
22     df_y=df_y.replace('0s=suspect Blood Donor',0)
23     df_y=df_y.replace('1=Hepatitis',1)
24     df_y=df_y.replace('2=Fibrosis',2)
25     df_y=df_y.replace('3=Cirrhosis',3)
26
27     #get the numerical attributes
28     df_X_num = df_X.drop(['Sex'], axis=1)
29     #convert single categorical column into numeric, label encoding
30     le = preprocessing.LabelEncoder()
31     df_X['Sex'] = le.fit_transform(df_X['Sex'])
32
33     #combine numerical attributes with a newly converted attribute (categorical to numeric, Sex column)
34     df_X_new = pd.concat([df_X['Sex'], df_X_num], axis=1)
35     #generate the model
36     print('Generate Decision Tree Model')
37     dt(df_X_new, df_y)

```

Gambar 0.5 Pembuatan Model DT Liver

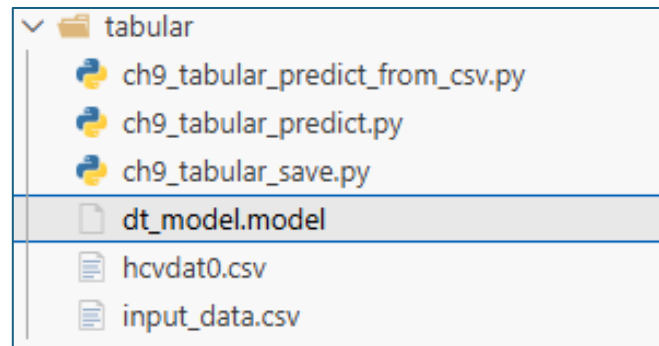
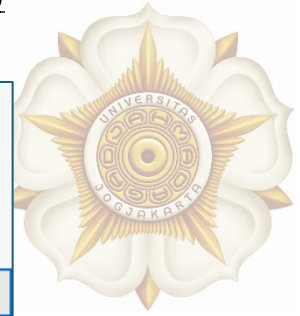
Jalankan script python pada Gambar 0.5. Jika berhasil maka akan terlihat seperti Gambar 0.6 dan akan menghasilkan file model seperti pada Gambar 0.7.

```

[13:54:05] (.venv) ~/test/flask/create_model/tabular
● $ python ./ch9_tabular_save.py
/home/ergeg/test/flask/create_model/tabular/./ch9_tabular_save.py:33: Future
sult.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.s
df_y=df_y.replace('3=Cirrhosis',3)
Generate Decision Tree Model

```

Gambar 0.6 Menjalankan script python



Gambar 0.7 File yang perlu disalin

3. Pembuatan Flask

Secara lengkap, pembuatan flask dapat dibaca pada dokumentasi resmi Flask.

<https://flask.palletsprojects.com/en/stable/quickstart/#a-minimal-application>

Flask memiliki *Web Server Gateway Interface* (WSGI) sehingga mempermudah dalam melakukan development. WSGI ini tentu berbeda dengan dedicated HTTP server. Untuk keamanan sebaiknya menggunakan dedicated HTTP server seperti NGINX, Apache, atau lainnya. Langkah-langkah yang perlu dilakukan jika ingin melakukan deployment secara nyata dapat dilihat pada dokumentasi berikut.

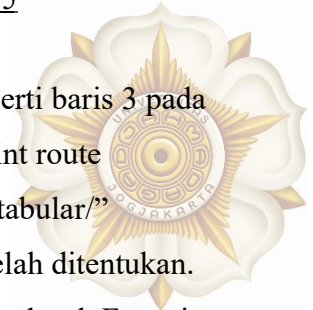
<https://flask.palletsprojects.com/en/stable/deploying/>

Dalam langkah-langkah berikut, akan digunakan WSGI yang telah disediakan oleh Flask. Perlu diingat bahwa WSGI ini mungkin tidak seaman dan tidak sebaik dedicated HTTP server.

Import pustaka-pustaka yang akan digunakan seperti Gambar 0.8.

```
1 import pandas as pd
2 import joblib
3 from typing import Optional
4 from flask import Flask, redirect, url_for, request, render_template
```

Gambar 0.8 Pustaka Deployment Flask



Untuk membuat aplikasi Flask Anda perlu membuat objek Flask seperti baris 3 pada Gambar 0.9. Objek flask ini dapat digunakan untuk membuat endpoint route menggunakan decorator route. Pada baris 5 Gambar 0.9, endpoint “/tabular/” didefinisikan. Endpoint ini akan mengembalikan sebuah aksi yang telah ditentukan. Dalam hal ini endpoint “tabular” akan mengembalikan halaman index.html. Fungsi `render_template` pada baris 7 Gambar 0.9 akan mencari file “index.html” yang berada di dalam folder “templates”. Baris 13 Gambar 0.9 akan digunakan untuk menjalankan Flask dengan WSGI. Kode tersebut bersifat opsional, Anda tetap dapat menjalankan script aplikasi ini dengan “`flask --app nama_file run`”.

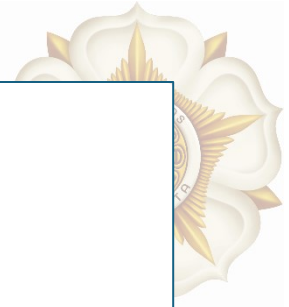
```
1 from flask import Flask, redirect, url_for, request, render_template
2
3 app = Flask(__name__)
4 #load index.html/ first page. receive input variable from user
5 @app.route("/tabular/")
6 def index():
7     return render_template('index.html')
8
9
10 if __name__ == "__main__":
11     #host= ip address, port = port number
12     #app.run(host='127.0.0.1', port='5001')
13     app.run()
```

Gambar 0.9 Minimal Flask Application

4. Integrasi Model

Melakukan integrasi model dilakukan pada aksi suatu endpoint. Anda dapat membuat endpoint tersebut mengirimkan JSON ataupun halaman html seperti yang akan dilakukan pada langkah-langkah berikut.

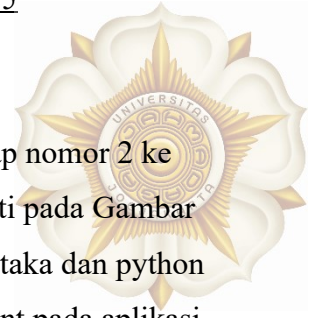
Tambahkan endpoint baru yang akan digunakan untuk menjalankan model AI. Endpoint ini akan menerima HTTP form, mengubah data input form tersebut ke bentuk input model, dan akan mengembalikan hasil prediksi serta memasukkan ke dalam `result.html`. Pastikan bahwa input telah sesuai dengan model yang dibuat. Endpoint form model Decision Tree klasifikasi penyakit hati dapat dilihat pada Gambar 0.10. Model AI di endpoint ini dipanggil pada baris 17-18.



```

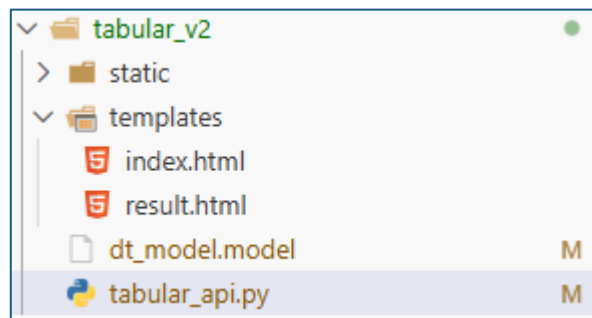
1 @app.route('/tabular/result/', methods=["POST"])
2 def prediction_result():
3     #receiving parameters sent by client
4     age = request.form.get('age')
5     sex = request.form.get('sex')
6     alb = request.form.get('alb')
7     alp = request.form.get('alp')
8     alt = request.form.get('alt')
9     ast = request.form.get('ast')
10    bil = request.form.get('bil')
11    che = request.form.get('che')
12    chol = request.form.get('chol')
13    crea = request.form.get('crea')
14    ggt = request.form.get('ggt')
15    prot = request.form.get('prot')
16    #load the trained model.
17    filename = 'dt_model.model'
18    loaded_model= joblib.load(filename)
19    # helper converters
20    def _to_float(x: Optional[str]) -> float:
21        return float(x) if x is not None and x != '' else float('nan')
22
23    def _encode_sex(x: Optional[str]) -> int:
24        if x is None:
25            return 0
26        s = str(x).strip().lower()
27        if s in ('m', 'male', '1', 'true'):
28            return 1
29        if s in ('f', 'female', '0', 'false'):
30            return 0
31        try:
32            return int(float(s))
33        except Exception:
34            return 0
35
36    sex_enc = _encode_sex(sex)
37
38    # create new dataframe matching training feature order
39    # Training used: df_X_new = pd.concat([df_X['Sex'], df_X[numb], axis=1)
40    columns_order = ['Sex', 'Age', 'ALB', 'ALP', 'ALT', 'AST', 'BIL', 'CHE', 'CHOL', 'CREA', 'GGT', 'PROT']
41    df_input = pd.DataFrame(columns=columns_order)
42    df_input.loc[0] = [
43        sex_enc,
44        _to_float(age),
45        _to_float(alb),
46        _to_float(alp),
47        _to_float(alt),
48        _to_float(ast),
49        _to_float(bil),
50        _to_float(che),
51        _to_float(chol),
52        _to_float(crea),
53        _to_float(ggt),
54        _to_float(prot),
55    ]
56    #make prediction
57    #print(df_input)
58    result = loaded_model.predict(df_input)
59    #print(result)
60    for i in result:
61        int_result = int(i)
62        if(int_result==0):
63            decision='Suspect Blood Donor'
64        elif (int_result==1):
65            decision='Hepatitis'
66        elif (int_result==2):
67            decision='Fibrosis'
68        elif (int_result==3):
69            decision='Cirrhosis'
70        else:
71            decision='Not defined'
72    #print('Disease is ', decision)
73    #return the output and load result.html
74    return render_template('result.html', age=age, sex=sex, alb=alb, alp=alp, alt=alt, ast=ast, bil=bil,
75                           che=che, chol=chol, crea=crea, ggt=ggt, prot=prot, status=decision)
76

```



Gambar 0.10 Endpoint Prediksi (tabular_api.py)

Jangan lupa untuk memindahkan model yang telah dibuat pada tahap nomor 2 ke direktori aplikasi Flask. Struktur lengkap proyek akan terlihat seperti pada Gambar 0.11 Struktur Proyek Flask. Perlu diperhatikan bahwa tiap versi pustaka dan python dapat menghasilkan hasil model yang berbeda, untuk itu environment pada aplikasi Flask dan pembuatan model dipastikan memiliki environment yang sama. Anda dapat menjalankan pembuatan model ulang jika terjadi error karena ketidakcocokan environment.



Gambar 0.11 Struktur Proyek Flask

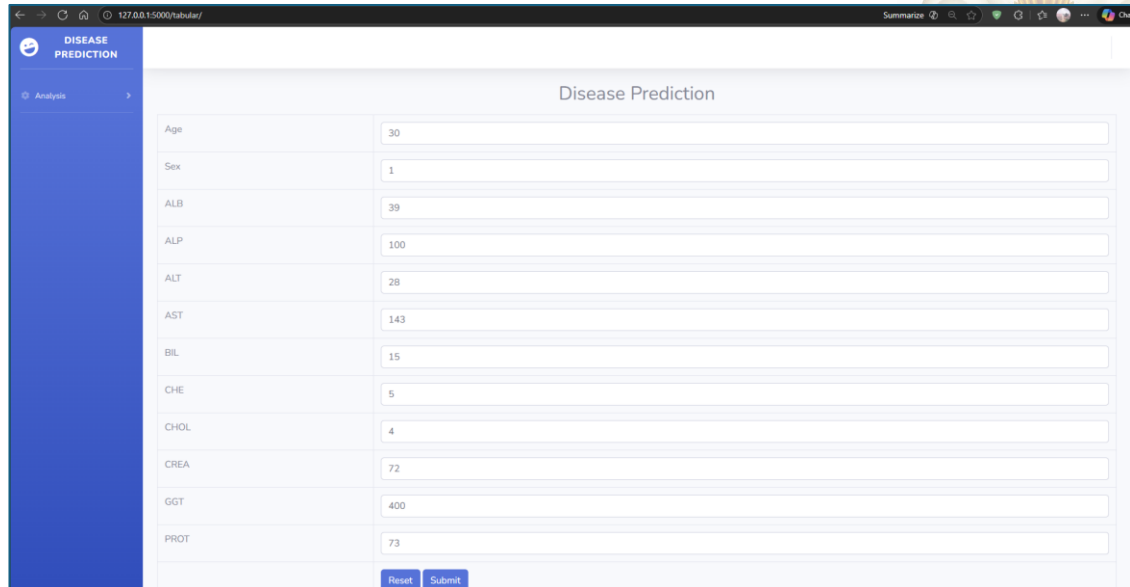
5. Menjalankan aplikasi Flask dengan WSGI

Jalankan script aplikasi Flask. Jika berhasil maka akan tertampil seperti cuplikan

```
[14:48:28] (.venv) ~/test/flask/web/tabular_v2
$ python ./tabular_api.py
* Serving Flask app 'tabular_api'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Gambar 0.12 Menjalankan Flask

Buka url 127.0.0.1:5000/tabular, maka halaman Gambar 0.13 Endpoint Tabular akan tertampil. Masukkan input-input data kemudian dapat mencoba menekan aksi submit.



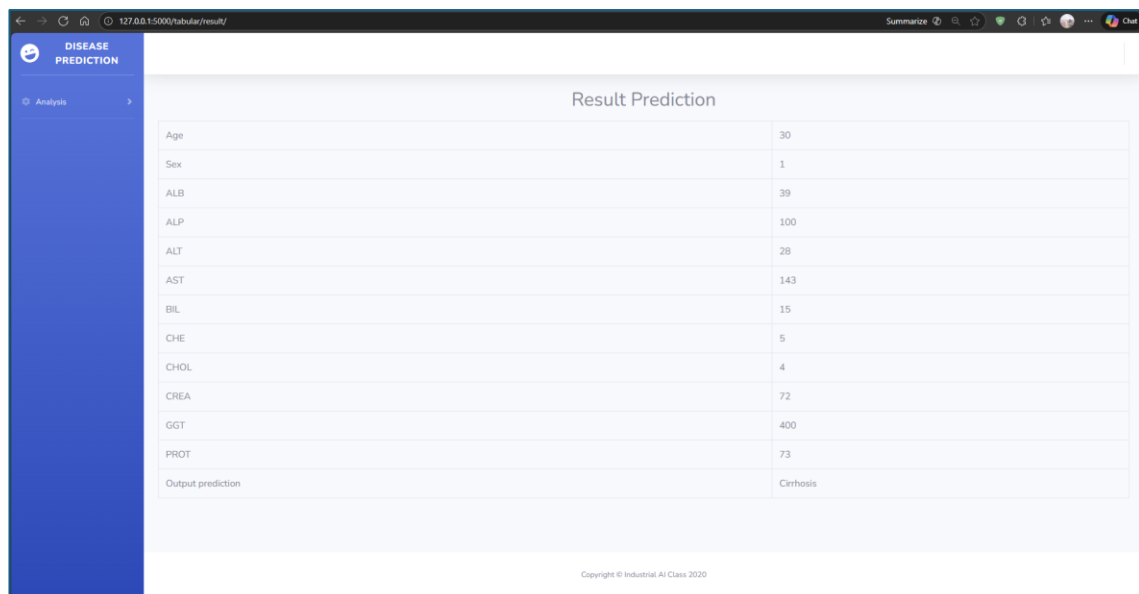
The screenshot shows a web application titled "Disease Prediction". On the left is a blue sidebar with a "DISEASE PREDICTION" header and an "Analysis" link. The main area contains a form with the following fields and values:

Parameter	Value
Age	30
Sex	1
ALB	39
ALP	100
ALT	28
AST	143
BIL	15
CHE	5
CHOL	4
CREA	72
GGT	400
PROT	73

At the bottom of the form are "Reset" and "Submit" buttons.

Gambar 0.13 Endpoint Tabular

Berikut hasil prediksi yang dihasilkan oleh model AI dan tampilan dibuat oleh Flask.



The screenshot shows the same web application, but now displaying the "Result Prediction". The input fields are populated with the same values as in the previous image, and the "Output prediction" field shows the result "Cirrhosis".

Parameter	Value
Age	30
Sex	1
ALB	39
ALP	100
ALT	28
AST	143
BIL	15
CHE	5
CHOL	4
CREA	72
GGT	400
PROT	73
Output prediction	Cirrhosis

At the bottom of the page, there is a small copyright notice: "Copyright © Industrial AI Class 2020".

Gambar 0.14 Endpoint Result Tabular

10.5. TUGAS & ANALISIS

Silakan menggunakan dataset dari proyek kelompok untuk melakukan pelatihan model, kemudian buatlah dashboard dengan Flask. Setelah itu, buatlah laporan praktikumnya.

10.6. REFERENSI

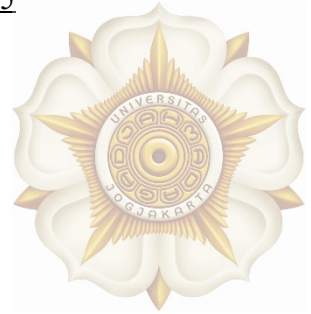
<https://flask.palletsprojects.com/en/stable/installation/>

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.





PERTEMUAN XI

MODEL DEPLOYMENT (STREAMLIT)

11.1. TUJUAN PEMBELAJARAN

- A. Memahami perbedaan paradigma deployment antara Web Service (Flask) dan Data Application (Streamlit).
- B. Menggunakan pustaka Streamlit untuk membuat antarmuka web interaktif berbasis Python.
- C. Membuat input widget (form, slider, dropdown) untuk menerima data dari pengguna.
- D. Mengintegrasikan model machine learning ke dalam dashboard untuk melakukan prediksi secara visual.

11.2. DASAR TEORI

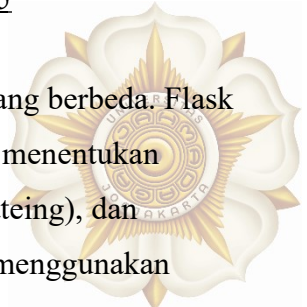
- **Mengenal Streamlit**

Streamlit adalah framework Python open-source yang dirancang khusus untuk Data Scientist dan Machine Learning Engineer. Streamlit memungkinkan pembuatan aplikasi web yang interaktif dan data-centric dalam waktu yang sangat singkat.

Berbeda dengan Flask yang mengharuskan kita menulis logika routing dan template HTML secara terpisah, Streamlit mengubah skrip data menjadi aplikasi web. Setiap kali pengguna berinteraksi dengan widget (misal: menggeser slider), Streamlit akan menjalankan ulang skrip dari atas ke bawah (rerun) untuk memperbarui tampilan.

- **Perbedaan Flask dan Streamlit**

Meskipun keduanya digunakan untuk membuat aplikasi berbasis web dengan Python, filosofi dan cara kerjanya sangat berbeda. Secara konsep kedua hal tersebut memiliki perbedaan di filosofi pengembangan dan juga arsitektur yang digunakan.



Secara filosofi pengembangan, flask dan streamlit memiliki tujuan yang berbeda. Flask memberikan kendali penuh pada pengembangan. Pengembang harus menentukan struktur URL (routing), bagaimana halaman HTML dirender (templateing), dan bagaimana data dikirim (HTTP Request/Response). Pengembangan menggunakan filosofi Flask sangat cocok untuk membangun sistem produksi yang kompleks atau API yang akan dikonsumsi oleh aplikasi lain. Berbeda dengan Streamlit, kecepatan pengembangan menjadi fokus utama. Streamlit “menyembunyikan” kompleksitas web (HTML/CSS/JS) di balik sintaks Python sederhana. Cocok untuk Data Scientist yang ingin mempresentasikan hasil model, membuat prototype cepat, atau dashboard internal tanpa perlu belajar front-end engineering.

Secara arsitektur, flask menggunakan model request-response dan streamlit menggunakan model script rerun. Arsitektur request-response akan berjalan ketika ada permintaan (request) ke URL tertentu. Variabel yang terbuat ketika menjalankan eksekusi request akan hilang setelah request selesai, kecuali disimpan di dalam database atau session. Sementara itu, Streamlit akan menjalankan seluruh skrip Python dari baris paling atas hingga paling bawah setiap kali pengguna berinteraksi dengan widget (misal menggeser slider atau mengetik di text box). Oleh karena itu, variabel biasa akan ter-reset, sehingga streamlit menyediakan fitur *Session State* dan *Caching* untuk mengatasi hal tersebut.

Fitur	Flask	Streamlit
Fokus Utama	Backend API & Web Development Umum	Data Dashboard & Prototyping Cepat
Front End	Perlu HTML/CSS/JS manual	Otomatis
Fleksibilitas	Sangat Tinggi	Terbatas pada pola Data App
Pengguna	Developer & Sistem Lain	End-User



- **Komponen Utama Streamlit**

Streamlit menyediakan berbagai fungsi ("komponen") yang dikategorikan berdasarkan kegunaannya. Berikut adalah komponen vital yang akan sering digunakan dalam Data Mining:

1. **Output dan Text Display (Menampilkan Informasi)**

Komponen ini menggantikan fungsi `print()` di Python biasa atau tag HTML seperti:

`<p>`, `<h1>`, dll.

- `st.write()`

Fungsi ini digunakan untuk menampilkan teks, dataframe, atau grafik secara otomatis tergantung dengan inputnya.

- `st.title()`, `st.header()`, `st.subheader()`

Fungsi ini digunakan untuk membuat hierarki judul untuk mengganti tag HTML `<h1>`, `<h2>`, dst.

- `st.markdown()`

Fungsi ini digunakan untuk menampilkan teks dengan format Markdown (bold, italic, link, latex).

- `st.metrics()`

Fungsi ini sangat berguna untuk dashboard, menampilkan angka besar dengan indikator delta (naik/turun). Contoh: menampilkan akurasi model.

2. **Input Widgets (Menerima Data Pengguna)**

Input widget adalah alat interaksi utama. Nilai kembali (return value) dari widget adalah data yang diinputkan pengguna.

- `st.text_input()`, `st.number_input()`

Fungsi ini digunakan untuk input data manual.

- `st.slider()`, `st.select_slider()`

Fungsi ini digunakan untuk input angka dalam rentang tertentu (cocok digunakan untuk hyperparameter tuning).

- `st.selectbox()`, `st.multiselect()`

Fungsi ini digunakan untuk membuat menu dropdown.

- `st.file_uploader()`



Fungsi ini digunakan untuk mengunggah file (CSV, Gambar, text) yang akan diproses model.

3. Layouts & Container (Mengatur Tata Letak)

Secara default, Streamlit akan menumpuk elemen dari atas ke bawah. Komponen ini membantu merapikan tampilan:

- `st.sidebar`

Fungsi ini digunakan untuk menampilkan sidebar dan dapat digunakan untuk menaruh kontrol input agar tidak mengganggu tampilan utama.

- `st.column()`

Fungsi ini dapat membagi layar menjadi beberapa kolom vertikal (seperti grid).

- `st.tabs()`

Fungsi ini dapat digunakan untuk membuat navigasi tab dalam satu halaman.

- `st.expander()`

Fungsi ini dapat digunakan untuk membuat area yang bisa diklik untuk dibuka-tutup.

4. Caching (Optimasi Performa)

Oleh karena Streamlit menjalankan ulang (rerun) skrip setiap kali ada interaksi, proses berat seperti membuat model Machine Learning atau membaca dataset besar akan membuat aplikasi menjadi lambat jika dilakukan berulang-ulang.

Untuk mengatasi hal tersebut, dekorator `@st.cache_resource` dapat digunakan secara khusus untuk menyimpan objek secara global agar hanya membuat objek tersebut sekali saja. Contoh penggunaan dekorator `@st.cache_resource` seperti berikut.

```
1 # Contoh implementasi Caching nanti di Langkah Percobaan
2 @st.cache_resource
3 def load_model():
4     return joblib.load('iris_model.pkl')
5
6 model = load_model() # Model hanya di-load sekali, tidak berulang-ulang
```

Gambar 0.1 Contoh penggunaan dekorator `st.cache_resource`



Dokumentasi komponen dan cara-cara penggunaan secara lengkap dapat diakses di website Streamlit.

<https://docs.streamlit.io/>

11.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Text Editor / IDE. Catatan: Tidak disarankan menggunakan Jupyter Notebook untuk menjalankan server Flask.
 - Browser
- **Pustaka Python**
 - streamlit
 - pandas
 - joblib atau pickle

11.4. LANGKAH PERCOBAAN

Pada pembuatan modul praktikum, environment yang digunakan adalah sistem operasi Linux Ubuntu di dalam WSL, Python 3.12, dan memiliki akses GPU. Oleh karena ini beberapa perintah dapat disesuaikan dengan target environment deployment masing-masing.

Dalam membuat proyek python alangkah baiknya menggunakan virtual environment yang sesuai dengan environment produksi yang ada di server. Pembuatan environment ini juga dapat membantu dalam manajemen pustaka agar tidak terjadi konflik dengan pustaka global lainnya. Untuk mempelajari lebih lanjut mengenai environment di python Anda dapat membaca referensi berikut:

- (vanilla) <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>



- (UV Astral) <https://docs.astral.sh/uv/>
- (Miniconda) <https://www.anaconda.com/docs/getting-started/miniconda/main>

Dalam percobaan ini, kita akan mencoba membuat sebuah aplikasi. Lakukan langkah-langkah berikut:

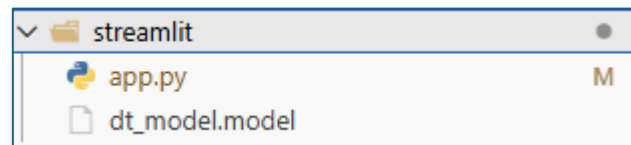
1. Unduh pustaka

Unduh streamlit, pandas, dan joblib. Berikut adalah kode instalasi dengan menggunakan PIP.

```
pip install streamlit pandas joblib
```

2. Struktur Proyek

Buatlah file app.py yang akan digunakan untuk menjalankan aplikasi Streamlit. Masukkan model yang telah dibuat pada pertemuan sebelumnya (dt_model.model) pada direktori yang sama seperti Gambar 0.1.



Gambar 0.1 Struktur Proyek Praktikum Streamlit

3. Jalankan Aplikasi

Jalankan aplikasi streamlit dengan menjalankan perintah berikut pada direktori dimana app.py streamlit berada. Jika berhasil maka tampilan terminal akan terlihat seperti Gambar 0.2. Oleh karena streamlit menggunakan arsitektur re-run, maka perubahan yang ada di dalam file app.py akan terimplementasikan secara langsung di browser setelah browser tersebut di refresh, sehingga terminal dapat dibiarkan saja ketika development aplikasi.

```
streamlit run app.py
```



```
[15:22:36] (.venv) ~/test/streamlit
$ streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.21.212.222:8501
```

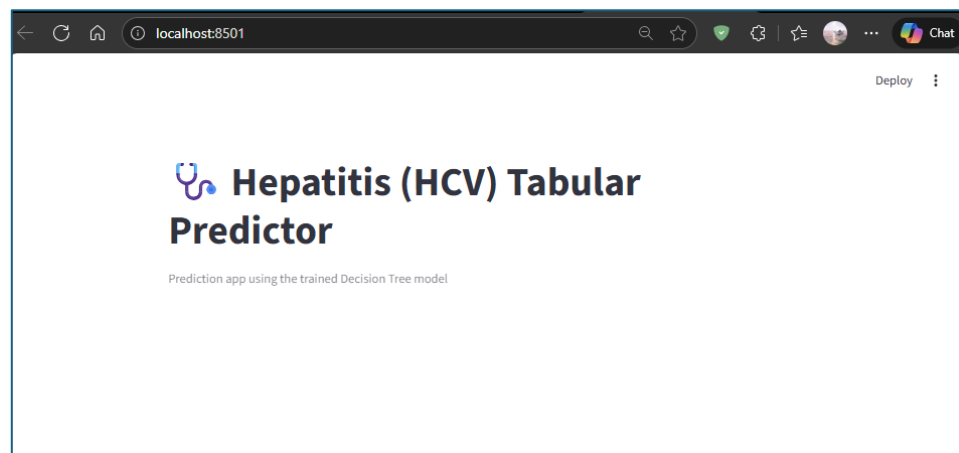
Gambar 0.2 Aplikasi Streamlit berhasil dijalankan

4. Menambahkan konfigurasi halaman web, title, dan caption

Menambahkan konfigurasi halaman web, title, dan caption dapat dilakukan dengan fungsi `set_page_config`, `title`, dan juga `caption`. Tambahkan kode Gambar 0.3 pada skrip `app.py`. Jika aplikasi streamlit dibuka, maka akan tertampil judul dan caption seperti Gambar 0.4.

```
1 import streamlit as st
2
3 st.set_page_config(
4     page_title="HCV Tabular Predictor",
5     page_icon="🩺",
6     layout="centered"
7 )
8
9 st.title("🩺 Hepatitis (HCV) Tabular Predictor")
10 st.caption("Prediction app using the trained Decision Tree model")
```

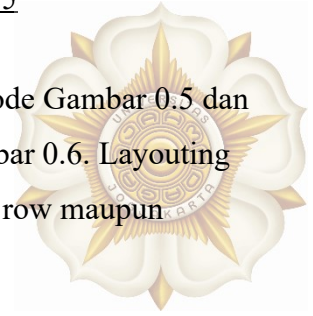
Gambar 0.3 Menambahkan Konfigurasi Web, Title, Caption



Gambar 0.4 Halaman yang tertampil di Streamlit

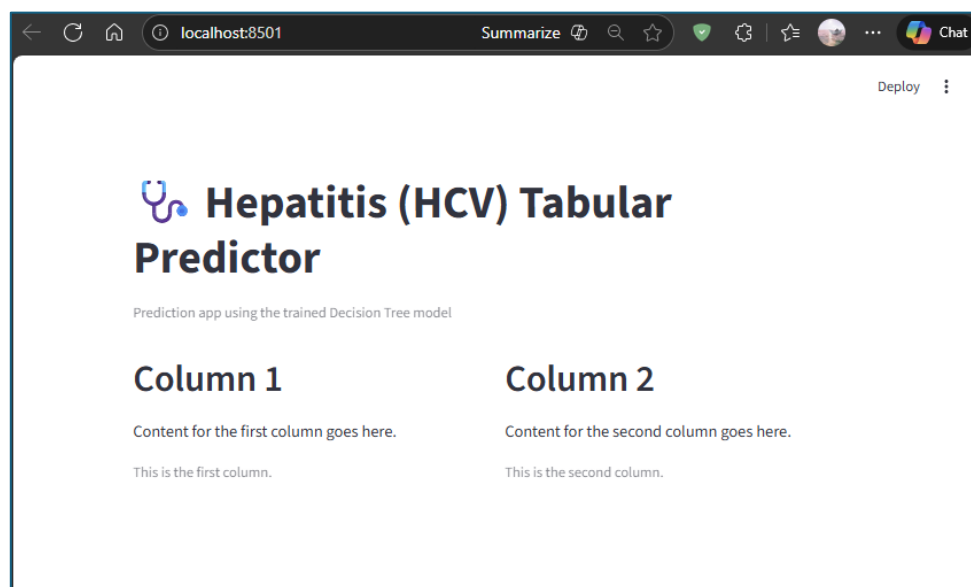
5. Membuat layout dasar

Pada tahap ini akan dibuat layout berupa dua kolom. Tambahkan kode Gambar 0.5 dan refresh halaman web sehingga halaman akan tertampil seperti Gambar 0.6. Layouting tidak terbatas pada column saja, tetapi bisa dikombinasikan dengan row maupun dilakukan nested row ataupun column.



```
1 top_col1, top_col2 = st.columns(2)
2
3 with top_col1:
4     st.header("Column 1")
5     st.write("Content for the first column goes here.")
6     st.caption("This is the first column.")
7 with top_col2:
8     st.header("Column 2")
9     st.write("Content for the second column goes here.")
10    st.caption("This is the second column.")
```

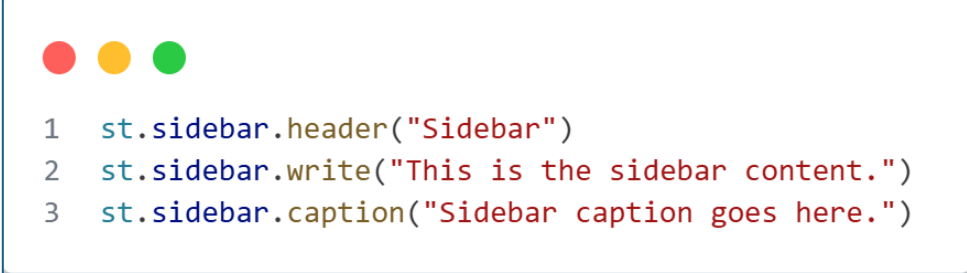
Gambar 0.5 Layouting 2 kolom



Gambar 0.6 Hasil layouting 2 kolom

6. Membuat Sidebar

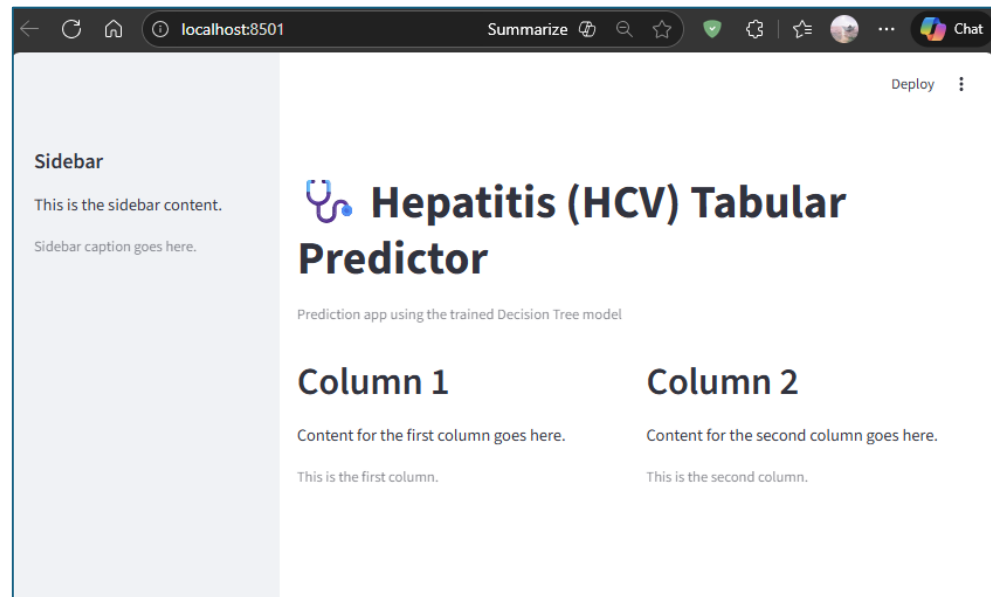
Untuk membuat sidebar sederhana, tambahkan kode Gambar 0.7. Lakukan refresh pada halaman web, maka sidebar akan terbentuk seperti Gambar 0.8.



```

1 st.sidebar.header("Sidebar")
2 st.sidebar.write("This is the sidebar content.")
3 st.sidebar.caption("Sidebar caption goes here.")
    
```

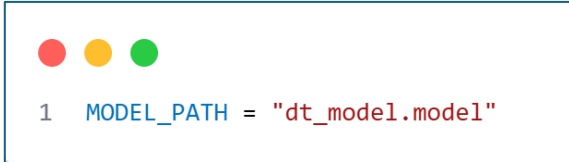
Gambar 0.7 Kode Streamlit Sidebar



Gambar 0.8 Halaman Streamlit diberikan sidebar

7. Melakukan integrasi dengan model AI

Buat variabel `MODEL_PATH` dan definisikan lokasi model berada. Letakkan kode berikut pada bagian atas. Sebagai contoh pembuatan variabel seperti Gambar 0.9.



```

1 MODEL_PATH = "dt_model.model"
    
```

Gambar 0.9 Model Path

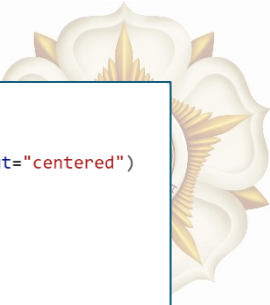
Tambahkan fungsi untuk memuat model seperti pada pertemuan sebelumnya. Berbeda sedikit dengan pertemuan sebelumnya, Pada streamlit, model dimuat dengan bantuan dekorator `cache` streamlit. Hal tersebut dilakukan agar model dapat dimuat dengan cepat di aplikasi streamlit. Kode memuat model dapat dilihat pada Gambar 0.10.



```
1 @st.cache_resource(show_spinner=True)
2 def load_model(path: str = MODEL_PATH):
3     return joblib.load(path)
```

Gambar 0.10 Load model dengan cache

Setelah itu buat logika aplikasi untuk manajemen state beserta fungsi bantuan lainnya yang telah dibuat pada pertemuan sebelumnya. Lengkapi kode yang dibuat menjadi seperti Gambar 0.11. Konstanta default yang didefinisikan pada baris ke 9 tersebut akan digunakan sebagai nilai untuk mereset ulang formulir dari aplikasi. Nilai default ini juga akan di muat ke dalam session pada baris ke 25.



```

1  st.set_page_config(page_title="HCV Tabular Predictor", page_icon="🦋", layout="centered")
2
3  st.title("🦋 Hepatitis (HCV) Tabular Predictor")
4  st.caption("Prediction app using the trained Decision Tree model")
5
6  MODEL_PATH = "dt_model.model"
7
8  # Default input values
9  DEFAULTS = {
10     "age": 30,
11     "sex": "Male",
12     "alb": 39.0,
13     "alp": 100.0,
14     "alt": 28.0,
15     "ast": 143.0,
16     "bil": 15.0,
17     "che": 5.0,
18     "chol": 4.0,
19     "crea": 72.0,
20     "ggt": 400.0,
21     "prot": 70.0,
22 }
23
24 # Initialize session state with defaults if missing
25 for k, v in DEFAULTS.items():
26     if k not in st.session_state:
27         st.session_state[k] = v
28
29 @st.cache_resource(show_spinner=True)
30 def load_model(path: str = MODEL_PATH):
31     return joblib.load(path)
32
33 def _to_float(x: Optional[float]) -> float:
34     try:
35         return float(x) if x is not None else float('nan')
36     except Exception:
37         return float('nan')
38
39 def _encode_sex(x: Optional[str]) -> int:
40     if x is None:
41         return 0
42     s = str(x).strip().lower()
43     if s in ("m", "male", "1", "true"):
44         return 1
45     if s in ("f", "female", "0", "false"):
46         return 0
47     try:
48         return int(float(s))
49     except Exception:

```

Gambar 0.11 Manajemen State dan Fungsi bantuan

Tambahkan tombol yang dapat digunakan untuk mereset nilai formulir ke nilai default. Kode implementasi hal tersebut akan terlihat seperti Gambar 0.12.



```

1 # Top-level Reset button must run before widgets are created
2 top_col1, top_col2 = st.columns([1, 1])
3 with top_col2:
4     if st.button("Reset to defaults", type="secondary"):
5         for k, v in DEFAULTS.items():
6             st.session_state[k] = v
7         st.rerun()

```

Gambar 0.12 Tombol Reset Formulir

Buatlah halaman formulir yang akan digunakan untuk menerima input. Dengan memanfaatkan ilmu layout yang telah dipelajari sebelumnya, praktikan dapat berekperimen untuk membuat formulir input. Sebagai contoh, kode formulir pada streamlit akan terlihat seperti Gambar 0.13. Pada baris ke 19, variabel submitted akan digunakan untuk melihat state tombol formulir apakah ditekan atau tidak. Variabel ini akan digunakan di tahap selanjutnya. Implementasi kode diatas hingga saat ini akan terlihat seperti Gambar 0.15.

```

1 with st.form("input-form"):
2     st.subheader("Enter Patient Lab Values")
3     col1, col2 = st.columns(2)
4     with col1:
5         age = st.number_input("Age", min_value=0, max_value=120, value=st.session_state["age"], step=1, key="age")
6         sex = st.selectbox("Sex", options=["Male", "Female"], index=0 if st.session_state["sex"] == "Male" else 1, key="sex")
7         alb = st.number_input("ALB (Albumin)", min_value=0.0, value=st.session_state["alb"], step=0.1, key="alb")
8         alp = st.number_input("ALP (Alkaline Phosphatase)", min_value=0.0, value=st.session_state["alp"], step=0.1, key="alp")
9         alt = st.number_input("ALT (Alanine Transaminase)", min_value=0.0, value=st.session_state["alt"], step=0.1, key="alt")
10        ast = st.number_input("AST (Aspartate Transaminase)", min_value=0.0, value=st.session_state["ast"], step=0.1, key="ast")
11    with col2:
12        bil = st.number_input("BIL (Bilirubin)", min_value=0.0, value=st.session_state["bil"], step=0.1, key="bil")
13        che = st.number_input("CHE (Cholinesterase)", min_value=0.0, value=st.session_state["che"], step=0.1, key="che")
14        chol = st.number_input("CHOL (Cholesterol)", min_value=0.0, value=st.session_state["chol"], step=0.1, key="chol")
15        crea = st.number_input("CREA (Creatinine)", min_value=0.0, value=st.session_state["crea"], step=0.1, key="crea")
16        ggt = st.number_input("GGT (Gamma-Glutamyl Transferase)", min_value=0.0, value=st.session_state["ggt"], step=0.1, key="ggt")
17        prot = st.number_input("PROT (Total Protein)", min_value=0.0, value=st.session_state["prot"], step=0.1, key="prot")
18
19    submitted = st.form_submit_button("Predict")

```

Gambar 0.13 Kode Formulir Streamlit

Buatlah output untuk menampilkan hasil prediksi beserta logika prediksinya seperti Gambar 0.14. Variabel submitted yang dibuat di tahap sebelumnya dipakai sebagai logika menampilkan output sehingga ketika tidak ada input, laman output tidak tertampil. Fungsi with spinner seperti pada baris 2 Gambar 0.14 digunakan untuk menampilkan UI loading. Di dalam fungsi tersebut terdapat aksi pembuatan model, prediksi (inference), dan pembaca hasil output. Logika ini tidak jauh berbeda dari

pertemuan sebelumnya. Implementasi tampilan output tersebut akan terlihat seperti Gambar 0.16.

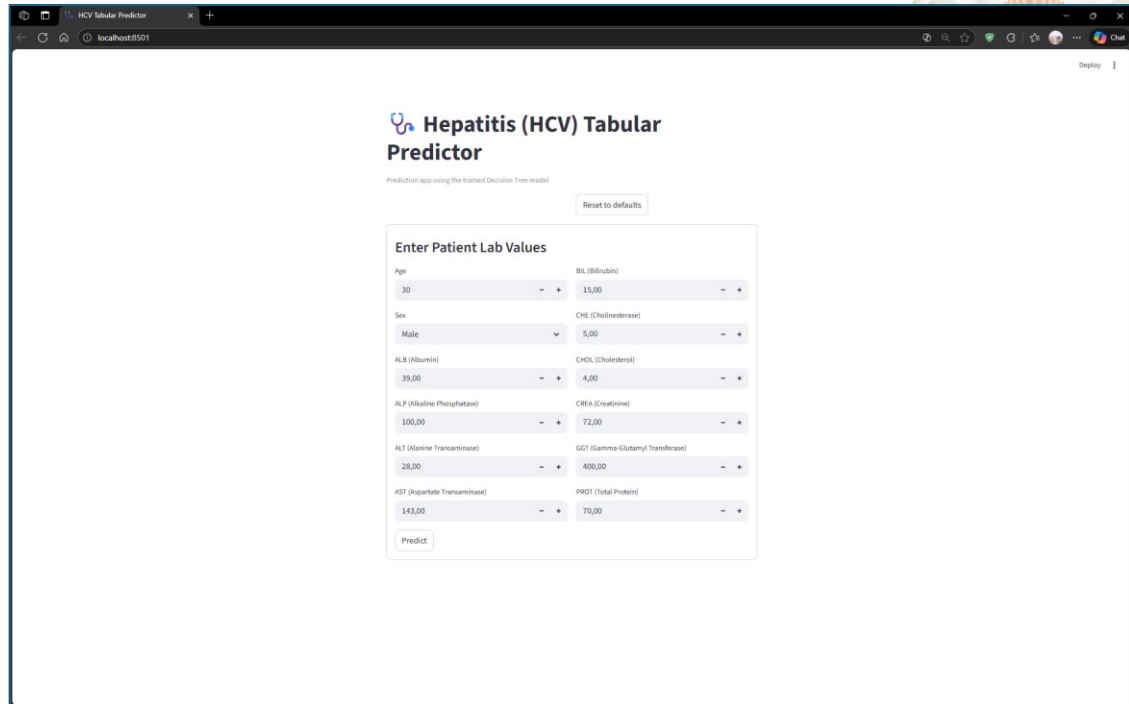


```

1  if submitted:
2      with st.spinner("Loading model and predicting..."):
3          model = load_model(MODEL_PATH)
4          sex_enc = _encode_sex(sex)
5
6          columns_order = ['Sex', 'Age', 'ALB', 'ALP', 'ALT', 'AST', 'BIL', 'CHE', 'CHOL', 'CREA', 'GGT', 'PROT']
7          df_input = pd.DataFrame(columns=columns_order)
8          df_input.loc[0] = [
9              sex_enc,
10             _to_float(age),
11             _to_float(alb),
12             _to_float(alp),
13             _to_float(alt),
14             _to_float(ast),
15             _to_float(bil),
16             _to_float(che),
17             _to_float(chol),
18             _to_float(crea),
19             _to_float(ggt),
20             _to_float(prot),
21         ]
22
23         result = model.predict(df_input)
24         int_result = int(result[0])
25         if int_result == 0:
26             decision = "Suspect Blood Donor"
27         elif int_result == 1:
28             decision = "Hepatitis"
29         elif int_result == 2:
30             decision = "Fibrosis"
31         elif int_result == 3:
32             decision = "Cirrhosis"
33         else:
34             decision = "Not defined"
35
36         st.success("Prediction complete")
37
38         st.metric(label="Predicted Status", value=decision)
39
40         with st.expander("Input Summary"):
41             st.write({
42                 "age": age,
43                 "sex": sex,
44                 "alb": alb,
45                 "alp": alp,
46                 "alt": alt,
47                 "ast": ast,
48                 "bil": bil,
49                 "che": che,
50                 "chol": chol,
51                 "crea": crea,
52                 "ggt": ggt,
53                 "prot": prot,
54             })

```

Gambar 0.14 Halaman Output Prediksi

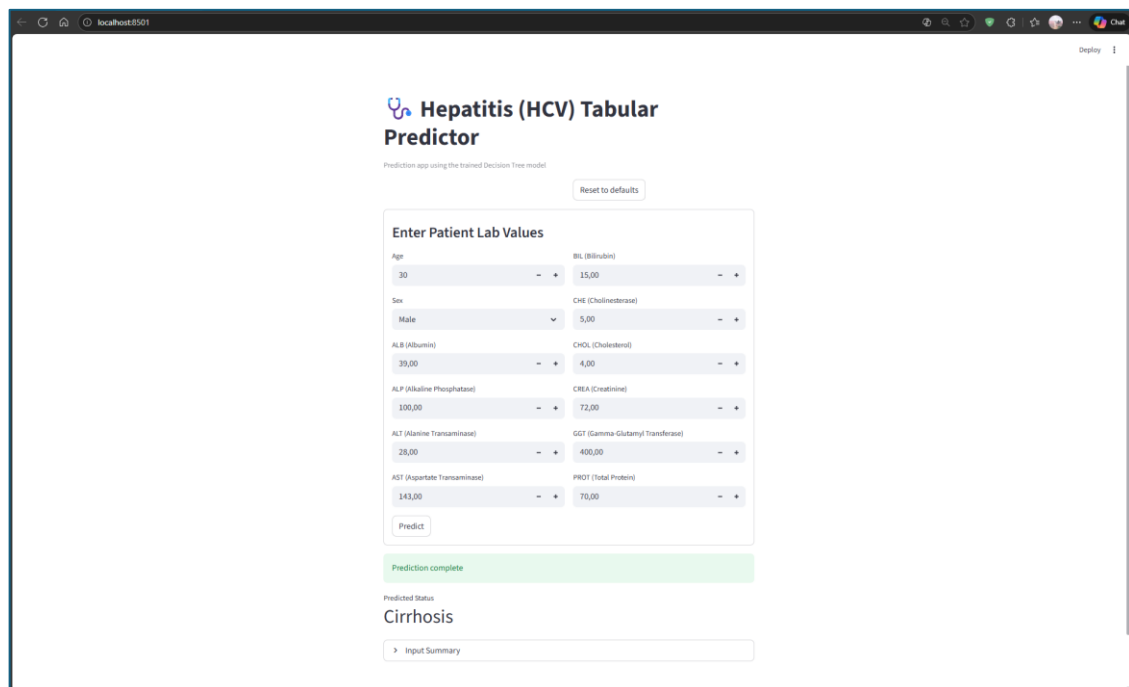


The screenshot shows a web browser window with the URL `localhost:8501`. The page title is "Hepatitis (HCV) Tabular Predictor". Below the title, it says "Prediction app using the trained Decision Tree model". There is a "Reset to defaults" button. The main section is titled "Enter Patient Lab Values" and contains a form with the following fields:

Field	Value
Age	30
Sex	Male
ALB (Albumin)	39,00
ALP (Alkaline Phosphatase)	100,00
ALT (Alanine Transaminase)	28,00
AST (Aspartate Transaminase)	143,00
BL (Bilirubin)	15,00
CHL (Cholesterol)	4,00
CHL (Cholinesterase)	5,00
CREA (Creatinine)	72,00
GGT (Gamma Glutamyl Transferase)	400,00
PROT (Total Protein)	70,00

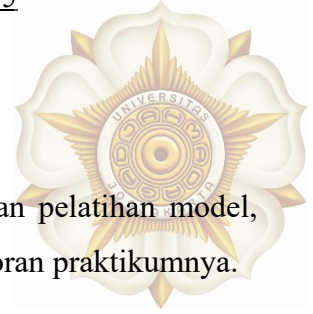
There is a "Predict" button at the bottom of the form.

Gambar 0.15 Halaman Input



The screenshot shows the same web browser window as before, but now the "Predict" button has been clicked. A green bar at the bottom of the form says "Prediction complete". Below the form, the "Predicted Status" is displayed as "Cirrhosis". There is a link to "Input Summary" below the predicted status.

Gambar 0.16 Halaman Input dan Output



11.5. TUGAS & ANALISIS

Silakan menggunakan dataset dari proyek kelompok untuk melakukan pelatihan model, kemudian buatlah dashboard dengan Streamlit. Setelah itu, buatlah laporan praktikumnya.

11.6. REFERENSI

<https://docs.streamlit.io/>

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.